



Programming

CD همراه شامل :

کلیه برنامه های داخل کتاب

مخصوص درس مبانی کامپیوتر

دانشجویان کامپیوتر و IT

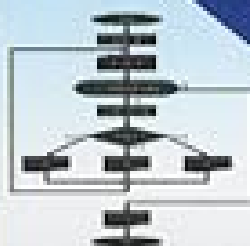


آموزش مبانی کامپیوتر و برنامه نویسی به زبان C++

www.naghoospress.ir

2008

```
int main(int argc, char *argv[])  
{  
    int ap, myid, myWindowOpened;  
    int allWindowsOpened;  
    Winspecs winspecs;  
    MPE_XGraph;  
    .  
    .  
    .  
}
```



نویسندگان :

دکتر ناصر قاسم آقایی

مهدی جابرواده انصاری

علی دهقان

آموزش مبانی کامپیوتر و برنامه‌نویسی به زبان

C++

دکتر ناصر قاسم‌آقائی

مهندس مهدی جابرزاده انصاری

مهندس علی دهقان



شرکت ناقوس اندیشه
(سهامی خاص)

سرشناسه : قاسم آقائی، ناصر، ۱۳۳۰ -
 عنوان و نام پدیدآور : آموزش مبانی کامپیوتر و برنامه‌نویسی به زبان C++ / ناصر قاسم آقائی، مهدی جابرزاده انصاری، علی دهقان.
 مشخصات نشر : تهران: شرکت ناگوس اندیشه: زانئیس، ۱۳۸۶.
 مشخصات ظاهری : [۵۲۸] ص. : مصور، جدول.
 شابک : ۸۵۰۰۰ ریال: ۹۷۸-۹۶۴-۳۷۷-۳۱۲-۰
 وضعیت فهرست‌نویسی: فیا.
 یادداشت : واژه‌نامه.
 یادداشت : کتابنامه: ص. [۵۲۸].
 موضوع : سی C++ (زبان برنامه‌نویسی کامپیوتر).
 موضوع : کامپیوترها—راهنمای آموزشی.
 شناسه افزوده : جابرزاده انصاری، مهدی، ۱۳۶۴ -
 شناسه افزوده : دهقان، علی، ۱۳۶۴ -
 رده بندی کنگره : ۹۳/۷۳/س QAV۶
 رده بندی دیویی : ۰۰۵/۱۳۳
 شماره کتابشناسی ملی : ۱۱۰۷۱۹۹



www.naghoospress.ir



چاپ اول

نام کتاب : آموزش مبانی کامپیوتر و برنامه‌نویسی به زبان C++
 مؤلفان : ناصر قاسم آقائی، مهدی جابرزاده انصاری، علی دهقان
 ناشر : شرکت ناگوس اندیشه
 ناشر همکار : انتشارات زانئیس
 چاپ اول : ۱۳۸۶
 تیراژ : ۲۰۰۰ جلد
 لیتوگرافی : فرازنگر
 چاپ : سعدی
 صحافی : صفحه‌پرداز
 حروفچینی و صفحه‌آرایی : واحد تولید
 حروف‌نگار : مریم رضائی
 قیمت با CD : ۸۵۰۰۰ ریال
 شابک : ۹۷۸-۹۶۴-۳۷۷-۳۱۲-۰
 ISBN : 978-964-377-312-0

کلیه حقوق برای ناشر محفوظ است. تکثیر تمامی یا قسمتی از این اثر به صورت حروفچینی یا چاپ مجدد، چاپ افست، پلی‌کپی، فتوکپی و انواع دیگر چاپ ممنوع است و پیگرد قانونی دارد.

مرکز پخش: شرکت ناگوس اندیشه

تهران: خیابان انقلاب، خیابان ۱۶ آذر، خیابان نصرت، پلاک ۱۵

تلفن: ۸۸۹۸۴۶۹۴-۵

فهرست

فصل اول : آشنایی با علم کامپیوتر و ویژگی‌های الگوریتم ۱۱	فصل سوم : مقدمات زبان ++C ۸۹
۱-۱ : پیشگفتار ۱۲	۱-۳ : ویژگی‌های زبان ++C ۹۰
۲-۱ : تاریخچه و نسل‌های کامپیوتر ۱۴	۲-۳ : کار با انواع داده‌ها در ++C ۹۴
۳-۱ : سخت‌افزار و ساختمان کامپیوترهای امروزی ۲۰	انواع داده اصلی در ++C ۹۵
۴-۱ : انواع کامپیوتر و کاربرد آن ۲۳	انواع داده فرعی در ++C ۹۹
۵-۱ : بررسی مفهوم الگوریتم ۲۴	نام‌گذاری متغیرها ۱۰۰
تعریف الگوریتم ۲۷	تعریف متغیر ۱۰۱
۶-۱ : تمرین ۳۰	مقداردهی به متغیرها ۱۰۱
فصل دوم : آشنایی با مقدمات زبان روندنما ۳۱	اعلان ثوابت ۱۰۲
۱-۲ : آشنایی با قواعد مقدماتی رسم روندنما ۳۲	۳-۳ : انواع عملگر در ++C ۱۰۳
جعبه‌های شروع و پایان ۳۳	بررسی عملگرهای حسابی ۱۰۳
ورود و خروج اطلاعات ۳۴	بررسی عملگرهای رابطه‌ای ۱۰۵
متغیرها و جعبه انتساب ۳۵	بررسی عملگرهای منطقی ۱۰۵
رسم اولین کارنما ۳۶	
۲-۲ : تصمیم‌گیری در الگوریتم‌ها ۳۸	
جعبه‌های تصمیم ۳۸	
بررسی عبارات رابطه‌ای ساده ۳۹	
۳-۲ : تمرین ۴۲	
۴-۲ : تکرار در الگوریتم‌ها ۴۴	
شمارنده‌ها و نگهدارنده‌ها ۴۴	
۵-۲ : تمرین ۴۸	
۶-۲ : مقدمه‌ای بر الگوریتم‌های جستجو ۵۶	
الگوریتم پیدا کردن بزرگترین عدد از میان چند عدد ۵۶	
الگوریتم شمارش نمرات ۵۸	
۷-۲ : تمرین ۶۰	
۸-۲ : تقدم عملگرها در عبارات جبری ۶۱	

۱۴۸	تمرین ۴-۴	۱۰۶	عملگرهای ترکیبی
۱۵۵	حلقه‌های تودرتو	۱۰۶	عملگر ()
۱۵۷	تمرین ۶-۴	۱۰۶	عملگر sizeof
فصل پنجم : ساختارهای تکرار و تصمیم‌گیری در C++		۴-۳	نخستین تجربه برنامه‌نویسی در محیط Visual C++ 6.0
۱۵۹	C++	۱۰۷	آماده‌سازی محیط VC6
۱۶۰	۱-۵: ساختارهای تکرار در C++	۱۱۰	اجرا کردن نخستین برنامه
۱۶۰	ساختار تکرار for	۱۱۱	۵-۳: ساختار کلی برنامه در C++
۱۶۴	حلقه‌های تودرتو	۱۱۱	تابع main
۱۶۵	ساختار تکرار while	۱۱۲	دستورات پیش پردازنده
۱۶۶	ساختار تکرار do ... while	۱۱۳	مستند سازی و توضیحات
۱۶۸	۲-۵: انتقال کنترل غیرشرطی	۱۱۴	۶-۳: ورودی و خروجی در C++
۱۶۸	دستور break	۱۱۴	شیء cout
۱۶۸	دستور continue	۱۱۵	شیء cin
۱۶۹	دستور goto	۱۱۶	دستور using
۱۷۰	۳-۵: ساختارهای تصمیم در C++	۱۱۷	۷-۳: تبدیل انواع
۱۷۰	ساختار تصمیم if	۱۱۸	تبدیل نوع اتوماتیک
۱۷۵	ساختار تصمیم else if	۱۲۰	تبدیل نوع موقت
۱۷۶	پاک کردن صفحه نمایش	۸-۳	نحوه ایجاد فایل اجرایی توسط کامپایلر
۱۷۷	انتقال مکان‌نما در مانیتور	۱۲۴	توابع و فایل‌های کتابخانه‌ای
۱۸۱	تولید اعداد تصادفی	۹-۳	تمرین
۱۸۲	ساختار switch	۱۰-۳	موارد مطالعاتی
۱۸۴	عملگر شرطی	فصل چهارم : ساختارهای حلقه‌زنی	
۱۸۷	تمرین ۴-۵	۱-۴	الگوریتم ساخت یافته
۱۹۱	۵-۵: موارد مطالعاتی	۲-۴	جعبه‌های تکرار
فصل ششم : نقش آرایه و متغیرهای لیستی در الگوریتم		۳-۴	مثال‌هایی بیشتر از به‌کارگیری جعبه‌های تکرار
۱۹۴	۱-۶: آرایه و کاربرد آن	۱۴۰	کار با فاکتوریل در الگوریتم
۱۹۴	مقدمه	۱۴۱	مسائل ساده‌سازی و تقسیم اعداد
۲۰۱	جستجوی ترتیبی	۱۴۳	هم عامل‌های یک عدد صحیح
۲۰۲	تمرین ۲-۶	۱۴۵	الگوریتم اقلیدس

۳-۶: عملیات بر روی مجموعه ها	۲۱۳
۴-۶: تمرین	۲۱۵
۵-۶: مرتب سازی و جستجو	۲۱۶
نخستین الگوریتم مرتب سازی	۲۱۸
مرتب سازی حبابی	۲۲۱
مرتب سازی درجی	۲۲۴
جستجوی دودویی	۲۲۶
۶-۶: تمرین	۲۲۸
۷-۶: کاربرد متغیرهای کمکی	۲۳۳
۸-۶: آرایه های دو بعدی	۲۳۴
۹-۶: تمرین	۲۳۹
فصل هفتم: آرایه ها و رشته ها در ++C ... ۲۵۱	
۱-۷: آرایه های یک بعدی	۲۵۲
تعریف آرایه	۲۵۲
دسترسی به عناصر آرایه	۲۵۳
فضای اشغال شده برای آرایه	۲۵۴
مقداردهی به آرایه یک بعدی	۲۵۴
۲-۷: آرایه های چند بعدی	۲۵۸
آرایه ای از آرایه ها	۲۵۸
مقداردهی به آرایه چند بعدی	۲۶۱
۳-۷: کانتینر بردار vector	۲۶۳
عملیات بر روی وکتورها	۲۶۴
۴-۷: وکتورهای چند بعدی	۲۷۴
بردارهای دندانه ای	۲۷۵
مقایسه آرایه و وکتور	۲۷۶
۵-۷: رشته های زبان C	۲۷۷
مقداردهی اولیه به رشته ها	۲۷۷
خواندن رشته از ورودی	۲۷۸
آرایه ای از رشته ها	۲۸۲
۶-۷: کلاس string	۲۸۳
توابع عضو کلاس string	۲۸۴
۷-۷: تمرین	۳۰۴
۸-۷: موارد مطالعاتی	۳۰۶
فصل هشتم: مفهوم رویه در الگوریتم و مقدمه ای بر رمزنگاری ... ۳۰۷	
۱-۸: تجزیه گام به گام	۳۰۸
روش حل کل به جزء	۳۰۸
الگوریتم تجزیه یک عدد به عامل های اول	۳۱۰
۲-۸: تمرین	۳۱۴
۳-۸: نگاره سازی با کامپیوتر	۳۱۸
۴-۸: تمرین	۳۲۱
۵-۸: گسترش تجزیه الگوریتم ها و پیدایش مفهوم رویه (تابع)	۳۲۲
۶-۸: تمرین	۳۲۶
۷-۸: مقدمه ای بر رمزنگاری	۳۲۷
رمزگذاری جانشینی	۳۲۷
رویه رمزگذاری	۳۳۲
رویه رمزگشایی	۳۳۷
رمزگذاری به روش جایگشت	۳۴۰
۸-۸: تمرین	۳۴۱
۹-۸: نمودار N-S	۳۴۲
۱۰-۸: تمرین	۳۵۰
فصل نهم: توابع و کلاس های حافظه در ++C ... ۳۵۱	
۱-۹: تعریف توابع در ++C	۳۵۲
نوشتن توابع	۳۵۲
دسته بندی توابع	۳۵۵
توابع نوع void	۳۵۶
توابع غیر void	۳۵۸
آرایه ها به عنوان آرگومان تابع	۳۶۱
حذف اعلان توابع	۳۶۵
توابع inline	۳۶۶
۲-۹: توابع ریاضی کتابخانه ای	۳۶۸

۴۱۸	تمرین ۴-۱۰	۳۷۲	سربارگذاری و قالب‌های تابع
۴۲۲	قضیه اصلی ۵-۱۰	۳۷۲	سربارگذاری توابع هم نام
۴۲۵	تمرین ۶-۱۰	۳۷۳	آرگومان‌های دارای پیش فرض
۴۲۶	حل رابطه‌های بازگشتی ۷-۱۰	۳۷۴	تعریف توابع به صورت قالب
۴۲۶	رابطه بازگشتی خطی همگن	۳۷۵	توابع کلی
۴۲۶	مرتبه رابطه بازگشتی	۳۷۷	کامپایلر چه کاری انجام می‌دهد؟
۴۲۹	تمرین ۸-۱۰	۳۷۸	تابعی با دو نوع کلی
۴۳۰	تکنیک تکرار در حل مسائل ۹-۱۰	۳۷۹	توابع قالب و انواع استاندارد
۴۳۲	تمرین ۱۰-۱۰	۳۸۰	تعریف مجدد تابع کلی
۴۳۳	روش تقسیم و حل ۱۱-۱۰	۳۸۱	چرا ماکروها نه؟
۴۳۹	تمرین ۱۲-۱۰	۳۸۲	تعریف مجدد قالب تابع
۴۴۰	برنامه‌نویسی پویا ۱۳-۱۰	۳۸۳	۴-۹: کلاس‌های حافظه
۴۴۶	تمرین ۱۴-۱۰	۳۸۳	متغیرهای محلی و سراسری
۴۴۷	تمرینات تکمیلی ۱۵-۱۰	۳۸۵	کلاس‌های حافظه
		۳۸۵	کلاس حافظه اتوماتیک
فصل یازدهم: اشاره‌گرها و مرجع ۴۵۱		۳۸۶	کلاس حافظه ثابت
۴۵۲	۱-۱۱: مبانی اشاره‌گرها	۳۸۶	کلاس حافظه استاتیک
۴۵۲	تعریف متغیر اشاره‌گر	۳۸۸	کلاس حافظه خارجی
۴۵۵	عملگر آدرس	۳۸۹	۵-۹: برنامه‌های چند فایل
۴۵۷	عملگر مقدار	۳۹۳	۶-۹: آرگومان‌های تابع main
۴۵۸	اعمال بروی اشاره‌گرها	۳۹۶	۷-۹: ماکروها و فراخوانی توابع با نام
۴۶۰	اشاره‌گر نوع void	۴۰۱	۸-۹: تمرین
۴۶۱	۲-۱۱: اشاره‌گرها و توابع	۴۰۳	۹-۹: موارد مطالعاتی
۴۶۱	آرگومان‌ها و اشاره‌گرها		فصل دهم: آشنایی مقدماتی با روش‌های
۴۶۳	اجرای توابع با آدرس آنها		طراحی و تحلیل الگوریتم‌ها ۴۰۵
۴۶۷	۳-۱۱: اشاره‌گرها و آرایه‌ها	۱-۱۰: آشنایی با مقدمات تحلیل الگوریتم‌ها	
۴۶۷	متغیرهای پویا	۴۰۶	
۴۶۷	تخصیص پویای حافظه	۴۰۶	تحلیل الگوریتم‌ها
۴۶۸	توابع malloc و free	۴۰۷	نماد O
۴۶۸	تعریف آرایه یک بعدی پویا	۴۱۰	نماد Θ
	بازگرداندن فضای یک آرایه پویا به حافظه	۴۱۱	نماد Ω
۴۷۰		۲-۱۰: تمرین	
۴۷۲	اشاره‌گر به اشاره‌گر	۳-۱۰: الگوریتم‌های بازگشتی	

فصل دوازدهم : بخش ضمیمه ها ۴۹۰	اشاره‌گرهایی با مراتب بالاتر ۴۷۵
پیوست ۱ : نصب VS98 ۴۹۱	ارسال آرایه به توابع ۴۷۶
پیوست ۲ : جدول کد اسکی ۴۹۴	عملگر const و اشاره‌گر ۴۷۶
پیوست ۳ : سرفایل limits.h ۴۹۶	اشاره‌گرها و رشته‌های زبان C ۴۷۹
پیوست ۴ : سرفایل math.h ۴۹۹	آرایه‌های دندانه‌ای ۴۸۰
پیوست ۵ : جدول کلیدهای ترکیبی ۵۱۳	طراحی منو و کلیدهای تابع ۴۸۱
پیوست ۶ : بهینه‌سازی در کامپایلرها ۵۱۵	۴-۱۱ : مرجع ۴۸۱
پیوست ۷ : کامپایل برنامه در لینوکس ۵۲۲	تعریف مرجع ۴۸۱
لغت‌نامه ۵۲۵	محدودیت‌های مرجع ۴۸۲
فهرست منابع و مآخذ ۵۲۸	مرجع به عنوان آرگومان تابع ۴۸۳
	مرجع به عنوان مقدار بازگشتی ۴۸۴
	عملگر const و مرجع ۴۸۵
	۵-۱۱ : تمرین ۴۸۶
	۶-۱۱ : موارد مطالعاتی ۴۸۹

مقدمه

هم اکنون که در حال نگاشتن مقدمه کتاب هستیم، ماه‌هاست که کار نگارش متن کتاب پایان یافته و مراحل تایپ، صفحه‌آرایی و ویرایش آن، همگی به اتمام رسیده‌اند. لذا بسیاری از مباحث و موضوعات مهم در خصوص نحوه مطالعه کتاب، ویژگی‌های آن و بسیاری از مطالب دیگر پیشتر در خلال سطور کتاب عنوان شده است. اما مقدمه هر کتاب جایگاه و اهمیت خاص خود را دارد، چرا که بسیاری از افراد قبل از آنکه تصمیم به مطالعه یک کتاب بگیرند، مقدمه آن را بررسی می‌کنند تا از ویژگی‌های کتاب مذکور مطلع گردند و در صورتی که آن کتاب را مفید یافتند، سعی در مطالعه آن می‌نمایند؛ لذا پوزش ما را به جهت مطرح کردن مباحثی که اندکی بعد در داخل متن کتاب خواهید یافت پذیرا باشید.

درس مبانی کامپیوتر نخستین درس تخصصی است که دانشجویان رشته‌های کامپیوتر و فناوری اطلاعات، پشت سر می‌گذارند. متأسفانه در برخی از مراکز آموزش عالی، برخلاف سر فصلی که وزارت علوم برای این درس مشخص کرده، و بر بررسی الگوریتم و آموزش زبان روندنما (فلوچارت) تأکید کرده است، دانشجو را به یکباره و مستقیم وارد مباحث برنامه‌نویسی می‌کنند، و سعی بر آن دارند که مفاهیم مرتبط با شیوه حل مسائل به روش الگوریتمی را به واسطه یک زبان برنامه‌نویسی به دانشجویان آموزش دهند. البته می‌توان این رفتار را ناشی از خلای دانست که نبود کتب مشخصی برای این واحد درسی به وجود آورده است. از طرفی دانشجویان نیز تنها مبادرت به تهیه کتب آموزش برنامه‌نویسی می‌کنند و در نتیجه اثرات ناشی از انتقال نیافتن یک دید الگوریتمی به دانشجو در سال‌های بالاتر و در واحدهای بعدی خود را نشان می‌دهد. با توجه به این اوصاف نیاز به تهیه کتابی جامع که هم مباحث علم کامپیوتر و طراحی الگوریتم و هم برنامه‌نویسی به یک زبان قدرتمند و قابل قبول در حد مبانی کامپیوتر را شامل شود، به شدت احساس می‌شد. لذا عمده‌ترین هدف در تهیه این کتاب بر طرف کردن نیاز دانشجویان و اساتید به یک کتاب واحد جهت تدریس در درس مبانی کامپیوتر است. در این کتاب به ازای هر یک فصل که در خصوص مباحث علم کامپیوتر و الگوریتم بحث شده است بلافاصله یک فصل نیز به بیان مطالب برنامه‌نویسی به روش ساخت یافته در ارتباط با همان مفاهیم الگوریتم-نویسی، و تحت زبان ++C پرداخته شده است. بدین ترتیب دانشجو در طول یک ترم به مرور طی یک برنامه جامع و هماهنگ هم مباحث مربوط به رسم روندنما و حل مسائل به شیوه الگوریتمی را فرا می‌گیرند و هم مباحث برنامه‌نویسی معادل آن‌ها را. با عنایت به آنکه ارزش کتب درسی به میزان تمرینات موجود در آن است در فصول مختلف تمرینات بسیاری را به فراخور بحث و با ترتیب خاص از آسان به دشوار تنظیم نموده ایم.

متن کتاب بسیار ساده و روان و همراه با توضیح ریزترین نکات می‌باشد، که قضاوت در این زمینه را به عهده خوانندگان عزیز می‌گذاریم. کتاب حاضر با این دیدگاه نگاشته شده است که خواننده هیچ آشنایی قبلی با کامپیوتر ندارد و باید او را قدم به قدم تا مرز یک برنامه‌نویس نیمه حرفه‌ای کامپیوتر پیش برد. لذا افراد مبتدی هیچ نگرانی در خصوص فهم مطالب کتاب نداشته باشند چرا که این کتاب تا حد بسیار زیادی به شکل خود آموز طراحی شده است. از نظر شکل و ساختار کتاب و دیگر ویژگی‌های آن در قسمت پیشگفتار مطالبی را عنوان خواهیم کرد و تنها این نکته را می‌افزایم که در حدود ۵۰ نفر ساعت کار کارشناسی از سوی دوست عزیز آقای مهندس محمد حقایق که از دانشجویان کارشناسی ارشد در رشته روانشناسی در دانشگاه اصفهان هستند صورت پذیرفته، تا به شکل و قالب فعلی کتاب رسیده‌ایم با این امید که بتوانیم میل خوانندگان را برای مطالعه صفحات بعدی کتاب هر چه بیشتر کنیم، و صفحات کتاب شکل خسته-

کننده‌ای را نداشته باشد. چه بسا کتبی وجود دارند که مطالب بسیار مفیدی را دربرمی‌گیرند اما به جهت صورت نا خوشایند صفحات و عدم تنظیم صحیح مطالب با اقبال عمومی روبرو نشده‌اند. به عنوان مثال در این کتاب قسمت‌های برنامه‌نویسی را به صورت رنگی وارد کرده‌ایم و به گونه‌ای رنگ کلمات انتخاب شده که در کامپایلرها انجام می‌گیرد تا بر یادگیری دانشجویان بیفزاید.

همچنین در ابتدای فصل سوم مطالب مفید و قانع‌کننده‌ای را در خصوص دلایل انتخاب زبان C++ (در برابر زبان جاوا) برای آموزش در این کتاب و برای درس مبانی عنوان نموده‌ایم و تنها این نکته را اضافه می‌کنم که اگر زبانی همچون جاوا را برای آموزش به افراد مبتدی در درس مبانی انتخاب کنیم، یک اشتباه برگشت‌ناپذیر خواهد بود. چرا که در آموزش جاوا از ابتدا باید مفاهیم شی‌گرایی را مطرح کرد همچون class، درحالی که دانشجوی درس مبانی هنوز با برنامه‌نویسی ساخت یافته نیز آشنایی ندارد، از طرفی این زبان برخی مفاهیم مهم شی‌گرایی همچون سربارگذاری عملگرها را شامل نمی‌شود که خود نقص بسیار بزرگی برای یک زبان آموزشی در درس برنامه‌نویسی پیشرفته نیز محسوب می‌شود. لذا در جلد دوم این مجموعه نیز به بیان مفاهیم شی‌گرایی و برنامه‌نویسی شی‌گرا به واسطه زبان C++ خواهیم پرداخت و در کنار آن زبان C# که دقیقاً دارای ساختاری مشابه زبان جاوا ولی تکامل یافته‌تر و نزدیک‌تر به زبان C++ است، مورد بررسی قرار می‌گیرد. بد نیست بدانید C++ مادر زبان جاوا و هر دوی این زبان‌ها مادر زبان C# هستند. همچنین جهت تهیه لوح‌های فشرده شماره ۲ و ۳ و ۴ که شامل کامپایلر Visual Studio 6.0 و MSDN مربوطه است به وب سایت کتاب مراجعه کنید. شایان ذکر است تمامی برنامه‌های موجود در کتاب ابتدا بر روی این کامپایلر تست و اجرا شده و به درستی وارد کتاب گردیده است و همچنین بر روی لوح فشرده شماره ۱ که همراه کتاب عرضه گردیده تمامی برنامه‌ها همراه با فایل‌های اجرایی آن‌ها موجود می‌باشد. احتمالاً تا یکی دو ماه پس از چاپ کتاب اسلایدهای لازم برای تدریس کتاب با فرمت PPS آماده می‌شود که برای دریافت آن‌ها می‌توانید به سایت کتاب با آدرس زیر مراجعه فرمایید:

<http://www.programmingbook.4t.com/>

در پایان وظیفه خود می‌دانیم از زحمات همه کسانی که ما را در این راه به هر شکلی یاری رساندند تشکر کنیم. در این بین از دوست عزیز آقای رضا چنگیز که انصافاً مردانگی به خرج دادند و کار تایپ قسمت اعظمی از کتاب و ویرایش اولیه آن را به عهده گرفت از صمیم قلبم تشکر می‌کنیم. همچنین از دانشجویان عزیز خانم‌ها سمیرا پویان فر و صالحه روانبخش که باز در زمینه تایپ فصول برنامه‌نویسی نه تنها خودشان بلکه برخی از اعضای خانواده محترم‌شان را نیز درگیر کرده بودند صمیمانه تشکر می‌کنیم. همچنین از دوست عزیز آقای احسان حنیف پور و خانم لیلا زاهدی که در صفحه‌آرایی ما را یاری رساندند نیز باید تشکر کنیم.

موفق و مؤید باشید!

دکتر ناصر قاسم آقائی

مهندس مهدی جابر زاده انصاری

مهندس علی دهقان



فصل اول

آشنایی با علم کامپیوتر و ویژگی‌های الگوریتم

اهداف فصل و چکیده مطالب :

۱. آشنایی با ویژگی‌های این کتاب و روش مطالعه آن
۲. آشنایی با تاریخچه کامپیوتر
۳. آشنایی با نسل‌ها و انواع کامپیوترها
۴. شناختی کلی از طریقه کار و ساختمان کامپیوترهای امروزی
۵. تفاوت حل مسائل به شیوه الگوریتمی و شیوه مکاشفه‌ای
۶. تعریف الگوریتم
۷. بررسی ویژگی‌های الگوریتم
۸. آشنایی با حل مسائل به شیوه الگوریتمی

۱-۱: پیشگفتار

شاید آخرین مطلبی که در هر کتابی نگاشته می‌شود مقدمه آن کتاب باشد، و غالب خوانندگان نیز بدون توجه به نکاتی که معمولاً نویسنده در خصوص ویژگی‌ها و روش مطالعه کتاب در مقدمه کتاب بیان می‌کند، از نخستین صفحات کتاب، مطالعه خود را آغاز می‌کنند؛ و در نهایت امر، به جای کسب حداکثر بهره لازم، به اندک مایه‌ای دست می‌یابند و همه تقصیرات را بر دوش نویسنده می‌اندازند که قلم بدی داشته و... .

لذا تصمیم بر آن گرفتیم که در نخستین صفحات این کتاب به‌طور بسیار مختصر و در قالب یک نشست دوستانه و صمیمی سخنانی در خصوص ویژگی‌های این کتاب و طریقه تدریس و مطالعه آن بیان کنیم، باشد که خوانندگان عزیز التفات لازم را بفرمایند و این چند صفحه را به دقت مطالعه نمایند. امید است که با به کار بستن نکات مطرح شده در خلال این صفحات، حداکثر بهره لازم را از این کتاب ببرید.

دوست عزیز! شاید شما یک دانشجوی رشته کامپیوتر یا یک دانش‌پژوه آزاد و علاقه‌مند به علم کامپیوتر باشید، در هر دو صورت شما در آغاز راه طویل و بی‌انتهایی قرار گرفته‌اید که برای آن انتهایی را نمی‌توان متصور شد. امروز که نگارش این کتاب را شروع کرده‌ایم، با اطمینان به شما می‌گوییم که، اگر روزگاری افرادی وجود داشتند که از یک تاصد این علم را آموخته بودند و می‌توانستند ادعا کنند که در علم کامپیوتر به حد نهایی رسیده‌اند دیگر وجود ندارند. اگر نیم‌نگاهی به علوم دیگری همچون ریاضیات، فیزیک، و... بیندازید متوجه خواهید شد که این علوم هر چند سابقه‌ای چند صد ساله دارند ولی دارای رشد علمی چندان شتابانی نیستند. و هنوز هم می‌توان افرادی را یافت که حداقل در یک بخش از این علوم، [هرچند برای دوره‌ای کوتاه از زمان] به حد نهایی دانش‌های موجود در رشته کاری خود رسیده باشند. اما علم کامپیوتر در شکل امروزی خود که شاید سابقه آن به یک قرن هم نمی‌رسد دارای چنان حجم گسترده و رشد فزاینده‌ای است که به واقع نمی‌توان انتهایی برای آن متصور شد.

لذا در این راه طویل و بی‌انتهای از یک طرف باید عزم و اراده‌ای محکم و پولادین داشته باشید و از طرف دیگر می‌بایستی به درستی و با بینش قدم بردارید. با آنکه در عرصه علم هیچ راهی به بی‌راهه نمی‌رود و تمامی راه‌ها شما را به مقصد و مقصود نزدیک‌تر می‌گرداند، اما نکته بسیار مهم این است که، در هر حالتی باید نزدیکترین راه را انتخاب کنید، و از مهمترین دلایل نگارش این کتاب هم، همین است که *راه را برای شما نزدیک‌تر گرداند*.

در اکثر دانشگاه‌ها کتب مختلفی برای درس "مبانی کامپیوتر" مورد استفاده قرار می‌گیرد و در کنار بیان اصول الگوریتم‌نویسی یک زبان ساخت یافته نیز تدریس می‌شود. لذا کتابی که پیش روی شماست در جلد نخست نه تنها اصول الگوریتم‌نویسی را به شما آموزش می‌دهد، بلکه آموزش برنامه‌نویسی ساخت یافته را نیز به واسطه زبان C++ تحت کامپایلر Microsoft Visual Studio 6.0 دربر دارد.

البته در خصوص دلیل انتخاب زبان C++ برای آموزش برنامه‌نویسی می‌توان گفت که، در اکثر دانشگاه‌های دنیا به دلیل قابلیت‌های این زبان و در دسترس عموم بودن کامپایلر آن، همین زبان تدریس می‌شود. در C++ گرچه اصل بر برنامه‌نویسی شیء‌گرا است، ولی می‌توان به راحتی برنامه‌هایی مبتنی بر برنامه‌نویسی ساخت یافته را نیز در این زبان نوشت؛ که از مهمترین ویژگی‌های C++ به شمار می‌آید. سال‌های متمادی اساتید دانشگاه‌ها برای آموزش برنامه‌نویسی ساخت یافته از زبان پاسکال استفاده می‌کردند که مسلماً برای آموزش سیستم شیء‌گرا مجبور بودند به زبانی همچون C++ رجوع

کنند، اما رفته رفته با منسوخ شدن زبان پاسکال، زبان C++ جایگزین آن شد و با توجه به آنکه زبان‌های برنامه‌نویسی که بعد از C++ آمدند مثل Java و C# از برنامه‌نویسی ساخت یافته به‌طور کامل پشتیبانی نمی‌کردند، زبان C++ همچنان به‌عنوان مهمترین و مناسب‌ترین زبان آموزش برنامه‌نویسی برای افراد مبتدی باقی ماند. البته در جلد دوم این مجموعه در خصوص برنامه‌نویسی شی‌گرا و تجزیه و تحلیل شی‌گرا که در کمتر کتاب برنامه‌نویسی به آن اشاره شده صحبت خواهیم کرد و مطالب مهمی را در این زمینه عنوان خواهیم نمود. همچنین در بخش ضمیمه‌ها تفاوت‌های برنامه‌نویسی در کامپایلرهای مختلف C++ تحت دو سیستم عامل ویندوز و لینوکس را مورد بررسی قرار خواهیم داد تا شما خود را مقید به کامپایلر خاصی نبینید.

از دیگر ویژگی‌های مهم این کتاب ترتیب خاص فصول آن است، که پس از بیان مطالبی در خصوص الگوریتم و رسم کارنمای چندین مثال مفید، بلافاصله در فصل بعدی آن، به بیان مطالب برنامه‌نویسی معادل الگوریتم‌ها می‌پردازد، لذا رعایت این ترتیب فصول در خصوص افراد مبتدی، چه از سوی اساتید و چه از سوی دانشجویان بسیار حائز اهمیت است. البته امکان دارد مثال‌های صفحات نخستین چندان مورد توجه افراد غیرمبتدی قرار نگیرد اما رفته رفته مثال‌های مفیدتر و جذاب‌تری را مخصوصاً در فصول برنامه‌نویسی خواهند یافت.

از دیگر ویژگی‌های این کتاب این است که، در فصول مختلف با توجه به مطالب بیان شده تمریناتی تحت‌عنوان "کار در کلاس" بیان گشته است. کار در کلاس‌ها باید بلافاصله پس از یادگیری مطالب مورد نظر، توسط دانشجویان حل گردد تا در صورت نیاموختن قسمتی از درس فوراً از استاد خود کمک بگیرند. لذا توجه به این تمرینات و حل به موقع آنها بسیار مهم است.

همراه با کتاب یک مجموعه لوح فشرده‌ای ارائه گردیده که دانشجویان می‌توانند با توجه به نکات مطرح شده در ضمیمه شماره ۱ محیط Visual Studio 6.0 را بر روی کامپیوتر خود نصب کنند و برنامه‌های موردنظر را در آن بنویسند و اجرا کنند. همچنین بر روی لوح فشرده شماره ۱ کد مربوط به کلیه مثال‌های کتاب قرار گرفته است. لوح شماره ۱ همراه با کتاب و لوح‌های شماره ۲ تا ۴ به‌صورت اختیاری عرضه شده است.

در پایان برخی از فصول، که نکاتی در خصوص برنامه‌نویسی بیان شده، صورت یک پروژه نیز بیان گردیده، که انجام به موقع این پروژه‌ها می‌تواند بسیار به فهم مطالب کمک کند. یادتان باشد که تمرینات برنامه‌نویسی این کتاب را باید به‌طور کاملاً انفرادی انجام دهید و از همکاری با دیگران در ترم نخست جداً اجتناب کنید!؟

آشنایی مقدماتی با سخت افزار کامپیوتر نیز، از مباحث درس مبانی کامپیوتر به‌شمار می‌رود، اما به دلیل حجم مطالبی که در زمینه الگوریتم و برنامه‌نویسی در این درس باید مطرح شود، اکثر اساتید تجربه بحث تفصیلی در این زمینه را به واحد درسی آزمایشگاه کامپیوتر محول می‌کنند و در مبانی کامپیوتر تنها به شرح مختصری در این زمینه می‌پردازند. لذا در ادامه این فصل به‌طور بسیار مختصر به بیان مطالبی در خصوص ساختار کامپیوترهای امروزی می‌پردازیم. سپس مفهوم الگوریتم را در خصوص چند مثال ساده و مقدماتی بررسی می‌کنیم. و ویژگی‌های الگوریتم‌ها را بر می‌شمریم، اما بحث تفصیلی در خصوص رسم کارنما را به فصل دوم واگذار می‌کنیم.

۲-۱: تاریخچه و نسل‌های کامپیوتر

احتمالاً نخستین سؤالی که از خود می‌پرسید این است که چه لزومی دارد با تاریخچه کامپیوتر آشنا شویم؟ جواب این سؤال بسیار روشن است! چرا که چگونه یک فرد می‌تواند ادعا کند که کارشناس کامپیوتر می‌باشد در حالی که نمی‌داند کامپیوتر از کجا آمده و چه سرگذشتی داشته؟! به همین جهت برای شمایی که قرار است در رشته کامپیوتر کارشناس بشوید آشنایی با تاریخچه کامپیوتر بسیار ضروری به نظر می‌رسد. با توجه به این مقدمه کوتاه به بیان تاریخچه کامپیوتر می‌پردازیم، امید است که شریینی این مطلب در خاطر شما بماند.

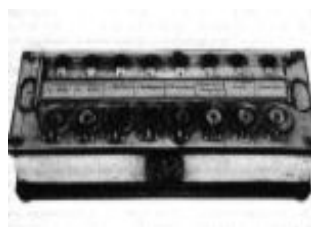
اگر به کتب فرهنگ لغت مراجعه کنید و به دنبال معنای کلمه رایانه یا کامپیوتر بگردید به اولین تعریفی که برمی‌خورید چنین است که :

«کامپیوتر ماشینی است که قادر به انجام برخی محاسبات ریاضی و منطقی می‌باشد»

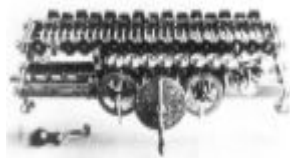
لذا می‌توان در یک کلمه معنای کامپیوتر را معادل عبارت ماشین محاسبه‌گر یا در معنی صحیح‌تر حسابگر دانست. بنابراین برای دنبال کردن تاریخچه کامپیوتر باید به دنبال وسایلی باشیم که انسان در طول تاریخ برای محاسبات ساخته و از آن بهره گرفته است.

از اولین وسایلی که انسان برای انجام محاسبات از آن بهره گرفت انگشتان دست خود بود و دقیقاً به همین دلیل است که انسان‌ها برای محاسبات از مبنای ده استفاده می‌کنند. اما تلاش انسان همواره در جهت ساخت وسایلی بوده که او را در زمینه شمارش یاری کند و کامپیوتر آخرین دستاورد این تلاش است، تاریخچه کامپیوتر مراحل طولانی را طی کرده است که چرتکه اولین مرحله آن بوده بنابراین به قولی می‌توان کشور چین را پیش‌گام در زمینه ساخت کامپیوتر دانست. اما محاسبات انسان تنها به شمارش ختم نشد و وسایلی در جهت محاسبات گوناگون همچون محاسبات نجومی، محاسبات دریانوردی و... ساخته شدند. از جمله این وسایل می‌توان به دو وسیله به نام‌های «لوح اتصالات» و «طبق المناطق» اشاره کرد که توسط دانشمند ایرانی «غیاث الدین جمشید کاشانی» جهت محاسبات نجومی و دریانوردی ساخته شدند.

اما تلاش‌های جدی در جهت ساخت ماشین‌های حساب به قرن هفدهم میلادی بازمی‌گردد که «ویلهلم شیکهارد» ریاضیدان آلمانی در سال ۱۶۲۳ طرح یک ماشین حساب را ارائه داد. پس از وی این تلاش‌ها ادامه یافت تا آنکه در سال ۱۶۴۲ «بلز پاسکال» که ریاضیدانی فرانسوی بود یک ماشین حساب مکانیکی (چرخ دنده‌ای) برای انجام عمل جمع و تفریق اختراع کرد که تصویر دو نمونه از آن را در زیر می‌بینید.



و بالاخره آخرین تلاش در جهت ساخت ماشین حساب مکانیکی منصوب به ریاضیدان آلمانی «لایب نیتز» می‌باشد که در سال ۱۶۹۴ توانست ماشین حسابی بسازد که چهار عمل اصلی را انجام می‌داد. تصویر این ماشین را نیز در زیر می‌بینید.

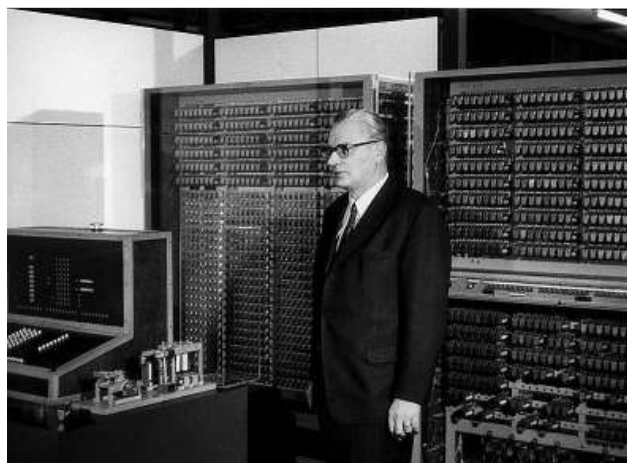


پس از آن، طرح ساخت ماشین‌های برنامه‌ریزی شده (خودکار) مطرح شد که اولین تلاش موفق در این زمینه مربوط به «ژوزف ژاکار» فرانسوی می‌شود. وی توانست در سال ۱۸۰۸ یک چرخ بافندگی خودکار بسازد که توسط کارت‌های مشبک طرح‌های متنوعی روی پارچه ایجاد می‌کرد.

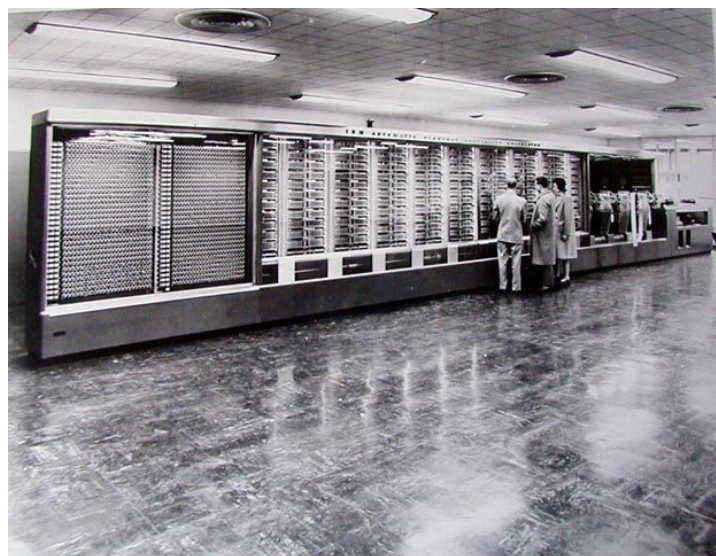
اما این موفقیت زمینه ساز این بود که با گسترش صنعت و تجارت و نیاز روز افزون به ماشین‌های محاسبه‌گر و ماشین‌های قابل برنامه‌ریزی جهت اداره کردن کارخانجات، ایده ساخت اینگونه ماشین‌ها در اذهان دو ابر قدرت صنعتی آن زمان یعنی آلمان نازی و ایالات متحده مطرح گردد.

در سال ۱۸۳۰ «چارلز بابیج» انگلیسی اولین نفری بود که طرح ساخت ماشینی با قابلیت برنامه‌پذیری را داد. وی وسیله‌ای با نام موتور تحلیل یا (Analytical) را طراحی کرد، هر چند که او به دلیل محدودیت امکانات نتوانست این دستگاه را بسازد ولی او را به پاس زحماتش پدر کامپیوتر می‌نامند.

اما نخستین تلاش در جهت ساخت کامپیوتر در مفهوم امروزی خود مربوط به سال ۱۹۳۹ می‌شود که مهندس «کنراد سوزه» از طرف سازمان تحقیقات فضایی آلمان مأموریت یافت یک ماشین محاسباتی بزرگ و پرتوان بسازد. تلاش وی در سال ۱۹۴۱ به ثمر رسید و ماشینی به نام Z_3 اختراع شد که براساس سیستم دودویی کار می‌کرد. دلیل اصلی موفقیت سوزه نسبت به طرح‌های ناموفق قبلی ابتکار وی در استفاده از همین سیستم دودویی بود. البته این ماشین در جنگ جهانی دوم به کلی نابود شد. در زیر تصویر مهندس سوزه را در کنار Z_3 مشاهده می‌کنید.



در آمریکا نیز همزمان با تلاش‌های سوزه «مهندس هوارد آیکن» که از اساتید برجسته دانشگاه هاروارد بود در سال ۱۹۳۷ یک ماشین حساب الکترومکانیکی ساخت. اما اندکی بعد، با موفقیت مهندس سوزه و بهره‌گیری آیکن از ایده‌ی منطق دودویی بکار رفته در Z_3 وی نیز توانست در سال ۱۹۴۲ ماشین حساب تماماً الکترونیکی به نام ASCC بسازد. اما به جهت تأثیری که این ماشین بر پیشبرد تکنولوژی گذاشت پس از مدتی آن را Mark I نام نهادند. این ماشین اولین کامپیوتر دیجیتالی محسوب می‌شود و قادر به ذخیره ۷۲ عدد ۲۳ رقمی در حین محاسبات بود و در هدایت موشک‌ها در جنگ جهانی دوم بکار گرفته شد. تصویر این ماشین را نیز در زیر مشاهده کنید.



آیکن و همکارانش پس از مدتی Mark II را ساختند که در آن ضمن آنکه به‌طور کامل از تئوری دودویی استفاده کردند بیش از ۱۳۰۰ رله به‌کار گرفتند. این ماشین قادر بود عمل وقت‌گیر ضرب را در ۰/۲۵ ثانیه انجام دهد که خود رکورد بزرگی محسوب می‌شد.

در دانشگاه پنسیلوانیا نیز دو دانشمند جوان به نام‌های «پراسپراکرت» و «جان ماچلی» در سال ۱۹۴۴ موفق به ساخت نخستین کامپیوتر نسل اول که از لامپ خلاء استفاده می‌کرد، شدند. کامپیوتر آنها که به سفارش مرکز آمار آمریکا ساخته شده بود و اینیاک (ENIAC) نام گرفت و تا سال ۱۹۵۵ از آن استفاده شد. آن دو به دلیل قرضی که جهت پرداخت جریمه تأخیر در تحویل ماشین اینیاک به سازمان آمار آمریکا متحمل شدند، مجبور به گرفتن قرض‌های متوالی و ساخت ماشین‌های دیگری شدند، و متأسفانه تا پایان عمر نه تنها لذت زحمات خود را نبردند بلکه در قرض و بدبختی و در سنین میانسالی به جهت کار طاقت فرسای جان دادند.

در سال ۱۹۴۷ «جان فون نویمان» دانشمند آلمانی مقیم آمریکا پیشنهاد کرد که کامپیوترها علاوه بر داده‌ها برنامه کار محاسبه را نیز در خود ذخیره سازند و قابلیت برنامه‌ریزی کردن را به کامپیوتر بافزایند.

در سال ۱۹۵۲ دو کامپیوتر به نام‌های^۱ EDVAC و^۲ EDSAC که براساس نظریه نویمان کار می‌کردند ساخته شدند. این ماشین‌ها قابلیت‌های زیر را داشتند، که به عبارتی می‌توان تحت‌عنوان فواید نظریه نویمان به آنها نگریست:

۱. سرعت ماشین‌هایی که منطبق با نظریه نویمان بودند صدها برابر شده بود.
۲. امکان ایجاد تغییرات لازم در برنامه‌هایی که در حین اجرا دچار اشکال شده بودند به وجود آمد. به بیان دیگر برنامه نویسی در این زمان متولد شد. چراکه تا قبل از این تمامی عملیاتی که کامپیوترها انجام می‌داند با طراحی مدارات الکترونیکی برای آنها میسر شده بود. لذا کامپیوترهای آن زمان به نوعی تک منظوره بودند، و یا اگر هم چند منظوره به شمار می‌آمدند، کارهای ثابتی را می‌توانستند انجام دهند. با ظهور کامپیوترهای منطبق با نظریه نویمان دیگر نیازی نبود برای تغییرات در کاربری کامپیوتر برای آن مدارات جدیدی را طراحی کنند بلکه برنامه جدیدی را طراحی می‌کردند و کامپیوتر وظیفه اجرای آن برنامه را به عهده داشت.

از حدود سال ۱۹۵۰ کامپیوتر به‌عنوان یک کالای تجاری توسط شرکت UNIVAC به بازار معرفی شد، پس از آن شرکت IBM که سازنده ماشین‌های تحریر و لوازمات اداری بود در سال ۱۹۵۴ (معادل ۱۳۳۰ شمسی) کامپیوترهای نسل اول را وارد بازار کرد که مدل ۷۰۱ و ۶۵۰ نام داشتند و به‌عنوان مهمترین سازنده کامپیوتر مطرح شد. در زیر تصویر مدل ۷۰۱ از سری کامپیوترهای IBM را مشاهده می‌کنید.



در ادامه به بیان نسل‌های کامپیوترها خواهیم پرداخت.

-
1. Electronic Discrete Variable Automatic Calculator
 2. Electronic Delay Storage Automatic Calculator

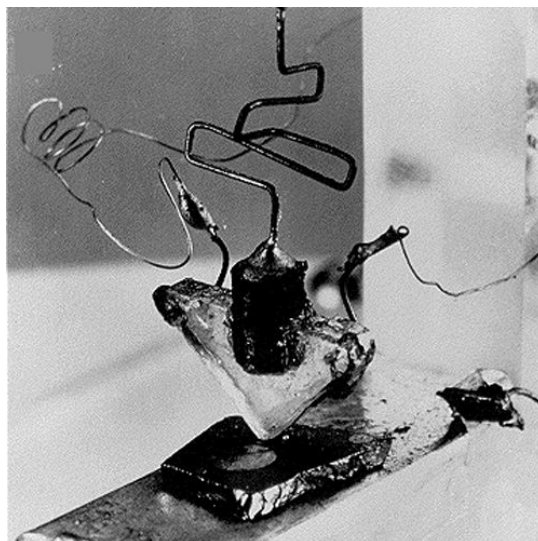
نسل اول (لامپ خلاء Vacume Tube)

همان‌طور که پیش‌تر بیان شد در ساخت این کامپیوترها از لامپ خلاء استفاده شد و از سال ۱۹۴۴ تا ۱۹۵۵ مورد استفاده قرار می‌گرفتند. یعنی قبل از دهه ۱۳۳۰ شمسی ساخته شدند. اما به دلایل زیر این کامپیوترها به سرعت کنار گذاشته شدند:

۱. اندازه این کامپیوترها بسیار بزرگ بود و فضای زیادی را اشغال می‌کردند.
 ۲. به جهت استفاده از لامپ خلاء به نیروی برق زیادی نیاز داشتند.
 ۳. به علت گرمای زیادی که در لامپ‌های خلاء ایجاد می‌شد به وسایل خنک‌کننده قوی نیاز داشتند.
 ۴. به علت گرمایی که در لامپ‌های خلاء ایجاد می‌شد امکان داشت در حین محاسبات این لامپ‌ها بسوزد و در نتیجه می‌بایستی تمامی محاسبات از سرگرفته می‌شد.
 ۵. مسلماً با استفاده از وسیله گران‌قیمتی مثل لامپ خلاء قیمت این محصول بسیار بالا تمام می‌شد.
- چنان‌که ذکر رفت یکی از اولین کامپیوترهای این نسل اینیاک بود که دارای ۳۰ تن وزن، ۱۷۰ متر مربع مساحت بود و ۱۸۰۰۰ لامپ خلاء در آن به‌کار رفته بود و به ۱۵۰ کیلو وات انرژی الکتریکی نیاز داشت.

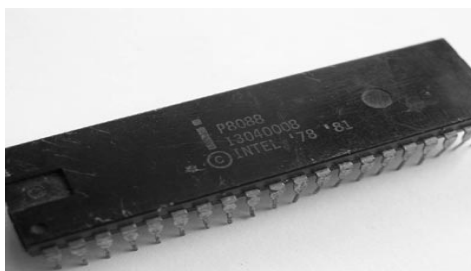
نسل دوم (ترانزیستور Transistor)

با اختراع ترانزیستور در سال ۱۹۵۰ تحول عظیمی در صنایع الکترونیکی رخ داد و در کامپیوترها نیز از ترانزیستورها به جای لامپ خلاء استفاده شد. این کامپیوترها دارای حجم کمتر و مصرف انرژی پایین‌تر و سرعت بالاتر بودند. در این کامپیوترها برای اولین بار از حلقه‌های مغناطیسی به‌عنوان حافظه اصلی (RAM) استفاده شد. گفتنی است این کامپیوترها در سال‌های ۱۳۳۰ تا ۱۳۴۰ شمسی مورد استفاده قرار گرفتند. در زیر تصویر نخستین ترانزیستور دنیا را مشاهده می‌کنید. آیا ترانزیستورهای کنونی را تا به حال دیده‌اید؟



نسل سوم (مدارات مجتمع IC)

با اختراع مدارهای مجتمع (Integrated Circuit) حجم کامپیوترها کاهش یافت و سرعت آنها افزایش یافت. چرا که این مدارات شامل بیش از صد عنصر منطقی بوده که هر عنصر منطقی خود شامل چندین عنصر الکترونیکی مثل دیود و ترانزیستور هستند و به روش خاصی بر روی صفحاتی از جنس سیلیکون در چند سانتی‌متر مربع کنار یک دیگر چیده شده‌اند. و این خود نشان‌دهنده میزان کوچک شدن کامپیوترها می‌باشد. در زیر تصویر یک نمونه IC را می‌توانید مشاهده کنید.



همچنین گسترش نرم افزارهای ساخت یافته نیز موجب افزایش سرعت این نسل و به طبع گسترش آنها شد. اولین کامپیوتر از این نسل را شرکت IBM در سال ۱۹۶۰ با نام IBM360 به بازار ارائه کرد و تا سال ۱۹۷۰ در مراکز تجاری مورد استفاده قرار می‌گرفت.

نسل چهارم (ریز پردازنده‌ها CPU)

در سال ۱۹۷۰ با متراکم تر شدن مدارات مجتمع به واسطه فن‌آوری VLSI ساخت ریزپردازنده‌ها آغاز شد و کامپیوتر به‌عنوان یک کالای خانگی راهی بازارها شد. از جمله ریزپردازنده‌ها می‌توان به خانواده ۸۰۸۶، ۸۰۲۸۶، ۸۰۳۸۶، ۸۰۴۸۶ و... اشاره کرد.

نسل پنجم (کامپیوترهای هوشمند)

از سال ۱۹۸۰ با گسترش نظریه منطق فازی ایده ساخت این کامپیوترها به‌طور جدی توسط ژاپن مطرح شد، که تلاشی در جهت ساخت کامپیوترهایی با ویژگی‌های انسانی است. براین اساس در تکنولوژی ساخت این کامپیوترها به‌گونه‌ای باید عمل شود که کامپیوتر قادر به اعمالی چون استنباط و استدلال کردن باشد. گرچه هنوز کامپیوتری با این ویژگی‌ها ساخته نشده است ولی ساخت آن دور از دسترس نیست؛ چنانکه شرکت‌ها و مؤسسات زیادی در حال کارکردن بر روی آنها هستند.

نسل ششم (کامپیوترهای انسان‌نما)

در این نسل هدف بر آن است که در ساخت کامپیوتر فعالیت‌های مغز انسان به‌طور کامل کپی برداری شود. چه بسا کامپیوترهای این نسل دارای قدرت درک حواس پنج‌گانه انسان و اعمالی از این دست باشند. مسلماً سرعت، دقت و هوشمندی این نسل از کامپیوترها قابل‌باور نبوده و دارای ساختمان و مدارات پیچیده‌تری نسبت به نسل‌های قبل از خود هستند.

۱-۳: سخت افزار و ساختمان کامپیوتر های امروزی

در این بخش هدف آن است که خوانندگان عزیز را با کلیاتی در خصوص ساختمان و نحوه کارکرد کامپیوترهای شخصی (Personal Computers) آشنا سازیم. البته به دلیل پیشرفت های مداوم در ساخت کامپیوترهای PC، از ذکر جزئیات و ارائه اعداد و ارقام در این بحث معذوریم و تنها به ذکر مباحث کلیدی و بیان مطالب به صورت انتزاعی می پردازیم. سعی خواهیم کرد مدلی برای کامپیوتر ارائه کنیم تا ما را در درک چگونگی عملکرد کامپیوتر در خصوص اجرای برنامه هایی که به زبان C++ می نویسیم یاری دهد. و همان طور که پیشتر گفته شد، ذکر جزئیات در این خصوص را به واحد درسی "آزمایشگاه کامپیوتر" واگذار می کنیم.

مسلماً هر فرد مطلعی بر این مطلب صحه می گذارد که مغز انسان کامپیوتری بسیار پیچیده تر و به مراتب قوی تر از کامپیوترهای امروزی است. گرچه مغز انسان از نظر سرعت انجام محاسبات بسیار ضعیف تر از کامپیوتر عمل می کند، اما امکاناتی که مغز انسان در زمینه تشخیص هوشمندانه مطالب دارد، بزرگترین کامپیوترها را نیز پشت سر می گذارد. کامپیوترها به خودی خود فاقد هرگونه هوش هستند. کامپیوتر ماشینی است که صرفاً قادر به انجام عملیات ریاضی و منطقی است، و این متخصصین کامپیوتر هستند که با برنامه ریزی کامپیوترها، آنها را قادر به درک هوشمندانه مطالب پیچیده می کنند. البته این مطلب را بدان جهت مطرح کردیم که برخی افراد به اشتباه فکر می کنند کامپیوتر ماشینی هوشمند است که قادر است به تنهایی جواب همه سؤالات را بیاورد!؟

آن چیزی که در این خصوص مهم به نظر می رسد این است که، اگر به نحوه انجام محاسبات توسط انسان و پردازش اطلاعات در مغز او توجهی داشته باشیم، می توانیم ادعا کنیم که بسیاری از قسمت های اصلی کامپیوترها که در پردازش اطلاعات درگیر هستند مشابه عملکرد انسان و مغز او [در انجام محاسبات] عمل می کنند.

هر انسانی برای انجام محاسبات بر روی تعدادی داده خام در ابتدا نیازمند آن است، که داده های خود را بر روی یکسری برگه نوشته و بر روی میزکار خود قرار دهد. سپس با رجوع به حافظه خود یا منابعی که در اختیار دارد، راه حل مناسب را پیدا کرده، و با به کارگیری توانایی مغز خود راه حل مذکور را بر روی داده های خام به طور مکرر اعمال می کند تا آنکه همه داده ها پردازش گردد و نتایج لازم حاصل آید. همچنین انسان قادر است نتایج به دست آمده یا هر مطلب دیگری را در حافظه کوتاه مدت و یا در صورت لزوم در حافظه بلند مدت خود به خاطر بسپارد. از طرفی انسان در حین محاسبات مدام عملیات انجام شده را چک می کند تا از صحت نتایج به دست آمده اطمینان حاصل نماید. حال تصمیم داریم با توجه به مدلی که در مورد انسان ارائه کردیم مدلی برای کامپیوتر ترسیم کنیم تا با نحوه عملکرد آن بهتر آشنا شویم.

۱. هر کامپیوتر در درجه اول نیازمند بخشی است که داده های خام را از کاربر خود دریافت کند. لذا در کامپیوتر اصلی ترین دستگاه ورود اطلاعات صفحه کلید است، که با آن اطلاعات عددی و کاراکتری را وارد کامپیوتر می کنند.

۲. همچنین هر کامپیوتر نیازمند بخشی است که نتایج به دست آمده از پردازش های خود را نشان دهد. از این رو صفحه نمایش (مانیتور) از متداول ترین وسایل جهت نمایش اطلاعات می باشد.

۳. از طرفی کامپیوتر نیازمند بخشی است که بتواند توانایی مغز انسان در انجام محاسبات ریاضی و اعمال منطقی را بر روی داده‌ها میسر سازد. برای این منظور یک واحد پردازشگر مرکزی به نام CPU در نظر گرفته شده است. این واحد قادر است عملیات چهارگانه ریاضی و همچنین اعمال منطقی را انجام دهد. در فصول بعدی با این عملیات بیشتر آشنا خواهید شد.

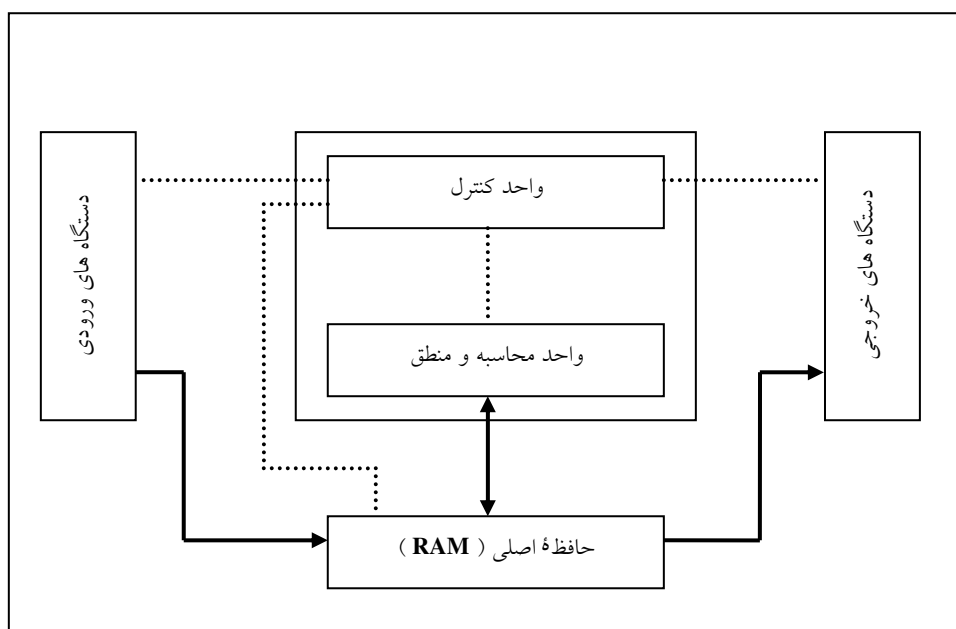
۴. همچنین هر کامپیوتر باید قادر باشد الگوها و راه حل‌ها و اطلاعات دیگر را در حافظه‌ای به صورت دائمی ذخیره سازد. لذا کامپیوتر دارای یک حافظه ثانویه مثل هارد دیسک یا فلاپی و امثال آن است که می‌توان مطالبی را بر روی آن ذخیره کرد تا در دفعات بعدی که کامپیوتر شروع به کار می‌کند، بتوان آن اطلاعات را بازیابی نمود و مورد استفاده قرار داد.

۵. اما حافظه اصلی چیست؟ کامپیوتر نیازمند داشتن مکانی مشابه میزکار انسان، جهت قرار دادن اطلاعات و داده‌های در حال پردازش بر روی آن، است. همچنین داده‌هایی که از صفحه کلید وارد می‌شود باید در حافظه موقت ذخیره گردد و داده‌هایی که بر روی مانیتور نمایش داده می‌شود ابتدا باید در مکانی قرار داشته باشد تا سپس توسط تراشه‌های تصویری به نمایش در آیند. همه این اطلاعات در حافظه RAM قرار می‌گیرند. این حافظه دائمی نبوده، و با خاموش شدن کامپیوتر تمامی اطلاعات آن پاک می‌گردد. به RAM حافظه اولیه نیز گفته می‌شود.

۶. از سوی دیگر در کامپیوتر باید مکانیسمی مشابه شبکه‌های عصبی انسان جهت کنترل صحت عملکرد قسمت‌های مختلف سیستم تعبیه گردد. همچنین فرمان شروع کار یا باز ایستادن از کاری باید توسط همین ارگان صادر گردد. لذا واحد پردازشگر مرکزی یعنی CPU از دو قسمت متفاوت به نام‌های ALU و CU تشکیل شده است که واحد محاسبه و منطقی یا همان ALU عملیات ریاضی و منطقی را انجام می‌دهد و واحد کنترل یا همان CU عملیات کنترل دستگاه‌ها و فرمان‌دهی را انجام می‌دهد.

۷. بنابراین با توجه به مطالبی که بیان شد هر کامپیوتر را می‌توان به صورت طرح وار همانند شکل ۱-۱ در نظر گرفت. چنانکه در این شکل می‌بینید، برخی از قسمت‌ها دارای یک ارتباط یک طرفه و برخی دیگر دارای ارتباطی دو طرفه با یکدیگر هستند، که نشان دهنده شکل جریان اطلاعات می‌باشد. از طرفی تمامی قسمت‌ها با خطوط نقطه چین با واحد CU ارتباط دارند. عملکرد این کامپیوتر طرح وار بدین صورت است که، داده‌های خام و دستورالعمل محاسبات از واحد ورودی با فرمان CPU به حافظه اصلی وارد می‌شود. سپس CPU با انجام محاسبات لازم، مرحله به مرحله طبق دستورالعمل از پیش تعیین شده جلو می‌رود تا در نهایت به حالت توقف برسد، و نتایج به دست آمده را در قسمت دیگری از حافظه قرار می‌دهد. در نهایت CPU نتایج به دست آمده را به واحد خروجی می‌فرستد تا بر روی مانیتور نمایش داده شوند. در حین انجام تمامی این مراحل واحد CU نه تنها بر درست کارکردن تمامی دستگاه‌ها نظارت دارد، بلکه این واحد CU است که طبق دستورالعمل داده شده به کامپیوتر، به بخش‌های مختلف

می‌گویند که چه کاری را و در چه زمانی انجام دهند، کدام داده را مورد پردازش قرار دهند و یا نتایج را در کجای حافظه ذخیره کنند، و... اما یک تفاوت اساسی میان عملکرد کامپیوتر و عملکرد انسان وجود دارد، به‌طوری که برای کارکردن کامپیوتر باید، راه‌حل مسئله از قبل توسط کاربر (انسان) به کامپیوتر داده شود و در کنار داده‌ها در RAM قرارگیرد تا CU بتواند از آن راه‌حل استفاده کند. بنابراین کامپیوترهای امروزی قادر نیستند به تنهایی راه حلی را بیابند بلکه کامپیوتر به یک انسان به‌عنوان کاربر نیازمند است تا نیازمندی‌های آن را برطرف سازد. کامپیوتر بدون حضور کاربر، همانند انسان با استعدادی است که سواد نداشته باشد، و لذا از استعداد خود هیچ بهره‌ای نمی‌تواند ببرد. بنابراین گرفتن دستورالعمل برای کامپیوتر همانند سوادمند شدن یک انسان با استعداد می‌ماند.



شکل ۱-۱: چگونگی ارتباط بین اجزای اصلی کامپیوتر

۸. اما مسلماً یک سؤال اساسی برای شما پیش آمده و آن اینکه چرا حافظه ثانویه در شکل فوق ترسیم نشده است؟ جواب این سؤال بسیار روشن است، چرا که حافظه ثانویه جزء واحدهای اصلی یک کامپیوتر به حساب نمی‌آید؛ چنانکه اگر ماشین حساب را به‌عنوان یک کامپیوتر کوچک در نظر بگیرید قسمتی مطابق با حافظه ثانویه در آن نمی‌یابید؛ ولی ماشین حساب بدون حافظه ثانویه به‌درستی کار می‌کند و این نشان‌دهنده عدم نیاز مدل کامپیوتری ما به حافظه ثانویه است. اما شما می‌توانید این حافظه را خودتان به شکل ۱-۱ اضافه کنید. فکر می‌کنید حافظه ثانویه چگونه رابطه‌هایی با اجزای شکل فوق باید داشته باشد؟

۴-۱: انواع کامپیوتر و کاربرد آنها

کامپیوترهای امروزی از نظر بزرگی، سرعت، دقت و کاربرد به دسته‌های مختلفی تقسیم می‌شوند:

۱. سوپر کامپیوترها (Super Computer)
۲. کامپیوترهای بزرگ (Main Frame)
۳. کامپیوترهای متوسط (Mini Computer)
۴. کامپیوترهای کوچک یا رایانه شخصی (Micro Computer or Personal Computer)

سوپر کامپیوترها بیشتر در مراکز نظامی و فضایی ابرقدرت‌ها مورد استفاده قرار می‌گیرند. قدرت محاسبات این کامپیوترها چنان گسترده و وسیع است که کشور فرانسه آزمایش‌های اتمی خود را به واسطه این کامپیوترها شبیه‌سازی کرده و در محیط کاملاً مجازی شرایط انفجار و تخریب بمب‌های اتمی خود را محاسبه و بررسی می‌کند. همچنین در کشور آمریکا در برخی مراکز بزرگ اطلاعاتی این کشور مثل NSA با این کامپیوترها، کوچکترین اطلاعاتی که توسط دستگاه‌های مخابراتی داخلی و خارجی آن کشور رد و بدل شود را مورد تجزیه و تحلیل قرار می‌دهد و در صورت لزوم رمزگشایی می‌کنند چنانکه ادعا می‌شود تا به حال هیچگونه پیام رمزی نبوده که توسط این ماشین رمزگشایی نشده باشد. لذا با توجه به کاربرد استراتژیک این کامپیوترها قدرت‌های بزرگ از دسترسی دیگر کشورها به این تکنولوژی جلوگیری می‌کنند.

کامپیوترهای بزرگ در مراکزی که با حجم اطلاعات بسیار زیاد سروکار دارند، به کار می‌روند، مثل دانشگاه‌ها و بانک‌ها و شرکت‌های هواپیمایی مادر و کامپیوترهای متوسط در مراکز آموزشی و تجاری کوچکتر و کامپیوترهای شخصی در منازل، ادارات، سازمان‌ها و شرکت‌های دولتی و غیردولتی کاربرد وسیعی دارند. البته شاید نتوان یک مرز بندی دقیق بین کامپیوترهای متوسط و کامپیوترهای شخصی قائل شد.

کار در کلاس ۱-۱:

در فصول موارد زیر تحقیق کنید و در صورت امکان با تهیه اسلاید، گزارشی از نتایج تحقیق خود را به کلاس ارائه دهید. مواردی که در زیر ذکر شده‌اند حتماً باید در درس آزمایشگاه کامپیوتر پوشش داده شوند، و یا آن که خود دانشجو به تحقیق در فصول آن‌ها بپردازد.

۱. قطعات یک کامپیوتر امروزی کدام‌ها هستند؟ نام قطعات لازم برای مونتاژ یک کامپیوتر را ذکر کنید و مختصری در فصول کارکرد هر یک شرح دهید.
۲. در مورد انواع لوازم جانبی کامپیوتر و کاربرد آن‌ها شرح دهید.
۳. سافت‌وار دافلی یک پردازنده اعم از ریسسترها، انبار، واهر مناسبه و منطق و... شرح دهید.
۴. در فصول نحوه ی انجام عملیات در پردازنده‌ها تحقیق کنید.
۵. اندکی در مورد برنامه‌های setup و سیستم عامل شرح دهید.

۱-۵: بررسی مفهوم الگوریتم

اولین مقوله‌ای که در عرصه علم کامپیوتر با آن برخورد می‌کنید "الگوریتم" می‌باشد. اما برای آنکه مفهوم الگوریتم را خوب و دقیق به شما انتقال دهیم و بتوانیم تعریفی دقیق از آن ارائه کنیم نیازمند پاسخگویی به یک سؤال مهم هستیم!

فرض کنید رباتی در اختیار داریم که به وسیله دستورات خاصی که برای آن قابل فهم است، قادر است دستورالعمل انجام کاری را از ما دریافت کند و آن کار را مطابق دستورالعمل داده شده انجام دهد. حال سؤال این است که چگونه می‌توانیم با بهره‌گیری از امکانات این ربات مسائلی که در روزمره با آنها روبرو می‌شویم را انجام دهیم؟ و یا به عبارت دیگر با چه روشی به ربات خود بفهمانیم که کار مشخصی را چگونه باید انجام دهد؟

این سؤال را می‌توانیم در حالت کلی به صورت زیر بیان کنیم :

چگونه مسائل را به واسطه کامپیوتر حل کنیم ؟

در جواب به این سؤال باید اذعان داشت هنگامی که مسئله‌ای را به ما می‌دهند مراحل زیر را در جهت حل مسئله باید انجام دهیم:

۱. **شناسایی مسئله :** منظور آنکه، بفهمیم مسئله چه داده‌هایی را در اختیار ما قرار داده است و چه مجهول‌هایی را از ما می‌خواهد؟!
 - شیوه مکاشفه‌ای
۲. **تحلیل راه حل :** منظور یافتن ارتباطی بین داده‌های مسئله و مجهول‌های آن است برای این منظور دو روش کلی برای عموم مسائل به کار گرفته می‌شود:
 - شیوه الگوریتمی

مقصود از شیوه مکاشفه‌ای این است، که به جای تفکر منطقی و سیستماتیک، به تفکرات جانبی روی آورده و به کوتاه‌ترین و عملی‌ترین راه حل برای حل مسئله دست پیدا کنید. به عنوان مثال هنگامی که به واسطه فرمول‌گرایی در حل برخی مسائل ریاضی به جای طی کردن مراحل علمی و دقیق ریاضی آن، مسئله را با برخی ترفندها و به قول معروف از راه حل‌های تستی حل می‌کنید از این روش بهره گرفته‌اید. به عنوان مثال تمرین زیر را ابتدا خودتان سعی کنید حل کنید و سپس پاسخ کتاب و تحلیل آن را به دقت مطالعه نمایید. هدف از حل این تمرین این است، که شما دریابید مسائلی را که با آن روبرو می‌شوید اصولاً به چه روشی حل می‌کنید. چه بسا به طور ناخودآگاه روش علمی معتبری را برای حل مسائل به کار می‌برید!؟

کارد در کلاس ۱-۲:

تعداد سی کشتی گیر در یک دوره مسابقات یک هزخی شرکت دارند. معین کنید در این دوره مسابقات چند مسابقه کشتی انجام می‌شود؟ راه حل خود را شرح دهید.

راهنمایی: (منظور از یک هزخی بودن مسابقات این است که دو نفر کشتی می‌گیرند و بازنده از دور مسابقات حذف می‌شود.)

یک راه حل منطقی این مسئله می‌تواند این باشد که از روش مسائل ترسیمی هندسه جلو برویم. بدان معنا که ابتدا

فرض کنیم مسئله حل شده است، بنابراین با توجه به آنکه فرضیات زیر را نیز داریم:

الف- فقط یک برنده و ۲۹ بازنده داریم.

ب- هر مسابقه فقط یک بازنده دارد.

ج- هر بازنده نیز فقط یک باخت دارد.

مشخص می‌شود که تعداد مسابقات با تعداد بازنده‌ها برابر است بنابراین ۲۹ مسابقه انجام شده است. به حل کردن مسائل بدین روش، شیوه مکاشفه‌ای گفته می‌شود. حتی حل این مسئله می‌تواند بدین صورت تعمیم پیدا کند که تعداد مسابقات یک حذفی برای n کشتی گیر برابر $n-1$ مسابقه می‌باشد. (فرمول گرایی)

اما شاید راه حل شما به این صورت باشد که گفته باشید: ابتدا ۱۵ مسابقه به صورت دوبه‌دو انجام شده و ۱۴ نفر از ۱۵ نفر باقی‌مانده در ۷ مسابقه شرکت می‌کنند و یک نفر نیز بدون مسابقه دادن، به قید قرعه بالا می‌آید، که در نهایت هشت نفر باقی می‌مانند. این ۸ نفر در ۴ مسابقه شرکت کرده و ۴ نفر بالا می‌آیند و برنده‌ها در ۲ مسابقه دو به دو شرکت کرده و فینالیست‌ها را مشخص می‌کنند و در نهایت ۱ بازی فینال انجام خواهد شد. بنابراین:

$$۱۵ + ۷ + ۴ + ۲ + ۱ = ۲۹$$

جمعاً ۲۹ مسابقه انجام شده است.

به شیوه حل اخیر شیوه الگوریتمی گفته می‌شود که در ادامه به اصول آن و مزایای استفاده از این روش بیشتر خواهیم پرداخت. فعلاً در همین حد بدانید که با شیوه الگوریتمی می‌توان بسیاری از مسائل پیچیده و بزرگ را به راحتی به واسطه کامپیوتر پیادسازی و حل کرد که شاید راه حل فرمولی و مکاشفه‌ای هم نداشته باشند. در کار در کلاس بعدی هدف بر آن است، که شما را بیشتر با شیوه الگوریتمی آشنا سازیم.

کار در کلاس ۱-۳:

فرض کنید مدلی که از کامپیوتر به عنوان ربات در ابتدای بخش ۱-۳ مطرح کردیم در اختیار شما قرار داده شده است. با صرف نظر از اینکه چگونه مفاهیم و دستورات را به ربات انتقال می‌دهیم، نحوه استفاده از تلفن عمومی (فقط گرفتن یک شماره تلفن به صورت موفقیت آمیز) را برای ربات شرح دهید.

راهنمایی: برای این منظور یک سلسله دستوراتی را به زبان فارسی برای کامپیوتر بنویسید.

شاید راه‌حلی که شما به صورت تعدادی دستورالعمل ارائه نموده‌اید به صورت زیر باشد:

۱. شروع کن.

۲. گوشی تلفن را بردار.

۳. یک سکه در تلفن بینداز.

۴. صبر کن تا صدای بوق آزاد را بشنوی.

۵. شماره مورد نظر را بگیر.

۶. پایان عملیات.

در تهیه یک دستورالعمل باید به نحوه اجرای آن توجه خاص داشت. منظور آنکه، در اجرای دستورالعمل هیچگونه سؤال و ابهامی برای مجری آن نباید پیش آید، و اجرای مراحل آن نباید بستگی به برداشت‌های متفاوت مجری داشته باشد. ضمناً ترتیب عملیات باید معین باشد، و دارای شروع و خاتمه مشخصی باشد. چرا که دستورات یک دستورالعمل توسط کامپیوترهای سریال که در اختیار ما قرار دارند، دستور به دستور به ترتیب از نقطه آغاز تا پایان اجرا می‌گردد. لذا نباید انتظار داشته باشید که کامپیوترهای سریال بتوانند چندین عمل را به‌طور همزمان انجام دهند. البته کامپیوترهای موازی وجود دارند که قادرند با چندین پردازنده به‌طور همزمان کارهای متفاوتی را به‌صورت همزمان انجام دهند که از بحث ما به‌کلی خارج است. فراموش نکنید که مبحث کامپیوترهای موازی با مقوله Multi Tasking بودن سیستم‌عامل‌هایی چون ویندوز و لینوکس به‌کلی متفاوت است در این خصوص در کتاب "آزمایشگاه کامپیوتر" مطالب روشنگری بیان شده است.

دستورالعملی با ویژگی‌های فوق یک الگوریتم نامیده می‌شود. توجه داشته باشید که الگوریتم مقوله‌ای خاص علم کامپیوتر نیست و در بسیاری از علوم دیگر نیز کاربرد دارد. اما گفته می‌شود که علم کامپیوتر، علم الگوریتم‌ها است. الگوریتم به معنای شیوه و روش حل یک مسأله است و از نام دانشمند بزرگ و گرانقدر ایرانی "محمد بن موسی خوارزمی" گرفته شده است، چرا که وی در آغاز قرن سوم هجری در کتاب "جبر و مقابله" برای حل مسائل با ابداع معادلات جبری شیوه و روش حل جدیدی ارائه کرد و علم جبر را پایه‌گذاری نمود. از آنجایی که در پردازش داده‌ها و محاسبات ریاضی ابداع شیوه و روش حل مناسب، و یا به عبارت دیگر طراحی یک الگوریتم خوب، نقش محوری و کلیدی در حل آن مسأله دارد، علم کامپیوتر را علم الگوریتم‌ها نامیده‌اند.

حال به ارائه تعریف دقیقی از الگوریتم می‌پردازیم.

تعریف الگوریتم:

الگوریتم عبارت است از تعدادی دستورالعمل پشت سرهم که مراحل مختلف یک کار را به زبان دقیق و با جزئیات کافی بیان نماید و در آن ترتیب مراحل و فاصله پذیر بودن عملیات کاملاً مشخص باشد.

البته در فصول بعدی ویژگی‌های دیگری را برای الگوریتم بیان خواهیم کرد و آن را در دو نوع الگوریتم‌های ساخت یافته و غیر ساخت یافته دسته‌بندی خواهیم نمود، اما فعلاً ویژگی‌هایی را که برای الگوریتم در تعریف فوق برشمردیم، مورد تحلیل قرار می‌دهیم.

۱ - زبان دقیق با جزئیات کافی

اگر از یک جمله، که در مرحله‌ای از یک الگوریتم آمده است، برداشت‌های متفاوتی شود و یا به سبب مهم بودن، سؤال برانگیز باشد گوئیم، بیان آن جمله دقیق نیست. مثلاً در الگوریتم بالا گفته نشده چه نوع سکه‌ای را در دستگاه تلفن بینداز؟! بنابراین از آنجایی که بیان جملات به‌صورت دقیق، به واسطه زبان فارسی یا انگلیسی بسیار مشکل است؛ از زبان‌های به اصطلاح صوری مثل زبان‌های برنامه‌نویسی استفاده می‌شود. همچنین جهت بیان الگوریتم‌ها، تعدادی قوانین و قواعد را به‌صورت استاندارد در آورده‌اند که به واسطه آن‌ها می‌توان الگوریتم‌ها را به‌صورتی فراگیر و همه فهم بیان نمود، به‌طوری که الگوریتم‌ها را می‌توان با رسم کارنما (فلوچارت) نمایش داد.

به‌طور کلی روش‌های زیر برای بیان الگوریتم وجود دارد:

۱. استفاده از زبان‌های طبیعی مثل فارسی و انگلیسی و ...

۲. استفاده از نمودار

۳. استفاده از زبان‌های برنامه‌نویسی

۴. استفاده از شبه‌کد (Pseudo code)

در این فصل از روش اول و در فصول آتی از روش‌های دوم و سوم استفاده می‌کنیم و در آنجا با این روش‌ها بیشتر آشنا خواهیم شد. اما در خصوص روش شبه‌کد باید گفت؛ در این روش با کمی دخل و تصرف در دستورات یک زبان برنامه‌نویسی الگوریتم را با آزادی عمل بیشتری بیان می‌کنند. چنانکه ممکن است دستوراتی را در شبه‌کد به‌کار ببرند که اصلاً در آن زبان وجود نداشته باشد ولی منظورشان را به روشی برسانند، و چگونگی پیاده‌سازی آن را به عهده فرد برنامه‌نویس می‌گذارند. در این خصوص در کتب طراحی الگوریتم مطالب مفیدی یافت می‌شود.

۲- ترتیب مراحل

مسلماً ترتیب انجام کارها در هر الگوریتمی مهم است. همان‌طور که پیشتر گفته شد کامپیوترهایی که شما با آنها سروکار دارید به‌صورت سریال عمل می‌کنند. یعنی تنها قادر به انجام یک عمل ریاضی یا منطقی در یک سیکل زمانی خود هستند، بنابراین کامپیوتر می‌تواند عملیات یک الگوریتم را پشت‌سرهم و به ترتیب انجام دهد.

۳- خاتمه پذیر بودن عملیات

شرط یا شروط خاتمه عملیات باید در الگوریتم به‌طور صریح یا ضمنی بیان شود. به‌خصوص وقتی که با انجام یکسری عملیات تکراری روبرو هستیم حتماً باید شرط پایان حلقه تکرار به‌طور صریح بیان شود. مثلاً در مثال قبلی اگر تلفن خراب باشد و بوق آزاد نزنند ربات ما باید تا ابد صبر کند!!!

حال با توجه به توضیحات بالا الگوریتم تلفن زدن را یک بار دیگر بازنویسی می‌کنیم. برای اینکه الگوریتم بدون ابهام باشد، فرض می‌کنیم که حداقل یک تلفن سالم در شهر وجود دارد.

" الگوریتم اصلاح شده تلفن زدن "

۱. شروع کن.

۲. گوشی تلفن را بردار و یک سکه سالم در تلفن بینداز.

۳. حداکثر ۳۰ ثانیه صبر کن تا صدای آزاد شدن خط را بشنوی اگر خط آزاد شد به مرحله ۵ برو در غیر این صورت به مرحله ۴ برو.

۴. گوشی را سرپایش بگذار، و اگر سکه پس داده شد آن را بردار. اگر دفعه سومی است که این مرحله را اجرا می‌کنی (منطقی است که نتیجه بگیرد تلفن خراب است) به مرحله ۶ برو و در غیر این صورت به مرحله ۲ بازگرد و مراحل را تکرار کن.

۵. شماره موردنظر را بگیر و به مرحله ۷ برو.

۶. تلفن دیگری در شهر پیدا کن و اجرای الگوریتم را از مرحله ۲ تکرار کن.

۷. پایان.

در ادامه به بیان یک الگوریتم عددی توسط زبان طبیعی می‌پردازیم و بحث این فصل را خاتمه می‌دهیم. در فصل آتی با رسم کارنما آشنا شده و به بیان ترسیمی الگوریتم‌هایی، یعنی کارنما می‌پردازیم.

کاردر کلاس ۱-۴:

الگوریتمی بنویسید که معدل سه عدد ۱۵ و ۱۷/۴۵ و ۱۹/۱۰ را حساب کند.

ما الگوریتم را در حالت کلی‌تری حل می‌کنیم. فرض کنید نمرات درسی یک دانش‌آموز را در متغیرهای x و y و z و معدل آنها را در متغیر M قرار می‌دهیم.؟! فقط به این نکته توجه داشته باشید که منظور از $U \leftarrow V$ این است، که مقدار متغیر U را در متغیر V قرار بده.

مثال ۱-۱: " الگوریتم محاسبه معدل سه عدد "

۱. شروع کن.

۲. مقدار متغیرهای x و y و z را از کاربر در یافت کن.

۳. مجموع آن‌ها را به صورت زیر حساب کن و در S قرار بده :

$$(S \leftarrow x + y + z)$$

۴. مقدار معدل را به صورت زیر محاسبه کن و در M قرار بده :

$$(M \leftarrow S / 3)$$

۵. مقدار معدل که در متغیر M قرار دارد را چاپ کن.

۶. پایان.

با توجه به معماری وان نویمان که در کامپیوترهای امروزی لحاظ شده است، ویژگی مهم زبان‌های برنامه‌نویسی دستوری، متغیرها، دستورالعمل‌های انتساب، ساختارهای تصمیم‌گیری و حلقه‌های تکرار است که در فصول آتی به تک تک این موارد می‌پردازیم.

۱-۶: تمرین

۱. شکل کلی از کامپیوتر رسم کرده و کار هر یک از بخش‌های آن را شرح دهید؟
۲. الگوریتم را تعریف کنید و ویژگی‌های آن را برشمرید؟
۳. الگوریتمی جهت تعویض چرخ پنجر یک خودرو بنویسید؟
۴. الگوریتمی جهت طرز تهیه کیک زیر بنویسید؟

مواد لازم عبارت است از :

- روغن دو فنجان،
- آرد شش فنجان،
- شکر دو فنجان،
- شیر دو فنجان،
- تخم مرغ شش عدد،
- وانیل نصف قاشق،

دستورالعمل: روغن را در ظرفی بریزید و شکر را به آن اضافه کنید، تخم‌مرغ‌ها را در آن بشکنید و خوب مخلوط کنید، سپس آرد و شکر را به آن اضافه کنید و به هم بزنید تا حالت کشش پیدا نماید. وانیل را اضافه کنید و مجدداً به هم بزنید. ظرف مخصوص کیک را چرب کنید کمی آرد به آن پاشید و مخلوط را در آن هم بریزید. سپس ظرف را به مدت ۳۰ دقیقه در حرارت ۱۲۰ درجه قرار دهید.

آیا می‌توانید الگوریتم فوق را به ابتکار خود به صورت یک نمودار نمایش دهید؟

۵. الگوریتمی برای تهیه چای به وسیله یک سماور برقی بنویسید؟

۶. الگوریتمی بنویسید که شعاع دایره‌ای را بگیرد و محیط و مساحت آن را چاپ کند؟

۷. الگوریتمی بنویسید که درجه حرارت را برحسب سانتی‌گراد (C) بگیرد و به فارنهایت (F) تبدیل کند؟ برای حل الگوریتم از فرمول زیر استفاده کنید:

$$F = 9.5 * C + 32$$

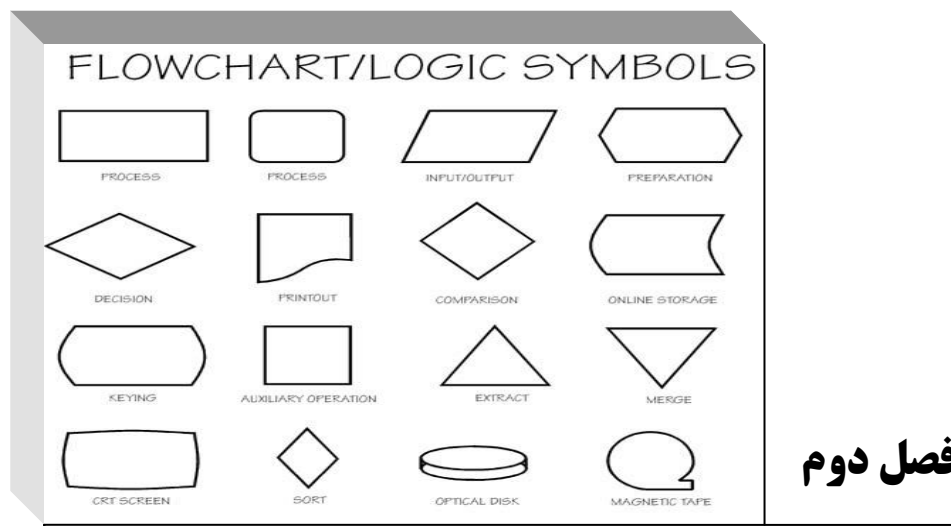
۸. الگوریتمی بنویسید که طول و عرض مستطیلی را دریافت کرده و محیط و مساحت آن را چاپ کند؟

۹. الگوریتمی بنویسید که قاعده و ارتفاع مثلثی را دریافت کرده و مساحت آن را چاپ کند؟

۱۰. الگوریتمی بنویسید که دو مقدار را بخواند و سپس مقدار بزرگتر را چاپ کند؟

۱۱. الگوریتمی بنویسید که عددی را خوانده و قدرمطلق آن را چاپ کند؟

۱۲. الگوریتمی برای پیدا کردن اولین جمله از دنباله فیبوناچی که از ۱۰۰۰ بزرگتر است به دست آورید؟ آیا می‌توانید این مسئله را از طریق مکاشفه‌ای حل کنید و برای آن یک فرمول عمومی ارائه کنید؟! پاسخ خود را توضیح دهید.



آشنایی با مقدمات زبان روندنما

اهداف فصل و چکیده مطالب :

۱. آشنایی با قواعد مقدماتی رسم کارنما
۲. آشنایی با تعریف متغیر و انواع عبارات در الگوریتم
۳. به کارگیری عبارات رابطه‌ای (شرطی) در الگوریتم‌ها
۴. آشنایی با مکانیسم‌های تکرار و تصمیم در الگوریتم
۵. آشنایی با تقدم عملگرها
۶. ارزیابی عبارات محاسباتی (جبری) بدون پرانتز
۷. خوانا ساختن عبارات محاسباتی به وسیله پرانتزگذاری
۸. آشنایی با چند تابع ریاضی پر کاربرد
۹. یافتن مهارت در چگونگی ساخت الگوریتم

۲-۱: آشنایی با قواعد مقدماتی رسم کارنما

در فصل گذشته با کلیاتی در خصوص ویژگی‌های الگوریتم آشنا شدیم، و الگوریتم‌هایی را توسط زبان فارسی نوشتیم. در این فصل به دنبال یافتن روش استاندارد برای بیان الگوریتم‌ها هستیم. معمولاً الگوریتم‌ها را توسط نمودارهای خاصی به نام کارنما (یا فلوچارت^۱) تحت یکسری قوانین استاندارد نشان می‌دهند. این بدان علت است که با رسم کارنما، هم می‌توان با به‌کارگیری قوه بصری افراد مفاهیم را به سرعت انتقال داد و در نتیجه بهره‌وری مخاطب را در یادگیری مطالب بالا برد؛ و هم اینکه با بهره‌گیری از قوانین استاندارد برای رسم کارنما، آن را برای هر فردی که با قوانین کارنما آشنایی داشته باشد قابل فهم کنیم. حتی برای شخص نویسنده الگوریتم نیز پیگیری الگوریتم و بررسی درستی آن به‌واسطه نمودار به مراتب راحت‌تر است. البته ما در این کتاب به دلایلی برخی از استانداردهای رسم کارنما را رعایت نخواهیم کرد که در جای خود شرح تک‌تک آنها داده خواهد شد. با توجه به نکاتی که در این فصل بیان می‌شود شما قادر خواهید بود بسیاری از الگوریتم‌های ساده را طراحی کنید و برای آنها کارنما رسم نمایید. اما در فصول آتی که مسائل پیچیده‌تری را مطرح می‌سازیم رفته‌رفته با مباحث پیشرفته‌تری از الگوریتم‌ها روبرو خواهید شد، که برای سهولت کار در ترسیم کارنما برای اینگونه الگوریتم‌ها، عناصری را به‌صورت یک قرارداد منحصر به این کتاب [و نه به صورت استاندارد] بیان می‌کنیم. به‌عنوان مثال در حلقه‌های تکرار از یک سری جعبه‌های فشنگی شکل استفاده خواهیم کرد، و این بدان جهت است که بتوانیم شرایط ساخت‌یافتگی را در مورد یک الگوریتم خاص مورد بحث و بررسی قرار دهیم.

خوشبختانه مجموعه قواعد زبان کارنما بسیار ساده و پیش پا افتاده است چنان که ظرف کمتر از یک ساعت می‌توان تمامی آنها را فراگرفت. اما آنچه که مشکل می‌نماید یادگیری چگونگی به‌کاربردن این قواعد برای ساخت یک الگوریتم است که تنها و تنها با تمرین و ممارست در این زمینه به‌دست می‌آید. الگوریتم نویسی و یافتن راه‌حل برای یک مسئله بیش از آنکه به آشنایی فرد با قواعد این زبان بستگی داشته باشد به هنر و قریحه فردی افراد بستگی دارد. چنانکه می‌توان گفت علم کامپیوتر و طراحی الگوریتم یک هنر است. البته این بدین معنا نیست که برخی از افراد به کلی در الگوریتم نویسی ناتوان هستند و برخی دیگر به‌طور ذاتی از آن برخوردار، بلکه منظور این است که، ممکن است راه‌حل‌های متفاوتی برای یک مسئله از سوی افراد مختلف ارائه شود ولی فقط یکی از آن راه‌حل‌ها بهترین راه‌حل به حساب می‌آید و بیشترین ارزش را دارد. مسلماً چنین راه‌حلی را فردی ارائه خواهد کرد که نهایت ذوق و قریحه را در طراحی الگوریتم خود به‌کار برده و به همه جوانب مسئله اندیشیده باشد.

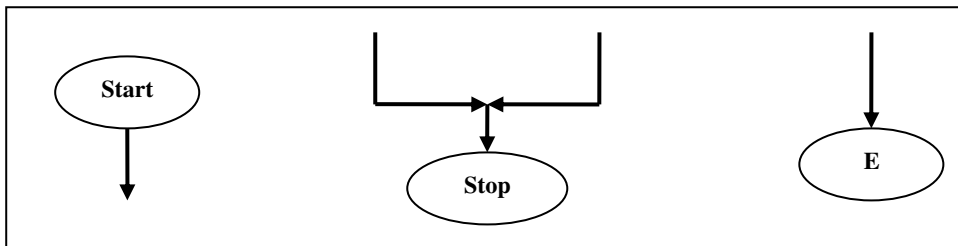
بنابراین در این فصل با در نظر گرفتن آنچه که در مورد ویژگی‌های الگوریتم در فصل قبل یادگرفتیم، قدم به قدم، برای قسمت‌های متفاوت الگوریتم‌هایی که تا اینجا نوشتیم، نمادهایی را در نظر می‌گیریم تا به کمک آن نمادها بتوانیم الگوریتم‌های خود را به‌صورت نموداری نشان دهیم. دوباره متذکر می‌شویم که حل تمامی تمرینات فصول مربوط به الگوریتم نویسی بسیار مهم است، چرا که بسیاری از نکات مهم در تمرینات آموزش داده شده است، تا خوانندگان با تلاش خود و به واسطه راهنمایی‌های کتاب به این نکات دست پیدا کنند و در نتیجه تأثیر مطالب در ذهن آنها ماندگارتر باشد. همچنین تمریناتی که دارای پیوستگی مطلب هستند با دقت مرتب شده‌اند.

1) flowchart

جعبه‌های شروع و پایان

همانگونه که دیدید در هر الگوریتم یک نقطه شروع و یک نقطه پایان وجود دارد. در استانداردهای انستیتوی ملی استاندارد امریکا (H.N.S.I) که تنها مرجع قانون‌گذار در این زمینه است پیشنهاد شده از نماد بیضی برای نشان دادن نقاط شروع و پایان الگوریتم‌ها استفاده شود، ما نیز از همین روش تبعیت می‌کنیم.

برای دکمه شروع الگوریتم از یک بیضی مدور با متن Start یا (S) و برای دکمه پایان نیز از یک بیضی به همان شکل اما با متن Stop یا End یا (E) استفاده می‌شود.



شکل ۱-۲: دکمه‌های شروع و پایان

نکات قابل توجه

۱. در هر الگوریتم تنها یک نقطه شروع وجود دارد. چرا که اگر نقطه شروع بیش از یکی باشد الگوریتم دارای ابهام خواهد بود. ولی ممکن است در الگوریتم چندین نقطه پایان وجود داشته باشد، در نتیجه می‌توان در هر کجا که نیاز باشد یک نقطه پایان در نظر گرفت. همچنین می‌توانید تنها یک نقطه پایان در نظر بگیرید و چندین پیکان ورودی به آن وارد کنید.

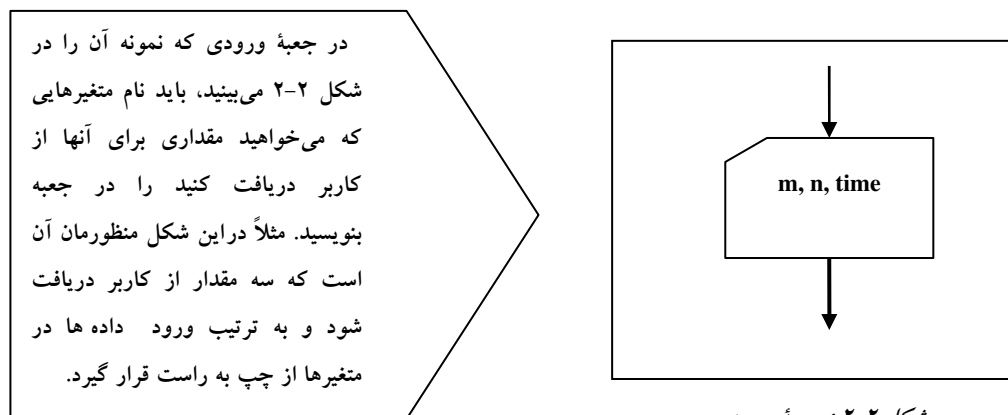
۲. در برخی از کتب به جای شکل بیضی برای دکمه‌های شروع و پایان از دایره استفاده می‌شود که مسلماً تفاوت چندانی در اصل موضوع ندارند. ما نیز در برخی از مثال‌ها از دایره استفاده کرده‌ایم. چنانکه بعداً خواهید دید در این کتاب از شکل بیضی تخت، برای جعبه‌های تصمیم استفاده شده است. لذا خوانندگان عزیز نباید شکل بیضی مدور که برای دکمه‌های شروع و پایان استفاده شده را با شکل بیضی تخت که برای جعبه‌های تصمیم استفاده شده است، اشتباه بگیرند.

۳. از هر مرحله می‌توان با یک پیکان خروجی به هر یک از مراحل قبلی کارنما، به غیر از دکمه شروع بازگشت. اما از دکمه‌های پایانی نمی‌توان به هیچ مرحله دیگری رفت.

۴. مقصود از پیگیری (یا آزمایش) الگوریتم آن است که از دکمه شروع، اجرای الگوریتم را آغاز کرده و مطابق با جهت پیکان خروجی از هر مرحله به مرحله بعدی بروید. تا اینکه در نهایت به یک دکمه پایانی برسید و اجرای الگوریتم متوقف گردد. در این حالت اگر الگوریتم نتایج مورد نظر را به دست داده باشد الگوریتم صحیح است در غیر این صورت باید مراحل الگوریتم را مورد بازبینی قرار دهید و دوباره آن را پی‌گیری نمود. به این عمل آزمایش الگوریتم یا اثبات درستی الگوریتم نیز گفته می‌شود.

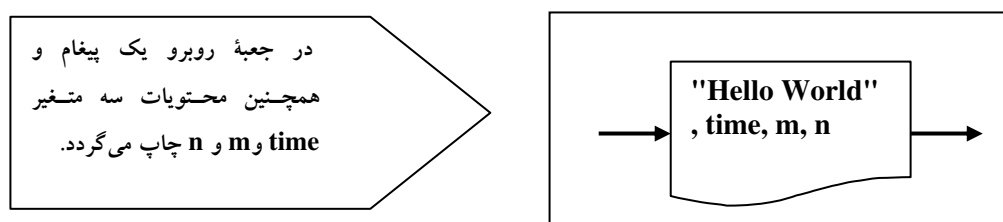
ورود و خروج اطلاعات

فرض کنید می‌خواهید از کاربر داده‌هایی را دریافت کنید و در متغیرهایی ذخیره نمایید. در این صورت از جعبه‌هایی مانند شکل ۲-۲ باید استفاده کنید. این جعبه ما را به یاد کارت منگنه که یکی از وسایل متداول ورود اطلاعات در گذشته به شمار می‌رفته می‌اندازد.



شکل ۲-۲: جعبه ورودی

همچنین جهت نمایش پیام‌ها و نتایج به دست آمده؛ از جعبه‌هایی مانند شکل ۲-۳ استفاده می‌کنیم که شبیه یک تکه کاغذ می‌باشد. نکته قابل توجه آنکه اگر می‌خواهید پیامی چاپ گردد باید آن را در بین دو علامت نقل قول به شکل " " قرار دهید. همچنین اگر متغیری را در داخل جعبه خروجی قرار دهید محتویات آن متغیر چاپ می‌شود نه نام آن متغیر.



شکل ۲-۳: جعبه خروجی

در جعبه‌های خروجی نیز اطلاعات به ترتیب از چپ به راست چاپ می‌گردد. به عنوان مثال در شکل ۲-۳ ابتدا پیام "Hello World" چاپ می‌گردد و سپس محتویات سه متغیر time و m و n چاپ می‌شود.

تذکر: در برخی از کتب؛ هم برای جعبه‌های ورودی و هم برای جعبه‌های خروجی، از جعبه‌هایی به شکل متوازی الاضلاع استفاده می‌کنند و در آنها از لفظ "بخوان" و "بنویس" برای نشان دادن عمل خواندن (یا نوشتن) استفاده می‌شود.

متغیرها و جعبه انتساب

همانگونه که در جبر از متغیرها در عبارات محاسباتی (جبری) استفاده می‌کنید در الگوریتم‌ها نیز می‌توانید از متغیرها برای این منظور استفاده کنید. هر متغیر با یک یا چند حرف لاتین به‌عنوان نام متغیر شناسایی می‌شود، و در مکان‌های مختلف می‌توان با استفاده از نام متغیر به محتویات آن دست یافت. می‌توان چنین فرض کرد که متغیر دارای مخزن است که قادریم در آن مخزن، مقادیری از اعداد حقیقی را جای دهیم، و با داشتن نام متغیر می‌توانیم به سراغ مخزن آن برویم و به محتویات داخل مخزن دست پیدا کنیم. به عبارت دیگر متغیر نامی است برای محتویات داخل مخزن خود. لذا ممکن است در طول الگوریتم محتویات مخزن به دفعات تغییر کند اما نام متغیر از اول تا پایان ثابت می‌ماند. اولین نکته‌ای که باید در این خصوص به خاطر بسپارید روش نامگذاری صحیح متغیرها است، به‌طوری که در نامگذاری متغیرها باید قواعد زیر را رعایت کنید:

۱. در نامگذاری متغیرها می‌توانید از حروف کوچک و بزرگ لاتین و ارقام و خط زیر استفاده کنید. بنابراین کاراکترهای مجاز در نامگذاری متغیرها عبارتند از : $a, b, c, \dots, z, A, B, C, \dots, Z, 0, 1, 2, \dots, 9, _$
۲. حرف اول نام یک متغیر نمی‌تواند رقم باشد. گرچه استفاده از خط زیر در نخستین کاراکتر نام متغیر بلامانع است ولی پیشنهاد می‌شود که حرف اول نام یک متغیر حتماً یک حرف لاتین باشد. این مسئله در برخی زبان‌های برنامه‌نویسی ضروری است.
۳. بهتر است نام متغیرها با معنا و توصیف‌گر محتویات خود باشند تا در پیگیری الگوریتم با مشکلی برخورد نکنید.
۴. طول نام یک متغیر می‌تواند از یک حرف آغاز شود و با ترکیب حروف و ارقام مختلف گسترش یابد. اما در زبان‌های برنامه‌نویسی اصولاً طول نام متغیرها از یک مقدار حداکثر نمی‌تواند بیشتر باشد. لذا ما نیز به تبعیت از این قاعده در الگوریتم‌های خود طول نام متغیرها را بیش از ۳۲ کاراکتر نمی‌گیریم.
۵. با توجه به آنچه گفته شد نام‌های زیر برای متغیرها مجاز شمرده می‌شوند:
`Time , m , n435p , odd_number_654j`
۶. فراموش نکنید که دو متغیر زیر با یکدیگر متفاوت هستند:
`M , m`

پس از تعریف کردن یک متغیر می‌بایستی یا در یک جعبه ورودی مقداری برای آن از کاربر دریافت کنید، و یا آنکه توسط یک دستور انتساب مقداری را به آن نسبت دهید. شکل کلی دستور انتساب به‌صورت زیر است:

(یک متغیر) یا (یک عبارت) یا (یک عدد حقیقی) $\leftarrow$ متغیر

در سمت چپ پیکان همواره تنها یک متغیر قرار می‌گیرد، بدین‌معنا که مقدار سمت راست در متغیر سمت چپ قرار داده شود. اما در سمت راست ممکن است از یک عدد و یا نام یک متغیر دیگر استفاده کنیم. مانند نمونه‌های زیر :

`Total  $\leftarrow$  17` در این مثال مقدار ۱۷ در متغیر سمت چپ قرار می‌گیرد.

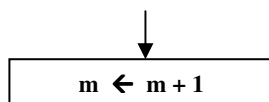
`m2  $\leftarrow$  m1` در این مثال محتویات متغیر `m1` در متغیر `m2` قرار می‌گیرد.

ولی گاهی هم پیش می‌آید که عبارتی را در سمت راست دستور انتساب قرار می‌دهیم. این عبارات بر دو گونه هستند. یا رشته‌ای از کاراکترها هستند، که عین عبارت رشته‌ای [که باید بین دو علامت " محصور شده باشد] در متغیر قرار می‌گیرد، مثل: $C \leftarrow \text{"Hello World"}$

و یا آنکه با یک عبارت محاسباتی (جبری) که ترکیبی از علائم ریاضی و نام تعدادی متغیر می‌باشد سروکار داریم. اگر یک عبارت جبری در سمت راست یک دستور انتساب قرار گیرد مقدار عبارت جبری محاسبه شده و حاصل آن در متغیر سمت چپ علامت پیکان قرار داده می‌شود. تنها نکته قابل توجه در مورد عبارات محاسباتی این است، که در این عبارات، به جای علامت ضرب ($\times$) از علامت ستاره ($*$) و به جای علامت تقسیم ($\div$) از علامت ممیز ($/$) استفاده می‌کنیم. بنابراین باید، برای نشان دادن اعداد اعشاری، از نقطه به عنوان ممیز عدد استفاده کنیم، نه علامت ممیز. به عنوان مثال عبارت زیر را در نظر بگیرید:

$$\Delta \leftarrow B^2 - 4 * A * C$$

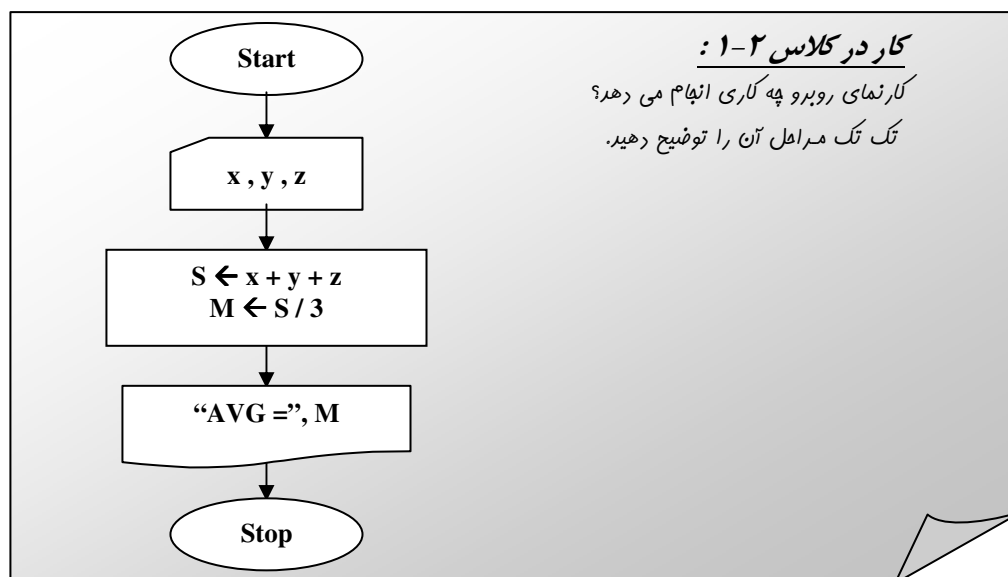
در دستور فوق ابتدا حاصل عبارت جبری سمت راست محاسبه شده و در متغیر Δ قرار داده می‌شود. دستورات انتساب از هر کدام یک از اشکال فوق که باشند می‌بایستی در داخل جعبه‌های مستطیلی شکلی به نام جعبه‌های انتساب قرار گیرند. به عنوان مثال در شکل زیر در داخل جعبه انتساب دستوری قرار گرفته که محتویات متغیر m را یک واحد افزایش می‌دهد:



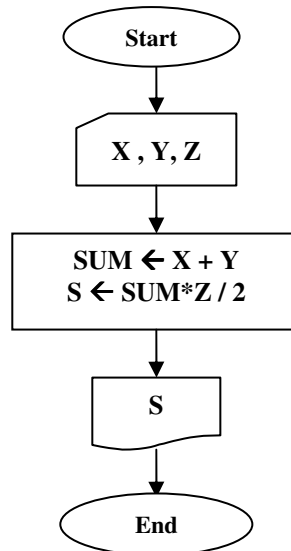
شکل ۲-۴: جعبه انتساب

رسم اولین کارنما

حال وقت آن رسیده است که اولین کارنما را با استفاده از آنچه که تا اینجا فراگرفتیم رسم کنیم.

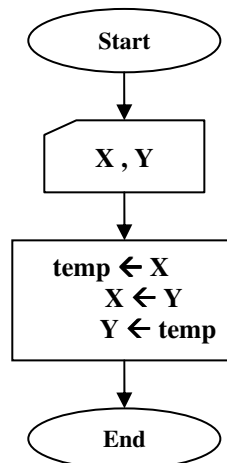


مثال ۱-۲: کارنمایی که با دریافت سه عدد به عنوان قاعده کوچک، قاعده بزرگ و ارتفاع یک دوزنقه مساحت آن را چاپ می کند.



شکل ۲-۵: کارنمای مثال ۱-۲

مثال ۲-۲: کارنمایی که دو مقدار را دریافت کرده و در دو متغیر X و Y ذخیره می کند و سپس محتویات این دو متغیر را به واسطه یک متغیر کمکی به نام $temp$ عوض می کند.



شکل ۲-۶: کارنمای مثال ۲-۲

آیا می توانید محتویات جعبه مستطیلی در کارنمای بالا را چنان تغییر دهید که دیگر نیازی به استفاده از متغیر $temp$ نباشد؟

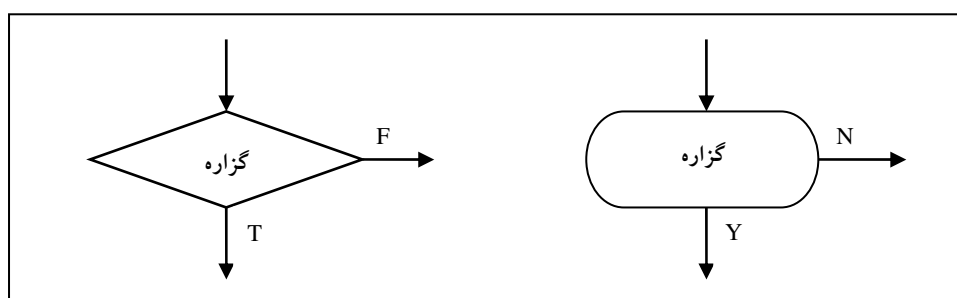
۲-۲: تصمیم‌گیری در الگوریتم‌ها (عملیات مقایسه‌ای و شرطی)

جعبه‌های تصمیم

در بسیاری از الگوریتم‌ها پیش می‌آید که بخواهیم روند آن الگوریتم را برحسب شرایط مختلف کنترل کرده و در شرایط گوناگون اعمال متفاوتی را انجام دهیم. به‌عنوان مثال در الگوریتم تلفن زدن دیدید که، در مرحله سوم با آوردن یک جمله شرطی خواستار آن شدیم، که الگوریتم بر حسب درستی یا نادرستی شرط مذکور به یکی از مراحل ۴ یا ۵ رفته و روند اجرای الگوریتم از یکی از این دو مرحله از سر گرفته شود.

بر این اساس جعبه‌ای با نام "جعبه تصمیم" در نظر گرفته شده است که در داخل آن یک گزاره ساده یا مرکب قرار می‌گیرد؛ و دو پیکان با بر حسب درست (T) و نادرست (F) از آن خارج می‌شود. بیشتر در ریاضیات گسسته با گزاره به‌عنوان "جمله‌ای خبری که دارای ارزشی درست یا نادرست است" آشنا شده‌اید. بنابراین در خصوص گزاره و ویژگی‌های آن بحثی نخواهیم کرد و به بررسی ویژگی‌های جعبه تصمیم می‌پردازیم. هنگامی که روند اجرای الگوریتم به جعبه تصمیم برسد، با بررسی ارزش گزاره داخل جعبه، براساس درست یا نادرست بودن گزاره مذکور، روند الگوریتم در جهت پیکان T [در صورت درست بودن گزاره] و یا در جهت پیکان F [در صورت نادرست بودن گزاره] حرکت می‌کند. گاهی به‌جای دو نماد فوق نمادهای Y و N به‌کار می‌رود.

در حالت استاندارد برای جعبه تصمیم از لوزی استفاده می‌کنند، اما در این کتاب در بیشتر مثال‌ها از بیضی تخت با طول دلخواه استفاده شده است، تا آزادی عمل بیشتری در توضیح و تفصیل گزاره تصمیم برای دانشجویان فراهم کرده باشیم، و بدین‌وسیله آنها هر چه لازم می‌دانند گزاره خود را شرح و تفصیل دهند. خصوصاً برای دانشجویانی که در ابتدای راه هستند شرح و تفصیل جعبه‌های تصمیم بسیار لازم و ضروری است. اما شما می‌توانید از هر یک از این دو جعبه استفاده کنید چنانکه ما نیز از هر دوی این جعبه‌ها در مثال‌های کتاب بهره گرفته‌ایم. شکل کلی این جعبه‌ها به‌صورت زیر است:



شکل ۲-۷ جعبه‌های تصمیم

از آنجا که اصولاً در اکثر الگوریتم‌ها از عبارات رابطه‌ای (شرطی) به‌عنوان گزاره استفاده می‌شود در ادامه قبل از بیان هرگونه مثالی به بررسی انواع عبارات رابطه‌ای می‌پردازیم.

بررسی عبارات رابطه‌ای (شرطی) ساده

منظور از یک عبارت رابطه‌ای (شرطی) ساده هر عبارتی به شکل کلی زیر است:

$$\begin{aligned} &\text{عبارت مساب۱} + ۲ \text{ یک علامت رابطه ای} + \text{عبارت مساب۱} \\ &\text{و مقصود از علائم رابطه‌ای علامت‌های زیر است که از قبل در ریاضیات با آنها آشنا شده‌اید:} \\ &=, <, >, \leq, \geq, \neq \end{aligned}$$

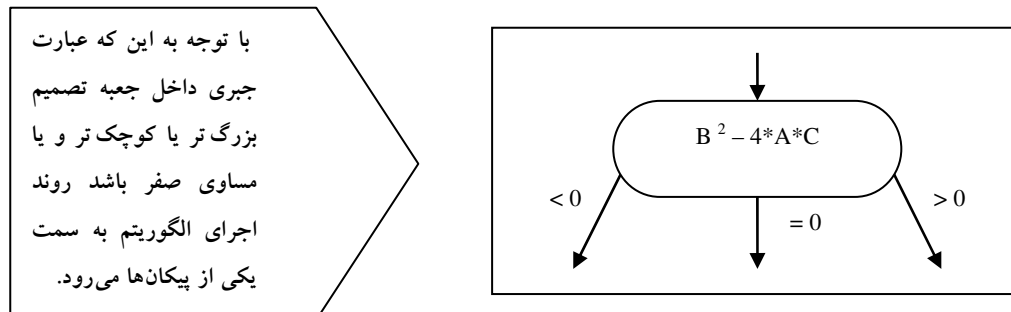
یک عبارت شرطی ساده می‌تواند دارای ارزشی درست یا نادرست باشد. به عنوان مثال عبارات شرطی زیر را در نظر بگیرید:

۱. $۵ < ۳$: این عبارت دارای ارزش نادرستی است و اگر در داخل یک جعبه تصمیم قرار بگیرد موجب می‌شود که روند اجرای الگوریتم به سمت پیکان F برود.
۲. $۴ \leq ۰$: این گزاره دارای ارزش درستی است و اگر در داخل یک جعبه تصمیم قرار بگیرد موجب می‌شود که روند اجرای الگوریتم به سمت پیکان T برود.
۳. $n \neq m$: این گزاره در صورتی که دو متغیر دارای محتویات متفاوتی باشند دارای ارزش درستی خواهد بود و روند اجرای الگوریتم را به سمت پیکان T می‌برد. و در صورتی که دارای محتویات مشابهی باشند دارای ارزش نادرستی بوده و روند اجرای الگوریتم را به سمت پیکان F هدایت می‌کند.
- با توجه به آنکه علائم رابطه‌ای بر روی اعداد تعریف شده‌اند، اگر در طرفین علامت رابطه‌ای یک عبارت محاسباتی (جبری) آمده باشد، ابتدا حاصل آن عبارت جبری محاسبه شده و جایگزین آن عبارت می‌گردد. سپس براساس حاصل به دست آمده ارزش گزاره شرطی معین می‌شود. به عنوان مثال به عبارت زیر توجه کنید:
۴. $B^2 - 4 * A * C > 0$: در این عبارت شرطی ساده ابتدا حاصل عبارت معروف دلتا که در سمت چپ علامت رابطه ای قرار دارد محاسبه می‌شود و سپس در صورتی که مقدار آن مثبت و بزرگتر از صفر باشد روند اجرای الگوریتم به سمت پیکان T و در غیر این صورت به سمت پیکان F می‌رود.
- البته بدین دلیل ابتدا باید مقدار عبارت جبری محاسبه شود که در جدول تقدم عملگرها، عملگرهای رابطه‌ای پس از عملگرهای حسابی قرار دارند و بنابراین دیرتر مورد ارزیابی قرار می‌گیرند. در این خصوص در بخش ۲-۱۴ بیشتر صحبت خواهیم کرد.
- بنابراین مفهوم الگوریتم شرطی را می‌توان به طور خلاصه چنین بیان کرد: هر گاه در الگوریتمی بخواهیم برحسب شرایطی، مجموعه‌ای از دستورات اجرا شوند و در صورت عدم تحقق آن شرایط مجموعه دیگر از دستورات اجرا شوند، باید از جعبه‌های تصمیم استفاده کنیم. همچنین شکل کلی زیر را می‌توان برای آن عنوان نمود:

اگر (شرط درست است) آنگاه (مجموعه دستورات ۱) وگرنه (مجموعه دستورات ۲)

تذکره: به کاربرد کلمه "اگر" در عبارت فوق خوب توجه کنید. رابطه‌ای مستقیم بین این کلمه و جمله شرط و به کارگیری جعبه تصمیم وجود دارد.

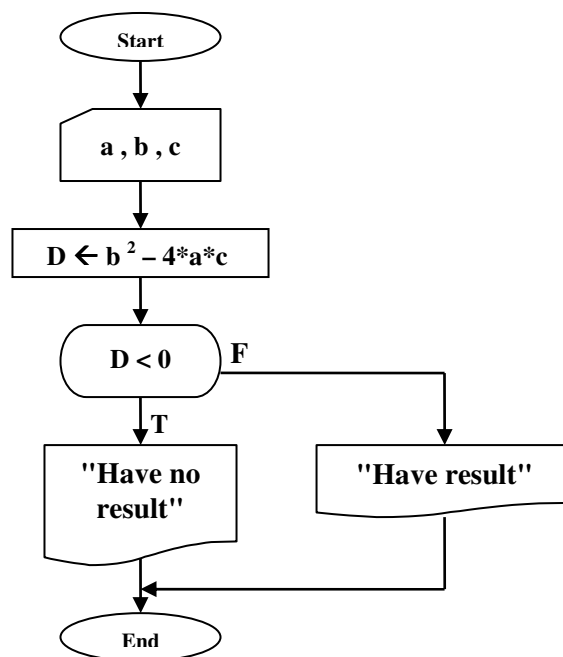
اما گاهی هم پیش می‌آید که با جعبه‌های تصمیم چندگانه (چند راهه) سروکار داشته باشیم. به عنوان مثال :



شکل ۲-۱: جعبه تصمیم چندگانه

در خصوص ویژگی‌های جعبه‌های تصمیم چندگانه (چندراهه) نیز در قسمت ۲-۱۴ بیشتر صحبت خواهیم کرد.

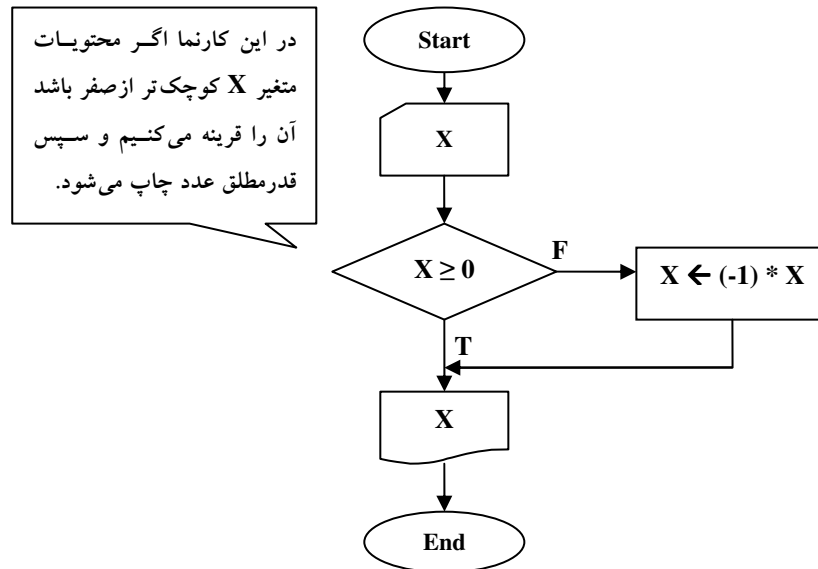
مثال ۲-۳: کارنمایی که با دریافت سه عدد به عنوان ضرایب یک معادله درجه دوم اعلام می‌کند که آیا این معادله درجه دوم ریشه حقیقی دارد یا نه.



شکل ۲-۹: کارنمای مثال ۳-۲

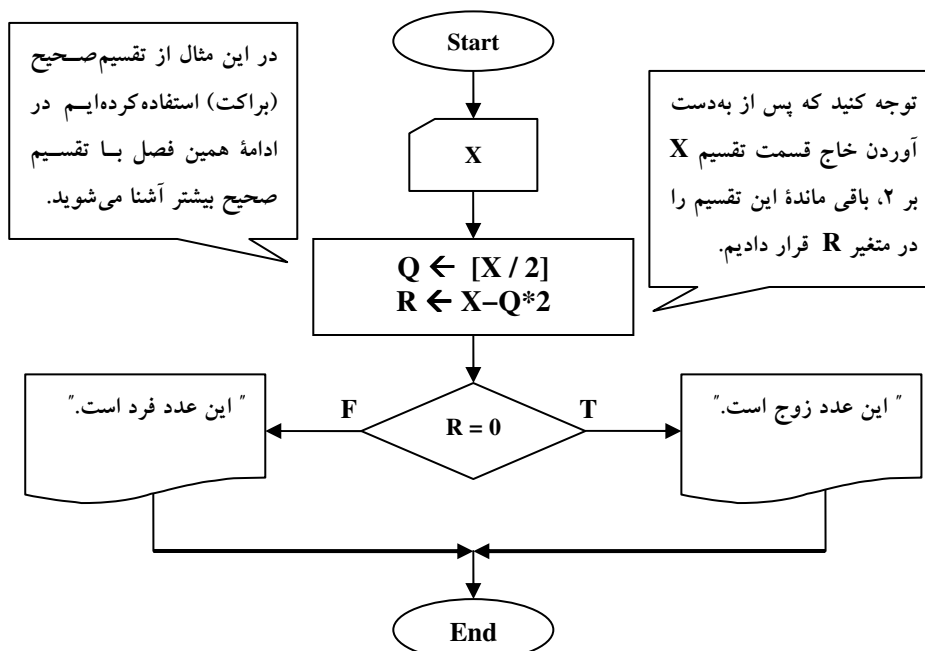
توضیح کارنما: در این کارنما پس از خواندن مقادیر سه متغیر a و b و c حاصل عبارت Δ محاسبه می‌شود و در صورتی که مقدار عبارت دلنا کوچکتر از صفر باشد، روند اجرای الگوریتم به سمت پیکان T می‌رود و پیغام عدم وجود جواب حقیقی را چاپ می‌کند و در صورتی که بزرگ‌تر یا مساوی صفر باشد، روند اجرای الگوریتم به سمت پیکان F خواهد رفت و پیغام وجود داشتن جواب چاپ می‌شود و در هر دو صورت در نهایت به حالت توقف می‌رود.

مثال ۲-۴: کارنمایی که با دریافت یک عدد، قدرمطلق آن را چاپ می‌کند.



شکل ۲-۱۰: کارنمای مثال ۲-۴

مثال ۲-۵: کارنمایی که با دریافت یک عدد مشخص می‌کند آن عدد زوج است یا فرد.

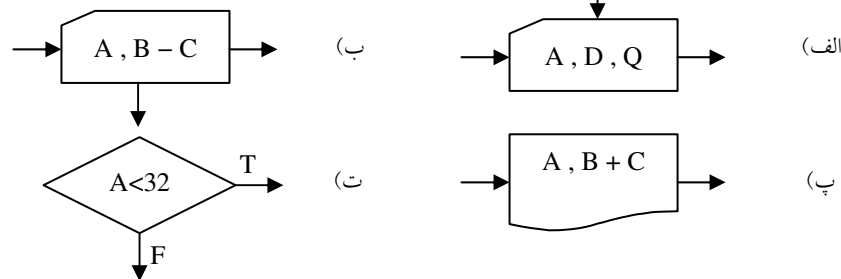


شکل ۲-۱۱: کارنمای مثال ۲-۵

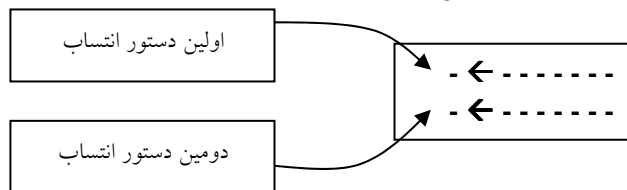
۳-۲: تمرین

۱. مزایای استفاده از کارنما به جای زبان‌های طبیعی چیست؟

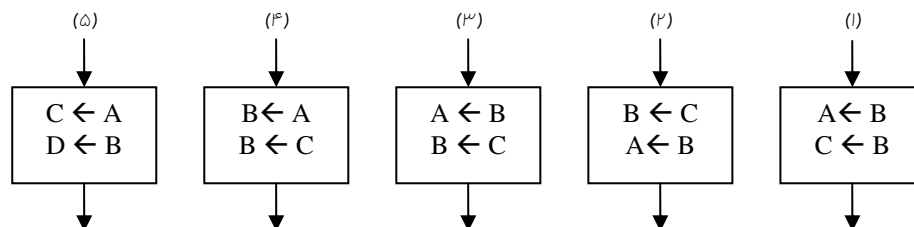
۲. از جعبه‌های زیر کدام جعبه‌ها درست و کدام جعبه‌ها غلط است؟ دلیل خود را شرح دهید.



۳. در شکل زیر، دو دستور انتساب در یک جعبه انتساب قرار گرفته‌اند. یک ضابطه کلی بیان کنید که تحت آن بتوان جای این دو دستورالعمل را، بدون اینکه تغییری در حاصل عملکرد جعبه ایجاد شود، عوض کرد.



راهنمایی: قبل از جواب دادن به سؤال فوق ۵ مورد زیر را در نظر بگیرید:



۴. درستی یا نادرستی هر یک از گزاره‌های موجود را با توجه به کارنمای روبرو تعیین کنید. (برای ۳ گزاره نخست فرض کنید مجموعه داده‌هایی که در نتیجه اجرای جعبه ۱ خوانده می‌شود، به ترتیب عبارت‌اند از ("F", "L", "O", "W", "1", "0") و برای ۲ گزاره آخر، فرض کنید در جعبه ۱ هر مجموعه دلخواهی از داده‌ها امکان دارد وارد شود.)

الف- مجموعه داده‌های فوق، اجرا فقط یکبار به جعبه ۵ می‌رسد.

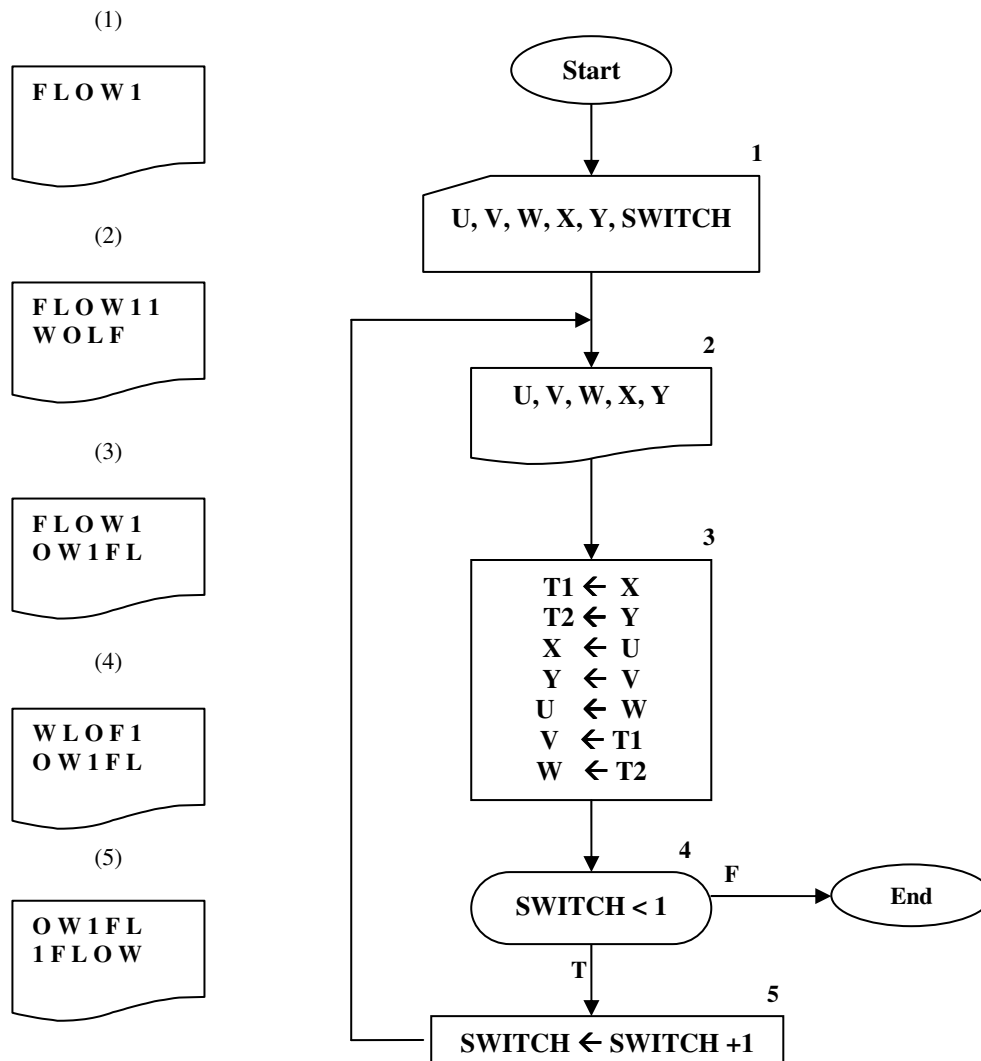
ب- برای مجموعه داده‌های ارائه شده، اجرا به جعبه ۶ خواهد رسید.

ج- اگر در داده‌های بالا به جای صفر عدد ۶ به کامپیوتر داده شود، جعبه ۲ شش مرتبه اجرا خواهد شد.

د- به ازای هر مجموعه داده‌ای، که در جعبه ۱ خوانده شود، جعبه ۲ همیشه دو بار اجرا می‌شود.

ه- فقط صفر و یک به عنوان داده برای متغیر SWITCH قابل قبول است.

۵. کدام یک از پنج جعبه خروجی که در زیر آمده، خروجی الگوریتم تمرین قبلی با همان ورودی‌ها است؟
۶. آیا می‌توان کارنمای مثال ۲-۲ را به گونه‌ای تغییر داد که بدون استفاده از متغیر کمکی محتویات دو متغیر X و Y را تعویض کند؟



شکل ۲-۱۲: کارنمای تمرین‌های ۴ و ۵

۲-۴: تکرار در الگوریتم‌ها**شمارنده‌ها و نگهبان‌ها**

مهمترین نقشی که ماشین‌ها از جمله کامپیوتر در زندگی روزمره انسان‌ها دارند، چنین است که انسان را از انجام امور تکراری و کسل‌کننده معاف می‌کنند. در این بین با بسیاری از مسائل برخورد می‌کنید که برای حل آنها نیازمند تکرار یک الگوی خاص هستید. برای حل اینگونه مسائل تنها کافی است مسئله را برای یک نمونه حل کنید و سپس با به‌کارگیری مکانیسم تکرار، الگوریتم را برای تعداد بیشتری داده گسترش دهید و بدین‌طریق قادر به حل کل مسئله خواهید بود. به‌عنوان مثال فرض کنید می‌خواهید معدل یک کلاس ۱۰۰ نفری که هر کدام از دانشجویان کلاس دارای ۵ نمره هستند را محاسبه کنید؟! اگر بخواهید چنین عمل خسته‌کننده‌ای را با دست انجام دهید، قطعاً در میانه راه باز خواهید ماند، و مسلماً با افزایش تعداد اعداد و ارقام عرصه سخت‌تر خواهد شد. پس بهتر است این قبیل کارها را به وسیله کامپیوتر انجام دهید. اما شما چه ایده‌ای برای حل اینگونه مسائل دارید؟ یا به زبان ساده‌تر شما چه الگوریتمی را برای این مسئله می‌توانید ارائه دهید؟

کار در کلاس ۲-۴:

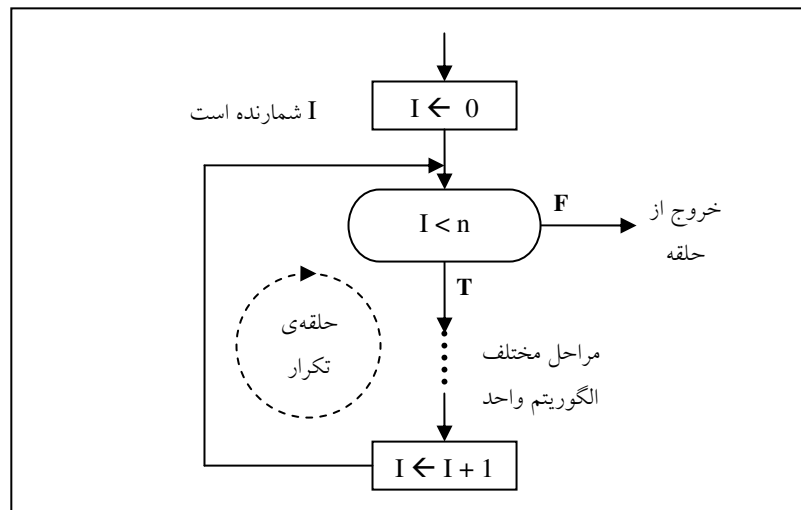
به ابتکار فورکارنمایی برای حل مسئله فوق ارائه دهید؟

بحث خود را با این سؤال ادامه می‌دهیم که: برای حل این مسئله نیازمند انجام چه کارهایی هستیم؟! ۱. ابتدا باید الگوریتمی بنویسیم که معدل یک دانشجو را بتواند حساب کند. به عبارت دیگر، باید مسئله را برای حالتی که تعداد موارد برابر یک است حل کنیم. اسم این الگوریتم را الگوریتم واحد می‌گذاریم.

۲. سپس باید با یک مکانیسم تکرار این عمل را آنقدر تکرار کنیم تا معدل همه دانشجویان حساب شود، و در نهایت از مجموعه معدل‌های به‌دست آمده معدل می‌گیریم تا معدل کل کلاس حساب شود.

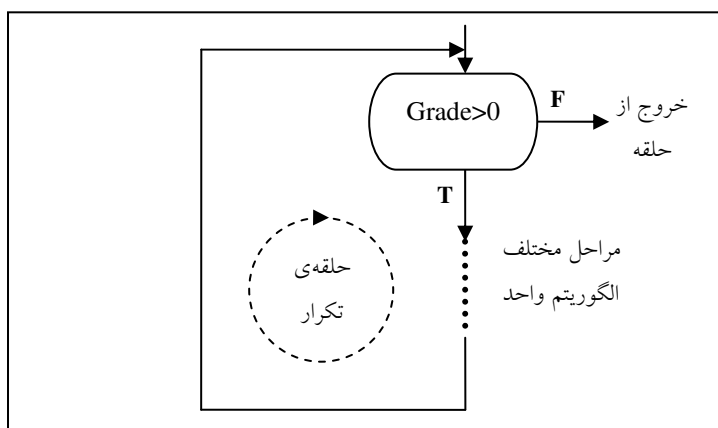
آنچه که تا اینجا مطرح شد استراتژی حل مسئله نامیده می‌شود. حال باید به دنبال روشی برای پیاده‌سازی این استراتژی باشیم. مهمترین مطلبی که می‌توانیم برای حل این مشکل از آن کمک بگیریم تعداد دانشجویان است. وقتی که ما تعداد دانشجویان را بدانیم و مسئله را برای یک دانشجو حل کرده باشیم، می‌توانیم الگوریتم را به‌گونه‌ای تغییر دهیم که الگوریتم واحد را برای n دانشجو تکرار کند؛ و در نتیجه معدل این تعداد دانشجو حساب می‌شود.

به این روش یعنی قراردادن یک متغیر برای محاسبه تعداد دفعات گذر از حلقه تکرار "روش شمارنده" گفته می‌شود. برای به‌کارگیری این روش ابتدا یک متغیر را به‌عنوان شمارنده تعداد دفعات گذر از حلقه، به صفر مقدار اولیه می‌دهیم، سپس به ازای هر بار گذر از حلقه تکرار یک واحد به آن متغیر می‌افزاییم، در نتیجه قبل از هر بار انجام دوباره الگوریتم واحد، چک می‌کنیم که آیا الگوریتم ما به تعداد دفعات موردنظر انجام شده است یا نه؟ اگر حلقه تکرار که دربردارنده الگوریتم واحد است به تعداد دفعات موردنظر انجام شده بود باید از انجام دوباره آن جلوگیری کنیم وگرنه باید یکبار دیگر مراحل بالا انجام شود. بنابراین باید شرط تکرار حلقه را در یک جعبه تصمیم قرار دهیم. بدین ترتیب اجرای مکرر الگوریتم واحد ادامه پیدا می‌کند تا در نهایت الگوریتم به حالتی برسد که از حلقه خارج شود. صورت کلی این روش در شکل ۲-۱۳ نشان داده شده است:



شکل ۲-۱۳: نمای کلی از حلقه تکرار با شمارنده

اما به روش دیگری نیز می‌توان تعداد دفعات تکرار حلقه‌های تکرار را کنترل کرد، که به "روش نگهبان" موسوم است. در این روش شرط حلقه را به‌گونه‌ای تنظیم می‌کنیم که به واسطه دستورات داخل خود حلقه پس از چندین بار تکرار حلقه، آن شرط نقض گردد و در نتیجه حلقه متوقف شود. به‌عنوان مثال می‌توان یک ورودی نامعتبر را به‌عنوان خاتمه دهنده حلقه در نظر گرفت. مثلاً از آنجایی که نمره زیر صفر، یک ورودی نامعتبر است، می‌تواند به‌عنوان شرط خاتمه‌دهنده حلقه تکرار مثال قبل به کار رود، بدین‌صورت که هر گاه کاربر یک نمره منفی را به‌عنوان ورودی وارد کرد، شرط حلقه دارای ارزش نادرست گردد و از حلقه خارج شود.



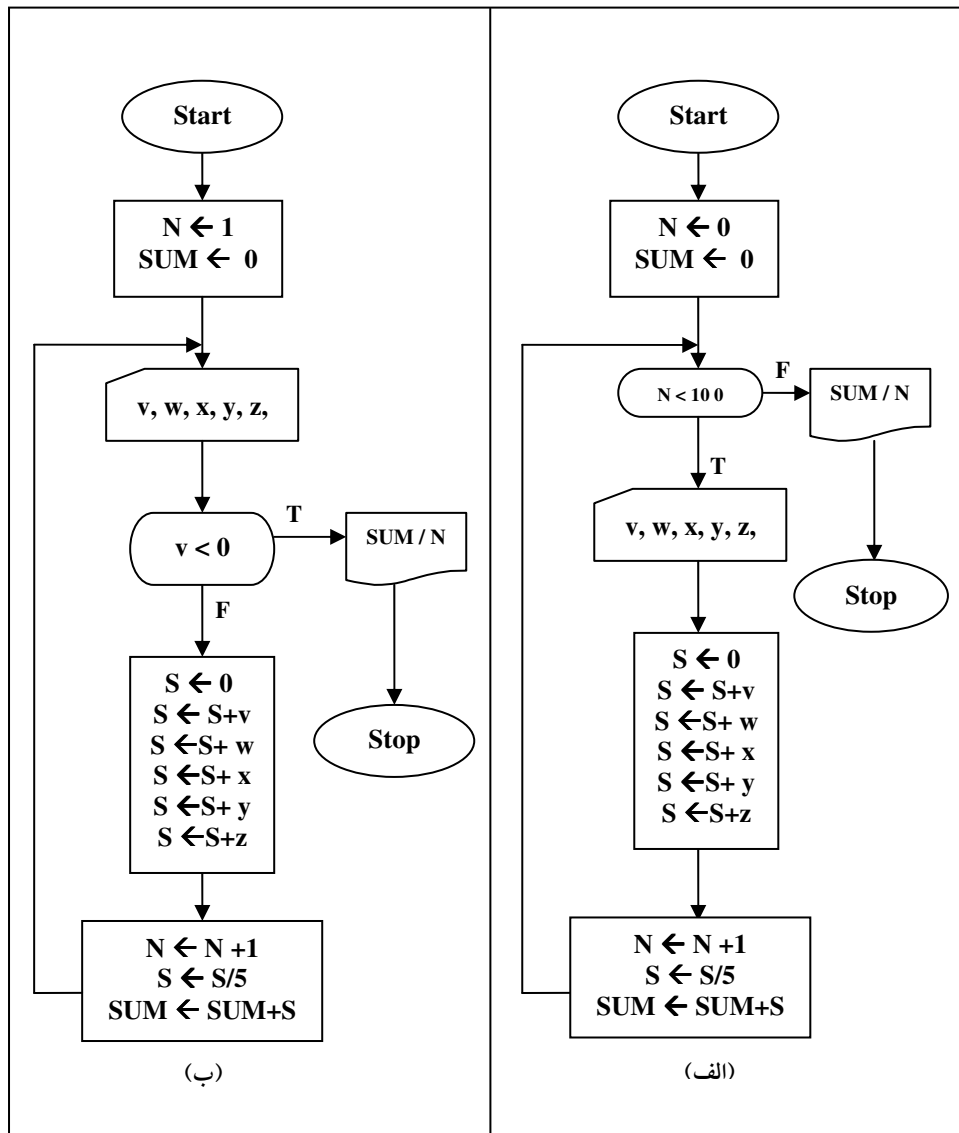
شکل ۲-۱۴: نمای کلی از حلقه تکرار با نگهبان

در شکل ۲-۱۵ کارنمای الگوریتم‌هایی را که برای مسئله محاسبه معدل بیان شد، ملاحظه می‌کنید.

کار در کلاس ۲-۳:

به سؤالات زیر در فصول کارنمای شکل ۲-۱۵ پاسخ دهید:

۱. هر یک از کارنماها را شرح دهید و نام روش هر کدام را بنویسید؟
 ۲. چرا در یک کارنما متغیر N را به صفر و در دیگری به یک مقدار اولیه داده ایم؟
 ۳. اگر در هر دو الگوریتم به‌طور مشابه با متغیر N رفتار می‌کردیم چه رخ می‌داد؟
 ۴. در مورد تعداد دفعات اجرای حلقه در هر یک از کارنماها بحث کنید؟
 ۵. اگر در کارنمای (ب) به جای شرط $v < 0$ از شروط دیگری استفاده می‌کردیم چه می‌شد؟ اصلاً چنین امری ممکن هست یا خیر؟
 ۶. نگهبان حلقه‌ی تکرار چه ویژگی‌هایی باید داشته باشد؟
- راهنمایی: فرض کنید دامنه نمرات بین ۰ تا ۲۰ می‌باشد.



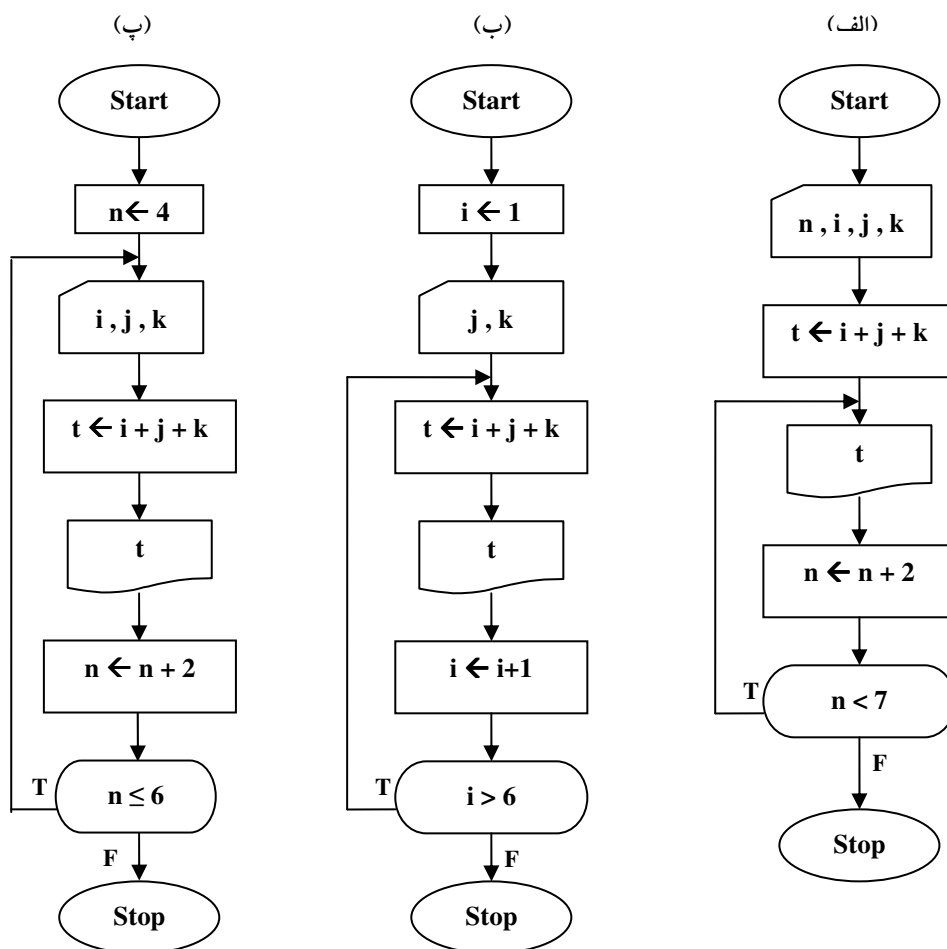
شکل ۲-۱۵ دو نوع کنترل حلقه تکرار

تنها نکته‌ای که از بحث الگوریتم‌های دارای نگهبان ناگفته ماند این است که، اگر نگهبان حلقه به واسطه داده ورودی نقض می‌شود، این ورودی نباید در دامنه داده‌های قابل قبول الگوریتم قرار داشته باشد. مثلاً در کارنمای (ب) در شکل ۲-۱۵ شرط نمرة منفی را در نظر گرفتیم، چرا که نمرة زیر صفر، معنی ندارد و کاربرد با وارد کردن عددی منفی موجب ایستادن حلقه تکرار می‌شود.

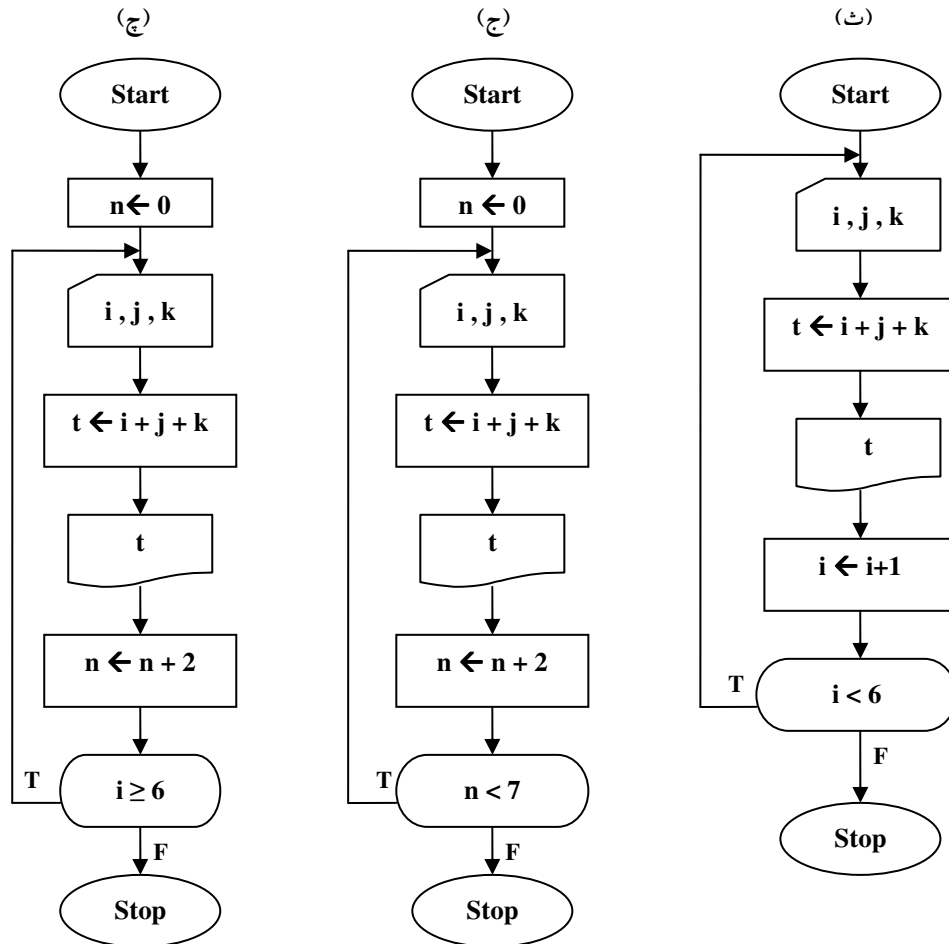
حال اگر شرط این حلقه $(v > 20)$ بود چه تغییری باید در الگوریتم و یا نحوه استفاده از آن می‌دادیم؟

۲-۵: تمرین

۱. در مورد حلقه‌هایی که با شمارنده کنترل می‌شدند، به‌خاطر دارید که تکرار محاسبات تعیین شده وقتی متوقف می‌شود که به تعداد دفعات موردنظر از حلقه عبور شده باشد. در هر کدام از کارنامه‌های زیر تعداد دفعات اجرای حلقه را تعیین کنید؟



شکل ۲-۱۶ (الف) - کارنامه‌های تمرین ۱



شکل ۲-۱۶ (ب)- کارنماهای تمرین ۱

آیا کارنمایی وجود داشت که نتوانید تعداد تکرار حلقه آن را محاسبه کنید؟ در صورت مثبت بودن پاسخ چه

نتیجه‌ای گرفته‌اید؟ به عبارت دیگر از این پس به چه نکته‌ای در ترسیم کارنماهای خود باید توجه داشته باشید؟

۲. استادی مسئله ساختن کارنمایی به شرح زیر را به دانشجویان کلاسی محول کرد. ورودی الگوریتم شامل طول و

عرض چند مستطیل است. هدف، تهیه فهرستی است از خطوطی که دارای شماره ردیف بوده و طول (L)، عرض

(W)، و مساحت (A)، آن دسته از مستطیل‌هایی را که محیط آنها از ۱۲ بیشتر است به دست دهد. کارنماهای

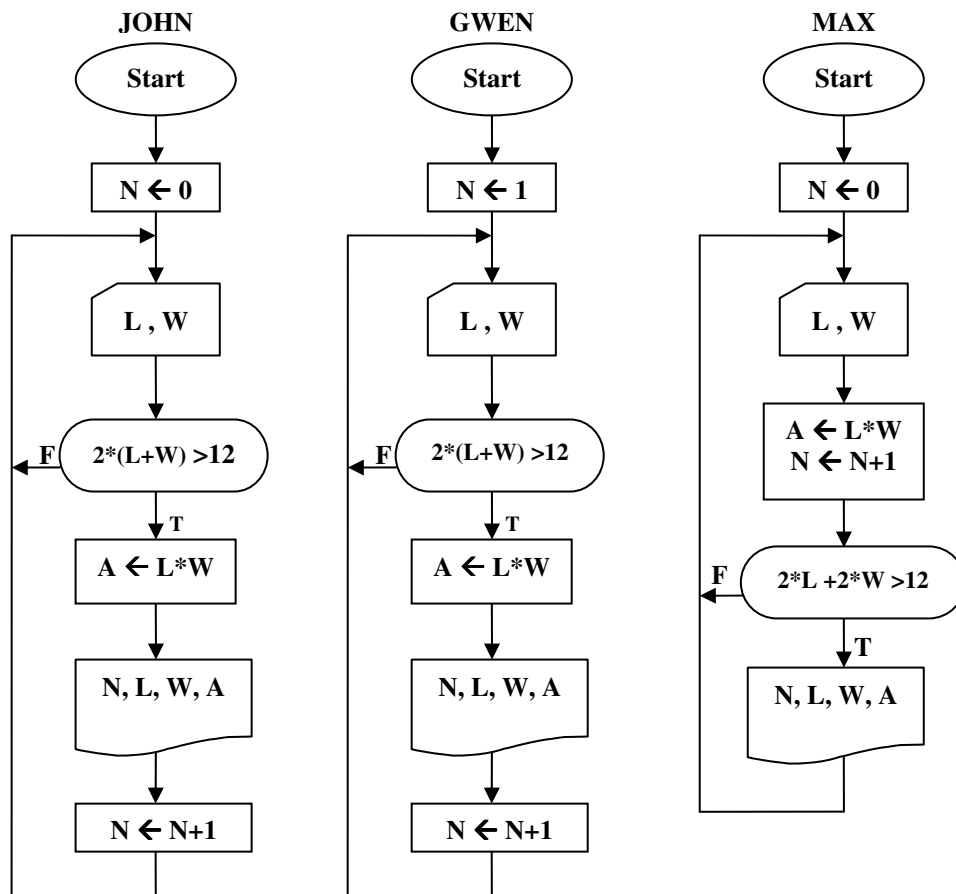
شکل ۲-۱۷ به عنوان حل این مسئله به معلم تحویل داده شد. وظیفه شما به شرح زیر است:

الف- تعیین کنید کدام راه حل‌ها صحیح و کدام‌ها غلط هستند. (فعلاً به راه‌حلی، راه‌حل صحیح می‌گوییم که

خروجی مورد نظر را تولید کند. هرچند که ممکن است کاراترین راه‌حل نباشد.)

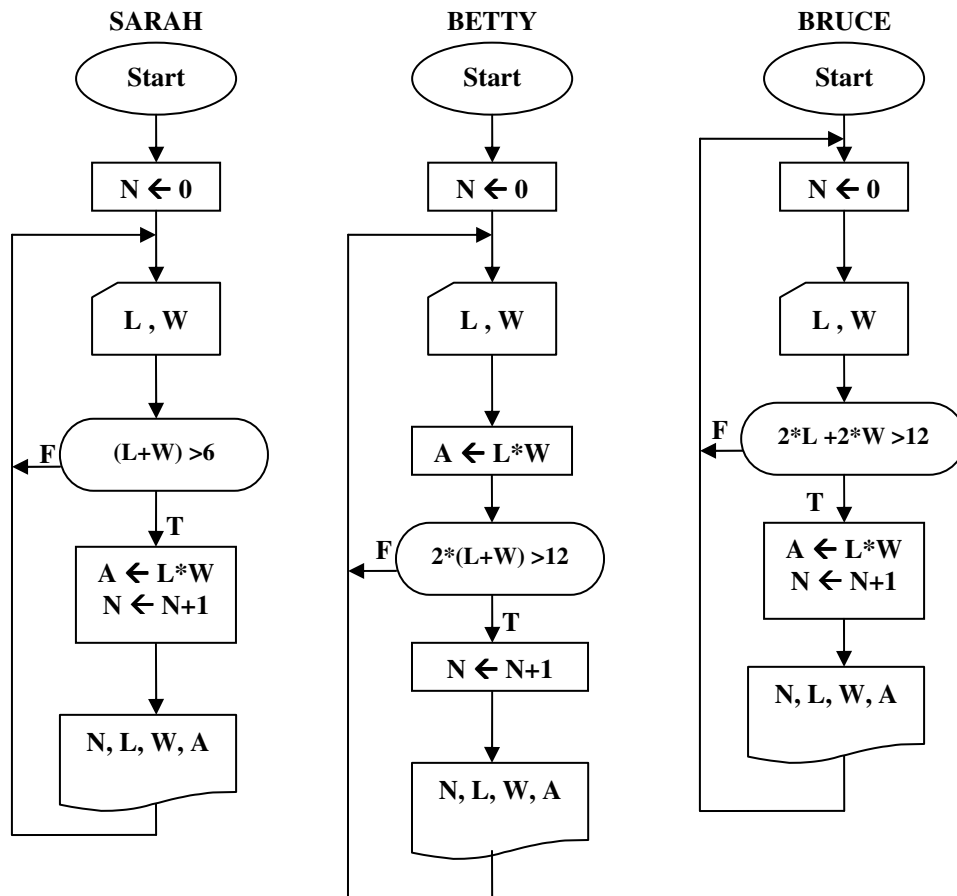
ب- در مورد راه‌حل‌های غلط، اشتباهات جواب‌هایی که به دست می‌آیند در چیست؟

ج- راه‌حل‌های صحیح را از نظر کارایی بررسی کنید و با استفاده از بهترین خصوصیات هر کدام، یک کارنما بسازید. (از دو برنامه، آنکه به تعداد کمتری از محاسبات نیاز دارد کارتر است).



شکل ۲-۱۷ (الف)- کارنماهای تمرین ۲

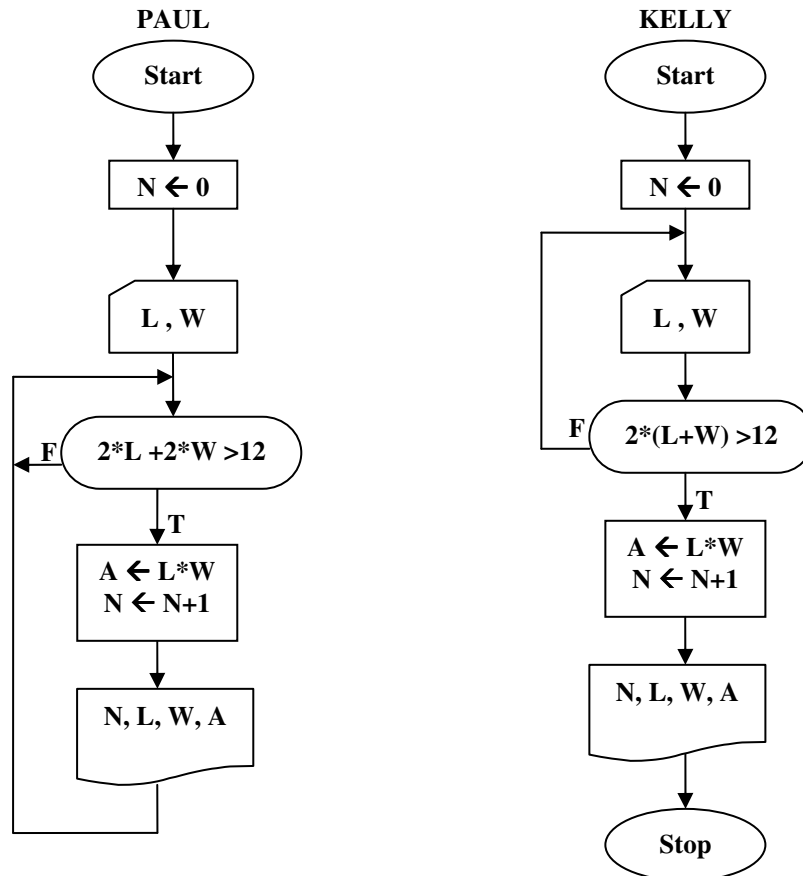
۳. مطابق مقررات پستی اگر طول جعبه‌ای به علاوه اندازه دور آن بیش از ۱۸۰ سانتیمتر باشد، آن را نمی‌توان به وسیله پست ارسال کرد. (اندازه دور جعبه عبارت است از طول کوتاه‌ترین نخ‌ای که بتوان به دور جعبه بست). سازنده‌ای تعداد زیادی جعبه دارد که می‌خواهد توسط پست ارسال کند. برای هر جعبه ابعاد سه گانه آن (به ترتیب از بزرگترین به کوچکترین) و یک شماره شناسایی ذخیره شده است. کارنمایی بنویسید که با خواندن این اطلاعات، برای هر جعبه که با مقررات فوق مغایرت دارد شماره مشخصه جعبه و همچنین مجموع طول و اندازه دور جعبه را چاپ کند. (تعداد جعبه‌ها را نیز از کاربر دریافت کنید).



شکل ۲-۱۷ (ب) - کارنماهای تمرین ۲

۴. کارنمایی رسم کنید که تمام جملات کوچکتر از ۱۰ میلیون دنباله فیبوناچی را چاپ کند. همچنین ترتیبی بدهید که سطرهای خروجی، مانند شکل زیر به ترتیب شماره گذاری و چاپ شوند.

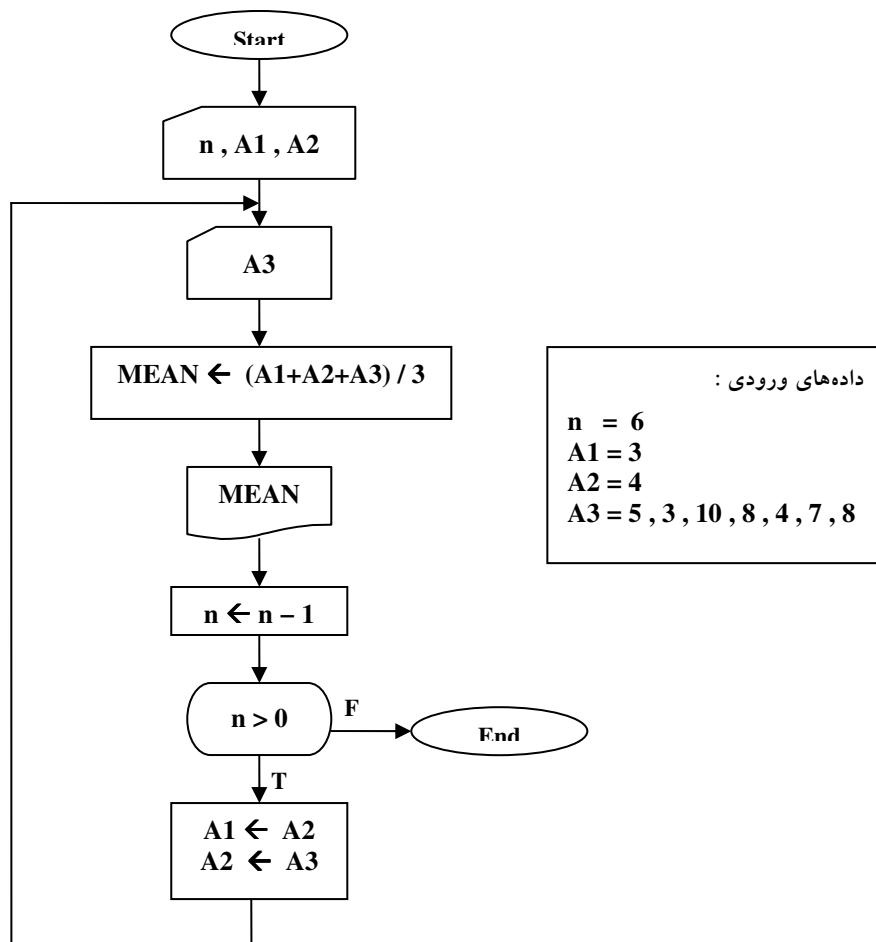
1	0
2	1
3	1
4	2
5	3
6	5
7	8



شکل ۲-۱۷ (پ) - کارنامه‌های تمرین ۲

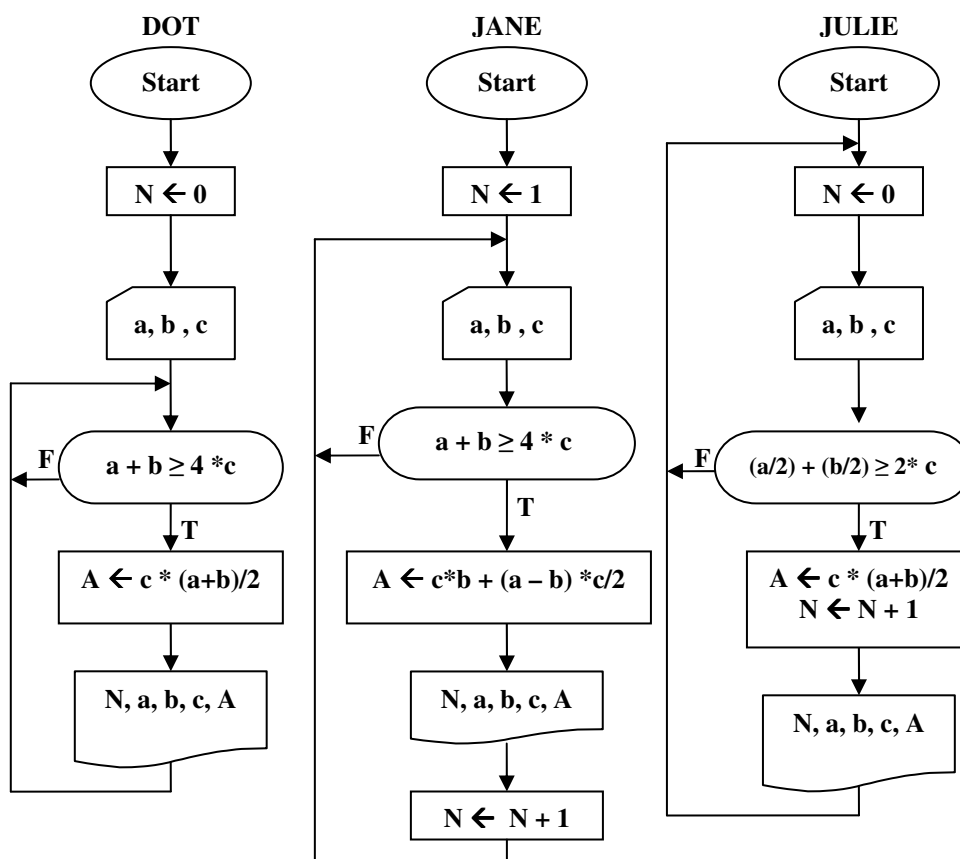
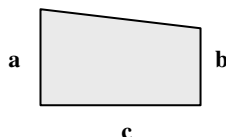
۵. یک کارنما برای الگوریتمی بسازید که یک مجموعه هزارتایی از مقادیر را یکی یکی قبول و مجموع مقادیر مثبت را حساب کند، صفرها را نادیده بگیرد و تعداد مقادیر منفی را بشمارد. وقتی محاسبات در مورد هر صد عدد انجام گرفت، مجموع مقادیر مثبت و تعداد مقادیر منفی باید چاپ شوند.
۶. کارنمایی رسم کنید که n عدد را به عنوان ورودی دریافت کند، سپس بزرگترین و کوچکترین عدد موجود در بین این اعداد را چاپ کند؟
۷. سه عدد a و b و c مفروض هستند. الگوریتمی بنویسید که معین کند آیا این سه عدد می‌توانند طول اضلاع یک مثلث باشند یا نه؟
۸. سه عدد a و b و c مفروض هستند. الگوریتمی بنویسید که معین کند آیا این سه عدد می‌توانند طول اضلاع یک مثلث قائم الزاویه باشند یا نه؟

۹. کارنمای شکل ۲-۱۸ را با استفاده از مقادیر داده شده پیگیری کنید، سپس درستی یا نادرستی هر یک از گزاره‌های (الف) تا (ت) را تعیین کنید.
- الف- دومین مقدار چاپ شده (در جعبه ۶) هفت است.
- ب- جعبه ۹ چهار بار اجرا می‌شود.
- پ- آخرین مقدار که برای A3 داده شده است، به داخل آورده نمی‌شود.
- ت - در این الگوریتم متغیر n به‌عنوان شمارنده‌ای برای کنترل حلقه مورد استفاده قرار می‌گیرد.



شکل ۲-۱۸: کارنماهای تمرین ۹

۱۰. در شکل ۱۹-۲ پنج کارنما نشان داده شده که به‌عنوان حل مسئله زیر توسط دانشجویان ساخته شده‌اند. به‌عنوان ورودی، مقادیر قاعده c و اضلاع a و b از چند دوزنقه داده شده است. (فرض کنید $a > b$) فهرستی تهیه کنید از سطرهایی که دارای شماره ردیف بوده و هر سطر متشکل از a و b و c و مساحت A ی آن دسته از دوزنقه‌هایی باشد که ارتفاع متوسط آنها یعنی $(a+b)/2$ حداقل دو برابر قاعده c است.

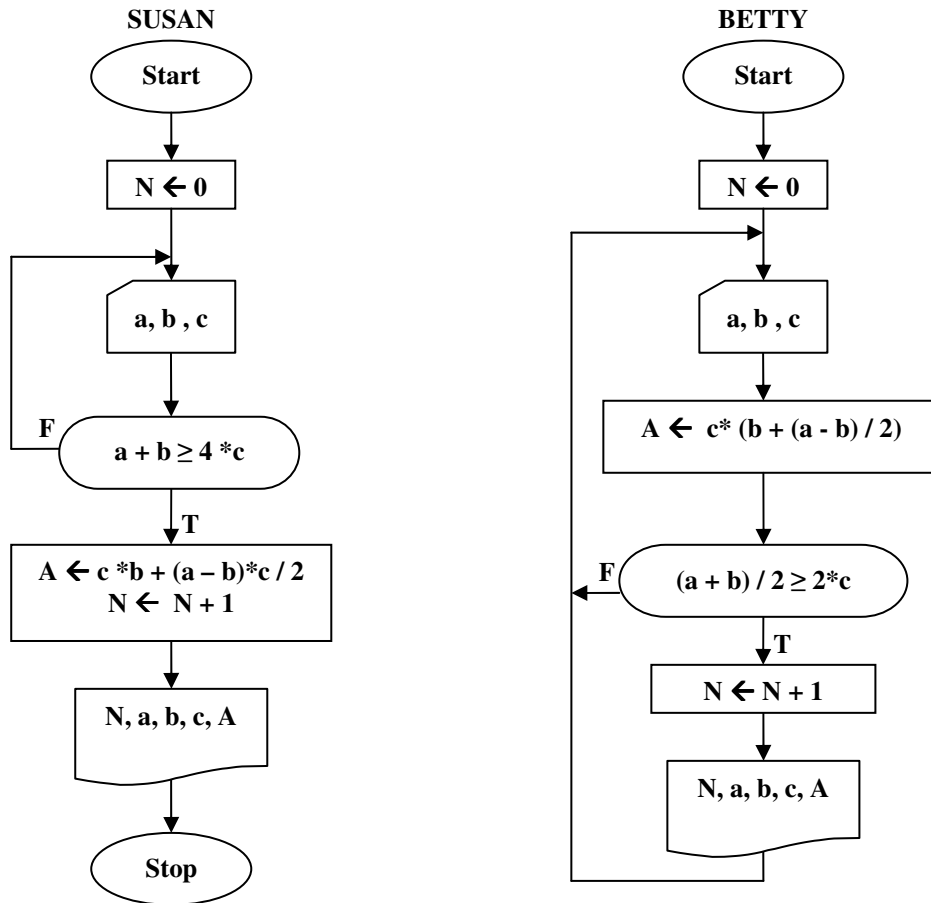


شکل ۱۹-۲ الف) - کارنماهای تمرین ۱۰

وظیفه شما به شرح زیر است:

الف- تعیین کنید راه حل کدام دانشجو صحیح و کدام غلط است.

ب- با استفاده از کاراترین خصوصیات هر یک از این ۵ راه‌حل یک کارنمای صحیح کارآمد بسازید.



شکل ۲-۱۹ (ب) - کارنماهای تمرین ۱۰

۱۱. کارنمایی رسم کنید که مجموع n عدد صحیح را محاسبه کند. بر روی روش‌های ممکن برای کنترل تعداد دفعات تکرار حلقه در این مسئله بحث کنید.

۱۲. الگوریتمی بنویسید که مقسوم‌علیه‌های عدد مفروض N را چاپ کند.

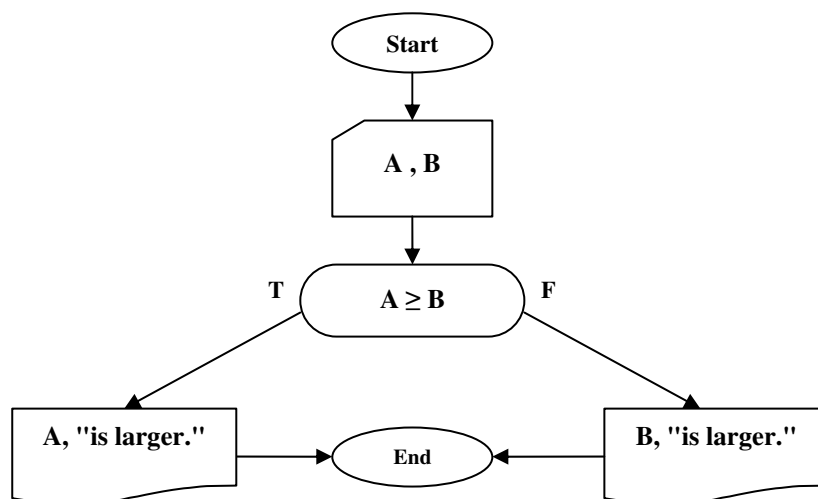
راهنمایی: در حل این تمرین از تقسیم صحیح باید استفاده کنید. یک عدد بر دیگری بخش‌پذیر است، اگر باقی مانده تقسیم عدد بزرگتر بر عدد کوچکتر صفر باشد. به مثال ۲-۵ توجه کنید.

۲-۶: مقدمه‌ای بر الگوریتم‌های جستجو

ساختن یک الگوریتم و کارنمای مربوط به آن، اساساً از مراحل حل مسئله است. برای آنکه شما را در ساخت الگوریتم‌های کارآمد، کارآموده‌تر سازیم باید با روش حل مسائل بهتر آشنا شوید. شما باید ببینید که برای حل یک مسئله چگونه یک نقطه شروع انتخاب می‌کنیم. شما باید برخی شروع‌های نادرست و غفلت‌ها و برخی الگوریتم‌های ناخوشایند را که در اولین کوشش‌هایمان به‌دست می‌آوریم را ببینید، تا بتوانید از این تجارب در حل مسائل پیچیده‌تر و مهم‌تر بهره بگیرید و آنها را به‌کار ببندید. از همه مهم‌تر باید بیاموزید که در ساختن الگوریتم، ابتدا می‌کوشیم تا یک نوع راه‌حل را برای مسئله به‌دست آوریم. سپس راه‌حل خود را با دیدی انتقادی مورد بررسی قرار می‌دهیم و سعی می‌کنیم آن را بهتر سازیم و در صورت لزوم به موارد کلی‌تر تعمیم دهیم. لذا در اکثر فصول کتاب سعی شده تا با مطرح ساختن برخی مسائل روشنگر شما را با شیوه‌های حل مسئله آشنا‌تر سازیم. آشنایی با الگوریتم‌های جستجو اولین قدم در این راستا است.

الگوریتم پیدا کردن بزرگترین عدد از میان چند عدد

بیایید مسئله پیدا کردن بزرگترین عدد در میان چند عدد را مدنظر قرار دهیم. این مسئله گرچه به خودی خود ساده است، لیکن بخشی از تعداد زیادی الگوریتم‌های دیگر را تشکیل می‌دهد. در واقع، ساده‌ترین الگوریتم از یک نوع کامل الگوریتم‌ها است که به نام "الگوریتم‌های جستجو" معروف هستند. برای حل این مسئله ساده‌ترین حالت را، که پیدا کردن بزرگترین عدد بین دو عدد است، در نظر می‌گیریم و سپس الگوریتم خود را برای تعداد N عدد تعمیم می‌دهیم. برای این منظور در شکل ۲-۲۰ کارنمایی ترسیم شده است که ساده‌ترین حالت را نشان می‌دهد.



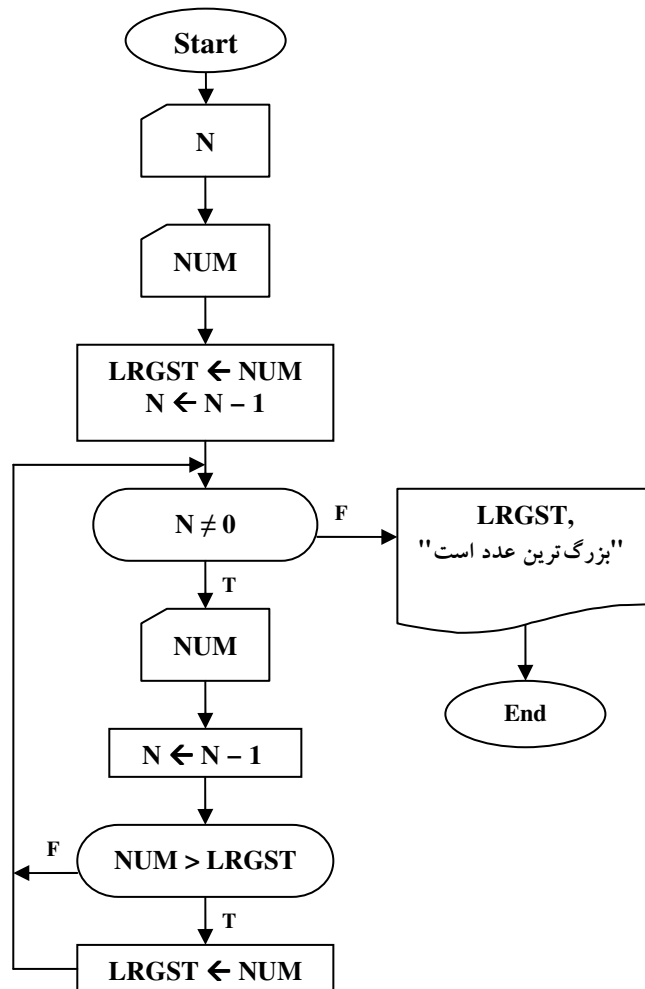
شکل ۲-۲۰: کارنمای یافتن بزرگترین عدد از بین دو عدد

حال بیایید به حالتی بپردازیم که در آن می‌خواهیم بزرگترین عدد را در میان N عدد پیدا کنیم. این مسئله یکی از مسائلی است که اگر بخواهیم با توجه به کارنمای ساده‌ترین حالت آن را حل کنیم دچار گمراهی خواهیم شد. فرض کنید بخواهید هر تغییری را که می‌خوانید به یک متغیر مانند A, B, C, D, E و ... نسبت دهید و سپس با توجه به روش ارائه شده در ساده‌ترین حالت کارنمایی بسازید که بزرگ‌ترین عدد را پیدا کند مسلماً به این نتیجه خواهید رسید که تعداد جعبه‌های موردنیاز کارنمای آن، بیشتر از تعداد مقادیر داده‌ها خواهد بود، و احتمالاً نتیجه خواهید گرفت که انجام این عملیات با دست راحت‌تر است. این مثال موردی است از اینکه در برخی مسائل عقل سلیم راهنمای بهتری برای حل مسئله است تا بررسی کارنمای ساده‌ترین حالت.

بیایید مسئله را با یک مسئله مشابه که در زندگی روزمره برای انسان پیش می‌آید شبیه سازی کنیم، تا شاید با این کار به روش مناسبی برای حل مسئله دست یابیم. در حقیقت داریم به عقل و تجربه خود رجوع می‌کنیم. فرض کنید دسته‌ای کارت که بروی هر کدام عددی یادداشت شده است و تعداد آنها بیشتر از دو عدد است، را به شما بدهند و بخواهند بزرگترین کارت را از میان این کارت‌ها پیدا کنید چگونه این عمل را انجام خواهید داد؟ احتمالاً روش زیر را برای حل این مسئله به کار خواهید برد:

۱. ابتدا دو کارت بخواهید داشت، کارت بزرگتر را انتخاب می‌کنید و کارت کوچکتر را کنار خواهید گذاشت.
۲. سپس کارت دیگری برمی‌دارید و آن را با کاردی که از مرحله قبلی به‌دست آمده مقایسه می‌کنید، دوباره با انتخاب کارت بزرگتر، کارت کوچکتر را کنار خواهید گذاشت.
۳. مرحله دوم را آنقدر تکرار خواهید کرد تا دیگر هیچ کاردی باقی نماند.
۴. هر کاردی که در نهایت باقی‌مانده باشد، بزرگ‌ترین کارت خواهد بود.

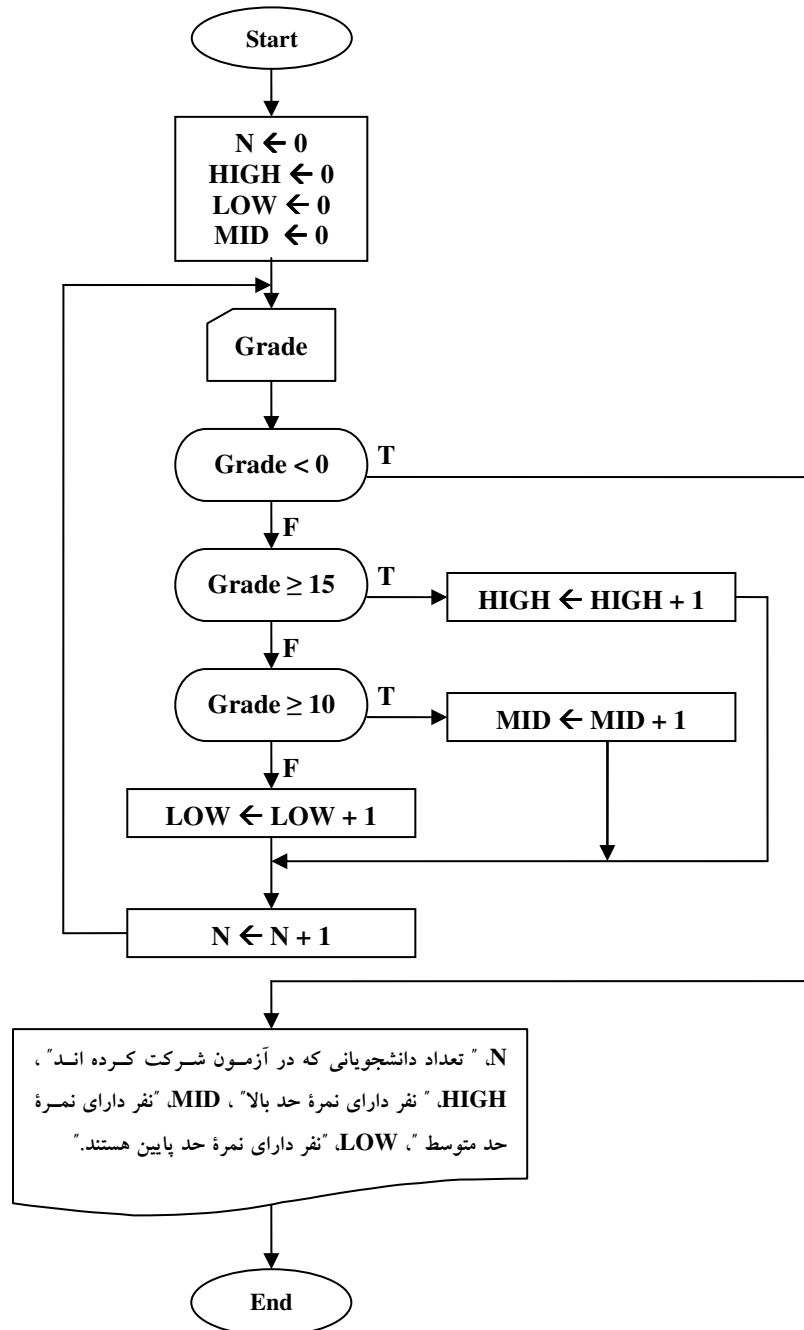
در شکل ۲-۲۱ با اقتباس از همین روش کارنمایی برای یافتن بزرگترین عدد از بین N عدد ارائه شده است. با توجه به روش فوق باید به این نتیجه رسیده باشید، که اصلاً نیازی به نگهداری داده‌هایی که کوچکتر از بزرگترین داده موجود هستند، نیست. چنانکه در هر مرحله کارت کوچکتر را کنار می‌گذاشتیم. کارنمای ما تنها از جهت مرحله اول با روش فوق متفاوت است، چرا که ما اعداد را یکی یکی می‌خوانیم و در مرحله نخست دو عدد کارت را به‌طور همزمان برنمی‌داریم. به‌طوری که می‌بینید ابتدا با خواندن اولین عدد فرض می‌کنیم که این عدد، بزرگترین عدد موجود است، لذا آن عدد را در متغیر کمکی **LRGST** قرار می‌دهیم. سپس با خواندن عدد بعدی، چک می‌کنیم که اگر عدد خوانده شده از بزرگترین مقداری که تا اینجا به‌دست آورده‌ایم، بزرگتر است عدد جدید را در متغیر **LRGST** قرار می‌دهیم در غیر این صورت عدد بعدی را می‌خوانیم. این روند تا جایی ادامه پیدا می‌کند که شمارنده حلقه تکرار به صفر نرسیده باشد. با توجه به استراتژی به‌کار رفته در حل این مسئله باید به نقش متغیرهای کمکی مثل **LRGST** پی برده باشید. در بسیاری از مسائل متغیرهای کمکی می‌توانند راه حل مسئله را بهینه سازند. متأسفانه هیچ معیار خاصی برای آنکه بگوییم در چه زمانی باید از متغیر کمکی استفاده کنید وجود ندارد، و چنانکه پیشتر گفته شد این مهارت رفته‌رفته با تجربه به‌دست می‌آید.

شکل ۲-۲۱: کارنامه‌های یافتن بزرگترین عدد از بین N عدد

الگوریتم شمارش نمرات

آخرین کارنمایی که در این مبحث ارائه می‌کنیم کارنمایی است که تعداد داده‌های متفاوتی را که خوانده شده است، را می‌شمارد. بر این اساس فرض کنید که تعداد نامشخصی نمره بین صفر تا ۲۰ را می‌خوانیم. الگوریتم باید تعداد نمرات بین ۱۵ تا ۲۰ را به‌عنوان نمرات دسته A، تعداد نمرات بین ۱۰ تا ۱۵ را به‌عنوان نمرات دسته B، تعداد نمرات پایین‌تر از ۱۰ را به‌عنوان نمرات دسته C و تعداد کل نمرات خوانده شده را لیست کند. متغیرهای HIGH، MID، LOW، متغیرهای شمارنده هستند که به ترتیب تعداد نمرات دسته‌های A و B و C را در خود ذخیره می‌کنند. برای هر مقدار ورودی متغیر Grade، به یکی از سه شمارنده فوق برحسب شرایط یک واحد اضافه می‌شود. همچنین متغیر دیگری مثل N، تعداد نمرات خوانده شده را نگه می‌دارد. در اینجا نیز برای متوقف شدن حلقه تکرار آخرین نمره باید یک نمره

خارج از دامنه تعریف شده نمرات باشد. چرا که از روش نگهدارنده حلقه تکرار استفاده شده است. مثلاً آخرین ورودی می‌تواند یک نمره منفی باشد.

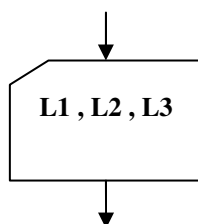


شکل ۲-۲۲: کارنمای شمارش حدود نمرات

۷-۲: تمرین

۱. کارنمای مقررات پستی خود، متعلق به مسئله ۳، از سری تمرینات ۲-۵ را طوری تغییر دهید که بتواند در موردی که ابعاد جعبه بدون ترتیب معینی وارد می‌شود (یعنی بزرگترین بُعد لزوماً اولی نباشد) کار کند.

راهنمایی: در هر دو مسئله، می‌توانید چنین فرض کنید که کارنما با قدم ورودی به شکل زیر شروع می‌شود به‌طوری که $L1$ و $L2$ و $L3$ طول سه یال جعبه را نشان می‌دهند. در مسئله اصلی از قبل معلوم بود که $L1$ باید بزرگترین باشد به‌طوری که اندازه دور جعبه برابر $2*(L1+L2)$ می‌شد. در مسئله حاضر اندازه دور جعبه برابر است با $2*(L1+L2+L3-X)$ که در آن X بزرگترین مقدار از بین $L1$ و $L2$ و $L3$ است.



۲. کارنمایی رسم کنید که مقادیر b و c را به‌عنوان ضرایب معادله $bX + c = 0$ دریافت کند و سپس در خصوص ریشه‌های این معادله برای حالت‌های زیر پیام خواسته شده را چاپ کند.

الف- اگر b برابر صفر و c مخالف صفر باشد، پیام "معادله $bX + c = 0$ ریشه ندارد." را چاپ کند.

ب- اگر b و c هر دو برابر صفر باشند، پیام "هر عدد حقیقی در $bX + c = 0$ صدق می‌کند." را چاپ کند.

ج- اگر b مخالف صفر باشد، ریشه معادله $bX + c = 0$ را حساب کند و عبارت "ریشه معادله $bX + c = 0$ برابر است با " و به دنبالش مقدار ریشه را چاپ کند.

۳. کارنمای الگوریتمی را رسم کنید که قد گروهی از مردم را دریافت کند و نزدیکترین قد به ۱۸۵ سانتی‌متر را چاپ کنید. فرض کنید که تعداد داده‌ها را ندارید اما آخرین داده ورودی صفر باشد.

۸-۲: تقدم عملگرها در عبارات محاسباتی (جبری)

قابل پردازش کردن عبارت‌های محاسباتی (جبری) برای کامپیوتر

چنانکه در بخش‌های قبلی دیدید، محاسبات اساسی در کامپیوتر مشتمل بر محاسبه مقدار عبارت‌های محاسباتی و رشته‌ای است که در جعبه‌های انتساب، جعبه‌های تصمیم و جعبه‌های خروجی یافت می‌شود. به عنوان مثالی برای عبارات محاسباتی، عبارت جبری زیر که مربوط به معادلات درجه دوم می‌باشد را در نظر بگیرید:

$$\frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

این نماد جبری ممکن است در زبان غیررسمی کارنما مورد قبول باشد ولی در هیچ زبان برنامه‌نویسی نمی‌توان چنین عبارتی را نوشت، در اینگونه زبان‌ها، عبارت‌ها باید به شکلی نوشته شوند که توسط ماشین قابل خواندن باشند. برای اینکه عبارت محاسباتی فوق را برای ماشین قابل خواندن کنیم، چندین تغییر به شرح زیر باید در عبارت مذکور اعمال کنیم:

۱. قبلاً مشاهده کردید که با توجه به قواعد مربوط به نامگذاری متغیرها، استفاده از کنار هم‌نویسی متغیرها برای نشان دادن عمل ضرب را غیرممکن می‌کند. چرا که برای کامپیوتر ac یک متغیر جدید محسوب می‌شود و مفهوم $a*c$ را ندارد. بنابراین برای نشان دادن صورت صحیح عبارت $4ac$ باید بنویسیم $4*a*c$.
۲. برای آنکه عبارت محاسباتی توسط کامپیوتر قابل خواندن باشد، باید به صورت یک رشته خطی از نمادها نوشته شود. نمادهایی که از حالت خطی خارج شوند مثل توان در b^2 قابل قبول نیستند. بنابراین در زبان کارنما و یا برخی زبان‌های برنامه‌نویسی مثل QBasic عملگرهایی مانند $^$ برای نشان دادن عمل به توان رساندن مورد استفاده قرار می‌گیرد. و عباراتی مثل b^2 را به صورت b^2 نشان می‌دهند.
۳. استفاده از نمادهای خاص برای نشان دادن برخی توابع ریاضی عمل خواندن توسط ماشین را پیچیده می‌کند، مثل علامت جذر در عبارت فوق. بنابراین می‌توان توابع ریاضی را با حروفی که معرف آن توابع باشند نشان داد. چنان که بهتر است به جای علامت جذر از حروف SQRT استفاده کرد. و در نتیجه عبارت ریاضی:

$$\sqrt{b^2 - 4ac}$$

به صورت :

$$\text{SQRT} (b^2 - 4*a*c)$$

نشان داده می‌شود. به همین نحو، به جای علامت قدرمطلق در عبارت جبری مثل $|R|$ در اکثر زبان‌های برنامه نویسی نوشته می‌شود $\text{ABS}(R)$. اما کامپیوتر از کجا بفهمد که SQRT و ABS نام یک متغیر نیستند بلکه نشان‌دهنده توابع اند؟! جواب این سؤال نیز بسیار روشن است. از آنجایی که علامت پرانتز باز جزء کاراکترهای مجاز جهت نامگذاری متغیرها نیست، می‌فهمد که این حروف نام تابع هستند. از طرفی ورودی‌های توابع را بین دو پرانتز باز و بسته قرار می‌دهیم تا کامپیوتر بداند بر روی چه مقدار یا مقداری باید عمل کند. مثلاً به جای عبارت جبری پر کاربرد $\sin X$ که در ریاضیات به همین صورت نوشته می‌شود، در الگوریتم‌های خود بنویسید $\text{SIN}(X)$. البته برخی

از توابع ممکن است به بیش از یک ورودی نیاز داشته باشند که در این صورت ورودی‌ها را با علامت ویرگول از یکدیگر جدا می‌کنیم. مثلاً تابع $\text{MIN}(a, b)$ می‌تواند تابعی باشد که مقدار کوچکتر را در بین دو ورودی a و b به دست می‌دهد. و یا تابعی مثل $\text{POW}(a, b)$ می‌تواند تابعی باشد که a را به توان b می‌رساند، و در نتیجه می‌توان عبارتی مثل b^2 را به صورت $\text{POW}(b, 2)$ نیز نشان داد. البته باید توجه داشته باشید که در چنین توابعی ترتیب آمدن ورودی‌ها حائز اهمیت است. چرا که مقداری که توسط تابع $\text{POW}(b, 2)$ به دست می‌آید با مقداری که توسط تابع $\text{POW}(2, b)$ به دست می‌آید به کلی متفاوت است. اما در مورد تابعی مثل MIN این چنین نیست و ترتیب آمدن ورودی‌ها اهمیت ندارد.

۴. همچنین در عبارت بالا تقسیم بر $2a$ را با نمادی که از حالت خطی خارج شده است نشان دادیم. هنگامی که قرار باشد ماشین این عبارت را بخواند، بایستی از نماد کسری صرف‌نظر کنیم و برای نشان دادن تقسیم از عملگری خطی مثل $(/)$ استفاده کنیم.

هنگامی که تمامی این تغییرات را اعمال کنیم، عبارت مربوط به ریشه‌های معادله درجه دوم به صورت زیر در می‌آید:

$$(-b + \text{SQRT}(b^2 - 4*a*c)) / (2*a)$$

۵. توجه داشته باشید هنگامی که خط کسری را به علامت $/$ تبدیل کردیم، مجبور شدیم دو جفت پرانتز اضافی نیز به کار ببریم، یک جفت برای دربرگرفتن صورت و جفت دیگر برای دربرگرفتن مخرج. اگر هر یک از این پرانتزها حذف شوند، عبارت به طور صحیح محاسبه نخواهد شد. مثلاً اگر پرانتزهای اطراف مخرج را حذف کنیم، آن وقت عبارت حاصله به معنای زیر خواهد بود:

$$\frac{-b + \sqrt{b^2 - 4ac}}{2 * a}$$

کار در کلاس ۲-۴:

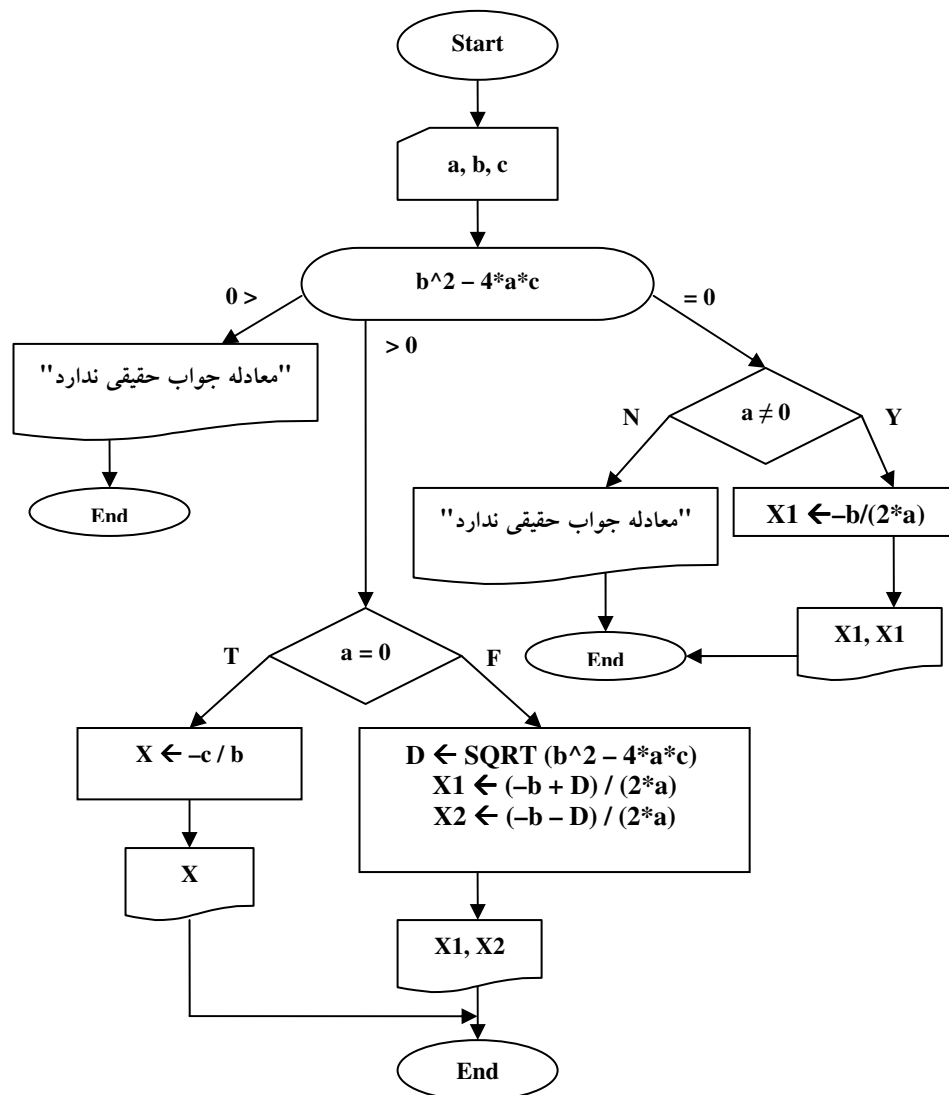
مطابق مثال فوق عبارت جبری حاصله از هر یک از موارد زیر را تعیین کنید؟
راهنمایی: باید به تقدم عملگرها توجه فاس داشته باشید.

۱. $-b + \text{SQRT}(b^2 - 4*a*c) / (2*a)$

۲. $-b + \text{SQRT}(b^2 - 4*a*c) / 2*a$

۶. نصیحت مفیدی که می‌توانیم در مورد قرار دادن یا قرار ندادن پرانتز بکنیم این است که: « اگر شک دارید پرانتز لازم است یا نه، بنا را بر لزوم آن بگذارید ». لزوم و یا عدم لزوم پرانتز را می‌توان با مراجعه به قواعد تقدّم برای انجام محاسبات در غیاب پرانتز، تعیین نمود که در مبحث بعدی به تفصیل در این خصوص صحبت خواهیم کرد.

مثال ۲-۶: کارنمایی که با دریافت ضرایب معادله درجه دوم $aX^2 + bX + c = 0$ ریشه‌های این معادله را چاپ می‌کند. به کاربرد جعبه تصمیم چندگانه و طریقه نوشتن عبارات جبری توجه کنید.



شکل ۲-۲۳: کارنمایی مثال ۲-۶

تقدم عملگرهای حسابی

قواعدی وجود دارد که تقدم عملگرهای حسابی را در عبارات محاسباتی در غیاب پرانتز مشخص می‌کند. این قواعد که به قواعد تقدم عملگرهای حسابی موسوم هستند در مواردی که نظیر آنها در قراردادهای معمولی ریاضی وجود داشته باشد با آنها مطابقت دارند. مثلاً در محاسبه عبارت محاسباتی

$$A + B * C$$

به ازای مقادیر $A=5$, $B=3$, $C=2$ ، از آنجایی که ضرب بر جمع تقدم دارد ابتدا B را در C ضرب کرده و سپس A را با حاصل ضرب به‌دست آمده جمع می‌کنیم. بنابراین داریم:

$$A + B * C = A + (3 * 2) = A + 6 = 5 + 6 = 11$$

اگر بخواهیم که ابتدا A با B جمع شود، باید از پرانتز استفاده کنیم به این ترتیب عبارت $(A + B) * C$ به‌صورت زیر محاسبه می‌شود.

$$(A + B) * C = (5 + 3) * C = 8 * C = 8 * 2 = 16$$

فهرست کامل سطوح تقدم عملگرهای حسابی در جدول ۱-۲ نشان داده شده است.

نشان عملگر	نام عملگر	سطح تقدم
$^$	به توان رساندن	بالا ترین تقدم
$*$	ضرب	بعدي
$/$	تقسیم	
$-$	منفی کردن (یک تایی)	
$+$	جمع	پایین ترین تقدم
$-$	تفریق (دوتایی)	

جدول ۱-۲: جدول سطوح تقدم عملگرهای حسابی

تذکره: همانگونه که در جدول فوق مشاهده می‌کنید، علامت منها در دو مکان به کار رفته است. در حقیقت منها دارای سه کاربرد مختلف به شرح زیر است: گاهی اوقات به‌عنوان جزئی از یک عدد منفی به کار برده می‌شود مثل عدد -5 ، برخی از اوقات نیز به‌عنوان یک عملگر یک‌تایی جهت منفی کردن یک متغیر به کار برده می‌شود مثل $-X$ و در نهایت می‌تواند به‌عنوان یک عملگر دوتایی تفریق به کار برده شود مثل $X-5$. اما ماشین از کجا می‌تواند تفاوت این سه عملگر را بفهمد؟ در واقع نقش منها را می‌توان با وضعیت قرار گرفتنش در عبارت تعیین کرد. بدین صورت که:

« منها همواره دوتایی است مگر وقتی که در شروع عبارت و یا بلافاصله پس از پرانتز باز بیاید. اگر منها دوتایی نبود و پس از آن یک رقم آمده باشد قسمتی از یک عدد و در غیر این صورت عملگر یک‌تایی منفی کردن است.» در عبارت زیر هر سه این حالت‌ها نشان داده شده است. آیا می‌توانید نقش هریک از علامت‌های منها را در عبارت زیر نام ببرید؟

$$X - (- (-5))$$

ارزیابی عبارت‌های محاسباتی بدون پرانتز

در این قسمت می‌خواهیم با ارائه مثالی ببینیم به واسطهٔ جدول ۱-۲ چگونه می‌توان عبارت‌های محاسباتی را مورد ارزیابی قرار داد.

کامپیوتر برای ارزیابی یک عبارت محاسباتی ابتدا عبارت را برای یافتن عملگری با بالاترین سطح تقدّم مورد پوشش (پردازش) قرار می‌دهد. ولی این پوشش چگونه صورت می‌گیرد؟ طبق قرار داد هر عبارت ریاضی از سمت چپ پوشش می‌شود و مثلاً در عبارتی به صورت $A + B + C$ با آنکه، دو عملگر جمع به کار رفته در آن، دارای تقدّم یکسانی هستند، ولی عملگر سمت چپ دارای تقدّم بالاتری است. چرا که در هنگام پوشش عبارت، عملگر سمت چپ زودتر خوانده می‌شود. به عبارت دیگر دو عملگر با سطح تقدّم یکسان دارای تقدّم مکانی نسبت به هم هستند. اما با آنکه عبارت جبری $A + B * C$ از سمت چپ پوشش می‌شود و عملگر جمع زودتر خوانده می‌شود این عملگر ضرب است که مطابق جدول ۱-۲ دارای اولویت بالاتری است و زودتر عمل می‌کند. اصولاً پرانتزها قادرند اولویت انجام محاسبات را تغییر دهند. بدین صورت که گفته می‌شود داخلی‌ترین پرانتز دارای اولویت بالاتری است. بنابراین در عبارتی مثل $(A * (B + (C - D)))$ ابتدا $(C - D)$ که داخلی‌ترین پرانتز است مورد پردازش قرار می‌گیرد سپس مقدار B با حاصل تفریق، جمع شده و در نهایت حاصل $(B + (C - D))$ با A در هم ضرب می‌شوند. اما در عبارتی که فاقد پرانتز هستند عملیات پوشش را به صورت زیر باید انجام دهید:

۱. اول عبارت را برای پیدا کردن عملگرهایی با بالاترین سطح تقدّم از چپ به راست ببینید.
۲. وقتی عملگری از این سطح پیدا کردید، عمل قید شده را انجام دهید و پس از جایگزین کردن حاصل به دست آمده، عمل پوشش را از همان محلی که رها شده بود برای عملگرهای همان سطح تقدّم ادامه دهید.
۳. وقتی که تمام عبارت محاسباتی را برای عملگرهایی با بالاترین سطح تقدّم مورد پوشش قرار دادید، پوشش را دوباره از چپ به راست برای عملگرهای سطح تقدّم بعدی تکرار کنید و به همین ترتیب ادامه دهید تا جواب نهایی عبارت به دست آید.

کار در کلاس ۲-۵:

مطابق آنچه که در بالا گفته شد مقدار عبارت $A * L - G * O^R / I + T / H * M$ را با توجه به جدول زیر محاسبه کنید. سپس عبارت فوق را به گونه ای پرانتز گذاری کنید که پیکوئگی مراحل محاسبه را نشان دهد.

راهنمایی: (اولین عملیاتی که صورت می‌گیرد باید در داخلی‌ترین هفت پرانتز قرار گیرد.)

A	L	G	O	R	I	T	H	M
5	6	8	2	3	4	6	2	7

در جدول ۲-۲ مراحل پردازش عبارت محاسباتی مربوط به کار در کلاس ۲-۵ به صورت قدم به قدم نشان داده شده است. علامت مثلث شکل کوچک ($\blacktriangle$) برای نشان دادن عملگری که باید در مرحله بعدی مورد محاسبه قرارگیرد، استفاده شده است. همان‌طور که می‌بینید مطابق سطوح تقدم آمده در جدول ۲-۱ ابتدا عملگرهای سطح اول، سپس عملگرهای سطح دوم و سپس عملگرهای سطح سوم مورد پردازش قرار گرفته‌اند.

عبارت حاصل بعد از عمل	عمل مورد پردازش	قدم	عملیات مربوط به سطح
$-A * L - G * O^R / I + T / H * M$	عبارت اولیه	۰	بالاترین سطح تقدم
$-A * L - G * 8 / I + T / H * M$	O^R	۱	
$-5 * L - G * 8 / I + T / H * M$	$-A$	۲	سطح تقدم میانی
$-30 - G * 8 / I + T / H * M$	$-5 * L$	۳	
$-30 - 64 / I + T / H * M$	$G * 8$	۴	
$-30 - 16 + T / H * M$	$64 / I$	۵	
$-30 - 16 + 3 * M$	T / H	۶	
$-30 - 16 + 21$	$3 * M$	۷	
$-46 + 21$	$-30 - 16$	۸	پایین‌ترین سطح تقدم
-25	$-46 + 21$	۹	

جدول ۲-۲: نمایش قدم به قدم محاسبه عبارت جبری مربوط به کار در کلاس ۲-۵

توجه کنید که در قدم ۲، $-A$ به -5 تبدیل شده و با وجودی که علامت منهای واقع در $-A$ یکتایی است، علامت منهای واقع در -5 قسمتی از عدد است و دیگر در محاسبات شرکت نمی‌کند. زیرا منهای مربوط به اعداد منفی عملگر نیست بلکه جزئی از خود عدد است.

تذکره مهم: اگر قواعد مندرج در جدول ۲-۱ همراه با پوش از چپ به راست رعایت شوند فقط یک مورد وجود دارد که، ترتیب محاسبه عبارت‌های محاسباتی، با قراردادهای معمولی ریاضی مطابقت ندارد. قواعدی که در این جدول ذکر شده‌اند عبارت A^B^C را به صورت $(A^B)^C$ محاسبه می‌کند درحالی که در ریاضیات، قرار داد است که عبارت A^B^C به صورت $A^{(B^C)}$ محاسبه شود. اگر این تفاوت را در نظر داشته باشید، همیشه می‌توانید مقصود خود را با استفاده از پرانتز اعمال کنید. علت این که با این مشکل برخورد کردیم از آن جهت است که تمامی عملگرها مثل جمع، تفریق، ضرب و تقسیم در عملگر سمت چپ، دارای اولویت بالاتری از نظر مکانی هستند ولی دو عملگر توان و منهای یکانی در عملگر سمت راست، دارای اولویت بیشتری هستند.

قواعد ارزیابی عبارت‌های محاسباتی دارای پرانتز

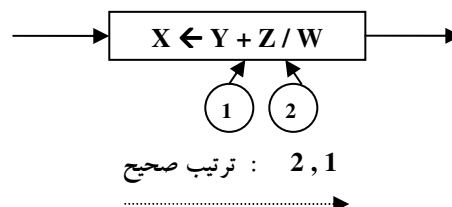
تمام روش‌های محاسبه عبارت‌ها، مستلزم داشتن درجات تقدم برای عملگرها نیستند. مثلاً می‌توانستیم صرفاً چنین توافق کنیم که عبارت‌ها را از چپ به راست (یا برعکس) پیوییم و به هر عملگری که رسیدیم نتیجه را درجا تعیین کنیم. مثلاً در زبان قدیمی APL پویش را از راست به چپ و بدون رعایت درجات تقدم انجام می‌دهد. مسلماً چنین روشی را راحت‌تر می‌توان در عمل پیاده‌سازی کرد. از طرف دیگر، این روش باعث می‌شود که ترتیب اجرای عملیات غالباً با قراردادهای معمولی ریاضی متفاوت باشد. مثلاً در APL عبارت $A * B + C$ به صورت $A * (B + C)$ محاسبه می‌شود. در برنامه‌های کامپیوتری مثل مفسرها و مترجم‌ها با اعمال تغییراتی بر روی یک عبارت محاسباتی، آن عبارت را به فرمی تبدیل می‌کنند که بدون پویش‌های مکرر، با یکبار پویش، عبارت موردنظر را مورد محاسبه قرار دهند، ضمن آنکه درجات تقدم هم رعایت می‌شود.

با توجه به آنچه که پیشتر در خصوص پردازش عبارات دارای پرانتز گفتیم می‌توانیم روشی را برای محاسبه عبارت‌های دارای پرانتز به صورت زیر بیان کنیم. با ذکر این نکته که عبارت موجود بین یک جفت پرانتز باز و بسته را زیر عبارت می‌گویند.

۱. عبارت را به منظور یافت اولین پرانتز بسته از سمت چپ پیوی.
۲. زیر عبارتی را که به پرانتز بسته فوق ختم می‌شود را طبق قواعد محاسبه عبارات بدون پرانتز (مطابق جدول ۲-۱) محاسبه کن.
۳. اگر قبل از این زیر عبارت نام یک تابع آمده است مقدار تابع ذکر شده را برای حاصل این عبارت حساب کن.
۴. در صورتی که به یک مقدار نهایی نرسیده‌ای به قدم یک بازگرد.

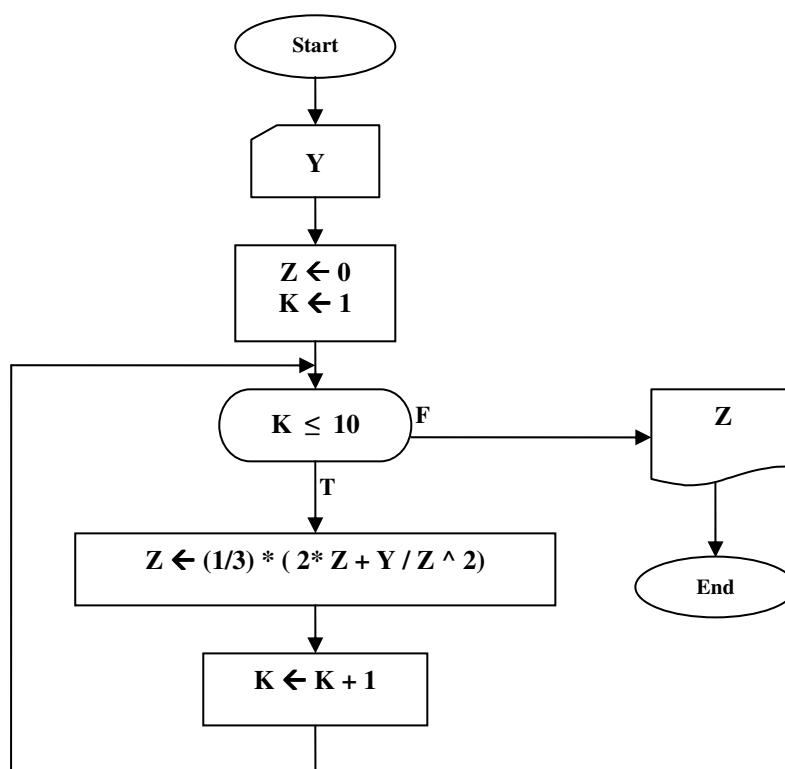
۲-۹: تمرین

۱. در هر یک از قسمت‌های (آ) تا (ت) این مسئله، جعبه انتسابی از یک کارنما به شما داده شده است. وظیفه شما این است، که با استفاده از قواعد تقدم (که در جدول ۲-۱ آمده است) ترتیب انجام عملیات حسابی را در هنگام اجرای جعبه تعیین کنید. برای سهولت در مراجعه، مطابق مثال زیر، شماره‌ای برای هر عملگر در زیر آن قرار دهید، سپس به ترتیب اجرای هر عملگر شماره عملگرها را از چپ به راست بنویسید.



- $I \leftarrow L \wedge O / V / E - B / E * T * T - Y$ → (الف)
- $I \leftarrow W + H \wedge E / A * T * P - E \wedge S \wedge K$ → (ب)
- $I \leftarrow S \wedge E * Y \wedge N / A * E \wedge S * S - B$ → (پ)
- $I \leftarrow B * C / D \wedge C / E \wedge C - F * G \wedge H$ → (ت)

۲. کارنمای زیر از تمام قواعد کارنمایی ما پیروی می‌کند. ولی اگر به یک برنامه کامپیوتری تبدیل و اجرا شود باعث می‌شود که کامپیوتر پیغام خطا دهد. توضیح دهید اشکال کار در کجاست.



شکل ۲-۲۴: کارنمای تمرین ۲

۲-۱۰: چند تابع ریاضی مهم

تابع جزء صحیح (براکت، کف)

با تابع براکت یا جزء صحیح یک عدد در دروس ریاضی دوره دبیرستان آشنا شده‌اید. تابع براکت یکی از مهمترین توابع در حل بسیاری از مسائل الگوریتم‌نویسی است. چنانکه کاربردهای متعددی از قبیل انجام تقسیم صحیح، یافتن باقی‌مانده تقسیم دو عدد بر یک دیگر و گردکردن اعداد دارد. در این کتاب از سه حرف FLR^1 به عنوان نمادی برای تابع جزء صحیح استفاده شده است. تنها به جهت یادآوری مقدار جزء صحیح چند عدد را به دست می‌آوریم:

1) $FLR(35.94) = 35$

2) $FLR(\pi) = 3$

3) $FLR(7\frac{2}{3}) = 7$

4) $FLR(-0.6) = -1$

5) $FLR(8) = 8$

6) $FLR(-2) = -2$

حال به بیان مثال‌هایی از کاربردهای تابع براکت می‌پردازیم.

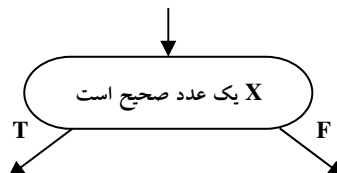
۱. انجام تقسیم صحیح

منظور از تقسیم صحیح آن است که با تقسیم مقسوم بر مقسوم‌علیه تنها، خارج قسمت تقسیم به دست آید و از باقی‌مانده آن صرف‌نظر شود. به عنوان مثال حاصل تقسیم صحیح ۱۴ بر ۵ برابر ۲ است که به صورت زیر نشان داده می‌شود:

$$FLR(14/5) = 2$$

۲. تشخیص عدد صحیح

غالباً در رسم الگوریتم‌ها آزمون‌هایی از قبیل



انجام می‌دهیم. برای تشخیص صحیح بودن یک عدد می‌توان از عبارت شرطی زیر در جعبه‌های تصمیم استفاده

$$X = FLR(X)$$

کرد:

در صورتی که متغیر X شامل عددی صحیح باشد مقدار آن با مقدار جزء صحیح خود برابر است.

۳. گرد کردن اعداد

در دوره دبیرستان با گردکردن آشنا شده‌اید. برای گردکردن اعداد اعشاری که قسمت اعشاری آنها کمتر از ۰.۵ بود تنها قسمت صحیح عدد را در نظر می‌گیریم. و اعدادی را که قسمت اعشاری آنها بیشتر از ۰.۵ بود را به عدد صحیح بعدی گرد می‌کردیم. به عبارت دیگر می‌توان از فرمول زیر برای گردکردن عدد تا رقم یکان استفاده نمود. در این فرمول

$$ROUND(X) = FLR(X + 0.5)$$

نام تابع گردکننده است.

۱. مخففی برای کلمه Floor به معنای کف. در کتب ریاضی از علامت $[]$ برای نشان دادن تابع جزء صحیح استفاده می‌کنند.

همانگونه که از ظاهر فرمول برمی‌آید روش فوق تنها قادر به گردکردن عدد تا رقم یکان است. اما در بسیاری از موارد پیش می‌آید که نیازمند گردکردن تا یک رقم اعشار، دو رقم اعشار، سه رقم اعشار و... باشیم. در اینگونه موارد ابتدا تعداد رقم‌های اعشاری موردنظر را با ضرب کردن عدد در توان‌های ۱۰ به قسمت صحیح عدد منتقل می‌کنیم. سپس با استفاده از فرمول فوق حاصل ضرب را گرد کرده، و در نهایت با تقسیم عدد گرد شده بر همان توان ده عدد را به صورت اعشاری تبدیل می‌کنیم. به عنوان مثال برای گرد کردن عدد اعشاری 12.7896354 تا سه رقم اعشار به صورت زیر عمل می‌کنیم.

$$\begin{aligned}\text{ROUND}(12.7896354) &= \text{FLR}(12.7896354 * 10^3) / 10^3 \\ &= \text{FLR}(12789.6354) / 1000 \\ &= 12789 / 1000 \\ &= 12.789\end{aligned}$$

یک تذکر: در کامپیوترهایی با سازمان کلمه‌ای (بایتی)، تمام محاسبات را به وسیله روشی مشابه روش فوق گرد می‌کنند تا نتایج به دست آمده در یک کلمه کامپیوتر بگنجد. ما در زبان کارنمای خود، معمولاً اثر گردکردن را به حساب نمی‌آوریم و چنین وانمود می‌کنیم که کلمه‌های کامپیوتر آنچنان ظرفیتی دارند که تمام عملیات و محاسبات به طور دقیق انجام می‌شود. ولی قبل از ترجمه کارنما به زبان‌های برنامه‌سازی غالباً لازم است که جعبه‌های تصمیم‌نظیر

$$B^2 - 4 * A * C = 0$$

را با شکل‌های تقریبی مثل شکل زیر جایگزین کنیم.

$$|B^2 - 4 * A * C| < 0.0001$$

شما چه نتیجه‌ای از بحث فوق می‌گیرید؟

کاردر کلاس ۲-۶:

کارنمایی رسم کنید که زمان دوییدن دونه ای را بر حسب ثانیه دریافت کند و سپس معین کند آن دونه چند ساعت و چند دقیقه و چند ثانیه دویده است؟ سپس الگوریتم خود را برای $S=8534$ آزمایش کنید.

تابع باقی مانده تقسیم

یکی از تکنیک‌های بسیار مهمی که ما را در حل کردن برخی از الگوریتم‌ها کمک می‌کند یافتن باقی مانده تقسیم بر یک عدد است. پیشتر در مثال ۲-۵ دیدید که با به دست آوردن باقی مانده تقسیم عدد X بر دو توانستیم زوج یا فرد بودن X را احراز کنیم. چنانکه در این مثال دیدید برای به دست آوردن باقی مانده تقسیم، از تابع براکت [ویژگی تقسیم صحیح آن] استفاده کردیم. به طور کلی برای به دست آوردن باقی مانده تقسیم X بر Y از روش زیر استفاده می‌کنیم:

$$Q = \text{FLR} (X / Y)$$

ابتدا خارج قسمت تقسیم این دو عدد را به دست می‌آوریم:

$$R = X - Q * Y$$

پس باقی مانده تقسیم را به صورت زیر به دست می‌آوریم:

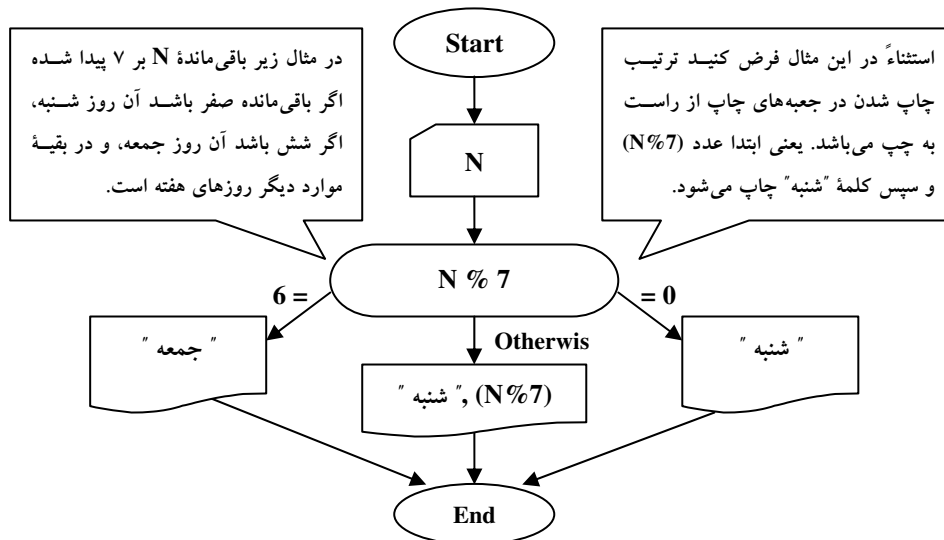
چنانکه می‌بینید به دست آوردن باقی مانده تقسیم یکی از کاربردهای تقسیم صحیح است. اما به جهت اهمیت و کاربرد زیادی که یافتن باقی مانده تقسیم در حل برخی مسائل دارد، در اکثر زبان‌های برنامه‌نویسی تابعی با نام MOD جهت یافتن باقی مانده تقسیم X بر Y تعریف شده است. این تابع دارای دو ورودی به صورت زیر است.

$\text{MOD} (X, Y)$

اما در زبان C++ چنانکه در فصل آتی خواهید دید نماد $\%$ را به عنوان یک عملگر برای یافتن باقی مانده تقسیم تعریف شده است. از آنجایی که در این کتاب زبان C++ نیز تدریس شده، ما نیز به جای تابع MOD از عملگر $\%$ برای نشان دادن عمل باقی مانده تقسیم استفاده کرده‌ایم. از نظر اولویت نیز این عملگر در ردیف ضرب و تقسیم قرار می‌گیرد. حال باقی مانده تقسیم چند عدد بر یکدیگر را به دست می‌آوریم:

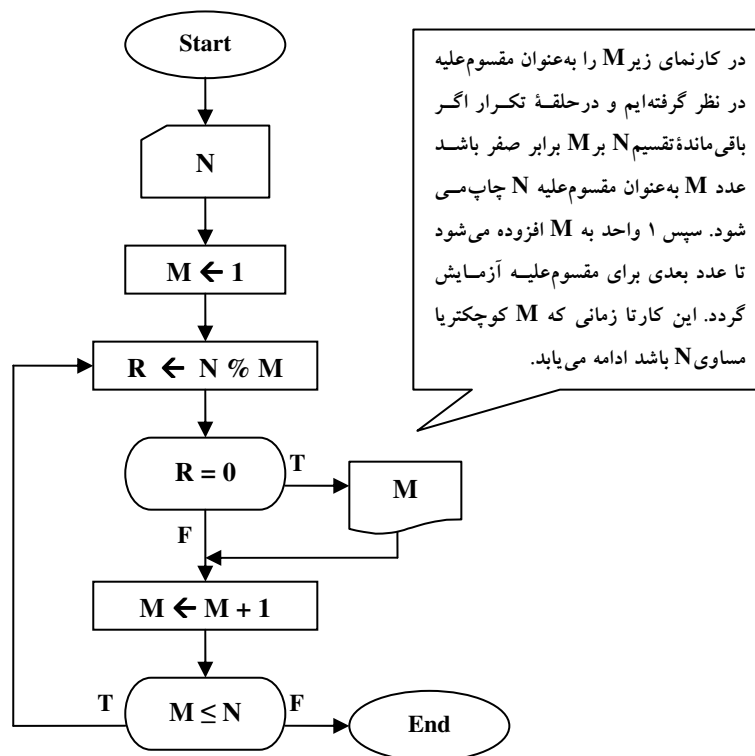
$$5 \% 2 = 1 \quad 8 \% 2 = 0 \quad 9 \% 3 = 0 \quad 11 \% 5 = 1 \quad 2 \% 4 = 2$$

مثال ۲-۷: روز اول سال، یکشنبه است. کارنمایی رسم کنید که معین کند روز N ام سال چه روزی از هفته است. (مثلاً، روز چهاردهم شنبه است).



شکل ۲-۲۵: کارنمای مثال ۲-۷

مثال ۲-۸: کارنمایی رسم کنید که با دریافت عدد طبیعی N کلیه مقسوم‌علیه‌های آن را چاپ کند؟



شکل ۲-۲۶: کارنمای مثال ۲-۸

کار در کلاس ۲-۷:

کارنمایی رسم کنید که با دریافت یک عدد بگوید آن عدد تام است یا نه؟
راهنمایی: عددی تام است که مجموع مقسوم‌علیه‌های کوچک‌تر از خودش برابر خود عدد باشد.

۲-۱۱: تمرین

۱. آیا می‌توانید کارنمای مثال ۲-۸ را به‌گونه‌ای بهینه‌سازی کنید که تعداد تکرار بیرونی ترین حلقه تکرار به نصف کاهش پیدا کند؟

۲. قسمت‌های (۱) تا (۶) این مسئله مربوط به کارنمایی است که در شکل ۲-۲۷ آمده است.

(۱) درست یا نادرست: به‌ازای هر بار که جعبه ۱ اجرا می‌شود، جعبه ۶ چهار بار اجرا می‌شود.

(۲) اگر ۶ جفت مقدار (هر جفت شامل یک مقدار برای N و یک مقدار برای L) به‌عنوان ورودی داده شود، تعداد مقادیر چاپ شده توسط برنامه:

(۱) بستگی به مقدار واقعی داده‌ها دارد.

(۲) دقیقاً ۱۸ تاست.

(۳) حداقل ۲۴ تاست.

(۴) هیچ مقداری چاپ نخواهد شد.

(۵) دقیقاً ۳ تاست.

(۳) فرض کنید که ورودی‌های این برنامه به ترتیب از چپ به راست به‌صورت زیر باشد:

۳۲۴ و ۷۹۲- و ۳۲۳ و ۷۹۱

کدام دسته از مقادیر زیر توسط برنامه چاپ خواهد شد:

(۱) ۱۹۷ و ۷۶۰ و ۴۳۷ و ۱۱۴

(۲) ۶۹۶ و ۹۰۵ و ۱۱۴

(۳) ۹۸۸ و ۵۵۱ و ۴۳۷ و ۱۱۱۴

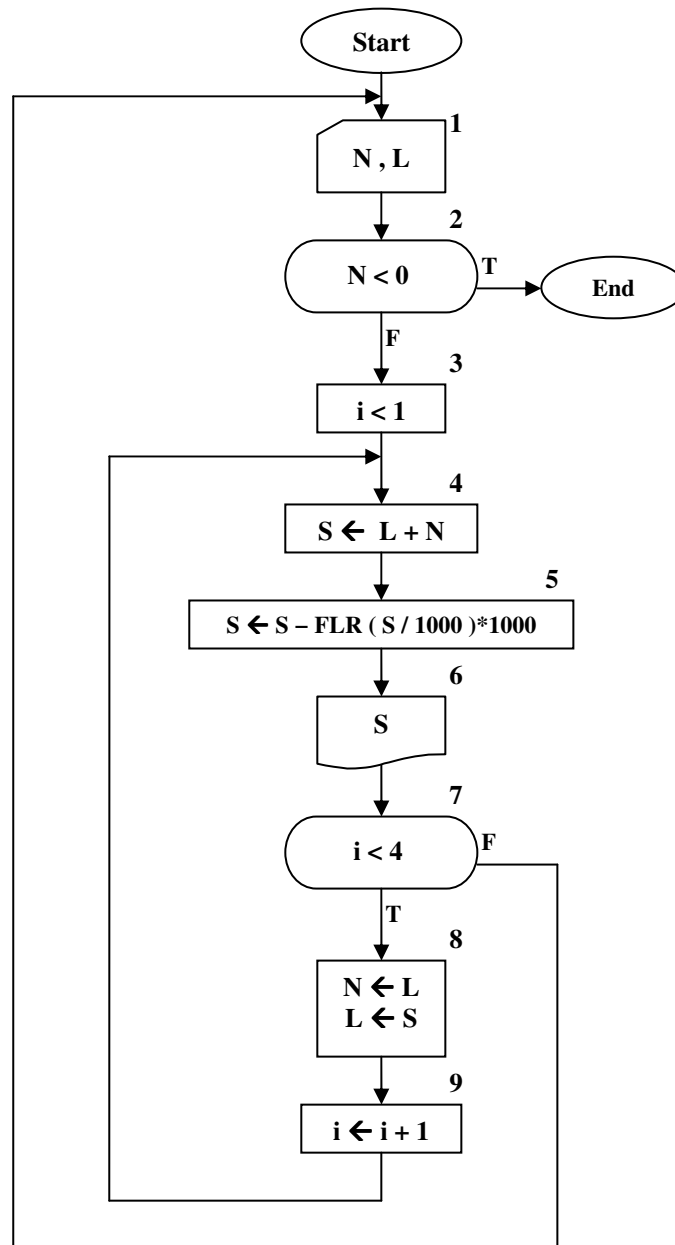
(۴) ۹۸۸ و ۵۵۱ و ۴۳۷ و ۱۱۴

(۵) ۹۸۸ و ۶۰۱ و ۶۹۶ و ۹۰۵ و ۱۱۱۴

(۴) درست یا نادرست: حلقه داخلی کارنما، یعنی جعبه‌های ۴ تا ۹، حلقه‌ای است که توسط شمارنده کنترل می‌شود.

(۵) درست یا نادرست: اگر فرض کنیم مقادیر منفی N غیر مجاز هستند، حلقه خارجی کارنما، یعنی جعبه‌های ۱ تا ۷، حلقه‌ای است که به وسیله نگهبان کنترل می‌شود.

(۶) چگونه می‌توان جعبه‌های ۴ و ۵ کارنما را به جعبه‌ای با یک انتساب واحد تبدیل کرد بدون اینکه اثر مهمی بر روی برنامه و یا نتیجه آن بگذارد؟



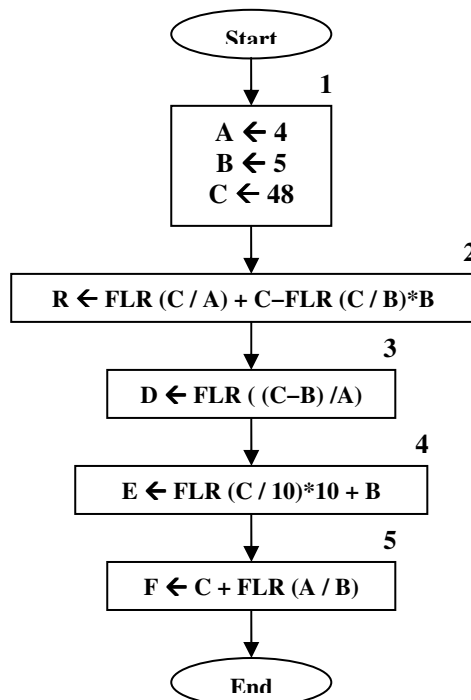
شکل ۲-۲۷: کارنمای تمرین ۲

۳. کارنمایی رسم کنید که یک مجموعه صدتایی از مقادیر داده را خوانده و تمام اعداد زوج به استثنای مضارب ۱۰ را در خروجی بنویسد. مطمئن شوید که کارنمای شما دقیقاً ۱۰۰ مقدار را می‌خواند.

۴. برای هریک از پنج دستور انتساب زیر مقادیر متغیرهای مربوط به قبل از اجرای دستور العمل انتساب داده شده است. وظیفه شما تعیین مقداری است که در هر دستور به T نسبت داده می‌شود.

- | | |
|--|----------------------|
| (1) $T \leftarrow I \times J + K$ | , I=11, J=3, K=75 |
| (2) $T \leftarrow \text{FLR}((I - J) / T)$ | , I=52, J=7.9, T=2.1 |
| (3) $T \leftarrow \text{FLR}(I / J + K)$ | , I=50, J=82, K=-1 |
| (4) $T \leftarrow N - \text{FLR}(N / D) * D$ | , N=10, D=2 |
| (5) $T \leftarrow N / 3 - \text{FLR}((N / 3) / D) * D$ | , N=14, D=7 |

۵. کارنمای زیر را پیگیری و جمله‌های زیر را کامل کنید:



الف- پس از اجرای جعبه ۲، مقدار R برابر است با ...

ب- پس از اجرای جعبه ۳، مقدار D برابر است با ...

پ- پس از اجرای جعبه ۴، مقدار E برابر است با ...

ت- وقتی به End برسیم، مقدار F برابر است با...

شکل ۲-۲۸: کارنمای تمرین ۵

۶. کارنمایی رسم کنید که تعدادی عدد صحیح را، یکی یکی به داخل آورده و به‌ازای هر مقدار، یک عدد دو رقمی چاپ می‌کند که تشکیل شده است از دو رقم سمت راست (یعنی رقم‌های یکان و دهگان) مقدار ورودی، تعدادی عدد صحیح مثبت به‌عنوان داده در اختیار داریم که یکی یکی باید خوانده شوند و آخرین ورودی یک عدد منفی است.

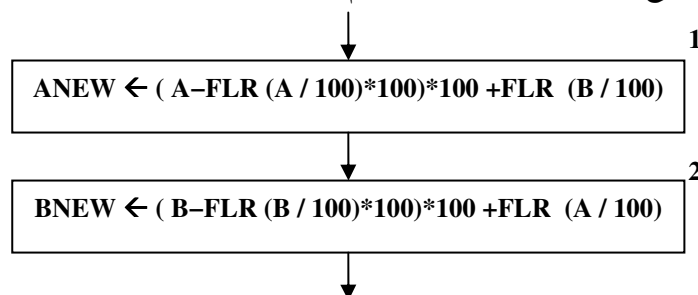
۷. آیا می‌توانید در مورد نحوه تشخیص قابل قسمت بودن یک عدد بر عدد دیگر توضیح دهید، کارنمایی رسم کنید که مقسوم‌علیه‌های هر عدد ورودی n را چاپ کند.

۸. الگوریتمی رسم کنید که مضارب کوچکتر از ۱۰۰۰۰ عدد ورودی n را محاسبه و چاپ کند.

۹. الگوریتم تمرینات قبل را به‌گونه‌ای گسترش دهید که بزرگترین مقسوم‌علیه مشترک دو عدد را یافته و چاپ نماید.

۱۰. الگوریتم تمرین قبل را به گونه‌ای گسترش دهید که کوچکترین مضرب مشترک دو عدد دریافتی را نیز بیابد.
راهنمایی: هدف از این تمرین آن است که توجه شما را به این نکته جلب کنیم که همیشه استفاده از روش‌های الگوریتمی و تکرار مناسب‌ترین راه حل نیست، به‌طوری که با توجه به قواعد ریاضی می‌توانید ک.م.م را با توجه به ب.م.م محاسبه کنید.

۱۱. A و B اعداد صحیح چهار رقمی هستند. کارنمای ناتمام شکل ۲-۲۹ داده شده است.



شکل ۲-۲۹: کارنمای تمرین ۱۱

الف- اگر مقدار A برابر ۱۴۶۸ و مقدار B برابر ۲۳۵۷ باشد، مقدار ANEW پس از اجرای جعبه ۱ چیست؟

ب- به ازای همین مقادیر A و B، مقدار BNEW پس از اجرای جعبه ۲ چیست؟

۱۲. مقدار ROUND(X) را وقتی که X مضرب فردی از ۰/۵ باشد تعیین کنید.

۱۳. عدد N مفروض است. کارنمایی رسم کنید که مجموع ارقام عدد N را به‌دست آورد.

۱۴. کارنمایی رسم کنید که مقادیر را دوتا دوتا خوانده و عدد جدیدی متشکل از دو رقم سمت چپ عدد اول و دو رقم سمت راست عدد دوم ساخته و چاپ کند. این پردازش برای جفت‌های بعدی اعداد ورودی نیز به همین نحو تکرار می‌شود. تعدادی عدد صحیح مثبت چهار رقمی به‌عنوان داده در اختیار داریم و آخرین ورودی یک عدد منفی است.

۱۵. هزینه ارسال نامه هوایی به ازای هر ۳۰ گرم ۱۰۰ ریال است و به‌منظور محاسبه هزینه پست، وزن‌نامه‌ها را با تقریب اضافی برحسب تعداد ۳۰ گرم‌های کامل تعیین می‌کنند. قدم‌های کارنمایی را رسم کنید که نشان دهنده تعیین هزینه ارسال یک برنامه هوایی به‌صورت تابعی از متغیر (حقیقی) وزن WT باشد. از یکی از توابع FLR یا ROUND استفاده کنید.

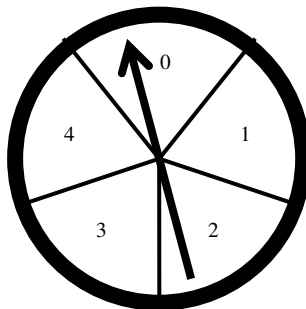
۱۶. روز اول سال، چهارشنبه است. الگوریتمی بنویسید که معین کند روز N-ام سال چه روزی از هفته است. (مثلاً، روز چهاردهم سه شنبه و روز صد و چهل و سوم جمعه است).

۱۷. فرض کنید در N-امین روز سال هستیم، الگوریتمی بنویسید که تاریخ روز را با دریافت N معین کند؟ (مثلاً، اگر در روز شصت و چهارم سال باشیم، تاریخ دوم خرداد ماه است ۳/۲، هدف تعیین شماره روز و شماره ماه مربوطه است).

۱۸. فرض کنید در روز D-ام از ماه شماره M-ام هستیم، الگوریتمی بنویسید که معین کند در روز چندم سال هستیم؟ (مثلاً، اگر D = ۲۱ و M = ۸ برای N مقدار ۲۳۷ به‌دست می‌آید).

۱۹. چرخ بازی‌ای که در شکل ۲-۳۰ نشان داده شده به پنج قطاع مساوی تقسیم شده است و این قطاع‌ها در جهت حرکت عقربه‌های ساعت به ترتیب از صفر به بالا شماره‌گذاری شده‌اند. عقربه‌ای وجود دارد که حول محوری که در وسط چرخ سوار شده است می‌چرخد.

فرض کنید S شماره قطاعی است که عقربه در حال سکون روی آن قرار گرفته است. حال با دست، ضربه‌ای در جهت حرکت عقربه‌های ساعت به عقربه می‌زنیم. عقربه از روی M قطاع عبور می‌کند و در داخل یک قطاع متوقف می‌شود (یعنی روی خط نمی‌ایستد).



شکل ۲-۳۰

الف- فرمولی بنویسید که با استفاده از یکی از توابع گرد کردن، شماره قطاع جدید NEWS را برحسب شماره قطاع اصلی در حال سکون S و تعداد قطاع‌هایی که عقربه از روی آنها عبور کرده است یعنی M، به دست دهد.

ب- فرمول قسمت (آ) این مسئله را برای حالتی که چرخ به K قطاع تقسیم و از صفر به بالا شماره‌گذاری شده باشد تعمیم دهید.

ج- قدم‌های کارنمای مورد نیاز برای رسیدن به شماره قطاع NEWS را، که طرز محاسبه آن در قسمت (آ) ذکر شد به نحوی ترسیم کنید که هم برای چرخش در جهت عقربه‌های ساعت و هم برخلاف آن قابل استفاده باشد.

۲۰. اگر قطاع‌های چرخ مسئله قبل را به جای ۰، ۱، ۲، ۳ و ۴ به صورت ۱، ۲، ۳، ۴ و ۵ شماره‌گذاری کنیم، چه تغییراتی لازم است در حل مسئله داده شود.

۲-۱۲: عبارات رابطه‌ای مرکب

عملگرهای منطقی

پیشتر با عبارات‌های رابطه‌ای ساده در بخش ۲-۲ آشنا شدید. حال تصمیم داریم با عملگرهای منطقی به عبارات رابطه‌ای مرکب و پیچیده‌تری دست پیدا کنیم، به‌طوری که می‌توان چندین عبارت شرطی ساده را با عملگرهای منطقی (و، and) و (یا، or) ترکیب کرد و عبارات شرطی مرکب را ساخت. در حالت کلی اگر p و q دو عبارت شرطی با ارزش درستی T یا نادرستی F باشند، نتیجه ترکیب آنها تحت شرایط مختلف با دو عملگر منطقی and و or در جداول زیر نشان داده شده است. r را به‌عنوان گزاره شرطی معادل ترکیب p و q در نظر بگیرید:

جدول درستی عملگر and

p	q	r
T	T	T
T	F	F
F	T	F
F	F	F

جدول ۲-۴

جدول درستی عملگر or

p	q	r
T	T	T
T	F	T
F	T	T
F	F	F

جدول ۲-۳

به‌عنوان مثال عبارت زیر مطابق جدول ۲-۴ زمانی درست است که هر دو عبارت شرطی ساده آن درست باشند، در غیر این صورت دارای ارزش نادرست است:

$$(A \neq 0) \text{ and } (B^2 - 4 * A * C > 0)$$

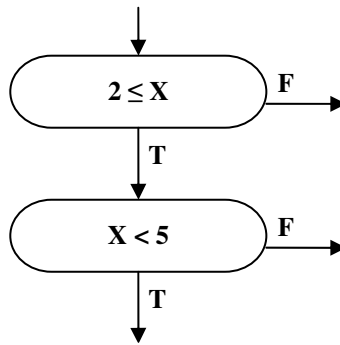
بهتر است در اطراف عبارات شرطی (رابطه‌ای) ساده پرانتز قرار دهید تا از خوانایی عبارت کاسته نشود. اما اگر پرانتز هم نگذارید اشکالی در محاسبات رخ نمی‌دهد. اگر بخواهید جدول ۲-۱ که مربوط به تقدم عملگرها بود را کامل کنید به جدول ۲-۵ خواهید رسید، و چنانکه می‌بینید در این جدول تقدم عملگرهای رابطه‌ای پس از عملگرهای محاسباتی می‌باشد و عملگرهای منطقی در انتهای جدول قرار دارند.

نشان عملگر	نام عملگر	سطح تقدم
$\wedge$	به توان رساندن	اول
$*, /$	ضرب و تقسیم	دوم
$\%$	باقی مانده تقسیم	
$-$	منفی کردن (یک تایی)	
$+$	جمع	سوم
$-$	تفریق (دوتایی)	
$=, \neq, \geq, >, <, \leq$	عملگرهای رابطه‌ای	چهارم
And, Or	عملگرهای منطقی	پنجم

جدول ۲-۵: تقدم تمامی عملگرها نسبت به هم در عبارات رابطه‌ای مرکب

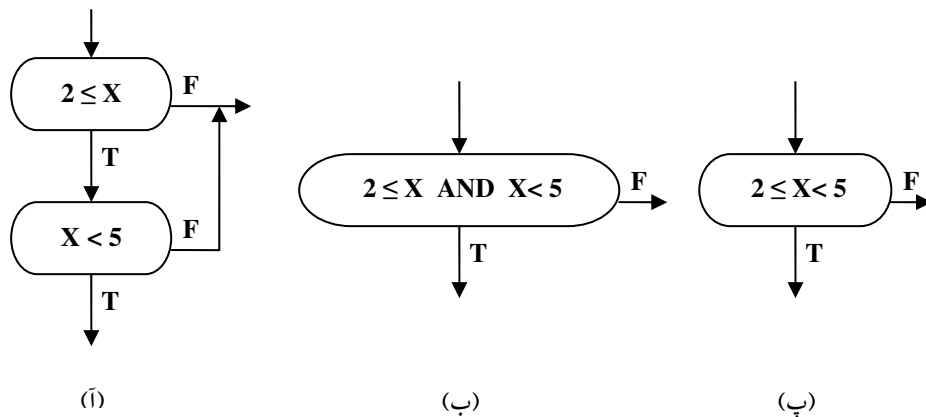
نماد خلاصه شده برای چند تصمیم پشت سرهم

یکبار دیگر به کارنمای شکل ۲۲-۲ مراجعه کنید و دقایقی آن را مدنظر قرار دهید. چنانکه در کارنمای شکل ۲۲-۲ می‌بینید در کارنهاها گاهی با یک سری جعبه تصمیم به همراه عبارات رابطه‌ای ساده به شکل زیر مواجه می‌شویم.



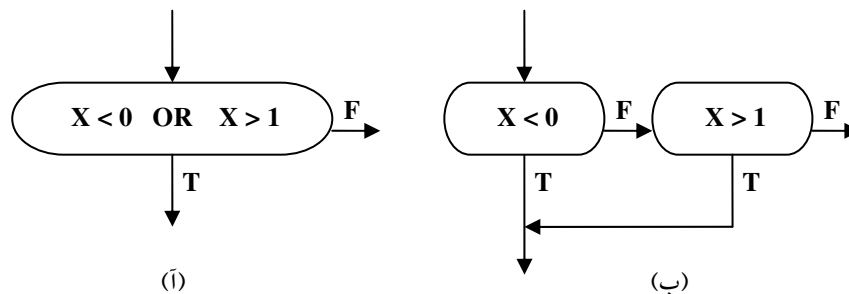
شکل ۲-۳۱: مجموعه جعبه‌های تصمیم ساده

غالباً به‌منظور خلاصه کردن کارنما و برای مشاهده تصویر کلی آن، مناسب است که یک جعبه تصمیم با عبارت رابطه‌ای مرکب معادل این سری از جعبه‌ها را جایگزین آنها سازیم. از این رو به‌جای شکل ۲-۳۵ (آ) می‌توان شکل ۲-۳۵ (ب) یا ۲-۳۵ (پ) را قرار داد.



شکل ۲-۳۲: مجموعه جعبه‌های تصمیم ساده و دو جعبه مرکب معادل آن

از طرف دیگر مواردی هم هست که می‌خواهیم در جهت عکس عمل کنیم و جعبه‌ای را که شامل یک شرط پیچیده‌تر یا مرکب است برداریم و به جای آن مجموعه‌ای از شرایط ساده را قرار دهیم. در این صورت مجموعه شرایط ساده شکل ۲-۳۶ (ب) می‌تواند جانشین شرط مرکب شکل ۲-۳۶ (آ) شود.



شکل ۲-۳۳: شرط مرکب و مجموعه شرایط ساده معادل آن.

نکاتی در خصوص جعبه‌های تصمیم چندگانه (چند راهه)

پیشتر در بخش ۲-۲ و در شکل ۲-۸ با جعبه‌های تصمیم چندگانه آشنا شدید. همچنین در کارنمای شکل ۲-۲۳ از این نوع جعبه تصمیم استفاده کردیم. در واقع جعبه‌های تصمیم چندگانه راهی دیگر برای ساده‌سازی کارنهاها هستند. اما به جهت رابطه این جعبه‌ها با خلاصه‌سازی کارنهاها و به جهت اینکه در بخش ۲-۲ سخنی در خصوص ویژگی‌های این جعبه‌ها به میان نیاوردیم در این قسمت به بیان ویژگی‌های این جعبه‌ها خواهیم پرداخت.

۱. در جعبه‌های تصمیم چندگانه هر مسیر خروجی باید به وضوح برجسب‌گذاری شود به نحوی که نشان دهد تحت چه شرط یا شرایطی آن مسیر انتخاب می‌شود.

۲. شرایطی که بر روی مسیرهای خروجی ذکر می‌شوند نباید همپوشانی بکنند. به عنوان مثال اگر برجسب یک خروجی $x > 5$ و برجسب دیگری $x < 8$ باشد و مقدار x برابر ۶ به دست آید از کدام یک از این مسیرها باید برویم؟

۳. همه حالات ممکن نیز باید در نظر گرفته شود، چنانکه اگر حالتی پیش آید که با هیچ یک از برجسب‌های خروجی سازگاری نداشته باشد، راهی برای خارج شدن از جعبه نخواهیم داشت.

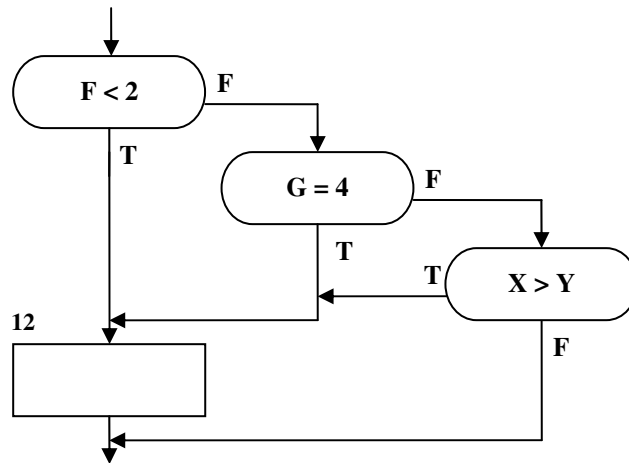
کار در کلاس ۲-۸:

کارنمای شکل ۲-۲۲ که در خصوص شمارش ورود نمرات امتحانی است را با جعبه‌ی تصمیم چندگانه بازنویسی کنید.

۲-۱۳: تمرین

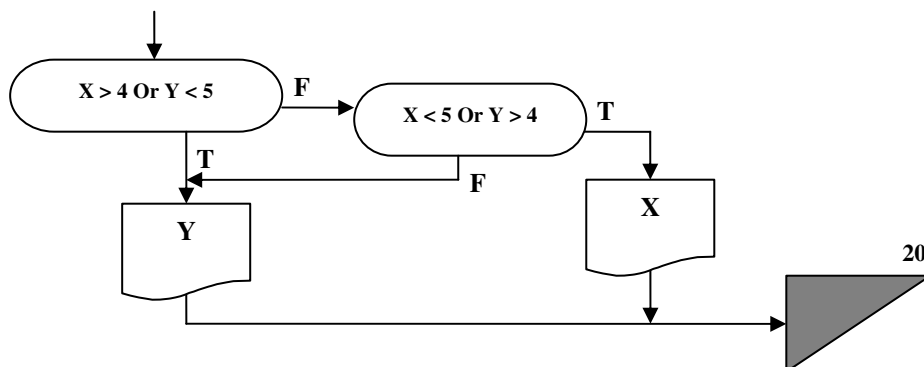
در مسائل ۱ تا ۳ یک قاعده تصمیم‌گیری همراه با کارنمای پیشنهادی معادل آن به شما داده شده است. متغیرهایی که در کارنما آمده‌اند خود بیانگر معانی مربوطه‌اند. تکلیف شما این است که تصمیم بگیرید که آیا هر کارنما معادل منطقی قاعده تصمیم‌گیری مربوطه هست یا نه، و در صورت منفی بودن جواب، کارنما را چنان اصلاح کنید که منطقاً معادل آن باشد.

۱. اگر X بیشتر از Y است و یا F کمتر از ۲ یا G مساوی ۴ است (یا هر دو)، آنگاه جعبه ۱۲ را اجرا کن در غیر این صورت جعبه ۱۲ را اجرا نکن.



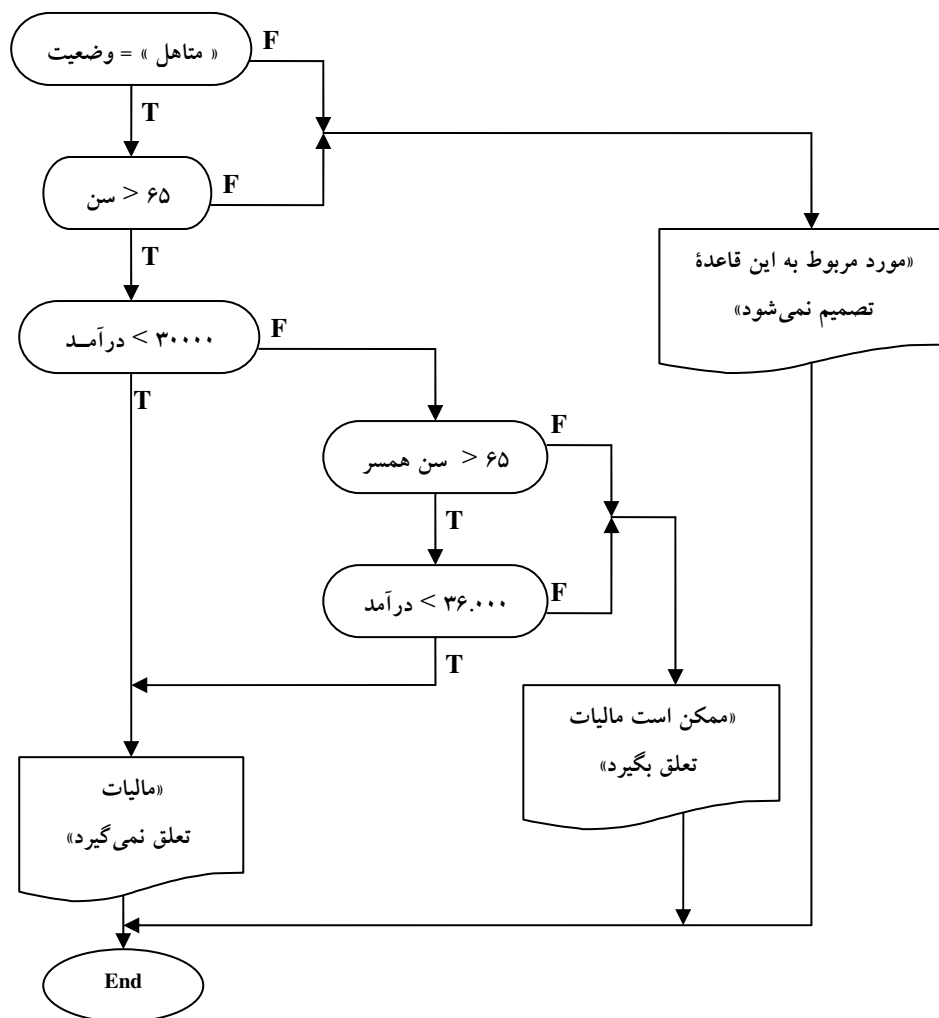
شکل ۲-۳۴: شکل تمرین ۱

۲. اگر X بیشتر از ۴ است در حالی که Y کمتر از ۵ است یا X کمتر از ۵ است، در حالی که Y بزرگتر از ۴ است، آنگاه مقدار Y را چاپ کن؛ در غیر این صورت مقدار X را چاپ کن و در هر دو صورت، پس از آن به جعبه ۲۰ برو.



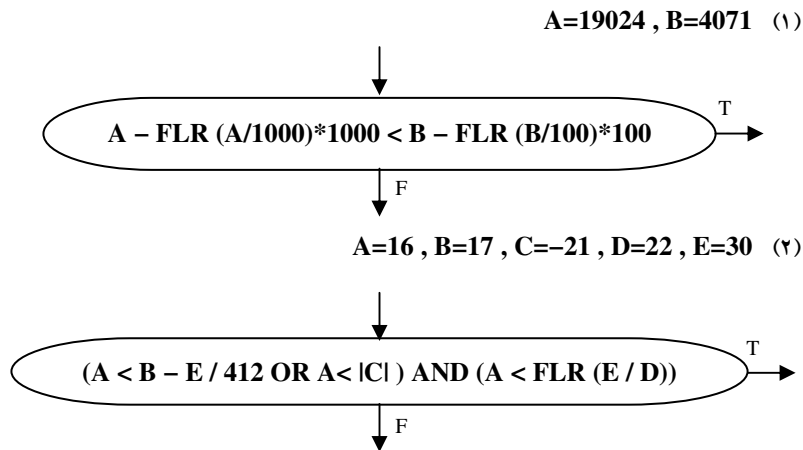
شکل ۲-۳۵: شکل تمرین ۲

۳. این قاعده مربوط به شخصی است که بیش از ۶۵ سال سن داشته، متاهل باشد و با همسر خود مشترکاً یک اظهارنامه برای مالیات برای درآمد پرمی کند. این شخص در صورتی که درآمد سالیانه‌اش کمتر از ۳۰.۰۰۰ تومان باشد یا در صورتی که درآمدش کمتر از ۳۶.۰۰۰ تومان و سن همسرش بیش از ۶۵ سال باشد، مالیاتی نمی‌پردازد.



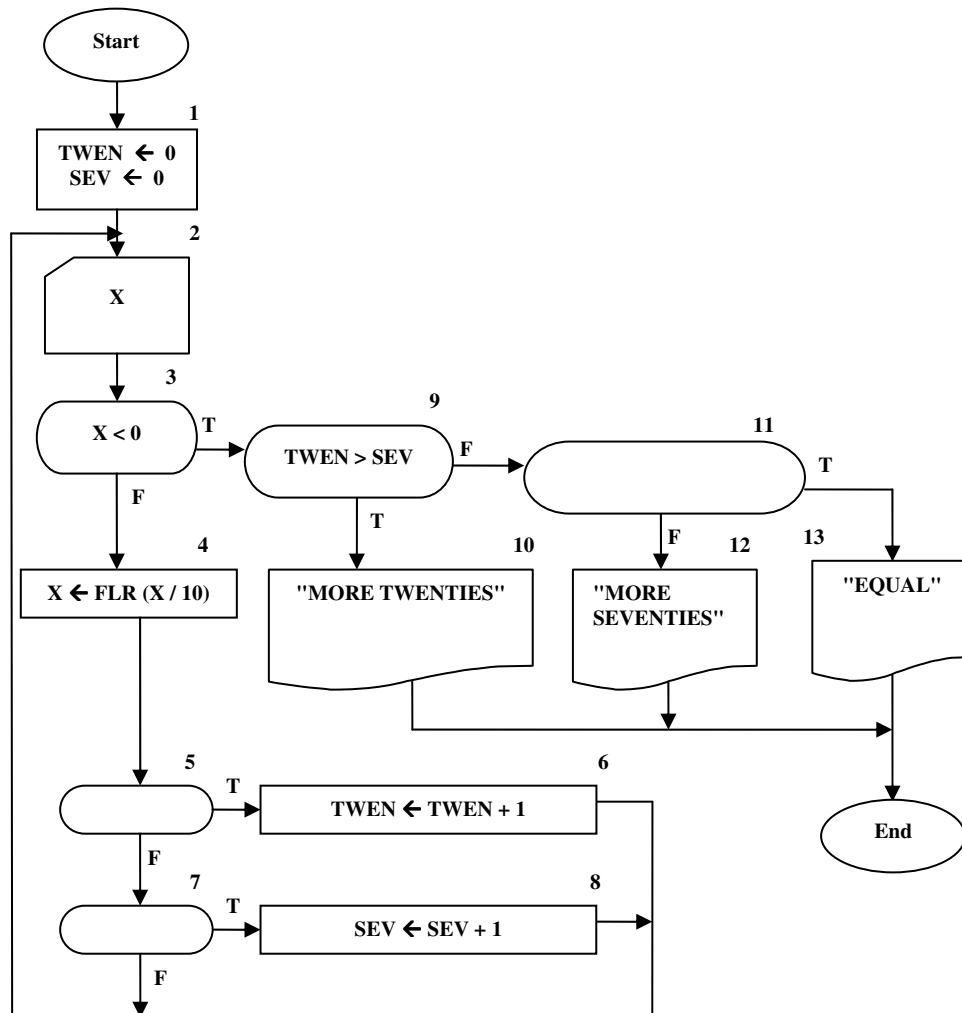
شکل ۲-۳۹: کارنمای تمرین ۳

۴. در قسمت‌های (۱) و (۲) زیر، عبارت‌های رابطه‌ای در کارنما داده شده‌اند. وظیفه شما این است که با استفاده از مقادیر داده شده، درستی یا نادرستی این عبارت‌ها را تعیین کنید.



۵. در شکل ۲-۴۷، کارنمای ناتمامی برای یک الگوریتم نشان داده شده است. این الگوریتم به منظور خواندن مجموعه‌هایی از اعداد مثبت از ورودی و تعیین اینکه آیا تعداد اعداد واقع در محدوده ۲۰ الی ۲۹ بیشتر از تعداد واقع در محدوده ۷۰ الی ۷۹ هست یا نه، طرح شده است. تعداد اعدادی که باید خوانده شود نامعلوم است ولی تمام شدن داده‌ها به وسیله عددی که شامل یک عدد منفی است، علامت داده می‌شود. در قسمت‌های (آ) و (ب) در زیر، شما باید تعیین کنید که چگونه باید کارنما را به طور صحیح تکمیل کرد. به اثر تابع FLR در جعبه ۴ توجه داشته باشید.

(آ)		(ب)	
	5		11
1.	$X = 0$	$X > 0$	$TWEN < SEV$
	5	7	11
2.	$X = 20$	$X = 70$	$TWEN = SEV$
	5	7	11
3.	$X = 2$	$X = 7$	$70 < 20$
	5	7	11
4.	$X < 30$	$X < 80$	$TWEN = 0$
5.	هیچ کدام		هیچ کدام



شکل ۲-۴۰: کارنمای تمرین ۵

۶. کارنمایی رسم کنید که محاسبات لازم برای تعیین ربع صفحه‌ای از محورهای مختصات که نقطه مفروض P در آن واقع شده است را انجام دهد و اعداد ۱، ۲، ۳ یا ۴ را به صورت پیامی برای نشان دادن ربع صفحه مربوطه چاپ کند. مختصات P ، $(X$ و $Y)$ مفروض‌اند. تصمیم در مورد نحوه عمل در مواردی که P روی یکی از محورها یا هر دوی آنها قرار گرفته باشد با شماست.

۷. کارنمایی برای ترسیم دایره ای به معادله $(x-a)^2 + (y-b)^2 = r^2$ در صفحه مختصات، رسم کنید. مسلماً ثوابت a و b باید از کاربر دریافت شود.

۸. کارنمایی ترسیم کنید که بتواند تشخیص دهد نقطه مفروض P نسبت به دایره فوق چه وضعیتی دارد.

۹. یک استاد ریاضی طالب الگوریتمی است که هرگاه دو عدد مثبت A و B که $(A < B)$ ، به آن داده شود، یا یک عدد صحیح C طوری پیدا کند که $A \leq C^2$ و $C^3 \leq B$ ، یا عدم وجود چنین عددی را گزارش کند. اگر بیش از یک C با خواص فوق وجود داشته باشد اهمیتی ندارد که کدام یک انتخاب می‌شود. دو خط مشی ممکن برای این کار در زیر عرضه می‌شود.

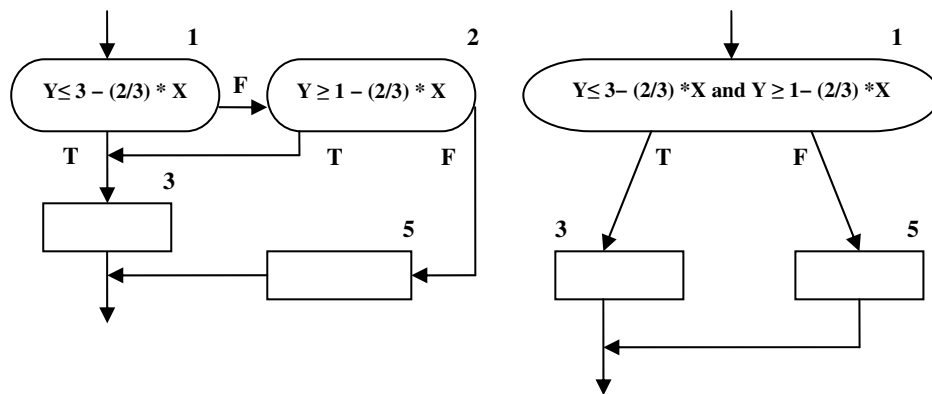
الف- مقادیر ... و ۳ و ۲ و ۱ $C =$ را امتحان کن تا اینکه یک مقدار قابل قبول پیدا شود یا اطمینان حاصل شود که چنین عددی نمی‌توان یافت.

ب- بزرگترین مقدار C را که به ازای آن $C^3 \leq B$ ، پیدا کن و بین آیا این مقدار C به اندازه کافی بزرگ هست که $A \leq C^2$ یا نه، و بدین ترتیب یک مقدار قابل قبول (در صورت وجود) برای C محاسبه کن. وظیفه شما به شرح زیر است:

(۱) برای هریک از دو خط مشی (الف) و (ب) کارنمایی رسم کنید. کارنماهای شما باید با چاپ پیام و مقدار، نتیجه را نشان دهند.

(۲) شما پیشنهاد می‌کنید که استاد فوق از کدام کارنما استفاده کند؟ چرا؟

۱۰. آیا دو ترکیب زیر معادل‌اند؟

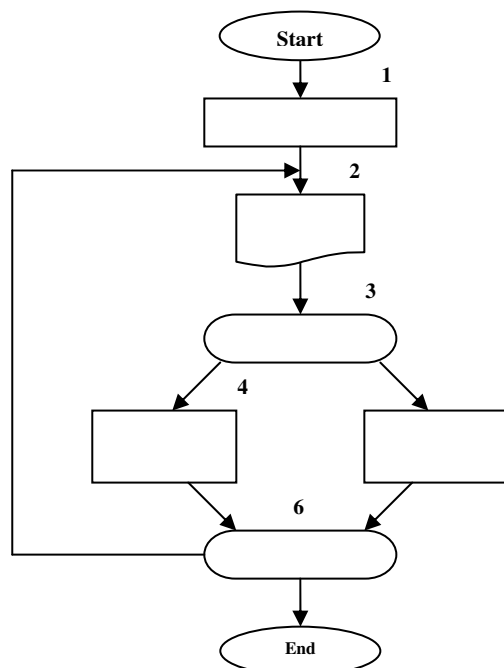


شکل ۲- ۴۱: کارنماهای تمرین ۱۰

۲-۱۴: تمرینات تکمیلی

۱. فرض کنید در کنار رودخانه‌ای هستید و دو سطل در اختیار دارید، یکی پنج لیتری و دیگری نه لیتری (سطل‌ها مدرج نیستند). اگر بخواهید چهار لیتر آب داشته باشید، به وسیله سطل پنج لیتری، پنج لیتر از آب سطل نه لیتری بر می‌دارید و در رودخانه خالی می‌کنید.

الف- وظیفه شما این است که کارنمایی بسازید که تمام تعداد لیترهایی را که با این قبیل پر و خالی کردن‌ها می‌توان جدا کرد، در خروجی بنویسد. فقط به یک متغیر، T ، نیاز دارید که حساب مجموع مقدار آب دو سطل را نگه دارد. مقدار T وقتی تغییر می‌کند که آب از رودخانه برداشته شده و یا به آن ریخته شود ولی اگر آب از سطلی به سطل دیگر ریخته شود مقدار T تغییر نمی‌کند. اگر T کوچک‌تر از ۵ باشد سطل نه لیتری را پرمی‌کنید و بدین ترتیب ۹ واحد به T می‌افزایید، در حالی که اگر T بزرگ‌تر یا مساوی ۵ باشد، سطل پنج لیتری را خالی کرده و در نتیجه پنج واحد از T کم می‌کنید. تمام مقادیر T را که محاسبه می‌شود در خروجی بنویسید. فراموش نکنید که مقدار اولیه صفر را به T نسبت دهید. برنامه را وقتی متوقف کنید که مقدار T دوباره صفر شود. ساخت کارنما در شکل ۲-۴۲ نشان داده شده است. تنها کاری که لازم است انجام دهید، پر کردن جعبه‌ها است.



شکل ۲-۴۲: کارنمای تمرین سطل آب

۲. کارنمای مسئله ۱ را طوری تغییر دهید که با هر مقدار T که چاپ می‌شود، تعداد دفعات پر شدن سطل نه لیتری و همچنین تعداد دفعات خالی شدن سطل پنج لیتری را که برای آن مقدار T لازم است، نیز چاپ کند. لازم است از دو متغیر جدید مثل F و E استفاده کنید. فراموش نکنید که به آنها مقدار اولیه نسبت دهید.
۳. کارنمای مسئله قبل را طوری تغییر دهید که با هر ظرفیتی (A و B) برای دو سطل کار کند (A و B اعداد صحیح و برحسب لیتر هستند). سطل A لیتری را پر کنید و سطل B لیتری را خالی کنید. پیش‌بینی لازم را برای خواندن A و B به عمل آورید. همچنین از متغیر N استفاده کنید که سطرهای خروجی را با شروع از صفر شماره‌گذاری کند به‌طوری که اولین سطر خروجی به‌صورتی که در شکل ۲-۳۳ نشان داده شده است بشود.

$$N=0, T=0, F=0, E=0$$

شکل ۲-۴۳: نمونه‌ی خروجی تمرین سه

۴. کارنمای مسئله ۳ را به ازای مقادیر ورودی زیر برای A و B پیگیری کنید (فقط سطرهای خروجی را بنویسید).

الف - $B=10$ و $A=7$

ب - $B=7$ و $A=10$

ج - $B=36$ و $A=54$

۵. در رابطه با مسئله ۴، توضیح دهید که چرا روابط زیر همواره صادق است.

الف - $N = F + E$

ب - $T = F \times A - E \times B$

۶. کارنمایی رسم کنید که با دریافت عدد N عبارت زیر را محاسبه کند.

$$1^2 + 2^2 + 3^2 + \dots + N^2$$

- آیا می‌توانید یک فرمول برای عبارت فوق پیدا کنید؟ در این مثال کدام یک از دو روش مکاشفه‌ای و الگوریتمی بهتر هستند؟

۷. کارنمایی رسم کنید که یک عدد در مبنای ۲ شامل ارقام صفر و یک را دریافت کند. سپس این عدد را به مبنای ۱۰ برده و چاپ کند.

۸. کارنمایی رسم کنید که محیط و مساحت یک مثلث را با داشتن مختصات رئوس آن چاپ کند. رئوس مثلث برابر A و B و C هستند و اضلاع آن برابر a و b و c هستند.

$$L = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2} \quad \text{طول یک ضلع مثلث}$$

در هر مثلث نصف مجموع اضلاع مثلث را با p نشان می‌دهند که برابر نصف محیط مثلث نیز می‌باشد. با توجه به p، مساحت و ارتفاع مثلث برابر است با:

$$p = \frac{a+b+c}{2}$$

$$S = \sqrt{p(p-a)(p-b)(p-c)}$$

$$h = \frac{2}{a} \sqrt{p(p-a)(p-b)(p-c)}$$

۹. کارنمایی رسم کنید که شماره پرسنلی، ساعات کار و دستمزد ساعتی کارکنان یک شرکت را خوانده و حقوق آنها را محاسبه کند. اگر کارمندی بیش از ۴۰ ساعت کار کرده باشد، اضافه کار به او تعلق می‌گیرد. به ازای هر ساعت اضافه کاری، یک و نیم برابر دستمزد ساعتی، به عنوان اضافه کاری پرداخت می‌شود. این الگوریتم را برای حالات زیر حل کنید.

الف- تعداد کارگران مشخص باشد.

ب - تعداد کارگران نامشخص باشد، اما اگر شماره پرسنلی نامعتبر وارد شد الگوریتم خاتمه یابد.

۱۰. کارنمایی رسم کنید با خواندن عدد N، مجموع N جمله اول سری زیر را محاسبه و چاپ کند.

$$S = 1 - 2 + 3 - 4 + \dots + (-1)^{N+1} N$$

۱۱. کارنمایی رسم کنید که دو عدد L و H را از ورودی خوانده و کلیه اعداد زوج مضرب ۷ بین این دو عدد را چاپ کند.

۱۲. کارنمایی رسم کنید که دو عدد صحیح را از ورودی خوانده و حاصل ضرب آنها را با استفاده از عمل جمع محاسبه می‌کند. (لزوماً این اعداد مثبت نیستند!)

۱۳. مسئله قبل را برای محاسبه خارج قسمت و باقی مانده تقسیم به واسطه عمل تفریق و جمع تکرار کنید. این مسئله را یکبار با فرض عدد طبیعی بودن ورودی‌ها یعنی مثبت بودن ورودی‌ها و دیگر بار با فرض دامنه اعداد صحیح یعنی هم مثبت و هم منفی برای ورودی‌ها حل کنید.

۱۴. کارنمایی رسم کنید که چه معادله درجه اول و چه معادله درجه دوم را به آن بدهند قادر به محاسبه ریشه‌های آن باشد.

راهنمایی: دو راه برای تفکیک معادلات دارید: (۱) صفر بودن ضریب a، (۲) پرسیدن نوع معادله از کاربر.

تصویر بیارنه استراوسزوپ مبتکر زبان C++



فصل سوم

مقدمات زبان C++

اهداف فصل و چکیده مطالب :

۱. آشنایی با تاریخچه زبان C++
۲. چرا زبان C++ را برای یادگیری انتخاب کنیم؟
۳. آشنایی با انواع داده در C++
۴. آشنایی با انواع عملگر در C++
۵. آشنایی با کامپایلر Microsoft Visual C++ 6.0
۶. بحثی کوتاه در مورد ساختار کلی برنامه در C++
۷. ورودی و خروجی در C++
۸. نوشتن نخستین برنامه‌ها در C++

۳-۱: ویژگی‌های زبان C++

مقدمه

زبان C++ از زبان C مشتق شده است. گفته می‌شود که زبان C++ یک ابر مجموعه^۱ زبان C می‌باشد. بدین معنا که در زبان C++ می‌توان اکثر دستورات زبان C را به کار برد، اما عکس این مطلب صادق نیست. لذا با توجه به اینکه ریشه زبان C++ در C قرار دارد، ابتدا به بررسی تاریخچه زبان C می‌پردازیم.

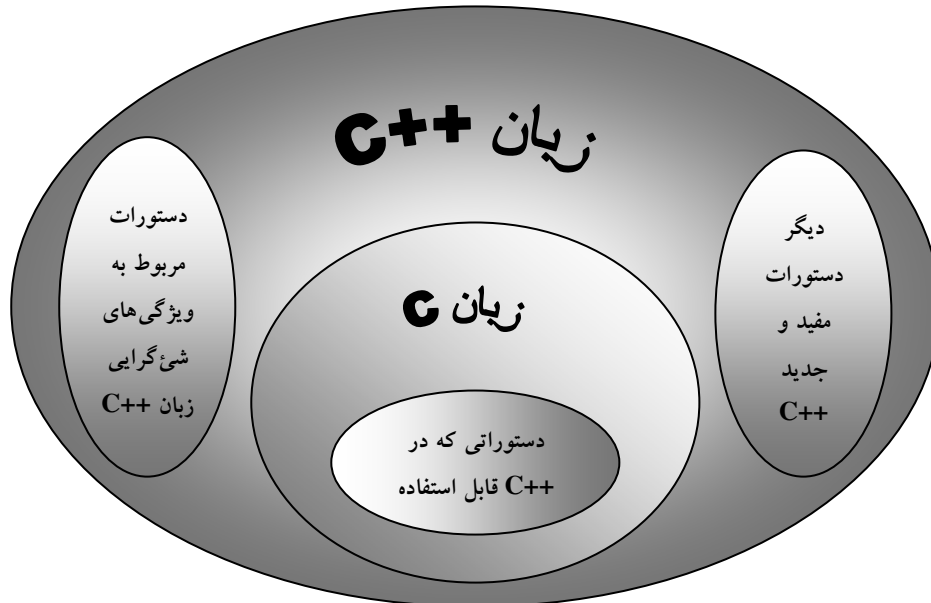
زبان C در سال ۱۹۷۲ توسط دنیس ریچی (Dennis Ritchie) در آزمایشگاه بل (Bell) طراحی شد. این زبان در ابتدا بر روی یک کامپیوتر DEC PDP-11 پیاده‌سازی شد، و تکامل یافته‌ی زبان B می‌باشد. علت نامگذاری C نیز این بوده است که بعد از B طراحی شد. کن تامپسون (Ken Thompson) برخی از ویژگی‌های زبان BCPL را در زبان ابداعی خود یعنی B مدل‌سازی کرد و در سال ۱۹۷۰ از زبان B برای ایجاد نسخه‌های اولیه سیستم‌عامل یونیکس در آزمایشگاه بل بر روی یک کامپیوتر DEC PDP-7 استفاده نمود. اما زبان B نیز خود تکامل یافته‌ی زبان دیگری به نام BCPL می‌باشد که طراح آن مارتین ریچاردز (Martin Richards) در سال ۱۹۶۷ این زبان را برای نوشتن نرم افزارهای سیستم عامل و کامپایلر نوشته بود.

زبان‌هایی چون B و BCPL از دسته زبان‌های "بدون نوع" (Type less) هستند. به عبارت دیگر هر داده‌ای تنها یک "کلمه" (Word) از حافظه را اشغال می‌کرد و کار با آن داده مثلاً به عنوان یک عدد صحیح یا یک عدد اعشاری به عهده برنامه‌نویس است. اما از مهمترین ویژگی‌هایی که به زبان C افزوده شد "نظارت بر انواع داده" بود که از دلایل محبوبیت و گسترش آن نیز به‌شمار می‌آید. تا اواخر دهه ۱۹۷۰ میلادی زبان C تکامل یافت. زبان C را در ابتدا عمدتاً به عنوان زبان توسعه و پیاده‌سازی سیستم‌عامل UNIX می‌شناختند. استفاده گسترده از زبان C توسط انواع گوناگون کامپیوترها، موجب به وجود آمدن گونه‌های مختلفی از آن شد. اینگونه‌ها غالباً با یکدیگر ناسازگار بودند و این پدیده برای برنامه‌نویسانی که نیاز بود برنامه‌هایی بنویسند که بر روی انواع گوناگون کامپیوترها قابل اجرا باشند، مشکلی جدی بود. از این رو ضرورت وجود یک نسخه استاندارد C احساس می‌شد. در سال ۱۹۸۳ کمیته فنی X3J11 تحت نظر "کمیته استانداردهای ملی امریکا (ANSI)" در خصوص کامپیوترها و پردازش اطلاعات (X3) تشکیل شد تا یک تعریف واضح (نامبهم) و مستقل از سیستم کامپیوتری را برای این زبان ارائه نماید. بالاخره در سال ۱۹۸۹ این استاندارد مورد تصویب قرار گرفت. کمیته ANSI با همکاری سازمان استانداردهای بین‌المللی (ISO) زبان C را در سراسر جهان استاندارد کرد. مستندات استاندارد مشترک در سال ۱۹۹۰ تحت عنوان ANSI/ISO 9899:1990 انتشار یافت. چنانکه پیشتر گفته شد زبان C++ گسترش یافته زبان C است، که در اوایل دهه ۱۹۸۰ توسط بیارنه استراوسروپ (Bjarne Stroustrup) در آزمایشگاه بل ایجاد گردید. اما مهمترین عناصری که به زبان C اضافه شدند تا زبان C++ ایجاد شود، کلاس‌ها، اشیاء و در یک کلام برنامه‌نویسی شیء‌گرا^۲ بود. در نتیجه نام اولیه زبان C++، "C کلاس‌دار" بود. با وجود این، زبان C++ ویژگی‌های بسیار زیاد دیگری دارد، که از جمله آن می‌توان به روش بهبود یافته‌ای برای ورودی و خروجی (I/O) اشاره کرد. در شکل ۳-۱ رابطه بین زبان‌های C و C++ نشان داده شده است. در نوامبر ۱۹۹۷ استاندارد

1. Super set

2. Object Oriented Programming (OOP)

C++ ANSI/ISO عرضه شد. ++C استاندارد، ویژگی‌های جدیدی به این زبان اضافه کرده است که از میان آنها می‌توان کتابخانه قالب استاندارد (STL) را نام برد.



شکل ۳-۱: رابطه زبان‌های C و ++C

زبان ++C یک زبان سطح میانی است. اصولاً زبان‌های برنامه‌نویسی را به سه دسته زبان‌های سطح بالا، زبان‌های سطح میانی و زبان‌های سطح پایین تقسیم می‌کنند. علت سطح میانی بودن زبان ++C آن است که هم مانند زبان‌های سطح پایینی مثل اسمبلی دسترسی مستقیم به حافظه دارد و می‌توان با مفاهیمی چون بیت، بایت و آدرس‌های حافظه کارکرد، و هم اینکه دستورات این زبان همچون دستورات زبان‌های سطح بالایی مثل پاسکال به زبان محاوره‌ای انسان نزدیک است و خوانایی بالایی دارد.

زبان‌های برنامه‌نویسی از نگاه دیگری بر دو دسته ساخت یافته و غیرساخت یافته تقسیم می‌شوند. در جدول ۳-۱ نام

تعدادی زبان ساخت یافته و غیرساخت یافته را مشاهده می‌کنید.

زبان‌های ساخت یافته	زبان‌های غیرساخت یافته
پاسکال	فورترن
ادا	بیسیک
C, ++C	کوبول

جدول ۳-۱: زبان‌های برنامه‌نویسی از نظر ساخت یافتگی

با وجود آنکه C++ به خاطر ویژگی شی‌گرای خود محبوبیت یافته، ولی با توجه به نزدیکی آن به زبان C از برنامه‌نویسی ساخت یافته نیز به‌طور کامل پشتیبانی می‌کند. در بسیاری از کتب آموزش برنامه‌نویسی C++ بدون توجه به سطح معلومات مخاطبین مبتدی از ابتدای کار شروع به گفتن مفاهیم اساسی برنامه‌نویسی شی‌گرا می‌کنند؛ اما چگونه می‌توان به خواننده‌ای که هنوز تفاوت برنامه‌نویسی ساخت یافته و غیرساخت یافته نیز را نمی‌داند مفاهیم شی‌گرا را تدریس کرد؟

برنامه‌نویسان در زبان‌های برنامه‌نویسی ابتدایی، که غیرساخت یافته به شمار می‌آمدند با مشکلات متعددی در زمینه برنامه‌نویسی برخورد کردند. در دهه ۱۹۶۰ میلادی، تولید بسیاری از نرم‌افزارها با مشکل مواجه شد. زمان‌بندی تولید نرم‌افزار به تأخیر می‌افتاد، هزینه‌ها بالا بود و در نتیجه بودجه تولید نرم‌افزار افزایش می‌یافت و نرم‌افزارهای تولیدی نیز از قابلیت اعتماد بالایی برخوردار نبودند. تحقیقاتی که برای برطرف کردن این مشکلات به عمل آمد، منجر به برنامه‌نویسی ساخت یافته شد، و در نتیجه زبان‌های ساخت یافته‌ای چون پاسکال، C و فورترن ساخته شدند. در این زبان‌ها ویژگی‌هایی افزوده شد مثل پیمانه‌ای بودن (ماجولار یا رویه‌ای بودن) تا برنامه‌نویسی ساخت یافته در این زبان‌ها امکانپذیر شد. در برنامه‌نویسی ساخت یافته از ایده "تفرقه بینداز و حکومت کن" استفاده شده است. به‌طوری که یک برنامه به‌صورت مجموعه‌ای از فعالیت‌ها تصور می‌شود که باید بر روی داده‌ها انجام گیرد. بنابراین در این نوع برنامه‌نویسی برای هر فعالیت یک تابع (رویه، ماجول) نوشته می‌شود، و سپس برای اجرای کامل برنامه باید به ترتیب توابع موردنظر را بر روی داده‌ها اجرا کرد. به عبارت دیگر برای حل یک مسئله آن را به مسائل کوچکتری تقسیم می‌کنیم و این روند را آنقدر ادامه می‌دهیم تا زیر مسئله‌ها قابل حل گردند. اما زبان‌های ساخت یافته نیز خالی از اشکال نبودند و مشکلات خاص خود را داشتند که به مرور به آنها اشاره خواهیم کرد تا به روند تکاملی زبان‌های برنامه‌نویسی واقف گردیم. برای برطرف کردن مشکلات مربوط به زبان‌های ساخت یافته نیز تحقیقات وسیعی صورت گرفت تا آنکه زبان‌های برنامه‌نویسی شی‌گرا ابداع شدند. برخی از زبان‌هایی که در این زمینه ساخته شدند مثل زبان اسمالتاک (Smalltalk) یک زبان برنامه‌نویسی کاملاً شی‌گرا (شی‌گرای محض) بودند. به عبارت دیگر در اینگونه زبان‌ها جزء به جزء شی‌گرا به روش دیگری نمی‌توان برنامه‌نویسی کرد. آموزش اینگونه زبان‌ها برای افرادی پیشنهاد می‌شود که از قبل با یک زبان ساخت یافته آشنایی داشته باشند. اما برای آموزش اصولی افراد مبتدی راهی جزء این نیست که ابتدا با برنامه‌نویسی ساخت یافته آشنا شوند و سپس برنامه‌نویسی شی‌گرا به آنها آموزش داده شود.

از طرفی به علت اینکه اشیاء و کلاس‌هایی که در برنامه‌های شی‌گرا ساخته می‌شوند، خود به‌عنوان بخشی از برنامه‌نویسی ساخت یافته به شمار می‌روند در جلد نخست این مجموعه تنها به برنامه‌نویسی ساخت یافته پرداخته شده است و در جلد دوم به سراغ سیستم‌های شی‌گرا خواهیم رفت. لذا در این فصل بیش از این در خصوص ویژگی‌های شی‌گرایی زبان C++ صحبت نخواهیم کرد. در فصل‌های آتی نیز تنها در خصوص ویژگی‌های برنامه‌نویسی ساخت یافته و تفاوت‌های آن با برنامه‌نویسی غیرساخت یافته به تفصیل بحث خواهیم کرد. لذا در حال حاضر نگران چگونگی ایجاد برنامه ساخت یافته و عدم آشنایی با ویژگی‌های آن نباشید. شایان ذکر است که مطالب فصول برنامه‌نویسی با این فرض تنظیم شده است که خواننده هیچگونه آشنایی قبلی با هیچ یک از زبان‌های برنامه‌نویسی ندارند، لذا سعی بر آن داریم که مثال‌های برنامه‌نویسی از هیچگونه پیچیدگی خاصی برخوردار نباشد. و این بدان علت است که از کیفیت آموزش کاسته

نشود. اما برای حفظ ارزش علمی کتاب برخی مثال‌ها و تمرینات پیچیده و ارزشمند را برای تکمیل بحث در انتهای فصول برنامه‌نویسی تحت عنوان "مورد مطالعاتی" آورده‌ایم. اما یک سؤال مهم همواره برای افرادی که به دنبال یادگیری یک زبان شی‌گرا هستند وجود دارد و آن اینکه:

از بین دو زبان شی‌گرای مطرح یعنی جاوا و ++C کدام یک ارزش بیشتری برای یادگیری دارد؟ و یا در اولویت قرار دارد؟

در جواب این دوستان باید گفت: از میان زبان‌های برنامه‌نویسی شی‌گرا، زبان ++C پر استفاده‌ترین زبان شی‌گرا محسوب می‌شوند. چرا که زبانی مثل جاوا فاقد بعضی از ویژگی‌های سطح پایین ++C مثل اشاره‌گرها است که از قدرت آن می‌کاهد و انعطاف‌پذیری آن نسبت به ++C کمتر است. اما با توجه به آنکه زبان‌هایی چون جاوا، C#، J# و امثال آن روزبه‌روز بر کاربریشان افزوده می‌شود در جلد دوم این مجموعه صفحاتی را به زبان C# اختصاص داده‌ایم، اما در این کتاب تنها به زبان‌های ++C و C خواهیم پرداخت. خوشبختانه برای یادگیری زبان ++C نیازی نیست که ابتدا زبان C را یاد بگیرید و برخی دستورات C تنها به این جهت مطرح شده است که ارزش دستورات آسان و کاربردی زبان ++C روشن گردد. اما دانستن زبان ++C در فراگیری هر چه بهتر زبان‌های شی‌گرای جاوا و C# بسیار مؤثر است.

به هر حال آنچه که در این کتاب در خصوص زبان ++C مطرح شده مطابق با زبان ++C استاندارد است. گرچه برخی شرکت‌های تولیدکننده کامپایلر ++C چون مایکروسافت و بورلند ویژگی‌هایی را به دلخواه خود به این زبان اضافه کرده‌اند اما هدف ما بیشتر آشنایی شما با قوانین استاندارد ++C است تا بتوانید در کامپایلرهای این زبان در سیستم‌عامل لینوکس نیز به راحتی و تغییراتی جزئی برنامه‌نویسی کنید. برای این منظور یک ضمیمه در خصوص نحوه برنامه‌نویسی به زبان ++C در سیستم عامل لینوکس در انتهای کتاب آورده شده است.

برای اجرای کدهای این کتاب نیازمند یک کامپایلر قابل اعتماد و دارای ویژگی‌های روز دنیا مثل امکانات دیداری (ویژوالی) هستید. چرا که در جلد سوم تصمیم داریم برنامه‌نویسی رویدادگرا را نیز آموزش دهیم لذا بهتر است با کامپایلری که دارای این ویژگی‌ها باشد کار را شروع کنید. برای این منظور در این کتاب تمامی برنامه‌ها با توجه به ویژگی‌های کامپایلر Microsoft Visual Studio 6.0 نوشته شده است. لذا ابتدا با مراجعه به ضمیمه شماره یک مطابق دستورالعمل ارائه شده این کامپایلر را به واسطه لوح‌های فشرده ۲ و ۳ و ۴ از مجموعه لوح‌های فشرده همراه کتاب^۱ نصب کنید. شاید بسیاری بر این اعتقاد باشند که بهتر بود کامپایلر جدیدتری مثل کامپایلر ۲۰۰۵ شرکت مایکروسافت را برای آموزش انتخاب می‌کردیم. اما به دو دلیل عمده از این کار خودداری نمودیم. اول آن که شرکت مایکروسافت ویژگی‌هایی به کامپایلرهای NET. خود افزوده که از زبان ++C استاندارد کمی فاصله گرفته است. دوم آن که برنامه‌هایی که در این کامپایلرها نوشته می‌شود برای اجرای، نیازمند وجود پلت‌فرم NET. بر روی کامپیوتر مقصد هستند که بدون شک بر روی ویندوزهای قدیمی مثل ویندوز ۹۸ این پلت‌فرم قابل نصب نیست. لذا از آن جا که امکان دارد شما روزی مجبور به کار کردن با کامپیوترهای قدیمی ارزان قیمت و به طبع با سیستم عامل‌های قدیمی شوید، مثل زمانی که بخواهید با یک ۸۰۸۶ یک خط تولید ساده را کنترل کنید، لزوم فراگیری کار با VS98 به خوبی روشن می‌شود.

۱. لوح فشرده شماره ۱ همراه کتاب است. اما لوح‌های فشرده ۲ تا ۴ به صورت اختیاری می‌باشد که شما می‌توانید جهت تهیه آنها به وب سایت کتاب به آدرس www.programmingbook.4t.com مراجعه فرمایید.

۲-۳: کار با انواع داده‌ها در C++

انواع داده در C++

چنانکه در فصل گذشته دیدید در الگوریتم‌ها هر جا لازم بود متغیری تعریف می‌کردیم و مقادیر صحیح، اعشاری، رشته‌ای و... را می‌توانستیم به آن نسبت دهیم. اما برای کامپیوترهای واقعی چنین امری ممکن نیست. یعنی باید نوع متغیر از نظر اینکه چگونه اطلاعاتی را می‌تواند در خود ذخیره کند مشخص باشد. لذا هنگامی که می‌خواهیم متغیری را تعریف کنیم باید قبل از نام آن متغیر، نوع متغیر را مشخص سازیم. در C++ انواع متغیرهایی که در جدول ۲-۳ آمده امکانپذیر است. به توضیحاتی که در خصوص جدول آمده خوب توجه کنید.

نوع داده	اسامی دیگر (انواع مشابه)	اندازه به بیت	بازه قابل قبول
int	signed , signed int	۱۶ یا ۳۲	بستگی به نوع سیستم عامل دارد
unsigned int	unsigned	۱۶ یا ۳۲	بستگی به نوع سیستم عامل دارد
__int8	char , unsigned char	۸	از ۱۲۸- تا ۱۲۷
__int16	Short , short int , signed short int	۱۶	از ۳۲۷۶۸- تا ۳۲۷۶۷
__int32	signed , signed int	۳۲	از ۲۱۴۷۴۸۳۶۴۷- تا ۲۱۴۷۴۸۳۶۴۸
__int64	ندارد	۶۴	از ۹۲۲۳۳۷۲۰۰۳۶۸۵۴۷۷۵۸۰۸- تا ۹۲۲۳۳۷۲۰۰۳۶۸۵۴۷۷۵۸۰۷
char	signed char	۸	از ۱۲۸- تا ۱۲۷
unsigned char	ندارد	۸	از ۰ تا ۲۵۵
short	short int , signed short int	۱۶	از ۳۲۷۶۸- تا ۳۲۷۶۷
unsigned short	unsigned short int	۱۶	از ۰ تا ۶۵۵۳۵
long	Long int , signed long int	۳۲	از ۲۱۴۷۴۸۳۶۴۷- تا ۲۱۴۷۴۸۳۶۴۸
unsigned long	unsigned long int	۳۲	از ۰ تا ۴۲۹۴۹۶۷۲۹۵
float	ندارد (دقت ۷ رقم اعشار)	۳۲	از $3/4 \times 10^{-38}$ تا $3/4 \times 10^{38}$
double	ندارد (دقت ۱۵ رقم اعشار)	۶۴	از $1/7 \times 10^{-308}$ تا $1/7 \times 10^{308}$
long double	ندارد (دقت تا ۱۹ رقم اعشار)	۸۰	از $1/2 \times 10^{-4932}$ تا $1/2 \times 10^{4932}$
bool	ندارد	۸	صفر یا یک

جدول ۲-۳: انواع داده‌ها و مقادیر قابل قبول آنها

در C++ استاندارد، چهار نوع داده اصلی وجود دارد که عبارتند از: int ، char ، float ، double. در ادامه به

بررسی انواع داده‌های اصلی و معادل‌های آنها مطابق با جدول فوق می‌پردازیم.

انواع داده‌ی اصلی در C++ استاندارد

۱. نوع داده صحیح: از آنجایی که می‌توان رقم صفر را به‌صورت عدم وجود جریان برق در یک خانه حافظه و رقم یک را به‌صورت وجود جریان برق در یک خانه از حافظه نشان داد، در کامپیوتر جهت ذخیره عدد آن را به مبنای دو می‌برند تا به یک رشته شامل صفر و یک تبدیل شود. در این صورت می‌توان نمایش دودویی عدد را که به‌صورت رشته‌ای از صفرها و یک‌ها است به راحتی ذخیره ساخت. چنانکه دیدید در الگوریتم‌نویسی در بیشتر موارد با اعداد صحیح سروکار داریم. به‌عنوان مثال شمارنده‌های حلقه‌های تکرار و یا متغیرهایی که معرف تعداد یک چیز هستند مثل تعداد دانش‌آموزان یک کلاس و هزاران نمونه دیگر از کاربردهای اعداد صحیح در الگوریتم‌نویسی به شمار می‌آیند. برای کارکردن با اعداد صحیح در C++ باید متغیر مورد نظر را از نوع `int` تعریف کنید.

چنانکه در جدول ۳-۲ نیز بیان شده طول نوع `int` بستگی به سیستم عامل دارد. در سیستم‌های ۳۲ بیتی مثل ویندوز ۹۸ و بعد از آن طول این نوع برابر ۳۲ بیت و دارای ظرفیتی از ۲,۱۴۷,۴۸۳,۶۴۸- تا ۲,۱۴۷,۴۸۳,۶۴۷ می‌باشد. اما در سیستم‌های ۱۶ بیتی مثل MS-DOS طول این نوع ۱۶ بیت بوده و محدوده‌ای از ۳۲,۷۶۸- تا ۳۲,۷۶۷ را شامل می‌شود. همان‌طور که می‌بینید اعداد صحیح در کامپیوتر مانند آنچه که در ریاضیات فرض می‌کردیم [که شامل هیچ حد و مرزی نبوده و از $-\infty$ تا $+\infty$ را شامل می‌شود] نیستند. چرا که خانه‌های حافظه کامپیوتر محدود بوده و با تخصیص ۴ بایت برای نوع `int` این بازه از اعداد را می‌توان در این نوع ذخیره کرد.

۲. نوع داده کاراکتری: مسلماً همیشه با ذخیره کردن اعداد روبرو نیستیم و گاهی اوقات پیش می‌آید که بخواهیم مثلاً حروف الفبای لاتین مثل حروف `a, b, c` و... یا علائمی مثل `[, ], $, (, )` و... را در کامپیوتر ذخیره کنیم. اما با توجه به آنکه در خانه‌های حافظه کامپیوتر فقط می‌توان صفر و یک ذخیره کرد چه راه‌حلی برای ذخیره این علائم و حروف به نظر شما می‌رسد؟

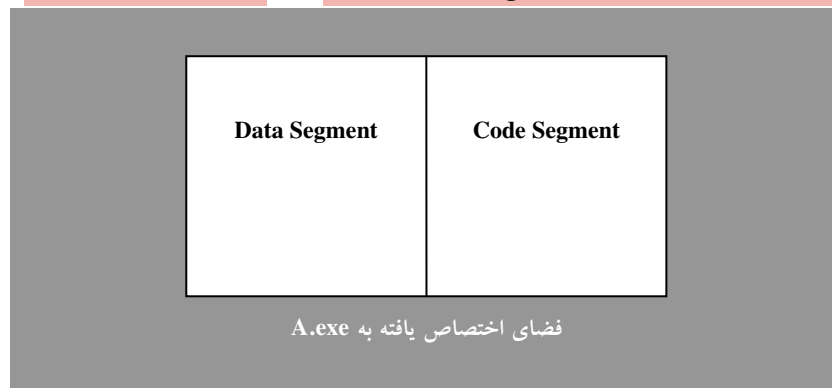
در ابتدا مشاهده شد که در یک بایت می‌توان از صفر تا ۲۵۵ را در حالت بدون علامت، و یا از ۱۲۸- تا ۱۲۷ را در حالت با علامت، ذخیره کرد. یعنی ۲۵۶ عدد مختلف، که می‌توانست به‌عنوان نماینده ۲۵۶ نماد مختلف در نظر گرفته شود. لذا در جدولی با نام جدول `ASCII Code` برای هر یک از حروف و علائمی که روی صفحه کلید موجود بودند و همچنین برخی علائمی که روی صفحه کلید موجود نیستند ولی برای اموری مثل کادر بندی و امثال آن مورد استفاده قرار می‌گرفت، یک عدد بین ۱۲۸- تا ۱۲۷ را در نظر گرفتند، مثلاً برای حرف `A`، عدد ۶۵ را در نظر گرفتند. حال اگر در یک بایت عدد ۶۵ را ذخیره می‌کردند، هنگام نمایش آن بایت به جای عدد ۶۵ حرف `A` نمایش داده می‌شد. اما در محاسبات و ذخیره‌سازی عدد ۶۵ به جای حرف `A` مورد استفاده قرار می‌گرفت. برای ذخیره کاراکترها باید متغیری را از نوع `char` تعریف کنید.

کار در کلاس ۳-۱:

اما یک مشکل هنوز حل نشده باقی ماند! کامپیوتر از کجا بفهمد که عدد ۶۵ ذخیره شده در یک بایت مربوط به کاراکتر 'A' است و یا خود عدد ۶۵ به‌صورت `int` است؟ در کلاس خود در این باره بحث کنید.

در جواب سؤال مطرح شده در کار در کلاس ۳-۱ می‌توان گفت:

در زمانی که می‌خواهیم یک فایل اجرایی (exe) را بروی کامپیوتر اجرا کنیم، سیستم عامل ابتدا مقداری از فضای آزاد حافظه (RAM) را به میزان مورد نیاز برای اجرای برنامه به آن اختصاص می‌دهد، مثلاً فرض کنید از کل فضای RAM که در شکل ۳-۲ نشان داده شده فضای مربع سفید رنگ به برنامه A.exe اختصاص داده شده است.



شکل ۳-۲: تقسیم بندی فضای حافظه

حال از فضای اختصاص یافته به برنامه یک سگمنت جهت داده‌ها و یک سگمنت جهت کد برنامه اختصاص داده می‌شود. مقصود از سگمنت داده‌ها فضایی از RAM است که متغیرها و داده‌های مربوط به برنامه‌ی A.exe را در خود ذخیره می‌سازد و مقصود از سگمنت کد آن قسمتی از RAM است که دستورات اجرایی برنامه در آن قرار می‌گیرد مثل دستورات انتساب، دستورات شرطی و... البته سگمنت‌های دیگری هم می‌تواند وجود داشته باشد که از بحث ما خارج است. سگمنت کد فعلاً مدنظر ما قرار ندارد، اما سگمنت داده‌ها به نوبه خود به قسمت‌های مختلفی تقسیم می‌شود؛ مثلاً یک بخش آن جهت داده‌های نوع صحیح (int) و بخش دیگری جهت داده‌های نوع کاراکتر (char) و ... تقسیم می‌شود. هنگامی که از کامپیوتر می‌خواهیم به متغیری از نوع char مراجعه کند [که در آن عدد ۶۵ را به عنوان نماد حرف A از قبل ذخیره کرده‌ایم] و محتویات آن را نمایش دهد، اساساً محل ذخیره کاراکترها با انواع دیگری مثل نوع int به کلی مجزا است، بنابراین کامپیوتر به راحتی تشخیص می‌دهد که این متغیر از نوع کاراکتر است نه نوع صحیح و عدد ۶۵ ذخیره شده در آن معرف کاراکتر A می‌باشد و نه خود عدد ۶۵. لذا در هنگام نمایش این محل از حافظه، حرف A نمایش داده می‌شود و نه عدد ۶۵. از طرفی عدد ۶۵ که در متغیری از نوع صحیح ذخیره می‌گردد در ۴ بایت ذخیره می‌شود ولی همین عدد وقتی در متغیری از نوع کاراکتر ذخیره شود تنها در ۱ بایت ذخیره می‌گردد. این نوع تقسیم‌بندی سگمنت داده‌ها در مباحثی مثل آرایه‌ها بسیار حائز اهمیت می‌باشد که در فصل آرایه‌ها و رشته‌ها به آن اشاره خواهیم کرد.

اما این تقسیم‌بندی ذاتاً توسط سخت افزار صورت نمی‌گیرد، بلکه این کامپایلر زبان C++ است که حافظه‌ی تحت اختیار برنامه را این چنین تقسیم بندی می‌کند. چنان که اگر شما خود کدهای اسمبلی برنامه‌ای را بنویسید می‌توانید سگمنت داده را بدین صورت تقسیم بندی نکنید و تمامی داده‌ها را بدون نظم در حافظه ذخیره کنید در آن حالت این برنامه نویس است که باید نوع داده و محتویات آن را تفسیر کند.

اما در برخی سیستم‌های کامپیوتری به جای آنکه در نوع `char` از ۱۲۸- تا ۱۲۷ را ذخیره کنند از صفر تا ۲۵۵ را ذخیره می‌سازند که در عمل تفاوت چندانی با حالت قبل ندارد. متأسفانه کاراکترهای بین ۱۲۸ تا ۲۵۵ استاندارد نبوده و مشکلات زمانی رخ می‌دهد که برنامه‌ها بخواهند بین سیستم‌های مختلف کامپیوتری جابه‌جا شوند. جدول کدهای اسکی استاندارد در پیوست ۲ آمده است. با مراجعه به این جدول، معادل اسکی کد هر یک از حروف لاتین، علائم و کلیدهای ترکیبی روی صفحه کلید و برخی کاراکترهای گرافیکی را مشاهده کنید. اما حتماً با مشاهده این جدول می‌پرسید حروف فارسی و دیگر زبان‌های غیرانگلیسی چگونه در کامپیوتر ذخیره می‌شوند؟ پس از گسترش کامپیوتر در سراسر جهان و احساس نیاز به ذخیره حروف دیگر زبان‌های طبیعی، به این نتیجه رسیدند که به جای استاندارد ASCII Code از استاندارد UNI Code استفاده کنند که در آن از دو بایت برای ذخیره سازی حروف زبان‌های مختلف و علائم گوناگون از جمله علائم ریاضی و... استفاده شد. لذا در C++ استاندارد، نوع وسیع تری نسبت به نوع `char` به نام `wchar_t` در نظر گرفته شده که برای نوشتن برنامه‌هایی با توزیع جهانی در نظر گرفته شده است. اما از آنجایی که فعلاً جدول اسکی کد نیاز ما را برای برنامه‌نویسی‌های مقدماتی برطرف می‌سازد و همچنین در حالت کنسول که ما در آن برنامه‌نویسی می‌کنیم از UNI Code پشتیبانی نمی‌شود امکان استفاده از این قابلیت `wchar_t` فعلاً برای ما وجود ندارد، و لذا مطالعه بیشتر در خصوص جدول UNI Code و نوع `wchar_t` را به خوانندگان عزیز محول می‌سازیم، البته در برنامه‌های کاربردی ویندوز امکان استفاده از `wchar_t` هست.

۳. انواع داده‌های اعشاری: نوع داده `float` جهت ذخیره‌سازی اعداد اعشاری ممیز شناور تا هفت رقم اعشار دقت مورد استفاده قرار می‌گیرد. اما مفهوم هفت رقم اعشار دقت چیست؟! آیا به معنای هفت رقم پس از ممیز اعشار است؟! مسلماً خیر. چراکه اگر چنین تعبیری صحیح بود هیچگاه نمی‌توانستیم عددی مثل $10^{-38} \times 3/4$ را در متغیری از این نوع ذخیره کنیم، زیرا تعداد ارقام اعشار این عدد ۳۹ رقم است. قبل از این که به رفع این ابهام بپردازیم باید دو مقوله نمایش اعداد به صورت نماد علمی و ممیز شناور را بررسی کنیم.

هنگامی که از سیستم ممیز شناور برای نمایش اعداد استفاده می‌کنیم بدین مفهوم است که تعداد ارقام معنی‌دار عدد ثابت است. به عنوان مثال اعداد زیر هر کدام دارای چهار رقم معنی‌دار می‌باشند:

$$10^2 \times 84/56, 0/0008540, 10^{-3} \times 5/623, 0/2000, -0/03$$

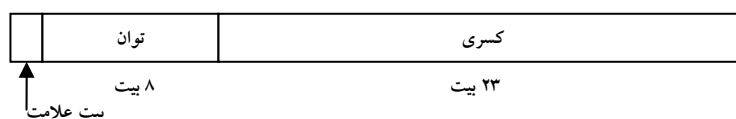
اما مفهوم هفت رقم معنی‌دار چیست؟ به تعداد هفت رقم، به غیر از صفرهای موجود قبل از اولین رقم ناصفر، هفت رقم معنی‌دار گفته می‌شود. به عنوان مثال عددی مثل $0/5623789$ دارای هفت رقم با معنی است. همچنین عدد $0/00052$ نیز دارای هفت رقم معنی‌دار است، زیرا می‌توان این عدد را چنین نوشت $10^{-3} \times 0/5200000$ ، و یا عدد $85/0023$ نیز چنین است. مسلماً متوجه شده‌اید که صفرهای سمت راست آخرین رقم ناصفر اعشاری نیز جزء رقم‌های معنی‌دار شمارش می‌شوند. اما عددی مثل $231/987564$ حداقل دارای ۹ رقم معنی‌دار می‌باشد. آیا می‌تواند ۱۰ رقم معنی‌دار نیز داشته باشد؟

اما بهترین روش برای نمایش اعداد ممیز شناور استفاده از نماد علمی است. در روش نماد علمی عدد به دو بخش تقسیم می‌شود. یک قسمت کسری یا مانیتس (شامل ارقام با معنی) و یک قسمت نیز حاوی عدد صحیحی است که توانی از ۱۰ می‌باشد. قسمت کسری عددی است بین صفر و ده، و قسمت توان یک عدد صحیح است. با نمایش عدد به صورت

نماد علمی به راحتی می‌توان تعداد ارقام معنی دار عدد را شمرد. به عنوان مثال اعداد زیر به صورت نماد علمی نمایش داده شده‌اند:

$$9/235000 \times 10^{32} , -2/365471 \times 10^{-24}$$

حال که با سیستم نمایش اعداد به صورت ممیز شناور در قالب نماد علمی آشنا شدید به راحتی می‌توانید بفهمید که اعداد ممیز شناور چگونه در ۴ بایت یک متغیر float ذخیره می‌شود، هر متغیر float دارای ساختاری مشابه ساختار زیر است:



شکل ۳-۳: شکل یک متغیر float

مسلماً با توجه به شکل ۳-۳ می‌توانید حدت بزنید، که یک عدد اعشاری ممیز شناور چگونه در یک متغیر از نوع float ذخیره می‌گردد. حال به نظر شما آیا عددی مثل ۰/۳۲۵۶۴۱۷۸ که ارقام اعشاری کمتری نسبت به حد پایین نوع float یعنی $10^{-38} \times 3/4$ دارد را می‌توان در متغیری از نوع float ذخیره کرد.

۴. نوع داده double: جهت ذخیره‌سازی اعداد اعشاری بزرگتری که در نوع float جای نمی‌گیرند مورد استفاده قرار می‌گیرد. نوع داده double که به ممیز شناور دقت مضاعف نیز موسوم است، اعداد اعشاری تا ۱۵ رقم معنی‌دار را می‌تواند در خود ذخیره کند.

۵. نوع داده منطقی: بسیاری از اوقات نیازمند ذخیره‌سازی دو مقدار یک و صفر به ترتیب به عنوان true و false هستیم. برای این منظور نوع داده bool در نظر گرفته شده است. در عمل برای ذخیره نوع داده bool به یک بیت از حافظه نیاز است اما در عمل، بسته به نوع کامپایلر، این نوع داده را به صورت اعداد صحیح در چهار بایت و یا در یک بایت ذخیره می‌کند. زیرا به اعداد صحیح به سرعت می‌توان دسترسی پیدا کرد، حال آن که یک بیت از یک عدد صحیح باید استخراج شود که نیازمند صرف زمان بیشتری است. بعداً خواهید دید که از نوع داده bool بیشتر برای ذخیره‌سازی نتایج عملیات منطقی (مقایسه‌ای یا شرطی) استفاده می‌شود و به عبارت دیگر می‌توان نتایج گزاره‌ها را در نوع داده منطقی ذخیره‌سازی کرد. توجه داشته باشید که این نوع جزء انواع استاندارد ++C نیست و شرکت‌های تولیدکننده کامپایلر ++C برحسب سلیقه و نیاز کاربران، این نوع را در کامپایلرهای خود ممکن است افزوده باشند ولی در هر صورت در کامپایلر مورد بحث این کتاب یعنی VC6 این نوع به خوبی پشتیبانی می‌شود.

در ادامه به بررسی انواع داده فرعی در ++C استاندارد و برخی از انواع که مختص کامپایلرهای شرکت مایکروسافت مثل VC6 هستند می‌پردازیم.

انواع داده فرعی در C++ استاندارد

با استفاده از کلماتی مثل **signed** (با علامت)، **unsigned** (بدون علامت)، **long** و **short** می‌توان انواع جدیدی را ایجاد کرد. این کلمات را می‌توان با انواع داده‌های اصلی به کار برد. مثلاً نوع **unsigned int** با حذف بیت علامت از نوع داده **int** قادر است بازه بزرگتری را در محدوده اعداد مثبت یعنی بین صفر تا ۴,۲۹۴,۹۶۷,۲۹۵ را در چهار بایت ذخیره سازد. اما از آنجایی که بسیاری از انواع داده مثل **int** از نوع علامت‌دار هستند به کار بردن کلمه **signed** در جلوی این انواع داده‌ای بی‌معناست. در کامپیوترهایی که در نوع **char** اعداد صفر تا ۲۵۵ را ذخیره می‌سازند به کارگیری کلمه **signed** در جلوی نوع **char** می‌تواند بازه قابل قبول این نوع را به ۱۲۸- تا ۱۲۷ تغییر دهد.

چنانکه پیشتر گفته شد طول نوع داده **int** بسته به ۱۶ یا ۳۲ بیتی بودن سیستم، متغیر است. به کارگیری کلمه **long** تضمین کننده ۳۲ بیتی بودن نوع صحیح و به کارگیری کلمه **short** تضمین کننده ۱۶ بیتی بودن نوع صحیح و به نوعی صرفه جویی در حافظه است. همچنین نوع داده **long double** جهت ذخیره سازی اعداد اعشاری با دقتی تا ۱۹ رقم اعشار است. البته مطابق جدول ۳-۲ تفاوتی بین کلمه **short** و نوع **short int** نیست و شما می‌توانید در به کارگیری انواع، از اسامی دیگر آنها به راحتی استفاده کنید. همچنین از حالت‌های مختلف نوع داده **char** می‌توان برای ذخیره اعدادی هم که در بازه ۱۲۸- تا ۲۵۵ قرار دارند نیز استفاده کرد، البته نحوه بکارگیری این نوع برای ذخیره سازی اعداد در فصول آتی بیان خواهد شد.

برای نوع صحیح در کامپایلرهای شرکت مایکروسافت انواعی تهیه شده است که اندازه هر کدام از این انواع بر حسب بیت به دنباله آن نوع ذکر شده است. و هدف از تهیه این انواع صرفه جویی در حافظه و همچنین پشتیبانی کردن از محدوده‌های بزرگتر نسبت به بازه قابل قبول **long int** می‌باشد. اصولاً در دیگر کامپایلرها نیز چنین انواعی پیش‌بینی شده و به کارگیری آنها به شدت توصیه می‌شود. این انواع که در ابتدای نام آنها از دو کاراکتر زیرواژه ' () استفاده شده است عبارتند از: **__int64** , **__int32** , **__int16** , **__int8** ، جهت اطلاعات بیشتر در خصوص بازه قابل قبول این انواع به جدول ۳-۲ مراجعه کنید.

دامنه اشغال شده توسط انواع داده‌ای مختلف C++، در سرفایل **limits.h** ارائه شده است. برای مشاهده محتویات این سرفایل می‌توانید به پیوست شماره ۳ مراجعه کنید. همچنین اگر بر فرض کامپایلر VC6 خود را در درایو C خود نصب کرده باشید می‌توانید به آدرس زیر مراجعه کنید و سرفایل **limits.h** را به واسطه ویرایشگر VC6 یا هر ویرایشگر دیگری باز کنید و محتویات آن را مشاهده کنید.

C:\Program Files\Microsoft Visual Studio\VC98\Include

دیگر سرفایل‌های C++ نیز در همین آدرس قرار دارند.

نامگذاری متغیرها

با قوانین نامگذاری متغیرها در فصل دوم آشنا شدید. در C++ نیز از همان قوانین برای نامگذاری متغیرها استفاده می‌شود، جز در دو مورد استثنائی که در زیر آمده است:

۱. نام متغیرها هر تعداد کاراکتری که می‌خواهید می‌تواند باشد اما در VC6 تنها ۲۴۷ کاراکتر اول و یا در بورلند C++

۲۵۰ کاراکتر اول نام متغیر مورد استفاده قرار می‌گیرد.

۲. در نامگذاری متغیرها نمی‌توان کلمات رزروی زبان C++ را به‌عنوان نام متغیر انتخاب کرد. لیست کلمات رزروی زبان C++ در جدول ۳-۳ آمده است. البته هیچ نیازی نیست که این جدول را حفظ کنید چرا که به محض تایپ شدن اکثر کلمات رزروی C++ در محیط ویرایشگر VC6، ویرایشگر این کلمات را شناخته و آنها را به رنگ آبی در می‌آورد که این خود نشانه خوبی برای تشخیص رزروی بودن یک کلمه است و خود مبین آن است که نباید این کلمه را به‌عنوان نام متغیر به کار برد.

align	enum	private	typedef	__finally
allocate	explicit	property	typeid	__forceinline
auto	extern	protected	typename	__inline
bool	false	public	Union	__int16
break	float	register	unsigned	__int32
case	for	reinterpret_cast	using declaration	__int64
catch	friend	return	using directive	__int8
char	goto	selectany	uuid	__interface
class	if	short	Virtual	__leave
const	inline	signed	Void	__multiple_inheritance
const_cast	int	sizeof	Volatile	__noop
continue	long	static	wchar_t	__pragma
default	mutable	static_cast	While	__ptr64
delete	naked	struct	__alignof	__sealed
deprecated	namespace	switch	__asm	__single_inheritance
dllexport	new	template	__assume	__stdcall
dllimport	noinline	this	__based	__super
do	noreturn	thread	__cdecl	__try / __except / __finally
double	nothrow	throw	__declspec	__unaligned
dynamic_cast	novtable	true	__expect	__uuidof
else	operator	try	__fastcall	__virtual_inheritance

جدول ۳-۳: لیست کلمات رزروی Microsoft Visual C++ 6.0

کلماتی که در جدول فوق با کاراکتر زیرواژه آغاز شده‌اند مخصوص مایکروسافت می‌باشند.

تعریف متغیرها

برای اعلان یک متغیر تنها کافی است نوع داده و سپس نام متغیر را ذکر کنید.

مثال ۳-۱: در مثال زیر تعدادی متغیر تعریف شده است.

```
int      x;
float    y, z, f;
unsigned long int  min_mark;
unsigned __int32   sq;
```

تذکره ۱: در پایان همه دستورات C++ باید یک علامت سمیکالن (;) به عنوان کاراکتر نشان‌دهنده پایان خط قرار دهید.

تذکره ۲: در تعریف متغیرها می‌توان با بکارگیری عملگر کاما (,) هم زمان بیش از یک متغیر را تعریف کرد. چنانکه در مثال فوق هم‌زمان دو متغیر y و z را به عنوان متغیرهایی از نوع float تعریف کرده‌ایم.

مقدار دهی اولیه به متغیرها

هنگامی که متغیری را تعریف می‌کنید، فضایی از سگمنت داده‌ها، به آن نسبت داده می‌شود. اما از آنجایی که امکان دارد بیت‌های آن بخشی از حافظه که به این متغیر اختصاص داده شده، از قبل دارای هر آرایشی از صفر و یک باشد، در صورت استفاده از این متغیر در برنامه بدون مقداردهی اولیه، ممکن است با خطای زمان اجرا برخورد کنید.^۱ لذا بهتر است در هنگام تعریف یک متغیر به آن مقدار اولیه بدهید، تا از خطای مذکور جلوگیری کنید. برای مقداردهی به متغیرها سه روش وجود دارد:

۱. در زمان تعریف متغیر، که به آن مقداردهی اولیه (Initialization) گفته می‌شود.

۲. پس از تعریف با دستور انتساب که به آن مقداردهی کردن (Assignment) گفته می‌شود.

۳. به وسیله دستورات ورودی

چنانکه ذکر رفت در هنگام تعریف یک متغیر می‌توان فوراً به آن مقدار اولیه داد. به عنوان مثال دستور زیر را در نظر بگیرید:

```
int      X, Z=0;
char     ch = 'h';
```

در مثال فوق به متغیر Z مقدار اولیه‌ای برابر صفر داده شده و متغیر X بدون مقدار اولیه دهی رها شده است. توجه کنید که برای به دست آوردن کد اسکی حرف انگلیسی h اطراف آن را با علامت ' محصور کرده‌ایم. همچنین می‌توان در هر کجای برنامه با دستور انتساب یک متغیر را به هر مقدار دلخواهی مقداردهی کرد. (که شامل مقداردهی اولیه نمی‌شود هرچند که آن متغیر پیشتر در هنگام تعریف مقدار دهی نشده باشد). دستور انتساب در الگوریتم‌ها به صورت ← به کار می‌رفت. اما در C++ به وسیله علامت (=) عمل انتساب صورت می‌گیرد و چنانکه در بخش ۳-۳ خواهید دید برای عملگر ریاضی تساوی از نماد (=) استفاده می‌شود.

۱. در کاردرکلاس ۳-۳ با ذکر مثالی این مطلب را بیشتر بررسی خواهیم کرد.

مثال ۳-۲: در مثال زیر متغیرهایی را که در مثال ۳-۱ تعریف کردیم مقداردهی می‌کنیم.

```
Sq = 204;
min_mark = Sq;
y = z = 35.21459f;
f = 3.25e-23;
```

توضیح مثال: در مثال فوق می‌بینید که متغیر Sq را با مقدار ثابت ۲۰۴ مقداردهی کرده‌ایم. و سپس متغیر min_mark را به وسیله متغیر Sq مقداردهی کرده‌ایم یعنی هر دو متغیر مذکور دارای مقدار مشابه‌ای شده‌اند. همچنین در خط سوم هم زمان دو متغیر y و z را مقداردهی کرده‌ایم، به طوری که ابتدا عدد اعشاری 35.21459 در متغیر z قرار می‌گیرد و سپس متغیر y به واسطه z مقداردهی می‌گردد. و اما در خط چهارم مقدار متغیر f با یک عدد علمی معادل 3.25×10^{-23} مقداردهی شده است. توجه کنید که کاراکتر f در انتهای خط سوم معرف نوع float است و هیچ ارتباطی با متغیر f تعریف شده در برنامه ندارد.

روش سومی که به وسیله آن می‌توان متغیرها را مقداردهی کرد، به کارگیری دستورات ورودی برای این منظور است. چنانکه در کارنما به واسطه جعبه‌های ورودی متغیرها را مقداردهی می‌کردیم در برنامه‌نویسی هم می‌توانیم به وسیله دستورات ورودی متغیرها را مقداردهی کنیم. در ادامه مثالی از این نوع را خواهید دید.

اعلان ثوابت

ثوابت مقداری هستند که در برنامه وجود دارند و قابل تغییر نیستند. برای اعلان ثوابت دو روش وجود دارد:

۱. استفاده از دستور #define که به صورت کلی زیر است:

#define مقدار ثابت نام ثابت

تذکرات مهم: نامگذاری ثوابت از قانون نامگذاری متغیرها تبعیت می‌کند. در هنگام تعریف ثابت با دستور #define مقداری که برای ثابت تعیین می‌شود، نوع ثابت را نیز مشخص می‌کند. دقت داشته باشید که در انتهای دستور #define علامت ; قرار نمی‌گیرد. علتش این است که این دستور، از دستورات پیش پردازنده است، نه دستور زبان C++. پیش پردازنده یک برنامه سیستم است که قبل از ترجمه برنامه توسط کامپایلر، تغییراتی در آن ایجاد می‌کند. پیش‌پردازنده مقدار ثابت را که در دستور #define آمده است، به جای نام ثابت در برنامه قرار می‌دهد و این دستور در زمان اجرا وجود ندارد. نام دیگر ثابتی که به این روش تعریف می‌شوند، ماکرو است. برای تفکیک ثوابت از متغیرهای برنامه، بهتر است تمامی نام آنها با حروف بزرگ انتخاب شود. البته به جهت عدم تعیین نوع به طور صریح، در دستور فوق امکان بروز خطا در برنامه با استفاده از این روش وجود دارد لذا در C++ استفاده از این روش توصیه نمی‌شود.

۲. استفاده از دستور const که به صورت کلی زیر است:

const مقدار ثابت = نام ثابت نوع داده ثابت

اصولاً برنامه‌نویسان C++ بیشتر از روش اخیر برای تعریف ثوابت استفاده می‌کنند، البته در این روش در صورت عدم ذکر نوع، نوع ثابت برابر int فرض می‌شود. مثال‌های این موارد را نیز در ادامه خواهید دید.

۳-۳: انواع عملگر در C++

با عملگرها و نقش آنها در الگوریتم‌ها در فصل قبل آشنا شدید، لذا در اینجا تنها به اشاراتی کوتاه بسنده می‌کنیم و تنها به بیان ناگفته‌ها می‌پردازیم. مقصود از عملگر هر علامتی است مثل علامت جمع که قادر باشد عملیاتی را بر روی متغیرها و یا مقادیر عددی انجام دهد. هر عملگر بر روی یک یا دو متغیر یا مقدار ثابت عمل می‌کند. متغیرها و یا مقادیری را که عملگرها بر روی آنها عمل می‌کنند، عملوند نامیده می‌شوند. برخی از عملگرها حتماً نیاز به دو عملوند دارند مثل عملگر جمع و برخی دیگر تنها نیاز به یک عملوند دارند مثل عملگر یکتایی منها. عملگرها در C++ بر چهار دسته زیر تقسیم می‌شوند:

۱. عملگرهای حسابی
۲. عملگرهای رابطه‌ای
۳. عملگرهای منطقی
۴. عملگرهای بیتی

از بین چهار نوع عملگر فوق سه نوع عملگر اول را در همین فصل بررسی می‌کنیم. اما به دلیل این که عملگرهای بیتی به ویژگی‌های سطح پایین زبان C++ مربوط می‌شوند و فعلاً کاربرد زیادی برای ما ندارند در فصل جداگانه‌ای [هنگامی که ارتباط زبان C++ را با زبان اسمبلی عنوان می‌کنیم،] مورد بررسی قرار خواهیم داد.

بررسی عملگرهای حسابی

عملگرهای حسابی، عملگرهایی هستند که اعمال جبری را روی عملوندها انجام می‌دهند. با اکثر این عملگرها در فصل قبل آشنا شده‌اید و از آنها در عبارات محاسباتی (جبری) استفاده کردید. چنانکه در بخش ۱-۲ دیدید عملگرهای $+$ ، $-$ ، $*$ ، $/$ جهت چهار عمل اصلی ریاضی به کار می‌روند. عملگر $\%$ برای محاسبه باقیمانده تقسیم به کار می‌رود. این عملگر، عملوند اول را بر عملوند دوم تقسیم می‌کند (تقسیم صحیح) و باقیمانده را برمی‌گرداند. اما بیایید با دو عملگر جدید آشنا شویم. عملگر کاهش ($--$) که یک واحد از عملوندش کم می‌کند و نتیجه را در آن عملوند قرار می‌دهد و عملگر افزایش ($++$) که یک واحد به عملوندش اضافه کرده نتیجه را در آن عملوند قرار می‌دهد.

مثال ۳-۳: دستورات زیر را در نظر بگیرید:

```
int x=10, m=15;
x++;
--m;
```

توضیح مثال: در مثال فوق ابتدا متغیرهای x و m با مقدار اولیه ۱۰ تعریف شده‌اند. دستور $x++$ یک واحد به x می‌افزاید و نتیجه را در x قرار می‌دهد و دستور $--m$ یک واحد از m می‌کاهد و نتیجه را در m قرار می‌دهد. در نتیجه در پایان در متغیر x مقدار ۱۱ و در متغیر m مقدار ۱۴ قرار دارد. بنابراین در این نوع دستورات، عملگر افزایش (یا کاهش) چه قبل از عملوند باشد چه بعد از آن، عملکرد آنها یکسان است. اما عملکرد آنها در عبارات محاسباتی با هم متفاوت است. به مثال ۳-۴ توجه کنید.

مثال ۳-۴: در مثال زیر چگونگی تفاوت عملگرهای افزایش و کاهش در عبارات محاسباتی نشان داده شده

است.

```
int x = 5 , y ;
y = 10;
int m = ++x + y--;
```

توضیح مثال: در مثال فوق ابتدا دو متغیر x و y از نوع صحیح تعریف شده‌اند، و متغیر x با مقدار ۵ مقداردهی اولیه شده است اما متغیر y بدون مقداردهی اولیه رها گشته. اما در خط دوم متغیر y به عدد ۱۰ مقداردهی شده است. و در نهایت در خط سوم با تعریف متغیر m از نوع صحیح آن را به واسطه یک عبارت محاسباتی مقدار اولیه داده‌ایم. اما چگونگی محاسبه حاصل این عبارت مهم است، چراکه بر حسب چگونگی ارزیابی عبارت محاسباتی می‌توان گفت چه مقداری در متغیر m قرار گرفته است. نکته قابل توجه در این خصوص آن است که، اگر عملگرهای ++ و -- در عبارات محاسباتی، قبل از عملوند قرار گیرند، ابتدا این عملگرها عمل کرده و نتیجه عملیات آنها در محاسبه عبارت شرکت می‌کند. ولی اگر بعد از عملوند ظاهر شوند، ابتدا عملوند با مقدار فعلی خود وارد محاسبات می‌شود، سپس عملگر بر روی عملوند خود عمل می‌کند. بنابراین در مثال فوق ابتدا مقدار x به عدد ۶ ارتقاء می‌یابد و این عدد با مقدار فعلی y [یعنی ۱۰] جمع شده و حاصل آنها در متغیر m قرار می‌گیرد، سپس مقدار y یک واحد کاهش می‌یابد، که البته هیچ اثری در محاسبه مقدار عبارت محاسباتی ما ندارد.

در جدول ۳-۴ عملگرهای محاسباتی C++ براساس درجه تقدم لیست شده‌اند. چنانکه در بخش ۲-۸ نیز متذکر شدیم عملگرهایی که تقدم آنها یکسان است، نسبت به هم تقدم مکانی دارند. یعنی، هر کدام که زودتر ظاهر شود، همان عملگر زودتر عمل می‌کند. [در C++ عبارات از سمت چپ ارزیابی می‌شوند].

ملاحظات	عملگرها	تقدم
در حالت پیشوندی دارای بیشترین تقدم هستند.	++, --	بالا ترین سطح
منظور منهای یکتایی است. (قرینه سازی).	-	
	*, /, %	
	+, -	پایین ترین سطح

جدول ۳-۴: عملگرهای حسابی C++ بر حسب تقدم (به ترتیب کاهش تقدم)

چنانکه در جدول ۳-۴ مشاهده می‌کنید در زبان C++ هیچ عملگری برای عمل به توان رساندن اعداد تعریف نشده است. پس مبادا به اشتباه عملگر $^$ را برای توان به کار گیرید، این عملگر صرفاً در الگوریتم‌نویسی برای ما اعتبار دارد. البته از این نماد در بسیاری از زبان‌های برنامه‌نویسی به عنوان عملگر توان استفاده شده است، اما متأسفانه در زبان C++ هیچ عملگری برای انجام عمل توان نداریم، اما بعداً در بخش توابع ریاضی در فصل توابع، تابعی کتابخانه‌ای را جهت عمل به توان رساندن معرفی خواهیم کرد.

بررسی عملگرهای رابطه‌ای

با عملگرهای رابطه‌ای نیز در فصل قبل آشنا شدید. تنها تفاوتی که عملگرهای رابطه‌ای C++ با عملگرهای رابطه‌ای ریاضی مطرح شده در فصل قبل دارد شکل ظاهری آنها ست. در جدول ۳-۵ لیست عملگرهای رابطه‌ای در زبان C++ را مشاهده می‌کنید.

عملگر رابطه‌ای در C++	معادل ریاضی عملگر رابطه‌ای	مثال
>	>	$x > y$
>=	$\geq$	$x \geq y$
<	<	$x < y$
<=	$\leq$	$x \leq y$
==	=	$x == y$
!=	$\neq$	$x != y$

جدول ۳-۵: عملگرهای رابطه‌ای در C++

بررسی عملگرهای منطقی

با عملگرهای منطقی و جدول درستی دو عملگر **and** و **or** در فصل قبل آشنا شدید. چنانکه می‌دانید عملگرهای منطقی بر روی گزاره‌ها عمل می‌کنند. در زبان C++، ارزش نادرستی یک عبارت منطقی با مقدار صفر و ارزش درستی یک عبارت منطقی با مقادیر غیر صفر نشان داده می‌شود. بنابراین هر مقدار غیر صفری در C++ برای ما ارزش درست دارد، اما اصولاً ارزش درستی را با مقدار یک نشان می‌دهند. دو ثابت **true** و **false** در زبان C++ برای نشان دادن مقادیر صفر و یک در عملیات منطقی به کار گرفته می‌شوند. چنانکه در بخش ۳-۲ در خصوص نوع داده منطقی مطرح شد، این نوع داده قادر است مقادیر صفر و یک حاصل از عملیات منطقی را در خود ثبت کند. در احکام انتساب، ثابت **true** به ۱ و ثابت **false** به صفر تبدیل می‌شود. به عنوان مثال با نسبت دادن یک متغیر از نوع **bool** که در آن مقدار **true** ذخیره شده است به یک متغیر از نوع صحیح، مقدار ثابت **true** به یک تبدیل می‌شود و در متغیر نوع **int** قرار می‌گیرد. عکس این مطلب نیز صحیح است. در جدول ۳-۶ عملگرهای منطقی به ترتیب کاهش تقدم، از بالا به پایین لیست شده‌اند.

عملگر منطقی در C++	معادل الگوریتمی عملگر	مثال
!	نقیض (not)	$x!$
&&	و (and)	$x > y \ \&\& \ m < n$
	یا (or)	$x != y \ \ m == n$

جدول ۳-۶: عملگرهای منطقی به ترتیب کاهش تقدم [از بالا به پایین]

پیشتر در جدول ۲-۵ عملگرهای رابطه‌ای و منطقی را از نظر تقدم در کنار یکدیگر ملاحظه کرده‌اید. سطوح تقدم مطرح شده در این جدول در خصوص زبان C++ نیز صادق است. اما در این جدول عملگر نقیض به طور استثناء پیش از عملگرهای رابطه‌ای قرار می‌گیرد.

عملگرهای ترکیبی

از ترکیب عملگرهای حسابی و عملگر انتساب (=)، مجموعه دیگری از عملگرها ایجاد می‌شود که عملیات محاسباتی و انتساب را به‌طور هم‌زمان انجام می‌دهد. در جدول ۷-۳ لیست این عملگرها و کاربرد آنها را می‌توانید مشاهده کنید. تقدم این عملگرها از تمامی عملگرها پایین‌تر است.

معادل	مثال	عملگر ترکیبی در C++
$x = x + y$	$x += y$	$+=$
$x = x - y$	$x -= y$	$-=$
$x = x * y$	$x *= y$	$*=$
$x = x / y$	$x /= y$	$/=$
$x = x \% y$	$x \% = y$	$\% =$

جدول ۷-۳: لیست عملگرهای ترکیبی در C++

عملگر ()

چنانکه در فصل دوم دیدید پرانتزها عملگرهایی هستند که تقدم عملگرهای داخل خود را بالا می‌برند.

عملگر sizeof

این عملگر می‌تواند طول یک متغیر یا طول یک نوع داده را برحسب بایت در زمان کامپایل (ترجمه) برگرداند. شاید برای شما فعلاً این عملگر بدون کاربرد باشد، اما در فصول آتی مثال‌هایی از نحوه به‌کارگیری این عملگر و اهمیت آن را خواهید دید. این عملگر برای تعیین طول یک متغیر و یا یک نوع داده به دو صورت متفاوت به‌کار می‌رود. شکل کلی کاربرد آن به‌صورت زیر است. البته توجه داشته باشید که در اطراف نام متغیر نیز می‌توان از علامت پرانتز بهره گرفت ولی به‌کارگیری آن اطراف نوع داده الزامی است.

نام متغیر `sizeof` ;

(نوع داده) `sizeof` ;

مثال ۳-۵: در مثال زیر چگونگی به‌کارگیری عملگر `sizeof` نشان داده شده است.

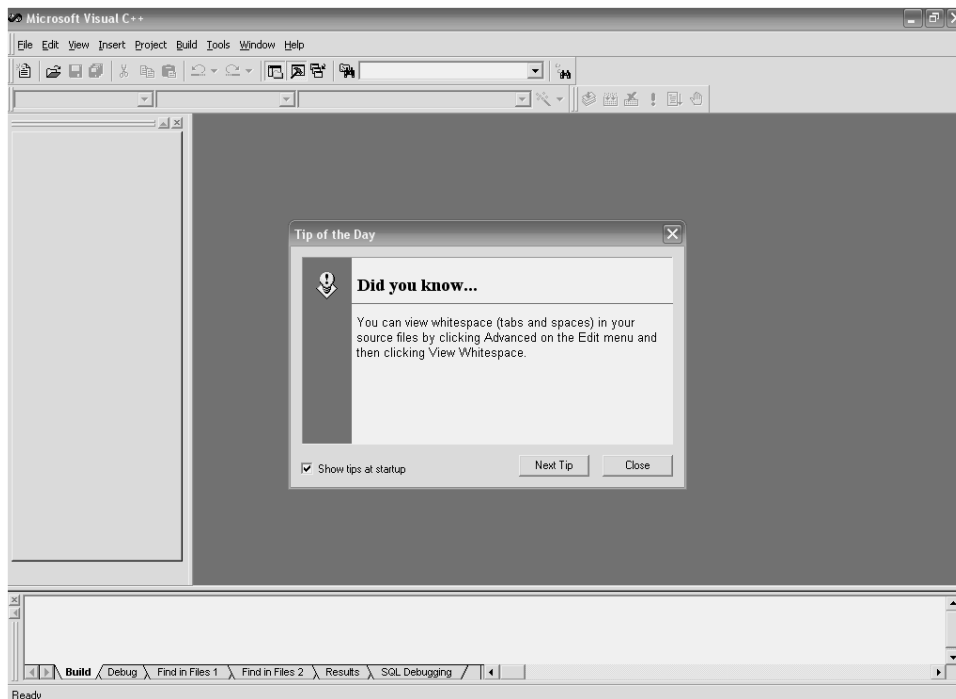
```
int size_var, size_type, x;
x = 5;
size_var = sizeof (x);
size_type = sizeof (char);
```

توضیح مثال: در مثال فوق ابتدا سه متغیر از نوع صحیح تعریف شده‌اند. در خط سوم طول متغیر صحیح `x` در داخل متغیر `size_var` قرار داده شده است. نکته قابل توجه آنکه مقداری که در متغیر `x` ذخیره شده است هیچ تأثیری بر عملگر `sizeof` ندارد چنانکه عدد ۴ را به‌عنوان طول متغیر `x` که از نوع `int` است برمی‌گرداند. در خط چهارم طول نوع داده‌ای `char` را در متغیر `size_type` ذخیره کردیم، که مسلماً عدد یک در این متغیر ذخیره شده است. به به‌کارگیری پرانتز در اطراف نوع داده‌ای و دلخواه بودن به‌کارگیری پرانتز در اطراف نام متغیر خوب توجه داشته باشید. اگر این قطعه کد را مطابق آن چه که در ادامه خواهید دید، در کامپایلر VC6 اجرا کنید و پرانتز اطراف نوع داده‌ای `char` را قرار ندهید با خطا روبرو خواهید شد اما در خصوص نام متغیر شما می‌توانید از پرانتز به دلخواه خود استفاده کنید یا نکنید.

۳-۴: نخستین تجربه برنامه نویسی در محیط Visual C++ 6.0

آماده سازی محیط ویرایشگر VC6

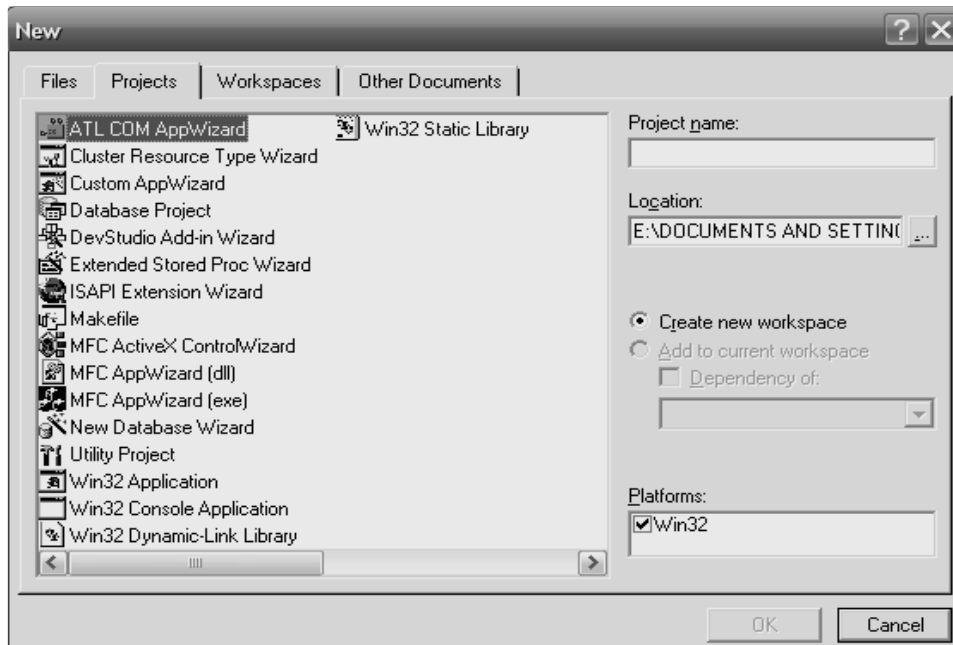
در صورتی که مراحل نصب محیط مجتمع Microsoft Visual Studio 6.0 - مطابق پیوست ۱- را با موفقیت طی کرده باشید، دقیقاً یک آیکون با همین نام در منوی All Programs در منوی Start ویندوز شما ایجاد شده است. همچنین در صورتی که با استفاده از لوح های فشرده ۳ و ۴ مجموعه MSDN را هم نصب کرده باشید، آیکون دیگری نیز با نام Microsoft Developer Network در منوی All Program ایجاد شده است. از طریق MSDN می توانید به مستندات در خصوص کامپایلر VC6 دست پیدا کنید و با جستجو در محیط MSDN مشکلات احتمالی خود را برطرف سازید. اما برای آغاز کار برنامه نویسی گزینه Visual C++ 6.0 Microsoft Visual Studio 6.0 را از داخل منوی Microsoft Visual Studio 6.0 انتخاب کنید تا صفحه ای مطابق شکل ۳-۴ پیش روی شما قرارگیرد.



شکل ۳-۴: صفحه آغازین محیط VC6

پس از باز شدن محیط VC6 پنجره ای با عنوان Tip of the day باز می گردد که در آن توضیحاتی در خصوص این کامپایلر ارائه شده است. با دکمه Next Tip می توانید توضیحات بعدی را مشاهده کنید و با دکمه Close می توانید پنجره را ببندید. در صورتی که تمایل دارید در دفعات بعدی که به VC6 مراجعه می کنید این پنجره باز نشود علامت تیک جلوی پیام Show tip at startup را قبل از بستن پنجره، بردارید.

با انتخاب گزینه New از منوی File پنجره‌ای را مطابق شکل ۳-۵ مشاهده خواهید کرد.

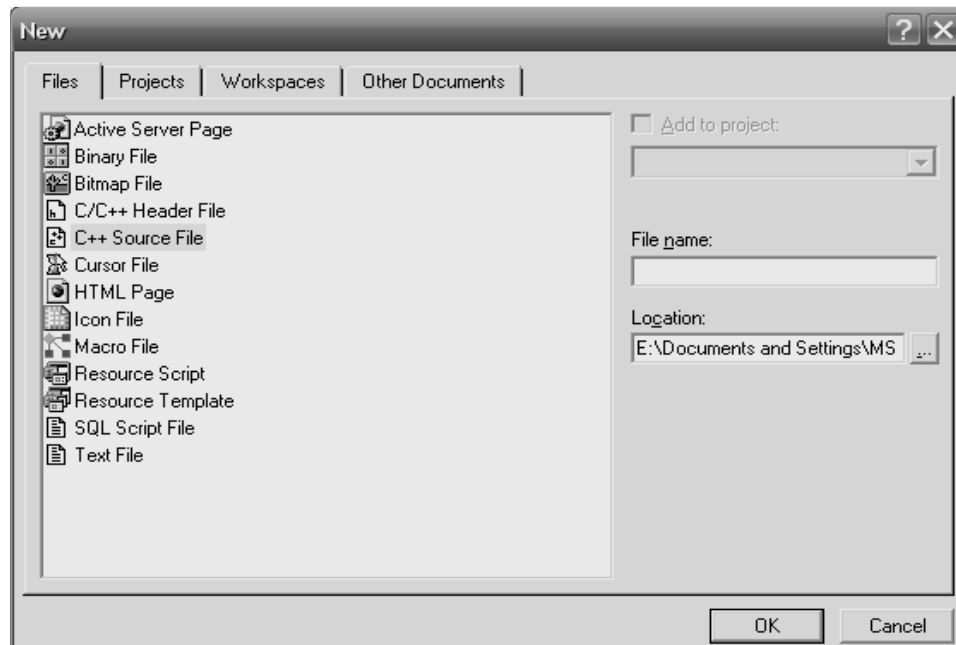


شکل ۳-۵: پنجره New در حالت Projects

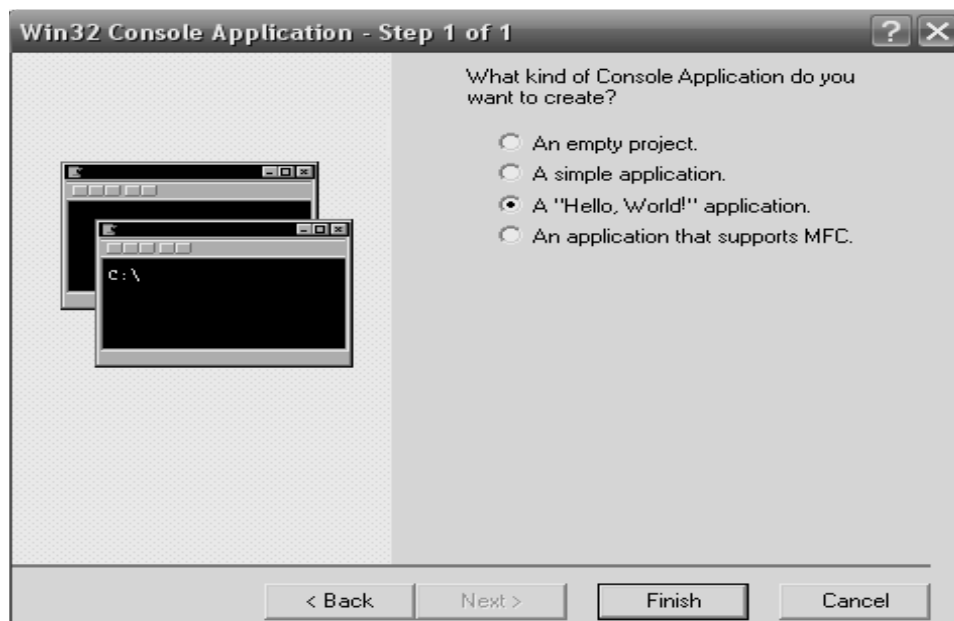
دو روش برای ایجاد برنامه‌های C++ در محیط VC6 وجود دارد. ابتدا روش ساده‌تر را بیان می‌کنیم، شما نیز برای ایجاد برنامه‌های C++ خود در جلد اول از این روش استفاده کنید.

۱. برای ایجاد برنامه C++ گزینه Files را از پنجره New انتخاب کنید تا پنجره‌ای را مطابق شکل ۳-۶ مشاهده کنید. در این پنجره ضمن انتخاب گزینه C++ Source File، نام برنامه خود را در قسمت File Name وارد کرده و سپس در قسمت Location محل ذخیره برنامه خود را تعیین کنید. با فشردن دکمه ok صفحه‌ای خالی جهت وارد کردن کد برنامه برای شما باز می‌شود. در این حالت یک فایل با پسوند .cpp و با نامی که شما انتخاب کرده‌اید ایجاد می‌شود. به خاطر داشته باشید پسوند فایل‌هایی که کد برنامه‌های خود را در آن قرار می‌دهیم .cpp باید باشد، تا توسط کامپایلر قابل ترجمه باشد.

۲. روش دیگر، ایجاد پروژه بر مبنای Win 32 Console Application است. در شکل ۳-۵ ضمن انتخاب گزینه مذکور و وارد کردن نام پروژه خود دکمه ok را بزنید تا پنجره‌ای مطابق شکل ۳-۷ باز شود. در این پنجره گزینه A application "Hello, World" را انتخاب کنید و دکمه Finish را بفشارید تا قالب پروژه موردنظر شما ایجاد گردد. پس از آنکه نوشتن برنامه در C++ استاندارد را فراگرفتید در خصوص نوشتن پروژه‌های Win32 هم صحبت خواهیم کرد. لذا فعلاً از این روش برای ایجاد برنامه‌های C++ خود استفاده نکنید. برای نوشتن هر برنامه‌ای ابتدا باید مرحله ۱ را برای ایجاد پروژه پیمایید.



شکل ۳-۶: پنجره New در حالت Files



شکل ۳-۷: پنجره Win 32 Console Application

اجرا کردن نخستین برنامه به زبان C++

مثال ۳-۶: برنامه‌ای که در آن پیام "Hello, World" بر روی صفحه چاپ می‌شود.

۱. مطابق آنچه گفته شد یک فایل `cpp` با نام `First` ایجاد کنید.

۲. کد زیر را به‌طور دقیق در آن تایپ کنید.

```
#include <stdio.h>
void main()
{
    printf("Hello,World.\n");//print the message
}
```

۳. با انتخاب گزینه `Compile` و یا زدن همزمان دکمه‌های `Ctrl` و `F7` برنامه خود را ترجمه (کامپایل) کنید.^۱ در این مرحله باید پیغام `0 error(s), 0 warning(s)` را در پنجره `Build`، در پایین صفحه دریافت کنید. اگر تعداد خطاها غیر از صفر باشد فایل اجرایی این کد قابل ساختن نخواهد بود، اما تعداد هشدارها فعلاً چندان مهم نیست. البته این بدان معنا نیست که از این پس به هشدارها توجه نکنید، چرا که برخی از اوقات توجه به هشدارها می‌تواند جلوی بسیاری از اشکالات در برنامه را بگیرد، لذا تنها در همین مثال به هشدارها بی‌توجه هستیم. اگر تعداد خطاها غیر از صفر است یک بار دیگر کد خود را کنترل کنید و با کتاب مطابقت دهید. مثلاً ممکن است فراموش کرده باشید علامت سمیکالن را در انتهای خط چهارم قرار دهید. از طرفی می‌توانید نوع خطا و محل آن را به واسطه پیغام‌هایی که در جعبه `Build` اعلام می‌شود مشاهده کنید. به‌عنوان مثال سمیکالن انتهای خط چهارم را بردارید و یک با دیگر کد خود را کامپایل کنید. در این صورت پیغام زیر را می‌توانید در این پنجره ببابید:

`error C2143: syntax error : missing ';' before '}'`

`1 error(s), 0 warning(s)`

با کلیک کردن بر روی این پیغام یک پیکان در کنار صفحه شما را به محل خطا راهنمایی می‌کند. که می‌توانید به آنجا مراجعه و خطا را رفع کنید.

۴. با انتخاب گزینه `Execute Program` از منوی `Build` و یا فشردن همزمان کلیدهای `Ctrl` و `F5` برنامه خود را اجرا کنید تا پیام "Hello, World" را بر روی صفحه کنسول ببینید.^۲ با زدن یک کلید اجرای برنامه را خاتمه دهید.

۱. اگر بر روی نوار ابزار جستجو کنید کلیدی را خواهید یافت که اگر چند لحظه اشاره‌گر را بر روی آن نگه دارید کلمه

`Compile(ctrl+F7)` بر روی آن نمایان می‌شود. به‌واسطه این کلید نیز می‌توانید عمل کامپایل را انجام دهید.

۲. برای اجرا کردن برنامه نیز می‌توانستید دکمه‌ای را که دارای علامت ! است را از داخل نوار ابزار بفشارید.

۳-۵: ساختار کلی برنامه در C++

مثال ۳-۶ را به این جهت عنوان کردیم که در این قسمت راحت تر بتوانید مطالب مرتبط با ساختار برنامه در C++ را دنبال کنید. مباحثی که در این بخش با عنوان ساختار کلی برنامه مطرح می‌سازیم بسیار محدود است، به طوری که هر چه جلوتر برویم با قسمت‌ها و ساختارهای دیگری از برنامه‌های C++ آشنا خواهید شد.

تابع اصلی (تابع main)

اصلی‌ترین قسمت هر برنامه در C++ تابع اصلی آن است که با نام `main` شناسایی می‌شود. در حقیقت سیستم عامل برای اجرای برنامه به دنبال تابع اصلی آن می‌گردد، و پس از یافتن این تابع اجرای برنامه از اولین خط این تابع آغاز شده و تا آخرین خط آن ادامه می‌یابد.

هر تابع در برنامه‌های C++ دارای یک بدنه است. بدنه هر تابع که شامل یک سری دستورات و فرمان‌هایی برای کامپیوتر است که در بین یک جفت آکولاد باز و بسته قرار می‌گیرد. فعلاً به پراگمات باز و بسته‌ای که پس از نام تابع اصلی آمده است کاری نداشته باشید، ولی در هنگام نوشتن کد حتماً آن را قرار دهید.

در C++ تمامی توابع باید دارای یکی از انواع موجود در C++ باشند. هر تابع بر اساس نوعی که دارد یک مقدار را به مجری خود برمی‌گرداند. مقصود از مجری یا فراخواننده هر تابع، کسی است که تابع را اجرا می‌کند. به عنوان مثال تابع اصلی توسط سیستم عامل فراخوانده می‌شود در نتیجه مجری تابع اصلی سیستم عامل است. همچنین ممکن است یک تابع فراخواننده تابع دیگری باشد که در این خصوص در فصل توابع بیشتر بحث خواهیم کرد. هر تابع باید مقداری را متناسب با نوعش به فراخواننده خود بازگرداند. مثلاً وقتی می‌گویند فلان تابع از نوع صحیح است منظور آن است که این تابع یک عدد صحیح را به فراخواننده خود بازمی‌گرداند. تابع `main` نیز از این قاعده مستثناء نیست، و باید مقداری را به سیستم عامل که فراخواننده آن است برگرداند. دلیل این امر آن است که سیستم عامل بفهمد تابع `main` با موفقیت به پایان رسیده است و تمامی دستورات آن با موفقیت اجرا گردیده. نوع تابع، قبل از نام تابع ذکر می‌گردد. به عنوان مثال اگر بخواهیم تابع اصلی از نوع صحیح باشد، این تابع دارای شکل کلی زیر خواهد بود.

```
int main()
{
    بدنه تابع اصلی
    return 0;
}
```

چنانکه در قطعه کد فوق ملاحظه می‌کنید، هنگامی که تابع `main` از نوع `int` تعریف گردد باید توسط یک دستور `return` در آخرین خط خود یک مقدار صحیح برگرداند. همواره با دستور `return` مقداری متناسب با نوع تابع به مجری تابع برگردانده می‌شود. همچنین با رسیدن به این دستور، کنترل اجرای برنامه نیز به مجری تابع بازگردانده می‌شود، بنابراین اگر دستور یا دستوراتی پس از دستور `return` قرار گیرد به هیچ عنوان اجرا نخواهند شد. در خصوص تابع `main` مقدار بازگشتی هیچ اهمیتی ندارد که چه مقداری است، مثلاً در مثال فوق می‌توانیم به جای عدد صفر، عدد یک یا

هر عدد صحیح دیگری را باز گردانیم اما به هیچ عنوان نمی‌توانیم عددی اعشاری را به‌عنوان مقدار بازگشتی برای یک تابع `int` بازگردانیم. اصولاً برای توابع `main` ای که از نوع `int` تعریف می‌شوند مقدار صفر بازگردانده می‌شود. چنانکه در مثال ۳-۶ نیز ملاحظه کردید اگر تابعی از نوع `void` تعریف شود نیازی به برگرداندن مقدار برای آن تابع نیست. اما در C++ استاندارد انجام این عمل برای تابع `main` خطا محسوب می‌شود. البته ممکن است برخی از کامپایلرها این خطا را نادیده بگیرند و یا آنکه به جای اعلام خطا یک هشدار صادر کنند ولی در هر صورت در مستندات مربوط به C++ استاندارد وجود نوع صحیح برای تابع `main` الزامی شمرده شده است.

دستورات پیش‌پردازنده و سرفایل‌ها (هدر فایل)

به خط اول مثال ۳-۶ خوب توجه کنید. یکبار این خط را پاک کنید و دوباره برنامه خود را کامپایل کنید. فکر می‌کنید با چه خطایی روبرو شده اید؟

اگر چنین عملی را انجام دهید با خطای عدم شناسایی دستور `printf` روبرو می‌شوید. در حقیقت دستورات پیش‌پردازنده که با علامت `#` شروع می‌شوند و به `;` نیز ختم نمی‌شوند از اجزای اصلی برنامه محسوب نمی‌گردند، اما یک ناحیه از حافظه را اشغال کرده و وجود آنها هنگام اجرای برنامه ضروری است. دستورات پیش‌پردازنده جزء دستورات برنامه نیز محسوب نمی‌شوند چرا که دستورات برنامه به کامپیوتر فرمان انجام کاری، مثل جمع کردن دو متغیر را می‌دهند. اما کاربرد این دستورات در برنامه چیست؟

قسمتی از کامپایلر به نام پیش‌پردازنده قبل از آغاز فرآیند واقعی کامپایل این دستورات را مورد ارزیابی قرار می‌دهد. دستور پیش‌پردازنده `#include` به کامپایلر می‌گوید، فایل دیگری را به فایل `.cpp` برنامه ما اضافه کند. به این ترتیب به جای دستور `#include` محتوای فایل مشخص شده قرار می‌گیرد. اما این فایل‌هایی که باید اضافه شوند چه نقشی در برنامه ما دارند؟

بسیاری از دستورات مفید و پرکاربرد در زبان C++ در فایل‌هایی موسوم به هدر فایل (سرفایل) قرار دارند. به‌عنوان مثال اگر می‌خواستیم بدون استفاده از دستور `printf` پیغام خود را روی خروجی استاندارد (یعنی مانیتور) بنویسیم باید خطوط زیادی شامل کد اسمبلی می‌نوشتیم تا بتوانیم چنین پیامی را بر روی صفحه نمایش چاپ کنیم. اما برای راحتی کار برنامه‌نویسان بسیاری از این کدها از قبل در فایل‌هایی نوشته شده و آماده استفاده می‌باشند. در واقع با دستورات پیش‌پردازنده ما می‌توانیم این فایل‌ها را به برنامه خود اضافه کنیم، و از امکانات موجود در آنها استفاده نماییم. به‌عنوان مثال برای استفاده از دستور `printf` باید هدر فایل `stdio.h` را به برنامه اضافه کنیم. در مثال ۳-۶ دستور `#include <stdio.h>` به کامپایلر می‌گوید قبل از کامپایل برنامه، هدر فایل `stdio.h` را به برنامه ما اضافه کند. سرفایل‌های جدید C++ استاندارد، دارای پسوند فایل نیستند اما سرفایل‌های قدیمی که از C++ مادر گرفته شده‌اند دارای پسوند `.h` هستند. در هنگام استفاده از دستور `#include` نباید بین کلمه `include` و `#` فاصله‌ای قرار داد. همچنین بین نام فایل و علامت‌های `<` و `>` نیز نباید فاصله قرار داد.

مستندسازی و توضیحات

معمولاً هر برنامه‌نویس پس از مدتی که به برنامه‌هایی که قبلاً نوشته مراجعه می‌کند با مشکلات متعددی در خصوص به یادآوری آنچه از قبل نوشته مواجه می‌شود. نمی‌داند فلان متغیر را برای چه منظوری در نظر گرفته بوده، فلان خط برنامه را برای چه نوشته و ... در این بین مستندسازی کردن^۱ در برنامه بسیار حائز اهمیت است. چرا که این مستندات هستند که می‌توانند ما را در فهمیدن نقاط کور برنامه کمک کنند. در C++ دو روش عمده برای مستندسازی و ارائه توضیحات در برنامه وجود دارد.

روش اول که نمونه آن را در مثال ۳-۶ دیدید، با قرار دادن // در انتهای هر خطی از برنامه قادر هستید، بعد از آن توضیحاتی را قرار دهید. به عبارت دیگر کامپایلر هرگاه به چنین علامتی برسد از آن نقطه تا پایان خط را نادیده می‌گیرد و به خط بعدی می‌رود. لذا علامت // باید پس از علامت ; قرار گیرد. به این نوع مستندسازی، توضیحات تک‌خطی گفته می‌شود.

اما روش دیگر مستندسازی، نوشتن توضیحات در چند خط، و یا به عبارت دیگر نوشتن بلوکی از توضیحات است. برای این منظور ابتدای توضیحات را با علامت /* آغاز کرده و انتهای آن را با علامت */ می‌بندیم. کامپایلر نیز محتویات کلیه خطوطی که بین این دو علامت قرار می‌گیرد را به رنگ سبز در می‌آورد تا قابل تمایز باشد. به عنوان مثال توضیحات زیر را ببینید.

```
/*This Variable is for save the number of students.
And initialize to zero.*/
```

مستندسازی در هنگام رفع عیوب برنامه، و همچنین به جهت خوانا کردن و قابل فهم کردن برنامه برای دیگر همکارانی که با شما بر روی یک پروژه کار می‌کنند بسیار راه گشا می‌باشد.

نکات متفرقه در خصوص زبان C++

۱. در C++ کلیه دستورات با حروف کوچک نوشته می‌شود. به عنوان مثال اگر تابع main را به صورت Main بنویسید برنامه با خطا روبرو خواهد شد.

۲. فضای خالی برای کامپایلر C++ بی اهمیت است. شما می‌توانید یک دستور C++ را در چندین خط بنویسید. تنها علامت ; است که انتهای خط و به عبارت بهتر انتهای دستور را برای کامپایلر مشخص می‌کند. با این حساب شما حتی می‌توانید دستورات برنامه را پشت سرهم در یک خط نیز بنویسید. اما از انجام چنین کاری به شدت خودداری کنید. اصولاً بهتر است هر خط را با کمی تو رفتگی نسبت به خط قبلی آغاز کنیم. در دو جا نمی‌توان از فضای خالی استفاده کرد. یکی در دستورات پیش‌پردازنده که قبلاً اشاره شد، دیگری در ثابت‌های رشته‌ای که اندکی بعد به آن اشاره خواهیم کرد.

۳-۶: ورود و خروج اطلاعات در C++

در زبان C ورود و خروج اطلاعات تا حدودی با پیچیدگی همراه بود. به‌عنوان مثال برای ورود اطلاعات باید به‌طور دقیق به کامپایلر بگوییم که چه نوع داده‌ای را باید بخواند. اما یکی از مهمترین مزیت‌های C++ نسبت به C افزوده شدن هدر فایل `iostream.h` بود که به وسیلهٔ اشیای موجود در این هدر فایل می‌توان به‌راحتی هر نوع داده‌ای را خواند و یا هر نوع اطلاعاتی را چاپ کرد.

چاپ اطلاعات به واسطهٔ شیء `cout`

به واسطهٔ شیء `cout` می‌توان هر نوع اطلاعاتی را به‌سادگی بر روی صفحه نمایش چاپ کرد. برای استفاده از شیء `cout` باید هدر فایل `iostream.h` را به برنامه اضافه کنید. شکل کلی این دستور به‌صورت زیر است:

```
cout << عبارت ۱ << عبارت ۲ << ... << عبارت n
```

در این دستور عبارت ۱ و عبارت ۲ و ... مقادیری هستند که باید در صفحه نمایش چاپ شوند، این مقادیر باید به واسطهٔ عملگر `<<` از یکدیگر جدا شوند. برای چاپ متن‌ها باید آنها را بین یک جفت کوتیشن قرار داد. همچنین می‌توان با استفاده از کاراکترهای کنترلی که لیست آنها در جدول ۳-۸ آمده شکل خروجی را بهبود بخشید. به‌عنوان مثال با کاراکترهای کنترلی می‌توان مشخص کرد که آیا تمام اطلاعات در یک سطر چاپ شوند یا در چند سطر، آیا اطلاعات با فاصلهٔ خاصی چاپ شوند یا پشت سر هم. به‌عنوان مثال کاراکتر کنترلی `\n` موجب می‌شود که سطر جاری رد شود و به ابتدای سطر بعدی برویم. اطلاعاتی که پس از `\n` بیاید در سطر جدیدی چاپ خواهد شد. نکتهٔ بسیار مهم در خصوص کاراکترهای کنترلی آن که این کاراکترها حتماً باید داخل یک جفت کوتیشن قرار گیرند و یا در داخل متون ثابت جای داده شوند.

کاراکتر کنترلی	کاری که انجام می‌دهد
<code>\n</code>	سطر جاری را رد می‌کند.
<code>\t</code>	کنترل خروجی را به ابتدای ۸ محل بعدی می‌برد.
<code>\a</code>	بوق سیستم را به صدا در می‌آورد.
<code>\\</code>	کاراکتر <code>\</code> را چاپ می‌کند.
<code>\"</code>	کاراکتر <code>"</code> را چاپ می‌کند.
<code>\v</code>	کنترل خروجی را به ابتدای ۸ سطر بعدی می‌برد.
<code>\b</code>	کاراکتر قبل از خودش را حذف می‌کند.
<code>\r</code>	عمل <code>carriage return</code> را مشخص می‌کند.
<code>\?</code>	کاراکتر <code>?</code> را چاپ می‌کند.
<code>\:</code>	کاراکتر <code>:</code> را چاپ می‌کند.

جدول ۳-۸: کاراکترهای کنترلی

خواندن اطلاعات به وسیله شی cin

به واسطه شی cin می توان هر گونه اطلاعات را از ورودی استاندارد (یعنی صفحه کلید) خواند و در متغیرهای مورد نظر قرار داد. برای استفاده از شی cin نیز نیازمند اضافه کردن هدر فایل `iostream.h` هستید. شکل کلی به کارگیری این دستور به صورت زیر است:

```
cin>> متغیر ۱ >> متغیر ۲ >> ... ;
```

وقتی چند قلم اطلاعات را توسط دستور cin می خوانید، هنگام ورود داده ها باید آنها را حداقل با یک فاصله از هم جدا کنید و پس از وارد کردن تمام آنها، کلید Enter را فشار دهید. نوع داده های ورودی باید مشابه با نوع متغیرهایی باشد که در دستور cin قرار گرفته اند و گرنه امکان دارد برنامه با خطای زمان اجرا مواجه شود.

تذکره: برخی اوقات افراد تازه کار فراموش می کنند که کدام یک از علائم << یا >> را برای شی cout و کدام یک را برای شی cin بکار می بردند. برای جلوگیری از این اشتباه این علائم را به مانند یک قیف در نظر بگیرید. وقتی که می خواهیم اطلاعات را بر روی صفحه نمایش چاپ کنیم باید اطلاعات را به داخل خروجی بریزیم در این حالت سر قیف باید به سمت خروجی که همان cout است باشد. از طرفی هنگامی که اطلاعات را می خوانیم باید اطلاعات خوانده شده را به داخل متغیرها بریزیم در این صورت باید سر قیف به سمت متغیرها قرار گیرد.

مثال ۳-۷: برنامه ای که شعاع یک دایره را خوانده و مساحت و محیط آن را چاپ می کند.

```
1. //This program shows the usage of cin and cout.
2. #include <iostream.h>
3. #define PI 3.1415927 // تعریف ثابت
4. int main ()
5. {
6.     cout<<"Please enter the radius of your Circle :";
7.     float rd;
8.     cin>>rd;
9.     double s= PI*rd*rd;
10.    double p= PI*2*rd;
11.    cout<<"\n\n\tThe circle's area = \t"<<s;
12.    cout<<"\n\n\tThe circle's prime = \t"<<p<<endl;
13.    return 0;
14. }
```

توضیح مثال: در مثال فوق با مواردی چون تعریف ثابت، به کارگیری کاراکترهای کنترلی، قرار دادن توضیحات در متن برنامه و دستکاری کننده endl روبرو هستید که جزء مورد آخر با تمامی موارد از قبل آشنایی دارید. دستکاری کننده endl همانند کاراکتر کنترلی \n عمل می کند با این تفاوت که نباید در داخل کوتیشن قرارگیرد و همچنین این دستکاری کننده موجب پاک شدن بافر خروجی می شود. از این پس در تمامی مثال ها برای بهتر ارجاع دادن خوانندگان به خطوط مختلف برنامه، خطوط برنامه ها را شماره گذاری می کنیم، که شماره ها جزء برنامه نیستند، و هنگام وارد کردن برنامه در کامپایلر از نوشتن شماره ها خودداری کنید.

دستور using

در C++ استاندارد، برنامه‌ها را می‌توان به فضاهای نام مختلفی تقسیم‌بندی کرد. مقصود از فضای نام^۱، بخشی از برنامه است که در آن، شناسه‌های مشخصی به رسمیت شناخته شده و در خارج آن، این شناسه‌ها به رسمیت شناخته نمی‌شوند. و مقصود از شناسه، اسامی متغیرها، توابع، کلاس‌ها و غیره است. مزیت بکارگیری فضاهای نام این است که، حوزه‌هایی در حافظه مورد استفاده برنامه، تشکیل می‌شود و هر حوزه به یک فضای نام اختصاص می‌یابد، تا متغیرها و داده‌های فضای نام مذکور در آن حوزه قرار گیرد در این صورت احتمال برخورد شناسه‌های موجود در حوزه‌های متفاوت با یکدیگر به صفر می‌رسد و از بروز بسیاری از خطاها جلوگیری می‌شود.

پس از استانداردسازی C++ بسیاری از هدرفایل‌ها بازنویسی شدند. هنگامی که می‌خواهید از هدر فایل‌های جدید C++ استفاده کنید باید در هنگام افزودن این هدرفایل‌ها به برنامه، از قرار دادن پسوند h. در دنباله نام فایل سرآیند خودداری کنید. بنابراین به جای استفاده از فایل قدیمی iostream.h می‌توانید از هدرفایل جدید iostream استفاده کنید. اما اگر برنامه مثال ۳-۷ را یک بار دیگر پس از حذف پسوند h. از انتهای فایل سرآیند دوباره کامپایل کنید با خطاهای متعددی برخورد می‌کنید. اولین خطایی که می‌توان مشاهده کرد عبارت است از:

error C2065: 'cout' : undeclared identifier

در حقیقت کامپایلر در پیام فوق به شما می‌گوید که شی cout را شناسایی نکرده است. این عدم شناسایی بدان علت است که پس از بازنویسی فایل‌های سرآیند در C++ استاندارد، تمامی شناسه‌های این فایل‌ها در حوزه‌ای با نام std^۲ قرار داده شد تا احتمال برخورد شناسه‌های موجود در آنها با سایر قطعات برنامه کم شود. برای استفاده از شناسه‌های موجود در این فایل‌ها باید قبل از تابع main عبارت زیر را تایپ کنید:

```
using namespace std;
```

اگر عبارت فوق را قبل از تابع main تایپ کنید و برنامه را دوباره کامپایل و اجرا کنید خواهید دید که دیگر با خطا برخورد نمی‌کنید. در واقع با قرار دادن عبارت فوق قبل از تابع main به کامپایلر می‌گویید که شناسه‌هایی نظیر cout در این فضای نام قرار دارد. اما می‌توان بدون قرار دادن عبارت فوق نیز کامپایلر را وادار به شناسایی شناسه‌های موجود در فضای نام std کرد. برای این منظور باید از عملگر حوزه تفکیک^۳ (یعنی ::) به همراه اسم فضای نام مورد نظر استفاده کنید. به عنوان مثال در این حالت خط شماره ۱۲ مثال ۳-۷ به صورت زیر باید بازنویسی شود:

```
std::cout<<"\n\n\tThe circle's area = \t"<<s;
```

۱. Namespace.

۲. std مخفف کلمه standard است.

۳. scope resolution.

۳-۷: تبدیل انواع

آیا تا به حال از خود پرسیده‌اید که کامپیوتر چگونه دو عدد را با هم جمع یا تفریق یا ضرب می‌کند؟ جواب این سؤال را باید با توجه به دستورات زبان سطح پایین اسمبلی داد. اگر در زبان اسمبلی [مربوط به پردازنده‌های اینتل] بخواهیم یکی از اعمال چهارگانه ریاضی را بر روی دو عدد صحیح انجام دهیم حتماً باید آن دو عدد با یکدیگر هم اندازه باشند. علت این محدودیت این است که پردازنده برای عملیات بر روی دو عدد، ابتدا باید آن دو عدد را در رجیسترهای خود قرار دهد، سپس با عمل کردن مدارات خاصی از پردازنده عمل ریاضی موردنظر صورت می‌گیرد. در حقیقت مشکل اصلی از مداراتی است که عمل ریاضی موردنظر ما را انجام می‌دهند. در ساختمان بسیاری از پردازنده‌ها اینگونه محدودیت‌ها وجود دارد چرا که طراحی مدارات داخلی آنها چنین محدودیت‌هایی را ایجاب می‌کند. به‌عنوان مثال هیچگاه نمی‌توان دو عدد که در حافظه RAM قرار دارند را با هم جمع یا تفریق یا ضرب و یا تقسیم کرد. چرا که مدارات پردازنده‌ها به‌گونه‌ای طراحی شده‌اند که برای عملیات کردن بر روی این دو عدد ابتدا باید حداقل یکی از آن دو عدد را به داخل رجیسترهای پردازنده آورد، تا مدارات قسمت ALU بتوانند عملیات مورد نظر ما را انجام دهند. این مثالی کوچکی از محدودیت‌های موجود در کارکردن با پردازنده‌ها است.

به همین صورت مدارات قسمت ALU تنها قادر به عملیات بر روی دو داده با طول مشابه هستند. مثلاً هر دو عدد باید دارای طولی برابر ۱۶ بیت باشند تا بتوان آنها را در رجیسترهای ۱۶ بیتی پردازنده قرار داد و مدار مربوطه را فراخوانی کرد. حال اگر دو عدد ۳۲ بیتی بودند بایستی از پردازنده‌هایی با طول ۳۲ بیت استفاده کنیم و مدار مربوط به عملیات مورد نظر خود را برای دو رجیستر ۳۲ بیتی فراخوانی کنیم.

اما سؤال بسیار مهمی که در اینجا مطرح می‌شود این است که اگر به‌عنوان مثال بخواهیم دو عدد را که یکی ۱۶ بیتی و دیگری ۳۲ بیتی است را با هم جمع یا ضرب کنیم با توجه به محدودیت‌های موجود در پردازنده‌ها تکلیف چیست؟ آیا باید از چنین اقدامی پرهیز کرد و یا راهی برای این مشکل وجود دارد؟

در پاسخ به این سؤال باید گفت با توجه به آنکه قرار دادن صفر در سمت راست یک عدد هیچ تأثیری بر ارزش آن عدد ندارد، می‌توان عدد ۱۶ بیتی را به عددی ۳۲ بیتی با همان ارزش تبدیل کرد. به‌عنوان مثال دو عدد ۰۰۱۰ و ۱۰ دارای ارزش مشابهی هستند. لذا با توجه به این مثال، می‌توان قاعده‌ای کلی به این صورت عنوان نمود که، در کامپیوتر برای انجام عملیات ریاضی بر روی دو عدد با طول متفاوت، ابتدا باید با قرار دادن صفر در سمت راست عدد کوچکتر، آن را هم طول عدد بزرگتر می‌سازد، و سپس قادر خواهد بود عملیات مورد نظر را بر روی آن دو عدد انجام دهد. لذا در اکثر زبان‌های برنامه‌نویسی مبحثی تحت عنوان تبدیل/انواع مطرح می‌شود که در ادامه به تشریح تبدیل/انواع در ++C خواهیم پرداخت. شایان ذکر است که تمام زبان‌ها دارای امکاناتی برای شرکت دادن انواع مختلف در عبارات محاسباتی نبوده و زبان C یکی از اولین زبان‌هایی بود که این امکان را فراهم نمود.

تبدیل نوع در C++ بر دو گونه است:

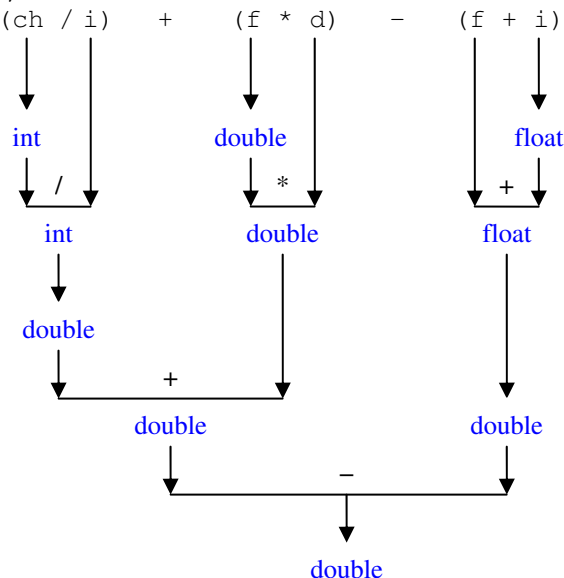
۱. تبدیل نوع اتوماتیک که بیشتر در پردازش عبارات محاسباتی و احکام انتساب رخ می‌دهد و به جهت آنکه برنامه نویس هیچ نقشی در انجام این تبدیلات ندارد به آنها تبدیل نوع اتوماتیک گفته می‌شود.
۲. تبدیل نوع موقت که خود دارای حالت‌های متفاوتی است و در ادامه به بررسی آن خواهیم پرداخت.

تبدیل نوع اتوماتیک (تبدیل نوع ضمنی)

در زبان C++ شما قادر هستید انواع مختلف موجود در C++ را بدون هیچ محدودیتی در عبارت محاسباتی به کارگیرید. به عنوان مثال شما می‌توانید یک عدد از نوع `int` را با یک عدد از نوع `float` جمع کنید. چنانکه پیشتر عنوان شد C++ برای محاسبه حاصل چنین عبارت محاسباتی، ابتدا نوع کوچکتر (`int`) را به نوع بزرگتر (`float`) تبدیل و سپس دو عدد `float` به دست آمده را با یک دیگر جمع می‌کند. بنابراین در تبدیل نوع اتوماتیک که در عبارات محاسباتی رخ می‌دهد، در صورتی که دو عملوند اطراف یک عملگر حسابی دارای انواع متفاوتی باشند، ابتدا نوع کوچک‌تر به نوع بزرگتر تبدیل می‌شود، و سپس عملگر برای نوع بزرگتر فراخوانی می‌گردد.

مثال ۳-۷: در قطعه کد زیر متغیر `result` باید از چه نوعی باشد تا حاصل عبارت محاسباتی سمت راست دستور انتساب بدون سرریز در این متغیر ذخیره گردد؟ مراحل پردازش و تبدیل انواع را به صورت نموداری رسم کنید؟

```
1. char ch;
2. int i;
3. float f;
4. double d;
5. result = (ch / i) + (f * d) - (f + i);
```



شرح مثال: در عبارت محاسباتی فوق پردازش از اولین پرانتز آغاز می‌گردد. چون متغیر `ch` دارای طول یک بایت و متغیر `i` دارای طولی برابر چهار بایت است. ابتدا متغیر `ch` به نوع صحیح تبدیل می‌گردد و سپس عمل تقسیم صورت

می‌گیرد، نتیجه این تقسیم هر چه باشد عددی هم نوع عملوندهای خود، یعنی از نوع صحیح خواهد بود. اگر روند نمودار رسم شده را به همین روش دنبال کنید خواهید دید که در پایان حاصل عبارت محاسباتی از نوع `double` می‌باشد، بنابراین متغیر `result` نیز باید از این نوع باشد.

حالت دیگری از تبدیل نوع اتوماتیک، زمانی رخ می‌دهد که در احکام انتساب دو نوع متفاوت به یکدیگر نسبت داده شوند. به‌عنوان مثال قطعه کد زیر را در نظر بگیرید:

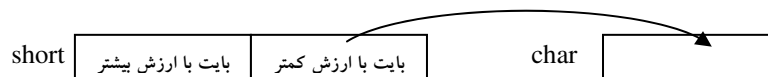
```
short    i=12654;
char     ch='k';
float    f=3.14;
ch=i;
i=f;
f=ch;
f=i;
```

تمامی انواع موجود در C++ را می‌توان توسط احکام انتساب به یکدیگر نسبت داد. اما دو حالت متفاوت در تبدیل نوع در احکام انتساب وجود دارد که به شرح زیر است:

۱. اگر نوعی با طول کوچکتر به نوعی با طول بزرگتر نسبت داده شود، در این حالت تبدیل نوع معمولاً بدون هیچگونه مشکلی صورت می‌گیرد. زیرا کامپایلر مانند حالت قبل به راحتی با قرار دادن صفر در سمت چپ نوع کوچکتر آن را به نوع بزرگتر تبدیل می‌کند. نمونه‌ای از این تبدیل نوع را در خطوط ۶ و ۷ مثال فوق مشاهده می‌کنید. اما کامپایلر برای برخی از این دست تبدیل نوع‌ها یک هشدار صادر می‌کند. مثلاً در تبدیل نوع `int` به `float` کامپایلر هشدار می‌دهد. به نظر شما چنین هشداری به چه دلیلی صادر می‌شود؟
۲. اگر نوعی با طول بزرگتر به نوعی با طول کوچکتر نسبت داده شود، کامپایلر یک پیغام هشدار تحت این مضمون صادر می‌کند که، "تبدیل کردن نوع بزرگتر به نوع کوچکتر ممکن است همراه با از دست رفتن اطلاعات باشد". به-عنوان مثال اگر خط پنجم از مثال فوق را داخل یک تابع `main` بنویسید و کد خود را کامپایل کنید هشدار زیر را دریافت می‌کنید:

warning C4244: '=': conversion from 'float' to 'int', possible loss of data

اما کامپایلر علی‌رغم هشداری که صادر می‌کند این تبدیل را انجام داده و حکم انتساب بدون هیچ مشکلی اجرا می‌گردد. فقط چنانکه گفته شد تبدیل از نوعی با طول بزرگتر به نوعی با طول کوچکتر، ممکن است با از دست رفتن اطلاعات همراه باشد. به‌عنوان مثال خط چهارم قطعه کد فوق را در نظر بگیرید. در این خط متغیری از نوع `short` با طول ۲ بایت به متغیری از نوع `char` با طول ۱ بایت نسبت داده شده است. در این حالت داده‌های موجود در بایت کم ارزش نوع `short` به داخل نوع `char` ریخته می‌شود و بایت با ارزش رهایمی گردد. حال اگر داده موجود در نوع `short` بقدری بزرگ باشد که از بایت کم ارزش فراتر رود و اطلاعاتی نیز در بایت با ارزش وجود داشته باشد، آن اطلاعات از بین خواهد رفت.



شکل ۳-۸: نحوه تبدیل اتوماتیک از نوع بزرگتر به نوع کوچکتر

در جدول ۹-۳، تبدیل انواع مختلف به یکدیگر، و اطلاعاتی که ممکن است از بین برود لیست شده است.

نوع منبع	نوع مقصد	اطلاعاتی که ممکن است از بین برود
char	signed char	اگر مقدار x بیش از ۱۲۷ باشد، در مقصد عددی منفی ذخیره می‌گردد. این عدد لزوماً (-x) نیست.
short int	char	۸ بیت با ارزش از بین می‌رود.
long int	char	۲۴ بیت با ارزش از بین می‌رود.
long int	float	اگر طول عدد موجود در مبدأ بیش از ۷ رقم اعشار باشد تبدیل نوع با از دست رفتن اطلاعات همراه است.
long int	short int	۱۶ بیت با ارزش از بین می‌رود.
float	long int	بخش کسری از بین می‌رود.
float	short int	نه تنها بخش کسری بلکه مقداری از قسمت صحیح نیز از بین می‌رود.
double	float	دقت علائم کم می‌شود و نتیجه گرد می‌گردد.
long double	double , int	دقت علائم کم می‌شود و نتیجه گرد می‌گردد.

جدول ۹-۳: تبدیل انواع در احکام انتساب

شما می‌توانید در جدول فوق انواع مشابه را نیز جایگزین کنید و به جدول کامل تری برسید. به‌عنوان مثال می‌توان در ردیف دوم، تبدیل نوع `int16` به نوع `char` را نیز در نظر بگیرید. اما ما به جهت صرفه‌جویی از نوشتن موارد تکراری خودداری کرده‌ایم.

تبدیل نوع موقت (تبدیل نوع صریح)

قبل از ادامه بحث در خصوص تبدیل نوع موقت از شما می‌خواهیم که کار در کلاس زیر را انجام دهید.

کار در کلاس ۲-۳:

برنامه ای به زبان C++ بنویسید که سه عدد را از کاربر دریافت کند، سپس معدل آن سه عدد را چاپ کند.

احتمالاً پاسخی که شما برای کاردرکلاس ۳-۱ داده‌اید چیزی شبیه برنامه زیر است.

```

1. //This program calculates average of 3 integer numbers.
2. #include <iostream.h>
3. int main ()
4. {
5.     int x,y,z;
6.     cout<< "Please enter 3 numbers , this "
7.     <<"program calculate average of these numbers:\n";
8.     cout<<"\n\tEnter X=";
9.     cin>>x;
10.    cout<<"\n\tEnter Y=";
11.    cin>>y;
12.    cout<<"\n\tEnter Z=";
13.    cin>>z;
14.    float avg=(x+y+z)/3;
15.    cout<<"\nThe average of these numbers is: "<<avg;
16.    return 0;
17. }
```

برنامه فوق از نظر منطق هیچ مشکلی ندارد. اما اگر این برنامه را در یک کامپایلر بنویسید و اجرا کنید انتظار دارید به عنوان مثال برای سه ورودی ۱۵ و ۱۹ و ۱۰ معدل را برابر ۱۴/۶۶۶۷ نشان دهد. اما چیزی که مشاهده خواهید کرد تنها عدد صحیح ۱۴ است. اگر برای هر گونه ورودی دیگری نیز آزمایش خود را تکرار کنید ورودی شما یک عدد صحیح خواهد بود در حالی که شما انتظار دارید جواب شما قسمت اعشاری نیز داشته باشد. به نظر شما اشکال این برنامه در کجاست و چه راه‌حلی برای اصلاح این مشکل وجود دارد؟ قبل از این که پاسخ این پرسش را بدهیم باید مطالبی را در خصوص تبدیل نوع موقت بیان کنیم.

بحث خود را با این سؤال آغاز می‌کنیم که تبدیل نوع موقت چیست و به چه منظوری به کار می‌رود؟ اصطلاح تبدیل نوع موقت برای تبدیلاتی به کار می‌رود که به طور صریح توسط برنامه‌نویس مشخص می‌گردد. به دیگر سخن هرگاه کامپایلر قادر به انجام تبدیل نوع به صورت اتوماتیک نباشد، برنامه‌نویس باید خود این تبدیل نوع را انجام دهد که به آن تبدیل نوع صریح یا موقت گفته می‌شود، و در اصطلاح برنامه‌نویس برای این منظور باید از یک `cast` استفاده کند. در C++ استاندارد، چند مدل تبدیل نوع موقت امکانپذیر است که از جمله می‌توان تبدیل موقت استاتیک^۱، تبدیل موقت داینامیک^۲ (پویا)، تبدیل موقت تفسیری^۳، و تبدیل موقت ثابت^۴ را نام برد. در این بخش تنها به تشریح تبدیل نوع استاتیک خواهیم پرداخت و دیگر انواع تبدیل نوع موقت را در فصول آتی مطرح می‌سازیم.

-
1. `static_cast`
 2. `dynamic_cast`
 3. `reinterpret_cast`
 4. `const_cast`

اما به بحث قبلی خود باز گردیم، و مشکل برنامه صفحه قبل را برطرف کنیم. چنانکه پیشتر گفته شد برای انجام اعمال ریاضی بر روی دو عملوند باید این دو عملوند از نظر نوع و اندازه با یکدیگر برابر باشند تا پردازنده قادر به انجام عمل ریاضی مورد نظر باشد. از طرفی در عملگر تقسیم هرگاه دو عملوند مربوط به عملگر تقسیم از نوع صحیح باشند، عملیات تقسیم نیز به صورت تقسیم صحیح صورت خواهد گرفت. لذا با توجه به آنکه هر دو عملوند عملیات تقسیم در خط ۱۴ از نوع صحیح است تقسیم صورت گرفته نیز از نوع صحیح خواهد بود. اما اگر به شکلی بتوانیم یکی از این دو عملوند را به نوع اعشاری تبدیل کنیم عملگر دیگر به طور اتوماتیک به نوع اعشاری تبدیل می‌گردد و در نتیجه به جای آنکه تقسیم صحیح صورت بگیرد تقسیم اعشاری صورت می‌گیرد و قسمت اعشاری پاسخ نیز به دست می‌آید. برای این منظور می‌توان به وسیله تبدیل نوع استاتیک یکی از عملوندها را به طور موقت به نوع اعشاری تبدیل کنیم. برای این منظور خط ۱۴ کد صفحه قبل را به صورت زیر بازنویسی کنید و دوباره آن را کامپایل و اجرا نمایید. این بار چه نتایجی را در خروجی می‌بینید؟

```
14. float avg=static_cast <float> (x+y+z)/3;
```

در حقیقت با تغییرات فوق کامپیوتر را مجبور می‌سازیم حاصل عبارت داخل پرانتز را به طور موقت به نوع float تبدیل کند. سپس به طور اتوماتیک عدد صحیح 3 نیز به نوع اعشاری تبدیل شده و عملیات تقسیم به صورت اعشاری انجام می‌پذیرد. در این صورت نتیجه مورد انتظار را مشاهده خواهید کرد. البته با توجه به مباحث قبلی بهتر است به جای نوع float از نوع double استفاده کنیم تا یک تبدیل نوع ایمن انجام دهیم. همیشه در هنگام استفاده از تبدیل نوع استاتیک باید نوع جدید را بین دو علامت < > قرار دهید. همچنین باید متغیر یا عبارتی را که می‌خواهید تبدیل نوع پیدا کند در داخل پرانتز قرار دهید.

کار در کلاس ۳-۳:

در برنامه زیر با نمونه دیگری از تبدیل نوع استاتیک آشنا خواهید شد. به نظر شما فروهی برنامه زیر چیست؟

```
1. //This program is a test for signed and unsigned int.
2. #include <iostream>
3. using namespace std;
4. int main()
5. {
6.     int int_var=1500000000;           //1,500,000,000
7.     cout<<"\nInt_var1 = "<<int_var;
8.     int_var=(int_var*10)/10;
9.     cout<<"\nInt_var2 = "<<int_var;    //wrong answer
10.    int_var=1500000000;               //1,500,000,000
11.    int_var=(static_cast <double> (int_var)*10)/10;
12.    cout<<"\nInt_var3 = "<<int_var;    //right answer
13.    cout<<endl;
14.    return 0;
15. }
```

اگر برنامه موجود در کاردر کلاس ۲-۳ را در کامپایلر VC6 کامپایل و اجرا کنید خروجی زیر را مشاهده خواهید کرد:

```
Int_var1 = 15000000000
Int_var2 = 211509811
Int_var3 = 15000000000
```

در خروجی فوق همه چیز قابل پیش‌بینی بود جزء دومین خط خروجی. چرا که ما انتظار داشتیم عدد ۱۵۰۰۰۰۰۰۰۰۰ چاپ گردد اما متأسفانه خروجی اشتباهی چاپ شده است. علت این امر این است که در خط نهم برنامه با ضرب کردن متغیر `int_var` در عدد ده مقدار آن از محدوده اعداد `int` فراتر می‌رود و در نتیجه منجر به خروجی اشتباه شده است. اما در خط دوازدهم برنامه ابتدا با یک تغییر نوع استاتیک متغیر `int_var` را به‌طور موقت به نوع `double` تبدیل می‌کنیم که قادر است عدد ۱۵۰۰۰۰۰۰۰۰۰ را در خود بگنجاند و در نتیجه خروجی مورد انتظار ما چاپ شده است.

قبل از C++ استاندارد، تبدیل نوع موقت با استفاده از قالب متفاوتی انجام می‌شد. مثلاً به‌جای نوشتن عبارت:

```
float_var=static_cast <float> (int_var);
```

عبارت:

```
float_var=(float) int_var;
```

و یا عبارت زیر:

```
float_var=float (int_var);
```

نوشته می‌شد. یکی از مزایای استفاده از فرم اول مشاهده آسان آن در متن یک کد بزرگ است. بنابراین جستجوی آن نیز به‌سادگی صورت می‌گیرد. البته این فرم از تبدیل نوع استاتیک، نیز در C++ استاندارد قابل قبول است. استفاده از تبدیل نوع موقت امنیت داده را به خطر می‌اندازد. چرا که گاهی منجر به خطاهایی می‌شود که کامپایلر قادر به کشف آن خطاها نیست. بنابراین بهتر است جزء در موارد ضروری از تبدیلات موقت استفاده نکنید.

۳-۸: نحوه ایجاد فایل اجرایی توسط کامپایلر

قبل از اینکه به نحوه ایجاد فایل اجرایی توسط کامپایلر اشاره کنیم باید اشاره‌ای کوتاه به مقوله توابع و فایل‌های کتابخانه‌ای در C++ بکنیم.

توابع کتابخانه‌ای و فایل‌های کتابخانه‌ای

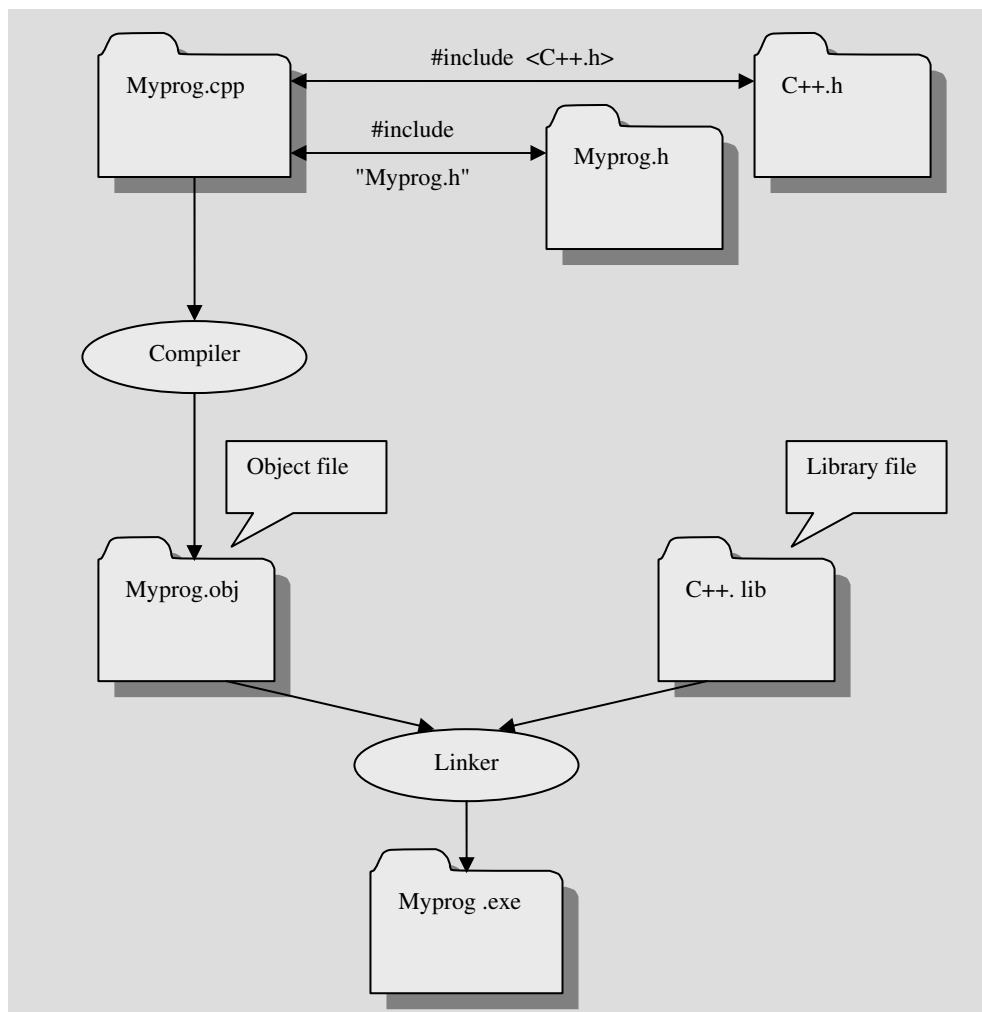
در زبان C++ برای بسیاری از کارها توابعی به صورت آماده نوشته شده است که شما می‌توانید از آنها در برنامه‌های خود استفاده کنید. به عنوان مثال برای محاسبه جذر تابعی تحت عنوان `sqrt` نوشته شده است. این توابع در فایل‌هایی موسوم به فایل‌های کتابخانه‌ای قرار دارند. این فایل‌ها حاوی برنامه اجرایی به زبان ماشین برای توابع کتابخانه‌ای هستند. فایل‌های کتابخانه‌ای دارای پسوند `.lib` هستند و هرکس می‌تواند آنها را تولید کند و در دسترس عموم قرار دهد. اما برای استفاده از توابع کتابخانه‌ای باید به کامپایلر مشخصات فایل کتابخانه‌ای حاوی تابع موردنظر خود را ارائه کنیم تا کد ماشین داخل توابع به کد برنامه ما پیوند زده شود. برای این منظور اطلاعات لازم در خصوص توابع کتابخانه‌ای در هدر فایل‌های خاصی قرار گرفته است. به عنوان مثال اطلاعات لازم در خصوص فایل کتابخانه‌ای که شامل برنامه اجرایی تابع `sqrt` است در هدر فایل `cmath` قرار دارد، و یا اطلاعات مربوط به توابع کتابخانه‌ای `scanf` و `printf` در سرفایل `stdio.h` قرار دارد. هنگام استفاده از توابع کتابخانه‌ای حتماً باید سرفایل مربوطه به برنامه اضافه شود. همان‌طور که قبلاً دیدید به واسطه دستور `#include` می‌توان انواع هدر فایل‌ها را به برنامه اضافه کرد.

همچنین در آینده خواهید دید که خود قادر به تولید هدر فایل هستید و می‌توانید کلاس‌ها اشیاء و توابع موردنظر خود را در هدر فایل‌هایی تولید و از آنها در برنامه‌های گوناگون استفاده کنید. اگر هدر فایلی را خود نوشته باشید به جای استفاده از دو نماد `<` در اطراف نام هدر فایل هنگام افزودن آن به برنامه باید از علامت کوتیشن در اطراف نام هدر فایل استفاده کنید. البته می‌توانید از علامت کوتیشن برای تمامی سرفایل‌ها بهره بگیرید.

نحوه تولید فایل اجرایی یک برنامه توسط کامپایلر

پس از آنکه برنامه خود را به طور کامل در یک ویرایشگر مثل Notepad و یا ویرایشگر خود کامپایلر زبان C++ تایپ کردید باید پسوند این فایل متنی را `.cpp` بنامید. هنگامی که فرمان کامپایل را به کامپایلر می‌دهید کامپایلر ابتدا کلیه هدر فایل‌هایی را که به برنامه خود اضافه کرده‌اید، چه آنهایی که مربوط به خود زبان C++ هستند و چه آنهایی که شما نوشته‌اید را به جای دستور `#include` مورد نظر قرار می‌دهد، اما شما تغییری در متن فایل `.cpp` خود نمی‌بینید چرا که این تغییرات در فایل‌های واسطه داده می‌شود. سپس فرآیند کامپایل آغاز شده و ابتدا اشکالات نحوی برنامه گرفته می‌شود. اگر خطایی از این نظر پیدا شود دارای عنوان `syntax error` خواهد بود. فراموش نکنید در صورت وجود خطا همواره باید اولین خطا را برطرف سازید، سپس دوباره فرآیند کامپایل را از نو اجرا کنید. همواره ممکن است یک خطا موجب اعلام تعدادی خطای نامعتبر دیگر شود در نتیجه همواره اولین خطا برای ما دارای ارزش است، و در اکثر مواقع با رفع کردن اولین خطا و کامپایل مجدد تعداد خطاها به نحو چشم‌گیری کاهش می‌یابد. پس از این مرحله اشکالات منطقی برنامه گرفته می‌شود. به خطاهایی که در این زمان توسط کامپایلر اعلام می‌شود خطای زمان ترجمه (کامپایل) گفته

می‌شود. اگر برنامه از مرحله کامپایل سربلند (بدون خطا) بیرون آید یک فایل با پسوند **.obj** ساخته می‌شود. سپس باید فرآیند ساخت فایل اجرایی را آغاز کنید. در این مرحله قسمتی از زبان C++ موسوم به پیوند دهنده^۱ شروع به کار می‌کند و فایل‌های کتابخانه‌ای لازم را که دارای پسوند **.lib** هستند به کد برنامه پیوند می‌زند. نتیجه این عملیات فایل اجرایی برنامه با پسوند **.exe** است که قابلیت اجرا شدن بر روی کامپیوتر را دارد. فایل **exe** شامل کد برنامه به صورت صفر و یک است. در شکل ۳-۹ تمامی این فرآیند را به صورت نمودار می‌توانید مشاهده کنید.



شکل ۳-۹: مراحل ایجاد فایل اجرایی

1. linker

۳-۹: تمرین

۱. تعیین کنید کدام یک از عبارات زیر درست و کدام یک نادرست هستند.
 - الف- توضیحات در برنامه، موجب می‌شود که متن بعد از // در صفحه نمایش چاپ شود.
 - ب- تمام متغیرها قبل از استفاده باید اعلان شوند.
 - ج- از نظر C++، متغیرهای `number` و `Number` یکی هستند.
 - د- تبدیل نوع `int` به `double` و یا `float` ممکن است با از دست رفتن اطلاعات همراه باشد.
 - و- اعلان متغیرها تنها باید در ابتدای تابع اصلی صورت گیرد.
 - ه- عملگر باقیمانده تقسیم، تنها می‌تواند با عملوندهایی از نوع صحیح به کار گرفته شود.
۲. در مورد شیء `cerr` تحقیق کنید.
۳. هر یک از خطوط زیر دارای چه اشکالات نحوی (`syntax error`) هستند. (در هر خط ۲ اشکال وجود دارد).


```
1. cout>>this is my first program>>'\\n';
2. cin>>67> >number;
3. int x=87; float f=56.7; x =< f++ ;
4. float f=45.6,g=0; f%=g;
```
۴. برنامه‌ای به زبان C++ بنویسید که شعاع یک دایره را بگیرد و قطر و مساحت و محیط آن را چاپ کند.
۵. در مورد خطاهای `fatal error` و خطاهای `nonfatal error` تحقیق کنید.
۶. برنامه‌ای به زبان C++ بنویسید که یک عدد پنج رقمی را بگیرد و تک تک ارقام آن را به دست آورد و با فاصله ۸ کاراکتر از هم چاپ کند. مثلاً عدد 98524 را در یافت کند و خروجی زیر را بدهد:


```
9      8      5      2      4
```
۷. ضرورت وجود توضیحات در برنامه را بیان کنید.
۸. تبدیل انواع چیست؟ با ذکر مثال‌هایی تبدیلی انواع را به طور کامل شرح دهید.
۹. تشریح کنید در چه زمانی ممکن است در تبدیل انواع اطلاعات از بین برود؟
۱۰. چگونگی تولید فایل اجرایی را از یک فایل متنی `cpp` توضیح دهید.
۱۱. با توجه به مقادیر تعیین شده هریک از عبارات زیر را ارزیابی کنید و مقادیر `k` و `L` را به دست آورید.


```
int x=8,y=10,m=6;
int k=x/4*y/2*m;
int L=x/y+ ++y/--m;
```
۱۲. تحقیق کنید که در چه مواردی ممکن است خطای `link` به وجود آید.
۱۳. برنامه‌ای بنویسید که وزن کالا را برحسب گرم بخواند و وزن آن را برحسب کیلوگرم چاپ کند.
۱۴. برنامه‌ای بنویسید که صورت و مخرج دو کسر را بخواند و حاصل جمع و تفریق و ضرب و تقسیم دو کسر را در خروجی بنویسد.

۱۵. در موارد زیر برنامه های داده شده را از روی لوح فشرده شماره یک پیدا کرده و در کامپایلر خود اجرا کنید و خروجی آن را ببینید. سپس در مورد تک تک خطوط هر برنامه توضیح دهید.

I.

```
1. #include "iostream.h"
2. int main()
3. {
4.     cout<<65<<endl;
5.     cout<<'A';
6.     cout<<'\\n'<<(int) 'A'<<"\\a\\n";
7.     return 0 ;
8. }
```

II.

```
1. #include <iostream>
2. using namespace std;
3. void main()
4. {
5.     int m,n;
6.     m=(n=66)+9;
7.     cout<<m<<" , "<<n<<endl;
8. }
```

III.

```
1. #include <iostream.h>
2. #include <limits.h>
3. int main()
4. {
5.     //prints the contents stored in limits.h
6.     cout<<"\\n minimum char = "<<CHAR_MIN;
7.     cout<<"\\n maximum char = "<<CHAR_MAX;
8.     cout<<"\\n minimum short = "<<SHRT_MIN;
9.     cout<<"\\n maximum short = "<<SHRT_MAX;
10.    cout<<"\\n minimum int = "<<INT_MIN;
11.    cout<<"\\n maximum int = "<<INT_MAX;
12.    cout<<"\\n minimum long = "<<LONG_MIN;
13.    cout<<"\\n maximum long = "<<LONG_MAX;
14.    return 0;
15. }
```

IV.

```
1. #include "iostream"
2. using namespace std;
3. int main()
4. {
5.     int x=6,y=8;
6.     cout<<"x="<<x<<" , y="<<y<<endl;
7.     cout<<"Squared hypotenuse is:"<<x*x+y*y<<endl;
8.     cout<<"(6/8)*8 is equal to "<<(6/8)*8<<endl ;
9.     return 0;
10. }
```

۳-۱۰: موارد مطالعاتی (Case Studys)

۱. در متن لاتین زیر نحوه کامپایل برنامه‌های C++ از طریق خط فرمان توضیح داده شده است. ضمن مطالعه این متن، مراحل گفته شده در آن را به‌عنوان یک آزمایش اجرا کرده و گزارشی از آن تهیه کنید.

Command Line Mode:

In order to use command line compiling, first copy this file:

C:\Program Files\Microsoft Visual Studio\VC98\bin\vcvars32.bat

to your *C:\WINDOWS* folder. This has already been done on the computer lab PCs.

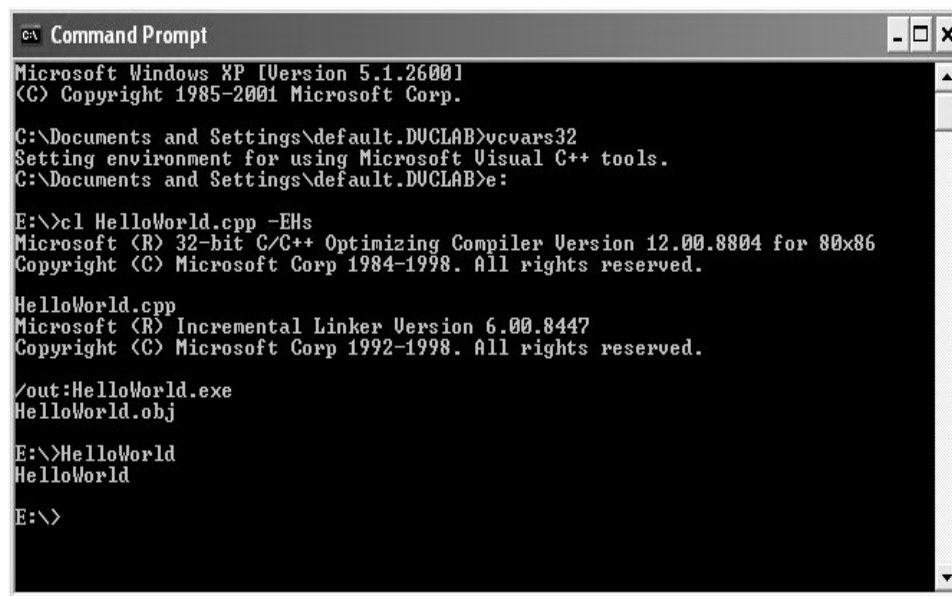
Use any text editor of your choosing, such as XP's Notepad, or JNotePad. Then when you go to a command prompt to compile and build, enter the command *vcvars32* once to prepare for using the command line editor. When saving CPP files from Notepad, put the filename in quotes to prevent Windows from adding .txt to the filename. *To compile*, use a command like the following (the command is "see-el", not "see-one") to create an EXE file:

```
cl HelloWorld.cpp -EHs
```

To compile and build projects consisting of *more than one CPP*, list the CPPs separated by spaces, like this:

```
cl main.cpp Time.cpp -EHs
```

The EXE has the same name as the first (or only) listed CPP, but with the extension "exe". *To run* the program, enter the name of the EXE on the command line -- you may leave off the trailing ".exe".



```

C:\> Command Prompt
Microsoft Windows XP [Version 5.1.2600]
(C) Copyright 1985-2001 Microsoft Corp.

C:\Documents and Settings\default.DUCLAB>vcvars32
Setting environment for using Microsoft Visual C++ tools.
C:\Documents and Settings\default.DUCLAB>e:

E:\>cl HelloWorld.cpp -EHs
Microsoft (R) 32-bit C/C++ Optimizing Compiler Version 12.00.8804 for 80x86
Copyright (C) Microsoft Corp 1984-1998. All rights reserved.

HelloWorld.cpp
Microsoft (R) Incremental Linker Version 6.00.8447
Copyright (C) Microsoft Corp 1992-1998. All rights reserved.

/out:HelloWorld.exe
HelloWorld.obj

E:\>HelloWorld
HelloWorld

E:\>

```

شکل ۳-۱۰: نمونه‌ای از کامپایل از طریق خط فرمان

To compile a CPP *without building*, include the `-c` flag -- this produces an OBJ file with the same name as the listed CPP, but with the extension "obj":

```
cl Time.cpp -c -EHs
```

To build an EXE from already-compiled OBJs, list the OBJs and do not use the `-EHs` flag, like this (creating *main.exe*):

```
cl main.obj Time.obj
```

When working with multiple CPP files in a single project, it is recommended to compile each CPP separately, using the `-c` flag during development. This makes debugging easier. Once the program is working, and you are making small code adjustments, then you should go back to compiling and building all in one command.

Using XP's Command Line Buffer

So that you do not have to retype the compile and run commands, use the up and down arrow keys to navigate through previously-typed commands. Use the F7 key to popup a list of commands in the buffer. The usual sequence is to type the compile and build command, followed by the run command. After that, *up-up* returns to the compile and build command, and *down* goes from there to the run command.

۲. در خصوص نحوه ورود و خروج اطلاعات در زبان C که توسط توابع `scanf` و `printf` صورت می گیرد تحقیق کنید.

۳. اگر سعی کنید با شیء `cout` متغیری که از نوع `int64`__ است را به خروجی ببرید خواهید دید که کامپایلر پیغام خطایی را به شما اعلام می کند، این یکی از اشکالات یا به اصطلاح `bug` های موجود در کامپایلر VC6 می باشد که در نسخه های بعدی اصلاح شد. اما روشی وجود دارد که می توانید این نوع متغیر را به خروجی ببرید. در شبکه جهانی اینترنت در این خصوص تحقیق کنید.

۴. در برخی سیستم ها و صفحه کلیدهای قدیمی برخی از کاراکترهای خاص مثل { یا } یا # یا & موجود نبوده است. لذا یک مجموعه کاراکتر به نام `Trigraph` (سه کاراکتر دنبال هم) مخصوص زبان C و `Digraph` (دو کاراکتر دنبال هم) مخصوص زبان C++ طراحی شدند که این نقیصه را برطرف کنند. مثلاً می توان به جای علامت { از علامت <?? و یا به جای { از علامت >?? استفاده کرد. و یا آنکه می توان به جای علامت & از کلمه `bitand` و به جای علامت && از کلمه `and` بهره گرفت. در مستندات MSDN و یا اینترنت در این خصوص تحقیق کنید. البته با توجه به آنکه استفاده از این علائم از خوانایی برنامه می کاهد و به علاوه اینگونه مشکلات در صفحه کلیدهای جدید برطرف شده است اکثر کامپایلرهای C++ از کاراکترها و کلمات رزروی تعریف شده در جدول `Digraph` پشتیبانی نمی کنند، و تنها از جدول `Trigraph` و آن هم تنها با هدف پشتیبانی کردن از تمامی امکانات زبان C پشتیبانی می کنند. شایان ذکر است که کلمات تعریف شده در جدول `Digraph` جزء کلمات کلیدی زبان C++ محسوب نمی شوند.

۵. بد نیست بدانید که اگر در تبدیل نوع، نوعی با تعداد بایت کمتر به نوعی با تعداد بایت بیشتر تبدیل شود به آن تبدیل نوع ارتقاء دهنده^۱ و در حالتی که نوعی با تعداد بایت بیشتر به نوعی با تعداد بایت کمتر تبدیل می‌شود به آن تبدیل نوع محدود کننده^۲ گفته می‌شود.

۶. تحقیق کنید که ساختار یک متغیر `double` و یا `long double` چگونه است؟ و تشریح کنید که چرا با توجه به این ساختار به کارگیری کلمه `unsigned` همراه این دو نوع موجب ایجاد نوع بدون علامت؛ و گسترش بازه اعداد مثبت قابل ذخیره در متغیرهای اعشاری دقت مضاعف نمی‌شود.

۷. در C++ ضمن انجام عمل انتساب، عملگر انتساب مقداری را که در عملگر سمت چپ خود جایگزین می‌کند، باز می‌گرداند. در فصل پنجم خواهید دید که چگونه می‌توان از یک عملیات انتساب در داخل عبارات شرطی استفاده کرد.

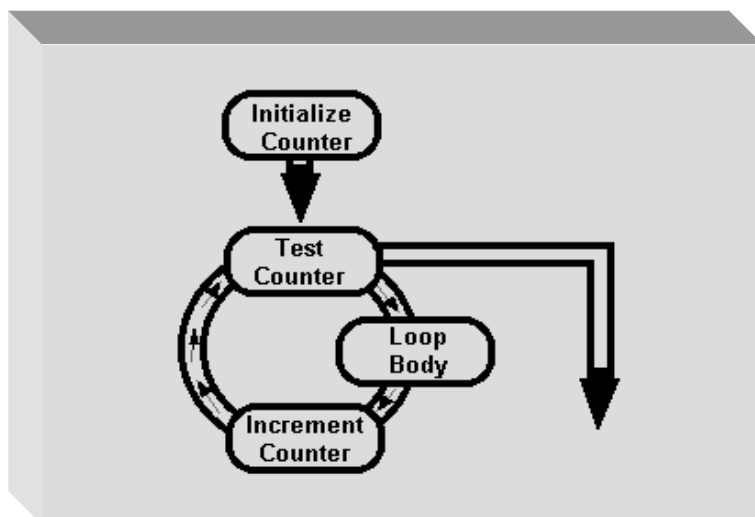
۸. در مورد این که چگونه اعمال اصلی ریاضی بر روی دو عدد ممیز شناور در کامپیوتر اعمال می‌شود تحقیق کنید؟
 ۹. به خاطر دارید که دقت متغیرهای `float` تا هفت رقم اعشار است. حال فرض کنید که می‌خواهیم حاصل جمع یازده عدد را به خروجی ببریم. از این یازده عدد یکی برابر 10^7 و ده تای دیگر برابر ۱ می‌باشند. سمیرا و سمیه هر یک دو روش متفاوت را برای یافتن این حاصل جمع به کار برده‌اند. سمیرا هر بار یکی از یک‌ها را به عدد بزرگتر که در ابتدا 10^7 می‌باشد می‌افزاید و در نهایت پس از انجام این عمل به تعداد ده مرتبه مقدار حاصل جمع را به خروجی می‌برد. اما سمیه ابتدا ده تا ۱ را با یکدیگر جمع می‌کند و سپس عدد ۱۰ را با عدد 10^7 جمع می‌کند و نتیجه را به خروجی می‌برد. به نظر شما روش کدام یک نتیجه صحیح‌تری را حاصل می‌کند. از این بحث چه نتیجه‌ای گرفتید؟

راهنمایی: برنامه‌ای به زبان C++ برای روش سمیرا و سمیه بنویسید و سپس نتایج را با یکدیگر مقایسه کنید.

۱۰. در خصوص گلوگاه وان نویمان^۳ که عاملی برای کاهش سرعت در معماری وان نویمان است تحقیق کنید.

۱۱. در مورد انواع روش‌های ترجمه و تولید فایل اجرایی توسط کامپایلرهای متفاوت مثل روش پیاده‌سازی کامپایلری، یا روش تفسیر محض و استفاده از مفسر^۴ و یا روش سیستم‌های پیاده‌سازی مختلط^۵ تحقیق کنید؟ همچنین از استاد خود بخواهید در خصوص ماشین زمان اجرا مثل ماشین زمان اجرای (مجازی) جاوا (Java Virtual Machine) اندکی اطلاعات در اختیار شما قرار دهد.

-
1. widening
 2. narrowing
 3. Von Neumann bottleneck
 4. interpreter
 5. hybrid implementation system



فصل چهارم

ساختارهای حلقه‌زنی

اهداف فصل و چکیده مطالب :

۱. مهمترین ویژگی الگوریتم‌های ساخت یافته
۲. آشنایی با انواع جعبه‌های تکرار
۳. مثال‌های کاربردی از به‌کارگیری جعبه‌های تکرار
۴. آشنایی با حلقه‌های تودرتو

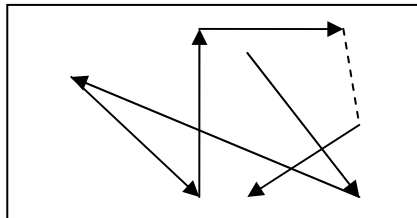
۴-۱: الگوریتم‌های ساخت یافته**مقدمه**

در فصل دوم با انواع جعبهٔ تصمیم و چگونگی ایجاد حلقه به واسطهٔ این جعبه‌ها آشنا شدیم. اما به جهت اهمیت حلقه زنی در الگوریتم‌ها در این فصل چند نماد جدید را برای ایجاد انواع حلقه، معرفی می‌کنیم. اهمیت این ساختارها زمانی مشخص می‌شود که بخواهیم یک الگوریتم را از جهت ساخت یافتگی تحلیل کنیم، و یا بخواهیم از حلقه‌های تودرتو استفاده کنیم. لذا ابتدا به بررسی مهمترین ویژگی الگوریتم‌های ساخت یافته می‌پردازیم، تا به جایگاه این ساختارهای حلقه زنی پی‌بریم. حال این سؤال را مطرح می‌سازیم که به چه الگوریتم‌هایی ساخت یافته و به چه الگوریتم‌هایی غیرساخت یافته گفته می‌شود؟ و یا به زبان ساده‌تر می‌توان چنین عنوان کرد که کارنمای الگوریتم‌های ساخت یافته چه ویژگی بارزی دارد؟ اما قبل از پاسخگویی به این سؤال از شما می‌خواهیم که کار در کلاس زیر را انجام دهید.

کاردر کلاس ۴-۱:

کارنمایی رسم کنید که تعدادی سه تایی (x, y, z) را از ورودی بفوائد و ضمن شمارش تعداد سه تایی‌ها بزرگترین مقدار هر سه تایی را چاپ کند. یک نماد را به عنوان نماد انتهای ورود داده در نظر بگیرید. در نهایت نیز تعداد سه تایی‌ها باید چاپ شود.

هدف ما از طرح این مسئله این است که شما را به نقش حلقه‌های تکرار در ساخت یافتگی و یا عدم ساخت یافتگی یک الگوریتم واقف سازیم. چنانکه از ظاهر این مسئله پیدا است باید یک حلقه تکرار برای دریافت تعداد نامشخصی سه تایی مرتب داشته باشیم. این حلقه باید با روش نگهبان کنترل شود اما الگوریتم واحد را چگونه پیاده‌سازی کنیم. یک راه پیاده‌سازی الگوریتم واحد به صورتی است که در شکل ۴-۳ مشاهده می‌کنید. کارنمای ارائه شده در این شکل شامل یک الگوریتم غیرساخت یافته است. در مقابل در شکل ۴-۴ کارنمای دیگری برای این مسئله ارائه شده که یک الگوریتم ساخت یافته را به تصویر کشیده است. به نظر شما چه تفاوت بارزی بین این دو کارنما مشاهده می‌شود؟ همان‌طور که ممکن است شما نیز حدس زده باشید نوعی نظم در کارنمای شکل ۴-۴ به وضوح قابل مشاهده است که در الگوریتم شکل ۴-۳ وجود ندارد. در کارنمای شکل ۴-۳ برای یافتن بزرگترین عدد از بین سه عدد x و y و z مدام به بالا و پایین رفته‌ایم و نوعی بی‌نظمی را بر روندنمای خود تحمیل کرده‌ایم. شاید تعبیر حرکت کاتوره‌ای که در علم فیزیک مطرح می‌شود و در شکل ۴-۱ نشان داده شده بی‌تناسب با روندنماهای غیرساخت یافته نباشد.



شکل ۴-۱: نمادی برای حرکت کاتوره‌ای در کارنماهای غیرساخت یافته

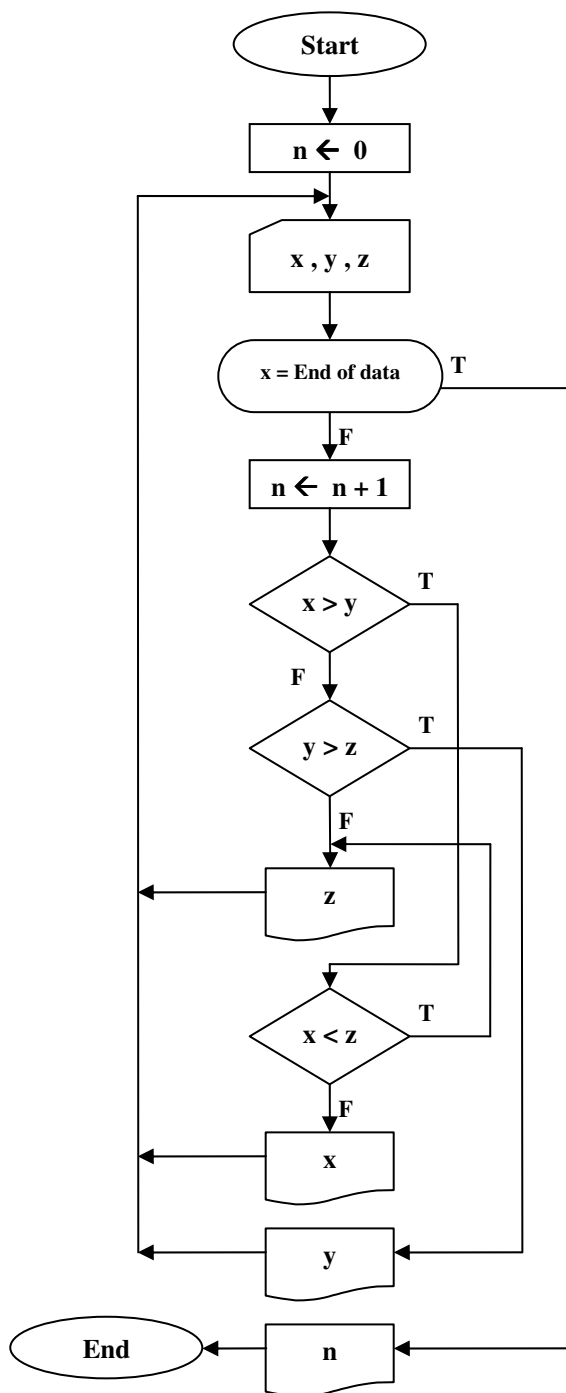
اما اگر با دقت بیشتری به کارنمای شکل ۴-۴ توجه شود، می‌توان نظم موجود در این کارنما را توصیف کرد، و با تعمیم این توصیف به قاعده‌ای جهت تمییز دادن کارنماهای ساخت یافته از کارنماهای غیرساخت یافته دست یافت. بنابراین با توجه به این کارنما قاعده زیر را به عنوان ویژگی بارز کارنماهای ساخت یافته مطرح می‌کنیم:

در کارنماهای ساخت یافته روند حرکتی کارنما دارای جریانی از سمت بالا به پایین است. البته ممکن است در برخی از نقاط کارنما با حلقه روبرو شویم اما در نهایت پس از خروج از حلقه نیز روند روبه پایین کارنما ادامه پیدا می‌کند. بنابراین در الگوریتم ساخت یافته تنها بازگشت به عقب، به منظور ساخت حلقه مجاز است.

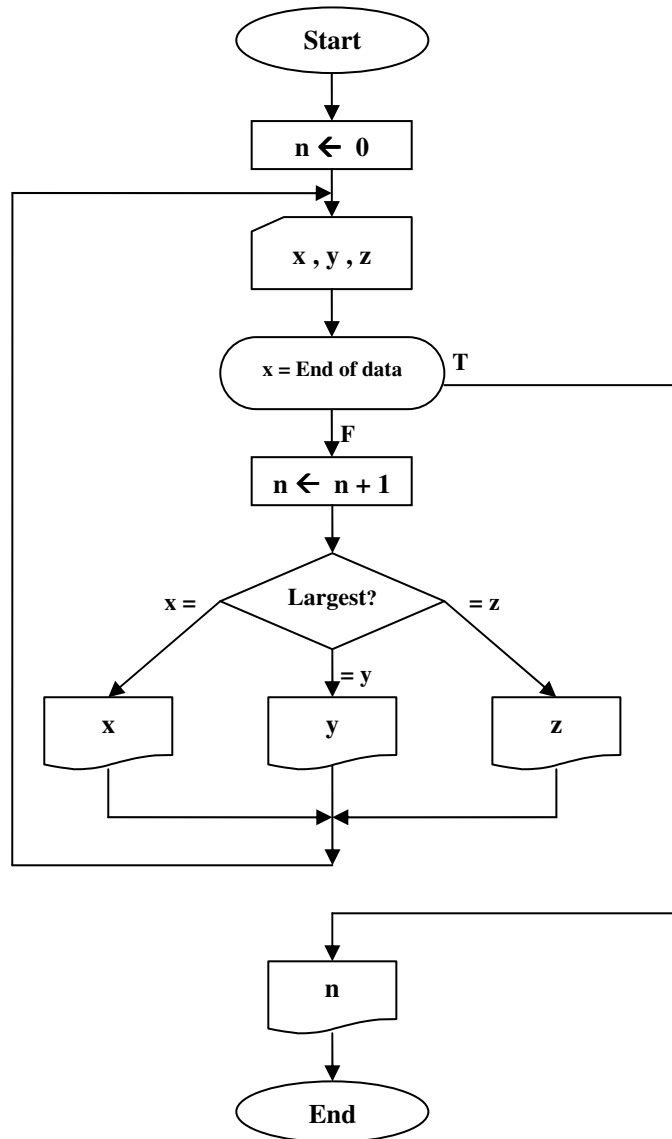
در شکل ۴-۲ نمادی برای روند روبه پایین کارنماهای ساخت یافته آمده است.



شکل ۴-۲: نمادی برای روند حرکتی کارنماهای ساخت یافته



شکل ۴-۳: الگوریتمی غیر ساخت یافته برای کار در کلاس ۴-۱



شکل ۴-۴: الگوریتمی ساخت یافته برای کاردرکلاس ۱-۴

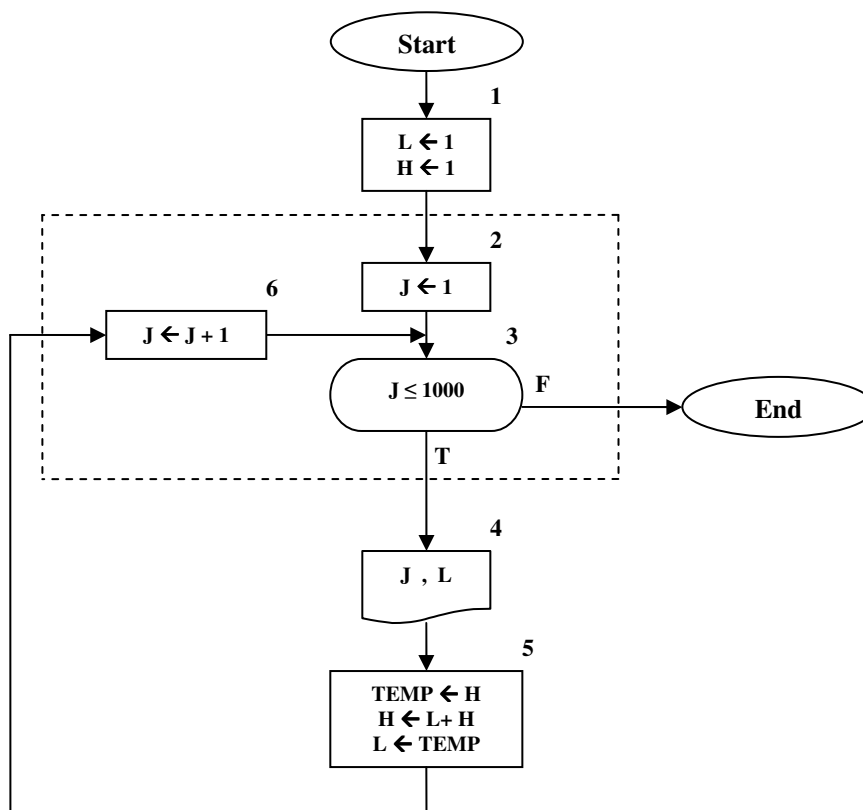
در فصل آتی یکبار دیگر به این دو الگوریتم بازخواهیم گشت و در آنجا از شما خواهیم خواست برنامه‌ای به زبان C++ برای این کارنها بنویسید، تا به تفاوت برنامه‌های ساخت یافته با برنامه‌های غیرساخت یافته نیز پی ببرید. اما برای آنکه از این پس، بازگشت به عقبی که در نتیجه قرار دادن حلقه در کارنها حاصل می‌شود را با حرکت‌های نامنظمی که الگوریتم را از ساخت یافتگی خارج می‌کند اشتباه نگیرید و قادر به تشخیص این دو از یکدیگر باشید؛ در بخش ۲-۴ ساختارهایی را برای حلقه‌زنی در کارنها ارائه می‌کنیم.

۴-۲: جعبه های تکرار

حلقه های دارای شمارنده

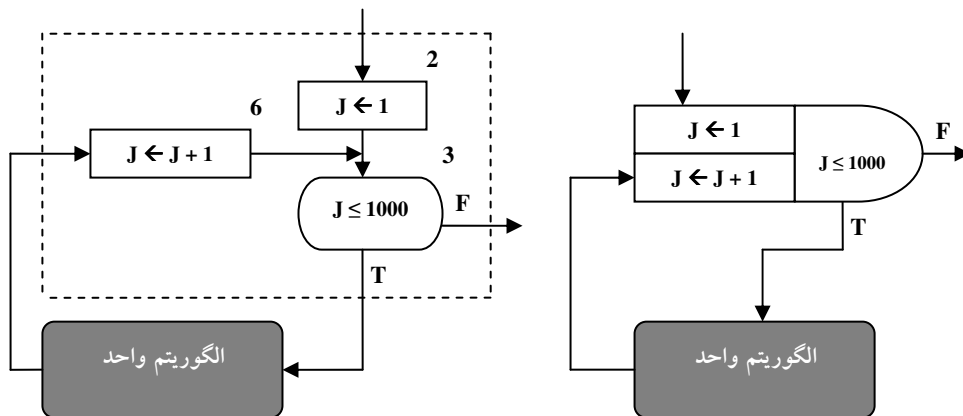
یکبار دیگر به بخش ۴-۲ بازگردید و شکل ۴-۲ که در خصوص الگوریتم‌های دارای حلقه تکرار با مکانیسم شمارنده بود را مورد بازبینی قرار دهید. ابتدا تصمیم داریم کارنمایی رسم کنیم که دارای چنین حلقه تکراری باشد سپس جعبه‌ای را برای اینگونه حلقه‌های تکرار ارائه می‌کنیم و سعی خواهیم کرد از این پس کلیه کارنماهای دارای حلقه تکرار با شمارنده را با جعبه‌هایی که برای مکانیسم تکرار در این قسمت ارائه می‌دهیم، رسم کنیم. لذا ابتدا به مثال زیر توجه کنید.

مثال ۴-۱: در زیر کارنمایی ارائه شده که هزار جمله اول دنباله فیبوناچی را چاپ می‌کند. از آنجا که در این مسئله با تعداد مشخصی جمله روبرو هستیم استفاده از حلقه تکرار دارای شمارنده اجتناب‌ناپذیر است. در دنباله فیبوناچی هر جمله برابر حاصل جمع دو جمله ما قبل خود است، و جملات اول و دوم برابر یک می‌باشند. با این تفاسیر کارنمای زیر را می‌توان برای این مسئله ترسیم کرد.



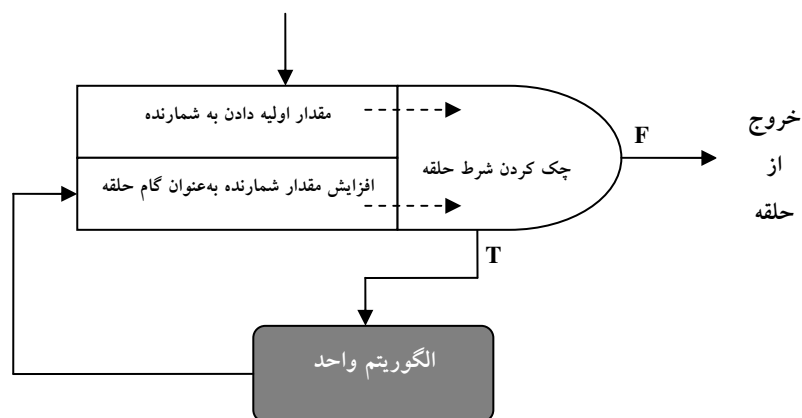
شکل ۴-۵: کارنمای مثال ۴-۱ (چاپ هزار جمله اول دنباله فیبوناچی)

یکبار دیگر به کارنامای شکل ۴-۵ توجه کنید. در این کارناما تمامی جعبه‌ها به ترتیب روند اجرا شماره گذاری شده‌اند. همچنین سه جعبه ۲ و ۳ و ۶ را با خطوط مقطع از دیگر بخش‌های کارناما جدا کرده‌ایم، چرا که این جعبه‌ها قلب عمل تکرار را تشکیل می‌دهند. در حقیقت جعبه‌های ۴ و ۵ را می‌توان به‌عنوان الگوریتم واحد در نظر گرفت. بنابراین تنها جعبه‌های ۲ و ۳ و ۶ هستند که موجب پیدایش گردش در کارناما می‌شوند. در جعبه شماره ۲ ابتدا شمارنده را با عدد یک مقدار اولیه داده‌ایم، در جعبه شماره ۳ شرط حلقه را چک می‌کنیم و در جعبه شماره ۶ مقدار شمارنده را یک واحد اضافه می‌کنیم که می‌توان تعبیر گام برداشتن را برای جعبه شماره ۶ در نظر بگیریم. در شکل ۴-۶ تمامی مراحل مقدار اولیه دادن به شمارنده، چک کردن شرط حلقه و گام برداشتن در جعبه‌ای فشنگی شکل موسوم به "جعبه تکرار با شمارنده" شبیه‌سازی شده است.



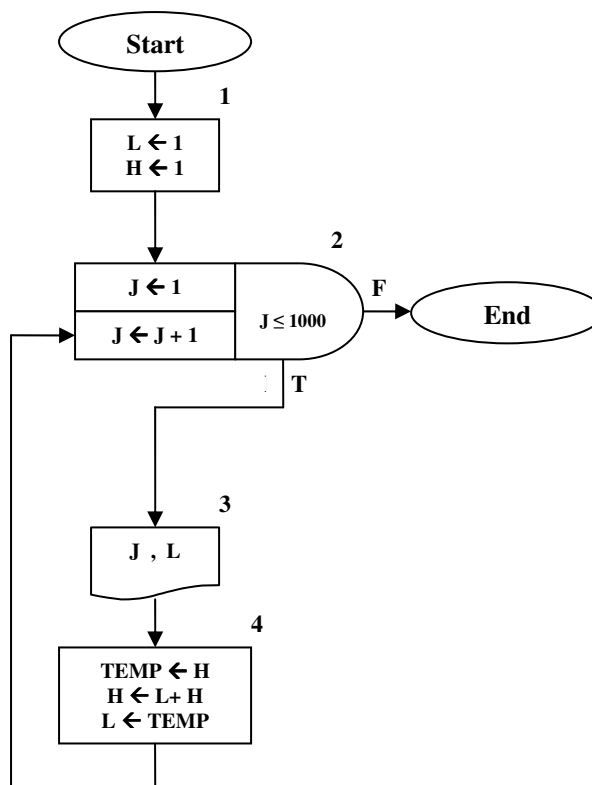
شکل ۴-۶: جعبه تکرار برای حلقه‌های دارای شمارنده

در شکل ۴-۷ مراحل حرکت در یک جعبه تکرار با شمارنده با جزئیات بیشتری نشان داده شده است.



شکل ۴-۷: ساخت حلقه تکرار به‌عنوان یک قدم واحد

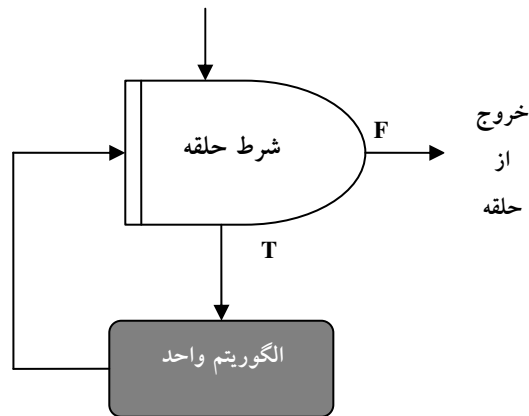
کارنمای مثال ۴-۱ را یکبار دیگر بازنویسی می‌کنیم اما این بار از جعبه تکرار برای ایجاد حلقه تکرار موجود در این الگوریتم استفاده می‌کنیم.



شکل ۴-۸: کارنمای دنباله فیبوناچی با جعبه تکرار با شمارنده

در کارنمای فوق پس از ورود به جعبه شماره یک، به قسمت مقدار اولیه جعبه تکرار وارد می‌شویم. سپس شرط حلقه چک شده و با توجه به درست بودن شرط، الگوریتم واحد، اجرا می‌گردد. پس از آن دوباره به جعبه تکرار باز می‌گردیم، اما این بار در قسمت گام حلقه وارد می‌شویم. در قسمت گام حلقه، مقدار گام حلقه به شمارنده اضافه می‌شود. توجه داشته باشید که گام حلقه همواره برابر یک نیست، به‌عنوان مثال در برخی موارد که مقدار شمارنده در محاسبات الگوریتم واحد شرکت می‌کند، ممکن است خواستار آن باشیم که مقدار شمارنده دوتا دوتا اضافه گردد. (یعنی گام حلقه را برابر ۲ می‌گیریم). لذا پس از افزایش مقدار گام حلقه به شمارنده دوباره شرط حلقه چک می‌گردد تا در صورت درست بودن شرط حلقه الگوریتم واحد، اجرا گردد و در صورت نادرست بودن شرط حلقه، از جعبه تکرار خارج می‌شویم.

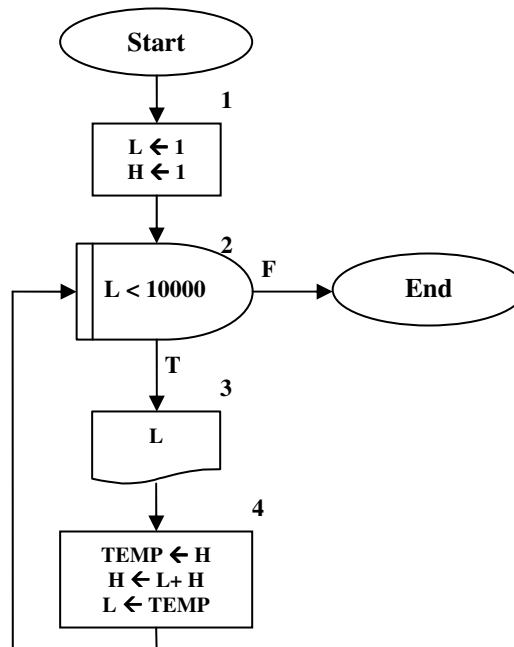
چنانکه در بخش ۲-۴ نیز دیدید نوع دیگری از حلقه‌های تکرار وجود دارد که بدون شمارنده و به واسطه نگهدارنده کنترل می‌شود. این حلقه‌ها تا زمانی که شرط حلقه درست باشد اجرا می‌شوند. برای اینگونه حلقه‌ها نیز نوعی جعبه تکرار را در شکل ۴-۹ ارائه کرده‌ایم که شباهت زیادی به جعبه‌های تصمیم دارد.



شکل ۴-۹: جعبه تکرار برای حلقه‌های دارای نگهدارنده

چنانکه پیشتر نیز متذکر شدیم باید شرایطی در داخل دستورات الگوریتم واحد در حلقه‌های دارای نگهدارنده وجود داشته باشد تا پس از اجرای حلقه به دفعات موردنظر، شرط حلقه نقض گردد و در نتیجه از حلقه خارج شویم. این نوع حلقه در بسیاری از زبان‌های برنامه‌نویسی به حلقه‌های `while` معروف است. در مثال زیر نمونه‌ای از کاربرد جعبه تکرار دارای نگهدارنده را می‌بینید.

مثال ۴-۲: کارنمایی که جملات دنباله فیبوناچی را تا جایی که کوچکتر از ۱۰.۰۰۰ باشد چاپ می‌کند.



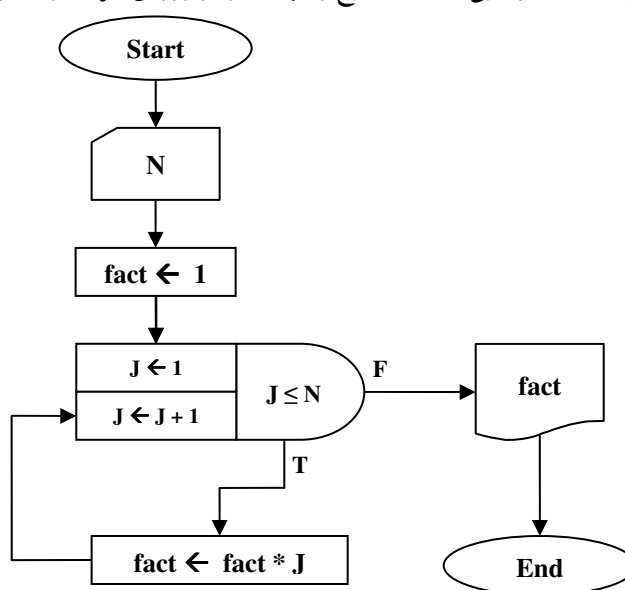
شکل ۴-۱۰: کارنمای دنباله فیبوناچی با جعبه تکرار با نگهدارنده

۳-۴: مثال‌هایی بیشتر از به‌کارگیری جعبه‌های تکرار

حلقه‌های تکرار دارای کاربردهای متعددی خصوصاً در زمینه کار با آرایه‌ها و متغیرهای لیستی هستند. اما از آنجا که بحث در خصوص آرایه‌ها را به فصل جداگانه‌ای موکول کرده‌ایم، در این بخش تصمیم داریم با مطرح کردن مثال‌های دیگری از کاربرد حلقه‌های تکرار در مسائل گوناگون، نحوه به‌کارگیری جعبه‌های تکرار را بیشتر بیاموزیم.

کار با فاکتوریل در الگوریتم‌ها

مثال ۳-۴: کارنمایی که عدد صحیح و مثبت N را از ورودی خوانده و مقدار $N!$ را محاسبه و چاپ می‌کند.



شکل ۴-۱۱: کارنمای محاسبه فاکتوریل

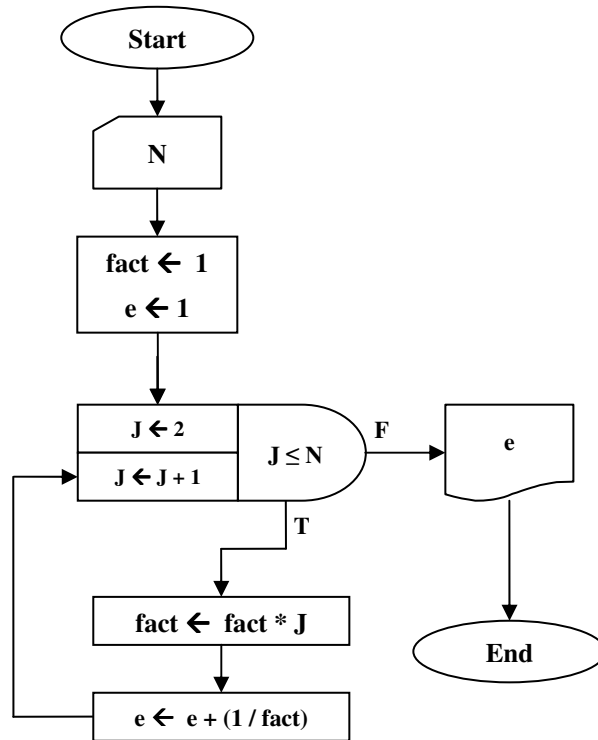
توضیح کارنما: با توجه به آن که مقدار $N!$ برابر $1 \times 2 \times 3 \times \dots \times N$ است. ابتدا متغیر $fact$ را با یک مقدار اولیه داده‌ایم. زیرا حتی $0!$ هم برابر یک است. اما در صورتی که $N > 0$ باشد به تعداد دفعات موردنظر، مقدار شمارنده را در متغیر $fact$ ضرب می‌کنیم. این عمل تا جایی ادامه پیدا می‌کند که شرط $J \leq N$ برقرار باشد، و در نتیجه مقدار $N!$ محاسبه می‌گردد.

مثال ۴-۴: کارنمایی که مقدار e (پایه لگاریتم نهرین) را با استفاده از رابطه زیر محاسبه می‌کند.

$$e = 1 + \frac{1}{2!} + \frac{1}{3!} + \dots + \frac{1}{n!}$$

توضیح الگوریتم: برای محاسبه سری فوق به یک متغیر مانند e نیازمندیم که مجموع به‌دست آمده را در خود نگهدارد. از طرفی به یک حلقه تکرار نیازمندیم که هربار عملیات محاسبه یک جمله از این سری را تکرار کند، و در نهایت مقدار

به‌دست آمده را با متغیر e جمع کند تا مجموع جدید به‌دست آید. در این کارنما نیز متغیر $fact$ مقدار فاکتوریل را برای جمله‌ای که هم اکنون در حال محاسبه آن هستیم، در خود نگه می‌دارد.



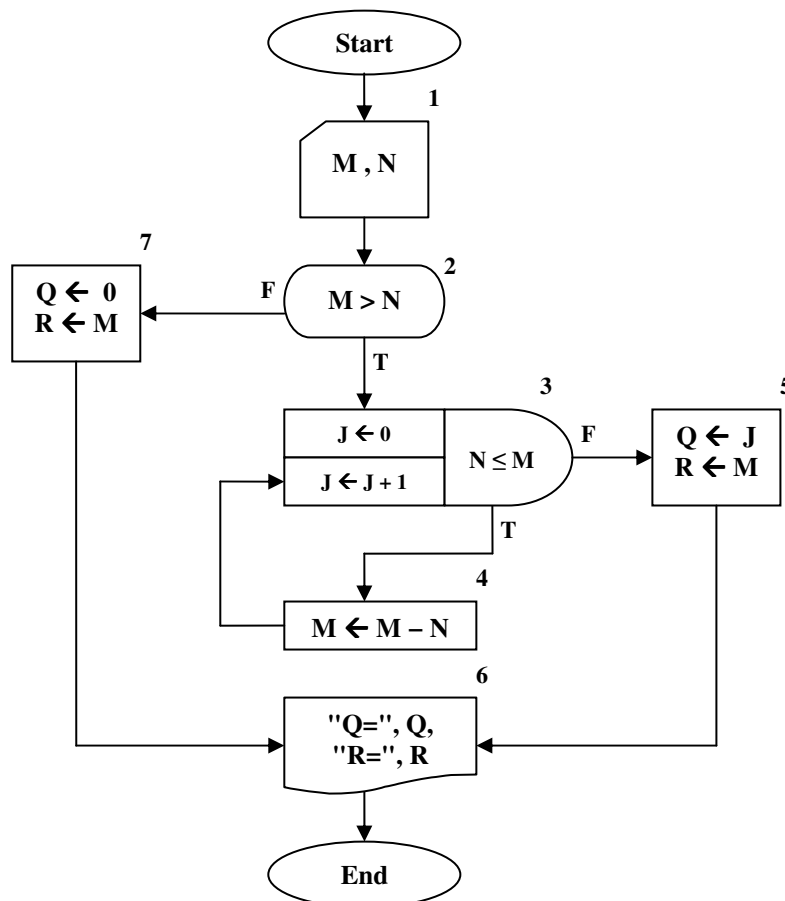
شکل ۴-۱۲: کارنمای محاسبه عدد e

مسائل مرتبط با ساده‌سازی و تقسیم اعداد

مثال ۴-۵: کارنمایی که دو عدد صحیح و مثبت را از ورودی خوانده، و آنها را با عمل تفریق بر هم تقسیم می‌کند.

توضیح الگوریتم: هنگامی که عدد M را بر عدد N تقسیم می‌کنیم، در حقیقت تعداد N ‌هایی که می‌توان در عدد M جدا کرد را می‌شماریم. بنابراین می‌توانیم هر بار که یک N را از M جدا می‌کنیم یک واحد به شمارنده بیفزاییم. این عمل تا جایی می‌تواند ادامه پیدا کند که N بزرگتر از باقی‌مانده M از مرحله قبل باشد. هنگامی که N از M کوچکتر شود مقدار شمارنده برابر خارج قسمت تقسیم و آنچه که در M باقی می‌ماند برابر باقی‌مانده تقسیم می‌باشد. کارنمای این مثال در شکل ۴-۱۳ ارائه شده است. در این کارنما R برابر باقی‌مانده و Q برابر خارج قسمت تقسیم می‌باشد.

به نظر شما آیا لزومی بر قراردادن جعبه‌های ۲ و ۷ در الگوریتم شکل ۴-۱۳ وجود داشت؟ آیا با حذف این دو جعبه اتفاق خاصی رخ می‌دهد؟ شما چه نتیجه‌ای از این بحث می‌گیرید؟

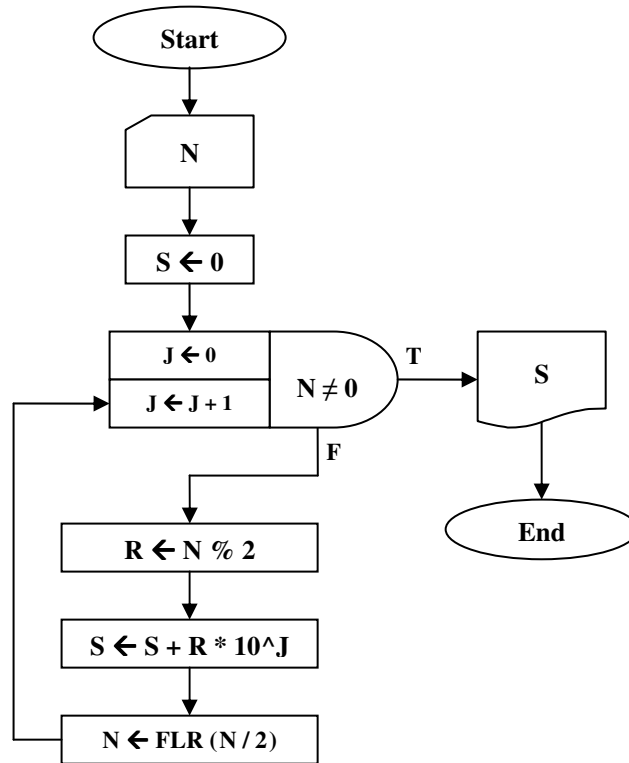


شکل ۴-۱۳: کارنمای محاسبه تقسیم به واسطه عمل تفریق

مثال ۴-۶: کارنمایی که یک عدد مبنای ۱۰ را از ورودی خوانده، و آن را به مبنای دو می‌برد و نتیجه را

چاپ می‌کند.

توضیح الگوریتم: تبدیل یک عدد از مبنای ۱۰ به مبنای ۲، با تقسیم‌های متوالی انجام می‌شود، به‌طوری که باقی‌مانده‌ها باید از آخرین باقی‌مانده به اولین باقی‌مانده چاپ شود. چنانکه می‌بینید اولین رقم خروجی در آخرین مرحله به‌دست می‌آید. یک راه برای چاپ این باقی‌مانده‌ها ذخیره آنها در یک مکان از حافظه است، تا در نهایت بتوان آنها را به ترتیب از آخر به اول چاپ کرد. اما این کار چندان عقلانی نیست چون ما را به یک تعداد خانه از حافظه محدود می‌کند، و اگر عدد ورودی از یک مقدار حداکثری، بیشتر باشد الگوریتم ما با مشکل مواجه می‌شود. اما یک راه دیگر آن است که نتیجه دودویی خود را به‌صورت یک عدد مبنای ۱۰ فرض کنیم، با این توضیح که این عدد تنها از ارقام صفر و یک تشکیل شده است. بنابراین از آنجا که باقی‌مانده هر مرحله باید به‌عنوان رقم متتعالیه سمت چپ عدد خروجی، قرارگیرد: می‌توان باقی‌مانده جدید را در توانی از ده ضرب کرد و با حاصل مرحله قبل جمع نمود تا در نهایت نتیجه در مبنای دو به‌دست آید.



شکل ۴-۱۴: کارنمای تبدیل یک عدد مبنای ۱۰ به عددی در مبنای ۲

هم عامل های یک عدد صحیح

پیشتر در مثال ۲-۸ الگوریتمی ارائه کردیم که تمامی مقسوم‌علیه‌های یک عدد صحیح را به‌دست می‌آورد. اما در این قسمت تصمیم داریم الگوریتمی ارائه کنیم که تمامی «هم‌عامل‌های» یک عدد صحیح مثل N را چاپ کند. مقصود از هم‌عامل‌های یک عدد صحیح مثل N ، تمامی جفت اعدادی است که حاصل ضرب آنها مساوی خود عدد N گردد. به‌عنوان مثال هم‌عامل‌های عدد ۲۰ شامل سه دوتایی (۱، ۲۰) و (۲، ۱۰) و (۴، ۵) می‌باشد. در حقیقت این اعداد مقسوم‌علیه‌های ۲۰ هستند که به‌صورت جفت‌های دوتایی انتخاب شده‌اند، و یا هم‌عامل‌های عدد ۲۵ عبارت است از (۱، ۲۵) و (۵، ۵). بنابراین رابطه‌ای تنگاتنگ بین این مسئله و پیدا کردن مقسوم‌علیه‌های عدد N وجود دارد، بنابراین از تجربه‌های خود در مثال ۲-۸ استفاده خواهیم کرد. یک راه ساده برای حل این مسئله چنین است: «تمامی مقسوم‌علیه‌های عدد مفروض N را به‌دست آور و هر مقسوم‌علیه را به‌همراه هم‌عاملش چاپ کن.»

قسمت اول مسئله یعنی یافتن مقسوم‌علیه‌های یک عدد را پیشتر حل کرده‌ایم. اما چنانکه در مسئله شماره یک بخش ۲-۱۱ نیز مطرح شد، الگوریتم مثال ۲-۸ کارایی لازم را ندارد. چرا که اگر به‌عنوان مثال عدد N را برابر یک میلیون در نظر بگیریم حلقه تکرار کارنمای مثال ۲-۸ یک میلیون بار تکرار می‌شود. ولی عملاً نیمی از این تکرارها بی‌هوده است. زیرا هنگامی که $M > N/2$ می‌شود دیگر هیچ مقسوم‌علیه‌ای یافت نمی‌شود جزء خود عدد N که بخش‌پذیری آن بر خودش بدیهی است. بنابراین نباید اشتباهی را که در این مثال مرتکب شدیم در این مسئله نیز تکرار کنیم، و باید با دیدی

باز شرط حلقه را به‌گونه‌ای تنظیم کنیم که از تکرارهای بی‌پایان جلوگیری شود. بنابراین یکبار دیگر راه‌حل خود و استراتژی حل مسئله را به‌طور موشکافانه مورد ارزیابی قرار می‌دهیم.

اگر فرض کنیم عدد صحیحی مثل K یک مقسوم‌علیه عدد N باشد خود به خود هم عامل آن با توجه به رابطهٔ روبه

$$N = K \times \frac{N}{K}$$

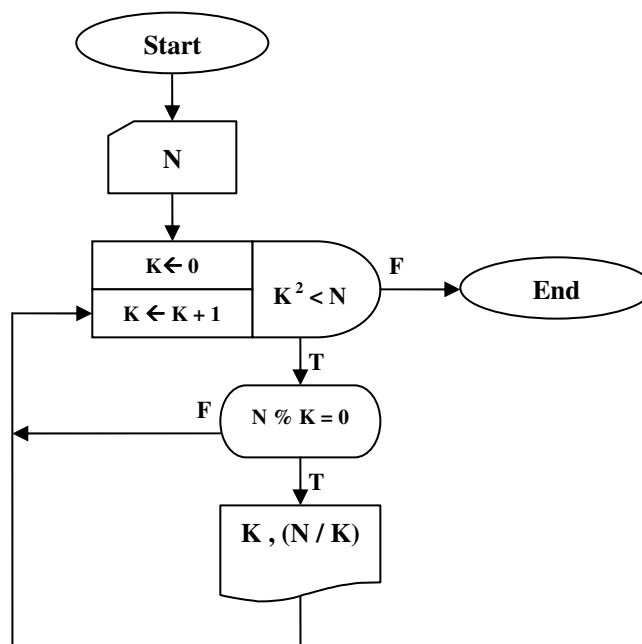
رو به‌دست می‌آید:

بنابراین در این مسئله تعداد بسیار کمتری از مقسوم‌علیه‌های عدد مفروض N را باید به‌دست بیاوریم. اما چگونه تعیین کنیم که رونر یافتن مقسوم‌علیه‌های عدد N را تا کجا باید ادامه پیدا کند؟ پاسخ این سؤال بسیار ساده است، عدد K و هم‌عاملش هر دو نمی‌توانند کوچکتر از $\sqrt{N}$ باشند. چرا که طبق رابطهٔ زیر اگر هر دوی آنها کوچکتر از $\sqrt{N}$ باشند به تناقض می‌رسیم:

$$K < \sqrt{N}, \frac{N}{K} < \sqrt{N} \Rightarrow N = K \times \frac{N}{K} < \sqrt{N} \times \sqrt{N} = N$$

بنابراین تنها کافی است تمامی مقسوم‌علیه‌های کوچکتر از $\sqrt{N}$ عدد مفروض N را به‌دست آوریم در این صورت

مقسوم‌علیه دیگر که بزرگتر از $\sqrt{N}$ می‌باشد خود به خود به‌دست می‌آید. حال به کارنمای این الگوریتم توجه کنید.



شکل ۴-۱۵: کارنمای پیدا کردن هم‌عامل‌های عدد مفروض N

اگر به شرط حلقهٔ تکرار توجه کرده باشید به جای عبارت $K < \sqrt{N}$ از عبارت $K^2 < N$ استفاده کردیم و این بدان جهت است که محاسبهٔ جذر در کامپیوتر همراه با تقریب است و می‌تواند دقت محاسبات ما را کاهش دهد در حالی که محاسبهٔ توان دوم یک عدد، با هیچ‌گونه تقریبی همراه نیست.

الگوریتم اقلیدس (یافتن ب.م.م دو عدد)

الگوریتم اقلیدس در کتاب پنجم اقلیدس آمده است و تاریخ آن حداقل به ۳۰۰ سال قبل از میلاد مسیح می‌رسد. این الگوریتم روشی برای پیدا کردن بزرگترین مقسوم‌علیه مشترک دو عدد صحیح است. قبل از بیان الگوریتم اقلیدس باید به ذکر برخی مقدمات بپردازیم.

با «بزرگترین مقسوم‌علیه مشترک» (ب.م.م) دو عدد در دروس ریاضی دوره راهنمایی آشنا شدید. ساده‌ترین راهی که می‌توان ب.م.م دو عدد را تعیین کرد بدین صورت است که ابتدا کلیه مقسوم‌علیه‌های هر دو عدد را تعیین کنیم. سپس مجموعه مقسوم‌علیه‌های مشترک دو عدد را یافته و در نهایت بزرگترین عضو این مجموعه را تعیین می‌کنیم. به عنوان مثال ب.م.م دو عدد ۵۴ و ۷۲ مطابق با این روش عبارت است از:

$$S = 54 = (2, 3, 6, 9, 18, 27, 54) = \text{مجموعه مقسوم‌علیه‌های } 54$$

$$T = 72 = (2, 3, 4, 6, 8, 9, 12, 18, 24, 36, 72) = \text{مجموعه مقسوم‌علیه‌های } 72$$

اشتراک این دو مجموعه برابر است با:

$$S \cap T = \{2, 3, 6, 9, 18\}$$

و از آنجا که بزرگترین عضو این مجموعه برابر ۱۸ می‌باشد، ب.م.م دو عدد ۵۴ و ۷۲ برابر ۱۸ می‌باشد. اما روش دیگری که غالباً از آن برای تعیین ب.م.م دو عدد استفاده می‌شود روش نردبانی است که از قبل با آن آشنایی دارید. اساس روش نردبانی بر قاعده ریاضی زیر استوار است. فرض کنید سه عدد صحیح X و Y و Z دارای رابطه زیر باشند:

$$Z = Y + X$$

حال اگر هر دوی این اعداد بر a بخش‌پذیر باشند، عدد سوم نیز بر a بخش‌پذیر خواهد بود. به عنوان مثال فرض کنید $Y=ay$ و $Z=az$ باشند آنگاه داریم:

$$a(z - y) = X \Rightarrow az = ay + X$$

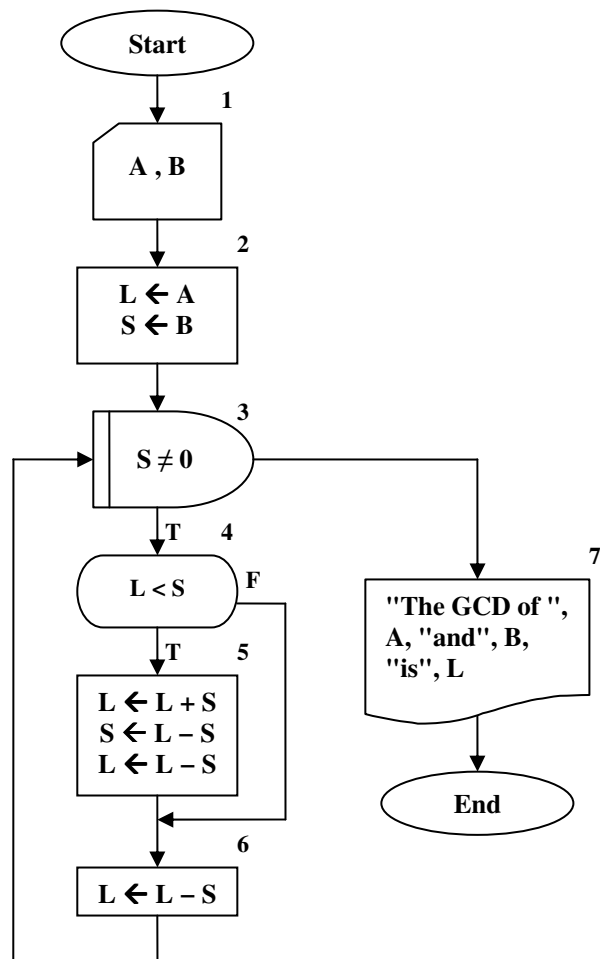
بنابراین عدد X نیز بر عدد a بخش‌پذیر می‌باشد. با توجه به رابطه اخیر هر دو عددی را که از میان این سه عدد انتخاب کنیم، مقسوم‌علیه‌های مشترک آنها همانند مقسوم‌علیه مشترک هر دو عدد دیگری است که از بین آنها انتخاب کنیم. برای روشن شدن مسئله سه عدد ۹۴۳ و ۴۳۷ و ۵۰۶ را در نظر بگیرید. این اعداد دارای رابطه زیر هستند:

$$943 = 506 + 437$$

با توجه به حکم فوق دو عدد ۵۰۶ و ۴۳۷ دارای همان مقسوم‌علیه‌هایی هستند که اعداد ۹۴۳ و ۵۰۶ دارند. حال برای یافتن مقسوم‌علیه مشترک دو عدد ۹۴۳ و ۵۰۶ می‌توانیم مقسوم‌علیه مشترک دو عدد ۵۰۶ و ۴۳۷ را بیابیم. این روند می‌تواند تا جایی ادامه پیدا کند که محاسبه مقسوم‌علیه مشترک دو عدد به سادگی امکان‌پذیر باشد. یعنی می‌توانیم برای محاسبه مقسوم‌علیه مشترک دو عدد ۵۰۶ و ۴۳۷، مقسوم‌علیه مشترک دو عدد ۴۳۷ و ۶۹ را بیابیم. زیرا:

$$506 - 437 = 69$$

اثبات می‌شود که مقسوم‌علیه مشترکی که از این روش به دست می‌آید همان بزرگترین مقسوم‌علیه مشترک دو عدد است. در شکل ۴-۱۶ کارنمایی ارائه شده که روند فوق را برای پیدا کردن ب.م.م دو عدد پیاده‌سازی می‌کند.



شکل ۴-۱۶: کارنمای الگوریتم اقلیدس با استفاده از تفریق

کار در کلاس ۴-۲:

۱. آیا می‌توانید بگویید چه عملی در پیچۀ شماره ۵ انجام می‌شود؟

۲. روندنمای فوق را برای ورودی‌های زیر امتحان کنید.

▪ $A = 177, B = 379$

▪ $A = 3363, B = 4465$

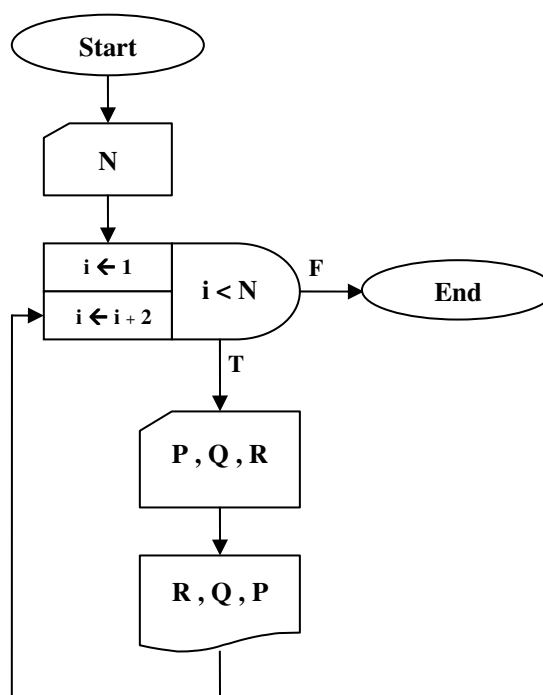
کار در کلاس ۳-۴:

کارنمای شکل ۴-۱۶ را به گونه ای بازنویسی کنید که الگوریتم اقلیدس را به واسطه تقسیم پیاده سازی کند. سپس الگوریتم خود را تعمیم دهید تا ک.م.م دو عدد را نیز مناسبه و چاپ کند. راهنمایی: برای مناسبه ک.م.م ساده ترین راه استفاده کردن از فرمول زیر است:

حاصل ضرب دو عدد = ک.م.م $\times$ ب.م.م

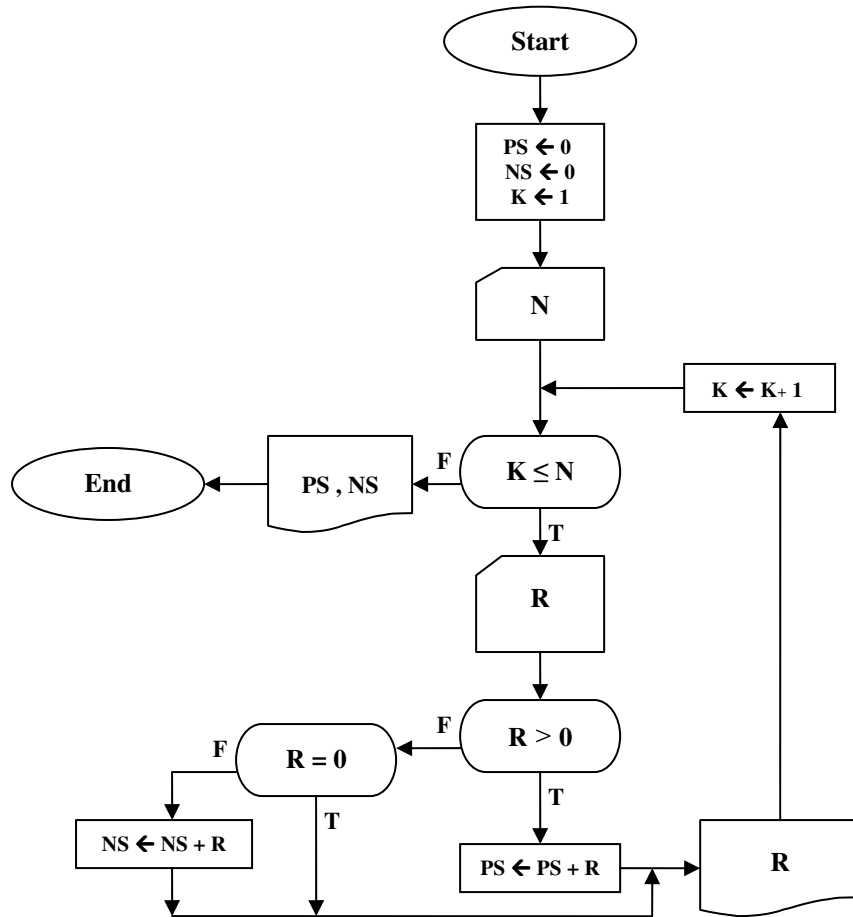
۴-۴: تمرین

۱. تفاوت الگوریتم‌های ساخت‌یافته با الگوریتم‌های غیرساخت‌یافته چیست؟
 ۲. یک روندنما و دنباله‌ای از داده‌های ورودی به شما داده شده است. وظیفه شما این است که تعیین کنید چه مقادیری به خارج داده خواهد شد.
- دنباله داده‌های ورودی: ۷، ۱، ۲، ۳، ۴، ۵، ۶، ۷، ۸، ۹.



شکل ۴-۱۷: کارنمای تمرین ۲

۳. کارنمایی رسم کنید که ب.م.م دو عدد را به روش زیر محاسبه کند.
- فرض کنید N از M کوچکتر است. مقسوم‌علیه‌های N را از بزرگ به کوچک به‌دست آورید و آزمایش کنید که آیا M نیز بر آن مقسوم‌علیه بخشپذیر است یا نه. اولین مقسوم‌علیه N که M نیز بر آن بخشپذیر باشد برابر ب.م.م دو عدد M و N است.
۴. کارنمایی رسم کنید که مجموع ارقام عدد مفروض N را به‌دست آورد.
۵. کارنمایی رسم کنید که تعداد N عدد صحیح را از ورودی بخواند، و اعدادی را که بر ۶ قابل قسمت هستند را به همراه تعدادشان چاپ کند.
۶. روندنمایی را که در شکل ۴-۱۸ نشان داده شده است مجدداً به‌گونه‌ای رسم کنید که، حلقه‌ای که توسط شمارنده K کنترل شده است، تحت کنترل یک جعبه تکرار باشد.



شکل ۴-۱۸: کارنمای تمرین ۶

۷. عدد طبیعی N مفروض است. کارنمایی رسم کنید که معین کند N چند رقم دارد.

راهنمایی: برای حل این مسئله از دو طریق می‌توانید عمل کنید. یکی از طریق عملگر باقی‌مانده تقسیم و جدا کردن ارقام N و شمارش تعداد ارقام آن. دوم از طریق رابطه زیر که ثابت می‌شود عدد طبیعی N دارای k رقم است.

$$(10^{(k-1)}) \leq N < 10^k$$

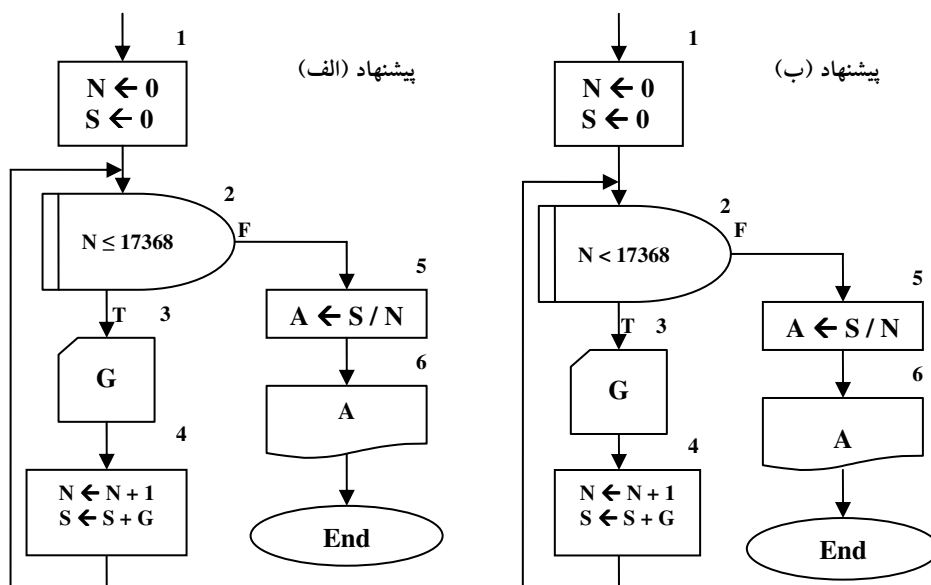
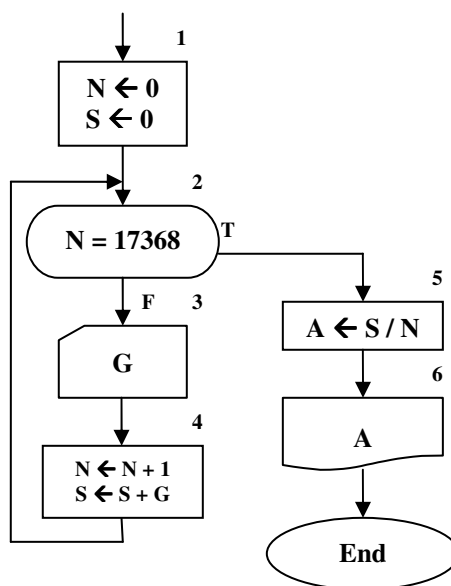
۸. مسئله قبل را به‌گونه‌ای تعمیم دهید که برای اعداد منفی و در نتیجه بر روی اعداد صحیح نیز جواب صحیح بدهد.

۹. کارنمایی رسم کنید که مقلوب عدد مفروض N را به‌دست‌آورد. مقلوب عدد ۳۴۲۸ برابر ۸۲۴۳ می‌باشد.

۱۰. کارنمایی رسم کنید که بدون محاسبه باقی‌مانده تقسیم بخشپذیر بودن عدد مفروض N را بر عدد ۱۱ به‌دست‌آورد؟

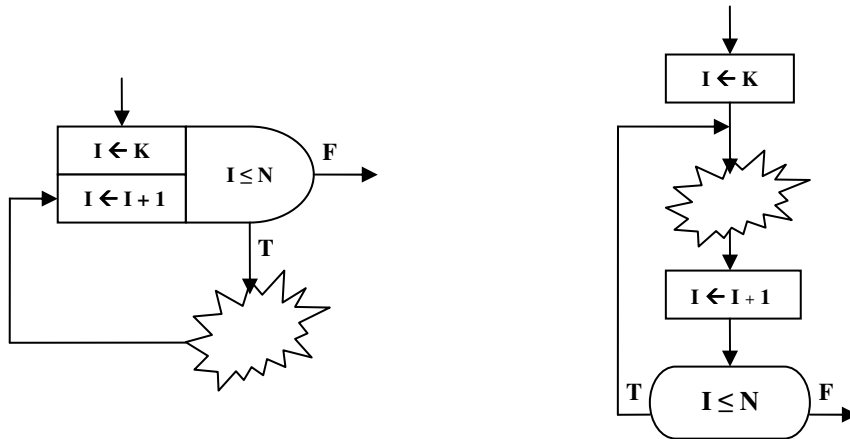
۱۱. الگوریتمی بنویسید که با دریافت یک عدد N اول بودن یا نبودن آن را تشخیص دهد.

۱۲. در این مسئله در ابتدا قطعه روندنمایی به شما داده شده است که دارای یک حلقه کنترل شده به وسیله شمارنده است. سپس چند قطعه روندنما، که در آنها از جعبه تکرار While استفاده شده، برای انجام همان کار پیشنهاد شده است. از شما می‌خواهیم تا پیشنهادی را که واقعاً همان کار را انجام می‌دهد انتخاب کنید.



شکل ۴-۱۹: کارنمای تمرین ۱۲

۱۳. دو قطعهٔ روندنما در زیر نشان داده شده و فرض بر این است که محتوای قسمت ابری در هر دو یکی است. تحت چه شرایطی این دو روندنما معادل یکدیگر خواهند بود؟



شکل ۴-۲۰: کارنمای تمرین ۱۳

۱۴. یک توزیع‌کننده، تعداد ۱۰,۰۰۰ اسباب‌بازی کوچک را که در جعبه‌های مکعب مستطیلی با ابعاد گوناگون بسته‌بندی شده است خریداری کرده است. وی خیال دارد به‌منظور مخفی نگه داشتن اسباب‌بازی‌ها و در نتیجه جلب‌توجه بیشتر بچه‌ها، این جعبه‌ها را در گُرهِ‌های پلاستیکی قرار دهد و مجدداً به فروش برساند. از این رو او باید بداند که چند عدد گُرهِ با قطرهای مشخص (۱۰، ۱۵، ۲۰، ۲۵، ۳۰ سانتیمتر) احتیاج دارد. چون بزرگترین اندازهٔ خارجی یک مکعب مستطیل به ابعاد A، B و C قطر آن است، لذا توزیع‌کننده باید با استفاده از رابطهٔ زیر:

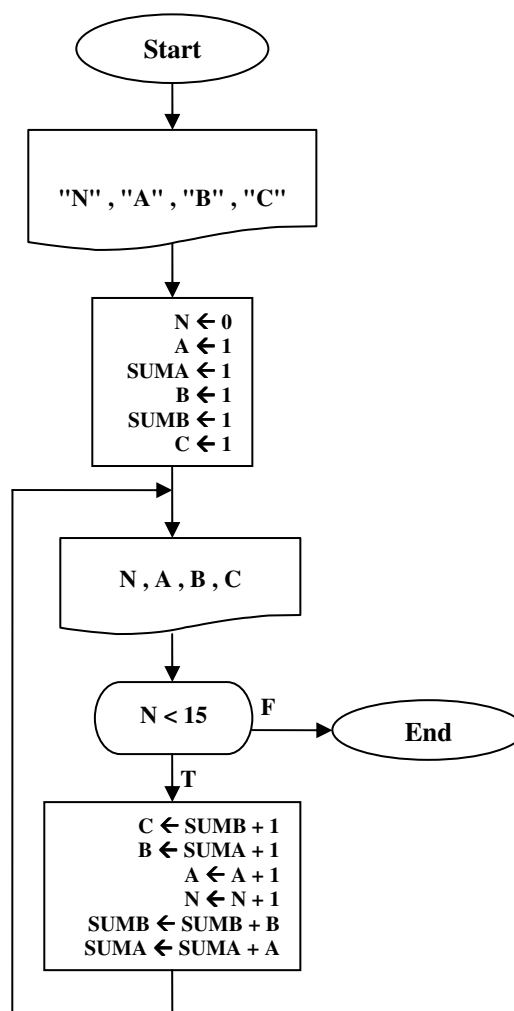
$$D = \sqrt{A^2 + B^2 + C^2}$$

قطر جعبه‌ها را حساب کند. سپس باید تعیین کند که تعداد جعبه‌هایی که قطرشان ۱۰ سانتیمتر یا کمتر است چقدر است، تعدادی که قطرشان بین ۱۰ و ۱۵ سانتیمتر است چقدر است، و به همین ترتیب. تصمیم توزیع‌کننده بر این قرار می‌گیرد که قطر هر ۱۰,۰۰۰ جعبه را حساب کند. هر جعبه دارای یک شمارهٔ مشخصهٔ ID و مجموعهٔ اندازه‌های A، B و C است. کار شما ترسیم روندنمایی است برای تهیهٔ یک جدول چاپی که هر خط آن دارای پنج قلم شامل مقادیر ورودی ID، A، B، C و مقدار حساب‌شدهٔ D باشد. هر خط از جدول باید نظیر یکی از بسته های اسباب‌بازی باشد. از جعبهٔ تکرار استفاده کنید. تعداد جعبه‌های مورد نیاز روندنما، بدون احتساب جعبه‌های شروع و پایان، فقط چهار تا است.

۱۵. الگوریتمی بنویسید که تعداد نامشخصی عدد را از ورودی بخواند و مشخص کند که آیا این اعداد مرتب هستند یا خیر؟ و اگر مرتب هستند، ترتیب صعودی است یا نزولی؟ الگوریتم باید در هنگام ورود داده‌ها مرتب بودن یا نبودن آنها را تشخیص دهد. آخرین ورودی نیز \$ است.

۱۶. این مسئله دنباله مسئله ۱۴ است. فرض کنید توزیع‌کننده اسباب‌بازی متوجه می‌شود که برای تعیین تعداد کره‌های پلاستیکی موردنیاز از هر قطر، راه ساده‌تری از پویش فهرست ۱۰,۰۰۰ خطی وجود دارد. روندنمایی را که در مسئله ۱۴ تهیه کرده‌اید چنان تغییر دهید که به‌جای چاپ آن جدول طولانی خود الگوریتم، حساب تعداد کره‌های مورد نیاز از هر سایز را نگه دارد و پس از پردازش تمامی ۱۰,۰۰۰ داده، تعداد کره‌های موردنیاز از هر سایز را چاپ کند.

۱۷. روند نمای شکل ۴-۲۱ را مطالعه کنید و به سؤال‌های زیر پاسخ دهید.
الف- مقادیری را که بر روی پنج خط اول خروجی الگوریتم چاپ می‌شود نشان دهید.
ب- این روندنما را از نو با استفاده از جعبه تکرار رسم کنید.



شکل ۴-۲۱: کارنمای تمرین ۱۷

۱۸. روندنمایی بسازید که جدولی شامل چهار ستون را محاسبه کند و در خروجی بنویسد. ستون اول باید شامل اعداد از ۱ تا ۱۰۰ باشد. ستون دوم باید شامل مربعات اعداد نظیر خود در ستون اول باشد. ستون‌های سوم و چهارم باید شامل مکعب‌ها و توان‌های چهارم اعداد ستون اول باشد. برای مثال، دومین خط خروجی باید چنین باشد:

$$۲ \quad ۴ \quad ۸ \quad ۱۶$$

۱۹. الگوریتمی بسازید که اولین ۵۰۰ عدد صحیح را آزمایش کند و فقط آنهایی را که کامل هستند در خروجی بنویسد. عدد کامل عددی است که مساوی حاصل جمع همهٔ عامل‌های خود باشد. کوچکترین عدد کامل ۶ است.

$$۶ = ۱ + ۲ + ۳$$

(بناباه تعریف، عدد یک، جزو اعداد کامل نیست.)

۲۰. لیست X که شامل N مقدار $X_1, X_2, \dots, X_N$ و یک مقدار A است داده شده است.

الف- روندنمایی برای محاسبهٔ NUM ، که تعریف ریاضی آن به صورت حاصل ضرب N جمله‌ای زیر داده شده است، ترسیم کنید.

$$NUM = (X_1 - A) \times (X_2 - A) \times (X_3 - A) \times \dots \times (X_N - A)$$

روندنمای شما باید شامل خواندن و نوشتن کلیهٔ داده‌های مورد نیاز و نتایج حاصله باشد. شما نیز یک متغیر با زیرنویس i بگیرید و در یک حلقهٔ دارای شمارندهٔ مقادیر را برای متغیر دارای زیرنویس مذکور بخوانید و یا بنویسید.

ب- مانند قسمت (آ) ولی با این تفاوت که جملهٔ $K - A$ از N جملهٔ حاصل ضرب حذف شده باشد.

۲۱. الگوریتمی بنویسید که تعدادی عدد را بخواند و هر عدد که بر ۹ بخشپذیر باشد را در خروجی چاپ کند. (از روش مجموع ارقام استفاده کنید.)

۲۲. الگوریتمی را بنویسید که عددی را با هر طول دلخواه بخواند و مشخص کند که آیا عدد متقارن است یا خیر. به عنوان مثال عدد ۱۲۳۲۱ متقارن است.

۲۳. الگوریتمی بنویسید که عدد N را از ورودی بخواند و ۱۰ مضرب اول N را چاپ کند.

۲۴. الگوریتمی بنویسید که بدون محاسبهٔ $b.m$ مقدار $k.m$ دو عدد را محاسبه و چاپ کند.

۲۵. الگوریتمی بنویسید که ابتدا دو عدد a و b را به عنوان دو کران بازهٔ $[a, b]$ از کاربر بگیرد، سپس با دریافت عدد مفروض N این بازه را به تعدادی زیر بازه با طول‌های مساوی تقسیم و هر زیر بازه را به طور جداگانه چاپ کند. به عنوان مثال برای بازهٔ $[0, 1]$ و عدد $n=4$ باید مقادیر زیر را چاپ کند.

$$[0, 0.25], [0.25, 0.5], [0.5, 0.75], [0.75, 1]$$

۲۶. کارنمایی رسم کنید که یک عدد و مبنای آن را دریافت و آن را از مبنای ۸ به مبنای ۱۰ و از مبنای ۱۰ به مبنای ۸ برود. این کار را برای تبدیل بین مبنای ۱۰ و ۱۶ نیز تکرار کنید.

۲۷. کارنمایی رسم کنید که عمل تبدیل بین مبنای ۲ و ۸ و ۱۶ را به یکدیگر انجام دهد. در این کارنما باید یک عدد و مبنای آن دریافت و معادل تبدیل یافته عدد در دو مبنای دیگر چاپ گردد.

۲۸. در مسئله ۱۹ از مجموعه تمرین ۲-۱۱، از شما خواسته شده بود تا چرخ بازی را N بار بچرخانید. در اینجا می‌خواهیم مفهوم شبیه‌سازی بازی به کمک کامپیوتر را که به‌منظور پیش‌بینی نتیجه انجام می‌گیرد، به‌طوری جدی‌تر ارائه دهیم. بازی موردنظر عبارت است از «بازی چرخ گردان». چنین تصور کنید که یک مقدار ورودی برای S محل عقربه در شروع بازی و یک منبع تمام‌نشده‌ی از مقادیر برای تعداد قطاع‌های طی شده، m ، به شما داده شده است.

این داده‌ها به نوعی نمایش‌گر آن چیزی هستند که یک شخص در صورتی که قرار باشد به دفعات و بدون دخالت بازیکن دیگری چرخ را بچرخاند، عملاً تجربه می‌کند. ما چنین تعریف می‌کنیم که یک «بازی» شامل یک سری چرخاندن توسط یک بازیکن معین است و پایان بازی موقعی است که قدرمطلق امتیازات وی، $|SUM|$ ، از یک مقدار بحرانی معین CV بیشتر شود. تعداد چرخاندن‌های انجام شده در سری مزبور را «طول» بازی می‌نامیم. سؤالی که واقعاً می‌خواهیم بکنیم این است: «قبل از آن که $|SUM| \leq CV$ شود، به‌طور متوسط چند بار می‌توان انتظار داشت که یک بازیکن چرخ را بچرخاند؟»

شما در این تمرین زمینه را، برای پاسخی که بعداً به سؤال فوق خواهید داد، آماده می‌کنید. وظیفه فعلی شما این است که روندنمایی برای شبیه‌سازی یک بازی کامل ترسیم نمایید. در زیر خطوط کلی یک راه‌حل داده شده است ولی تا هنگامی که طرح ساخت خودتان را آزمایش نکرده‌اید به آن مراجعه نکنید.

الف- مانند تمرین‌های قبل، ابتدا چهار مقدار تشکیل‌دهنده «قاعده محاسبه امتیازات» را که باید از آن استفاده شود، بخوانید.

ب- بعد مقداری برای CV ، مقدار بحرانی، بخوانید.

ج- سپس مقداری برای S و یک سری از مقادیر m را بخوانید.

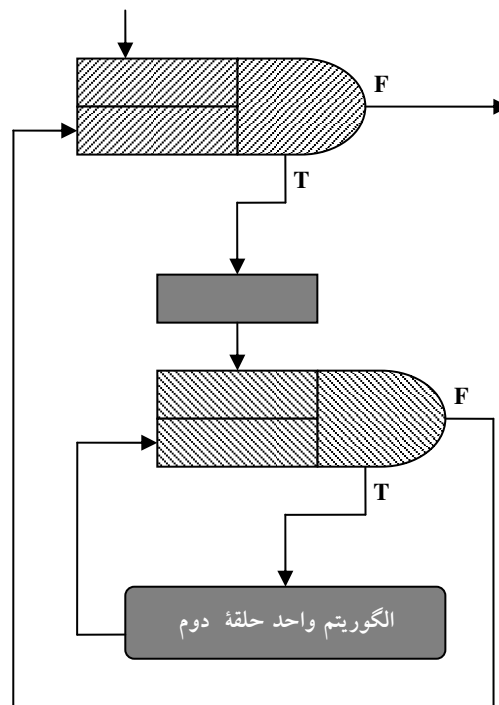
د- پس از خواندن هریک از مقادیر m ، مقدار جدید امتیازات حاصله (SUM) را محاسبه کرده و شمارنده تعداد چرخاندن‌ها (L) را به‌هنگام در بیاورید.

و- هرگاه قدرمطلق SUM بیش از CV شود، مقادیر L ، CV و SUM را چاپ کرده و کار را متوقف کنید.

ه- به‌منظور اجتناب از به‌وجود آمدن حلقه بی‌پایان، هرگاه مقدار L بیش از ۱۰۰۰ شود ضمن چاپ یک پیغام خطا، کار را متوقف کنید. استفاده از جعبه تکرار در جایی که به نظرتان موجب سادگی ساخت روندنما خواهد شد، از یاد نبرید.

۴-۵: حلقه‌های تودرتو

اصطلاح حلقه‌های تودرتو^۱ به الگوریتم‌هایی اطلاق می‌شود که مانند آنچه در شکل ۴-۲۲ نشان داده شده دارای حلقه‌ای در داخل حلقه دیگر هستند. به عبارت دیگر اگر در داخل الگوریتم واحد یک حلقه از حلقه تکرار دیگری استفاده شود به مجموعه این حلقه‌ها «حلقه‌های تودرتو» می‌گویند.



شکل ۴-۲۲: نمایی از حلقه‌های تودرتو

کاربرد حلقه‌های تودرتو و مثال‌های متنوعی از این مبحث را پس از مطرح کردن مبحث متغیرهای لیستی و آرایه‌ها خواهید دید. اما در حال حاضر صرفاً جهت تکمیل مبحث حلقه‌ها، این بحث را مطرح ساختیم و بحث مفصل در خصوص کاربرد حلقه‌های تودرتو را به فصل آرایه‌ها موکول می‌سازیم. لذا در ادامه به طرح مثال مقدماتی در این زمینه می‌پردازیم.

1. Nested loop

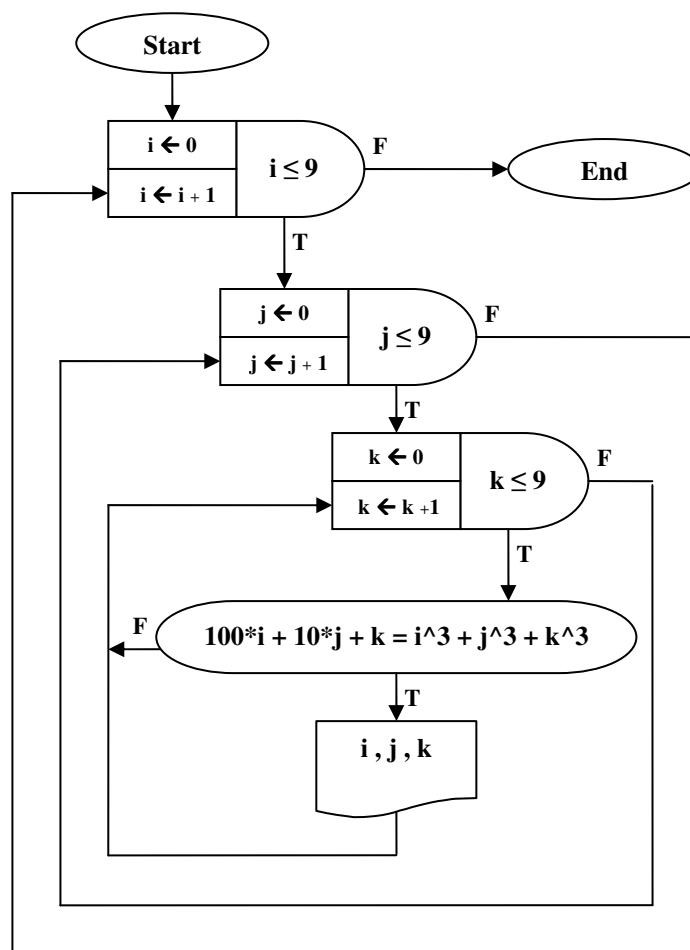
مثال ۴-۷: کارنمایی که تمام اعداد محدوده ۰۰۰ تا ۹۹۹ را که برابر مجموع مکعب‌های ارقام‌شان هستند

پیدا کنید.

توضیح الگوریتم: اگر ارقام اعداد مورد نظر را با i و j و k نشان دهیم، آنگاه مسئله تبدیل می‌شود به پیدا کردن تمام ارقام سه‌گانه مذکور که در شرط زیر صدق کنند.

$$100*i + 10*j + k = i^3 + j^3 + k^3$$

در حقیقت این تنها محاسبه‌ای است که در این الگوریتم انجام می‌شود. بقیه الگوریتم مقادیر مختلف را برای متغیرهای i و j و k به واسطه حلقه‌های تودرتو مهیا می‌کند. به‌ازای هر بار گذر از حلقه i ، حلقه j به تعداد ده بار و به‌ازای مقادیر ۰ تا ۹ تکرار می‌شود. همچنین به‌ازای هر بار گذر از حلقه j ، حلقه k نیز به تعداد ۱۰ مرتبه اجرا می‌گردد. حال به کارنمای این الگوریتم توجه کنید.



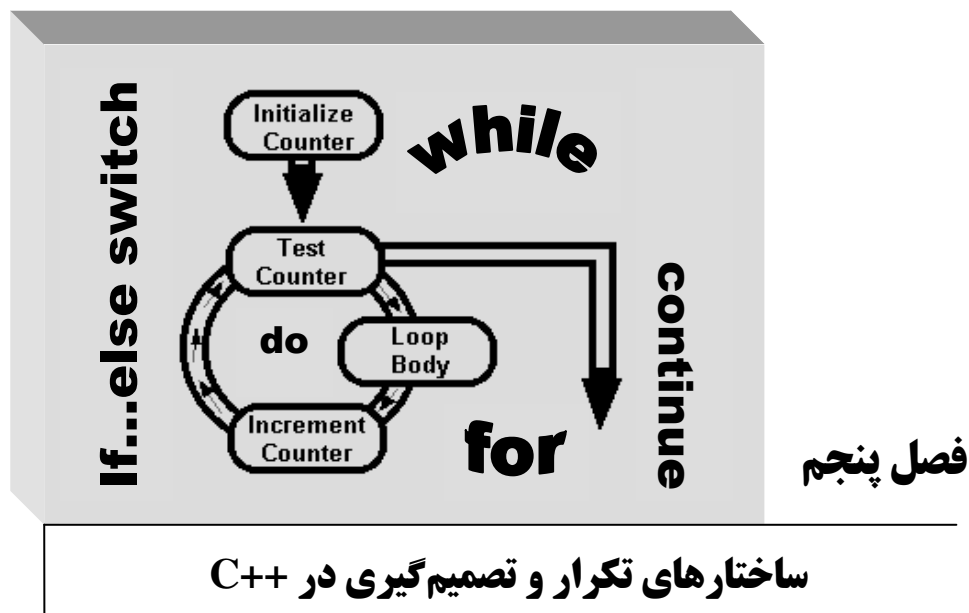
شکل ۴-۲۳: کارنمای مثال ۴-۷

۴-۶: تمرین

- شکل ۴-۲۳ را بررسی کنید و به سؤالات ۱ تا ۶ پاسخ دهید.
- از دکمه شروع تا پایان چند عمل ضرب صورت می‌گیرد؟
 - از دکمه شروع تا پایان چند مقدار مختلف i^3 محاسبه می‌شود؟
 - از دکمه شروع تا پایان چند مقدار مختلف j^3 محاسبه می‌شود؟
 - الگوریتم را طوری تغییر دهید که هیچ مقدار i^3 بیش از یک مرتبه و هیچ مقدار j^3 بیش از ده مرتبه محاسبه نشود.
 - چگونه می‌توانیم الگوریتم را تغییر دهید به قسمی که هیچ مقدار i^3 ، j^3 و k^3 بیش از یک مرتبه محاسبه نشود؟
 - با توجه به آن که عمل ضرب عملی وقت‌گیر برای کامپیوتر است، عملی پیشنهاد دهید که سعی کنید ضمن استفاده از حداکثر نه جعبه مختلف در روندنما، تعداد ضرب‌ها را به ۱۲۰ عدد تقلیل دهید. سپس در تلاش بعدی تعداد ضرب‌ها را به ۱۸ عدد کاهش دهید. آیا قادرید تعداد ضرب‌ها را از ۱۸ عدد هم کمتر کنید؟
 - با استفاده از حلقه‌های تودرتو الگوریتمی طراحی کنید که یک اسکناس ۱۰۰ دلاری را به واسطه اسکناس‌های ۵۰، ۲۰، ۱۰، ۵ و ۲ دلاری طوری خرد کند که تمام حالت‌های ممکن را شامل شود.
 - الگوریتمی طراحی کنید که عددی را دریافت کند و بگوید آن عدد اول است یا خیر؟
 - الگوریتمی طراحی کنید که مجموع N جمله اول دنباله زیر را به‌ازای عدد حقیقی x محاسبه کند.

$$s = \frac{1}{x^1} - \frac{1}{x-2x^2} + \frac{1}{x-2x^2+3x^3} - \frac{1}{x-2x^2+3x^3-4x^4} + \dots$$
 - الگوریتمی بنویسید که مقدار زاویه x را برحسب رادیان بخواند و مقدار $\sin(x)$ را به واسطه رابطه زیر تا ۱۰ رقم اعشار تعیین کند. (برای تشخیص تعداد ارقام اعشار چه راه حلی را پیشنهاد می‌کنید؟)

$$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$$
 - فرض کنید سکه‌های رایج کشور عبارتند از ۱ و ۲ و ۵ و ۱۰ و ۲۵ و ۵۰ تومانی. الگوریتمی بنویسید که یک اسکناس N تومانی را دریافت کند و با توجه به این سکه‌ها آن اسکناس را با حداقل تعداد سکه خرد کند.
 - منحنی $f(x) = x^2$ مفروض است. سطح تقریبی زیر منحنی را در فاصله a تا b به روش زیر محاسبه کنید. (توجه کنید که انتگرال‌گیری مد نظر نیست. اعداد a و b و N را از کاربر دریافت می‌کنید.)
- فاصله a تا b را به N قسمت مساوی تقسیم کنید و با رسم خطوط موازی محور y ها N دوزنقه به‌دست خواهد آمد. مساحت هر یک از دوزنقه‌ها را حساب کنید و مجموع مساحت آنها را به‌عنوان سطح زیر منحنی چاپ کنید.



اهداف فصل و چکیده مطالب:

۱. کاربرد ساختار تکرار for
۲. کاربرد ساختارهای تکرار while و do...while
۳. کاربرد دستورات break و continue
۴. آشنایی با دستور goto
۵. برنامه‌های ساخت یافته و غیر ساخت یافته
۶. کاربرد ساختار تصمیم‌گیری if
۷. کاربرد ساختار تصمیم‌گیری else if
۸. کاربرد ساختار switch
۹. شبیه‌سازی توابع clrscr و gotoxy
۱۰. تولید اعداد تصادفی و شبیه‌سازی توابع random و randomize

۵-۱: ساختارهای تکرار در C++

مقدمه

بیشتر مطالب کلیدی لازم برای کارکردن با ساختارهای تکرار و تصمیم‌گیری را در فصول دوم و چهارم آموختید. لذا در این فصل سعی خواهیم کرد از تکرار مطالب قبلی خودداری کنیم، و بیشتر با ذکر مثال‌های متنوع به ذکر نکات برنامه‌نویسی در این زمینه بپردازیم. پس از آنکه بحث خود را در خصوص ساختارهای تکرار به پایان رساندیم، به بررسی برنامه‌های ساخت‌یافته و غیرساخت‌یافته خواهیم پرداخت.

به‌طور معمول دستورات برنامه باید از اولین دستور تا آخرین دستور به‌طور متوالی و پشت سرهم اجرا گردند. ولی چنانکه پیشتر نیز دیدید، گاهی نیازمند تکرار یک الگو و یا عدم انجام یک الگو تحت یک شرایط خاص و اجرای الگوی دیگری در شرایطی دیگر هستیم. لذا در تمامی زبان‌های برنامه‌نویسی دستوراتی جهت تصمیم‌گیری و یا ایجاد حلقه در برنامه وجود دارد. در C++ نیز دستورات متنوعی جهت عملیات تکرار و تصمیم‌گیری تدارک دیده شده که در ادامه به بررسی تک‌تک آنها خواهیم پرداخت.

ساختار تکرار for

پیشتر در فصل چهارم با ساختارهای تکرار دارای شمارنده آشنا شدید. دقیقاً ساختار تکرار for را می‌توان معادل ساختارهای تکرار دارای شمارنده در کارنها در نظر گرفت. بنابراین هرگاه تعداد دفعات تکرار حلقه را از قبل بدانیم می‌توانیم از این ساختار استفاده کنیم. شکل کلی این ساختار به‌صورت زیر است:

(گام حرکت شمارنده؛ شرط حلقه؛ مقدار اولیه شمارنده) **for**

```
{
    دستور یکم
    ⋮
    دستور n-ام
}
```

در این ساختار متغیری وجود دارد که شمارنده یا اندیس حلقه نام دارد، و تعداد دفعات تکرار حلقه تکرار را در خود نگه می‌دارد. چنانکه می‌بینید در داخل پرانتز سه قسمت جداگانه وجود دارد که با سمیکالن از هم جدا می‌شوند. در اولین قسمت باید شمارنده حلقه را مقدار اولیه داد. در قسمت دوم باید یک عبارت رابطه‌ای ساده و یا مرکب را به‌عنوان شرط حلقه قرار داد، و در آخرین قسمت باید به وسیله یک عبارت محاسباتی مقدار شمارنده را به میزان گام حلقه افزایش دهیم، و چنانکه می‌بینید دستورات الگوریتم واحد نیز به ترتیب از اولین دستور تا آخرین دستور بین دو نماد آکولاد باز و بسته قرار می‌گیرند، به عبارت دیگر دستورات مربوط به حلقه در داخل یک بلاک قرار می‌گیرند. قرار گرفتن دستورات حلقه در بلاک به ما این امکان را می‌دهد که متغیرهایی را در داخل خود بلاک تعریف کنیم به‌طوری که در بیرون از بلاک قابل شناسایی نباشند. در یک حلقه تکرار که تنها دارای یک دستور است قرار دادن آکولاد باز و بسته الزامی نیست.

هنگامی که کنترل اجرای برنامه به دستور `for` می‌رسد ابتدا به اولین قسمت حلقه رفته و شمارنده را مقدار اولیه می‌دهد. متغیر شمارنده می‌تواند قبل از حلقه اعلان شده باشد. در این صورت می‌توان شمارنده را مقدار اولیه نداد و مقدار کنونی شمارنده را به عنوان مقدار اولیه شمارنده پذیرفت، در نتیجه در این حالت هیچ چیزی در قسمت اول حلقه نمی‌نویسیم. همچنین می‌توان شمارنده را در همین قسمت تعریف کرد و مقدار اولیه داد. در این صورت برای اعلان یک شمارنده به نام `i` می‌توان به صورت زیر عمل کرد:

```
int i=0;
```

همچنین در صورت لزوم می‌توانید در این قسمت بیش از یک متغیر را تعریف کنید. در این صورت باید متغیرها را با علامت کاما از یکدیگر جدا کنید. متغیرهای تعریف شده در این قسمت برخلاف متغیرهای تعریف شده در بلاک حلقه، از علامت `{` به بعد نیز قابل شناسایی هستند. نکته قابل توجه آنکه قسمت اول حلقه تنها یک بار اجرا می‌گردد و در دفعات بعدی که حلقه تکرار می‌شود شمارنده مقداردهی اولیه نمی‌شود.

پس از مقدار اولیه دادن به شمارنده کنترل اجرای برنامه شرط حلقه را چک می‌کند و در صورت درست بودن این شرط دستورات حلقه به ترتیب از اولین دستور تا آخرین دستور اجرا می‌گردد. پس از پایان یافتن آخرین دستور حلقه، کنترل اجرای برنامه به قسمت گام حلقه رفته و عبارت محاسباتی آمده در این قسمت را اجرا می‌کند. دوباره شرط حلقه چک شده و در صورت درست بودن آن، دستورات داخل بلاک اجرا می‌گردد و در صورت نادرست بودن شرط حلقه کنترل اجرای برنامه به خط بعد از آکولاد بسته می‌رود.

در صورتی که در قسمت شرط حلقه هیچگونه عبارتی نوشته نشود، کامپایلر مقدار `true` را به طور پیش فرض به جای شرط حلقه قرار می‌دهد، که در این صورت حلقه تکرار به یک حلقه بی‌نهایت تبدیل می‌شود و متوقف نخواهد شد. برای خاتمه دادن به اجرای حلقه تکرار بی‌نهایت باید دکمه‌های `Ctrl` و `Break` را به طور همزمان فشار داد. همچنین اگر در داخل قسمت شرط یک عبارت همیشه درست بنویسید، حلقه بی‌نهایت ایجاد می‌شود.

مثال ۵-۱: در زیر نمونه‌های متنوعی را از تعریف حلقه‌های `for` مشاهده می‌کنید.

۱. ساده‌ترین تعریف از یک حلقه `for` که ۱۰۰ بار اجرا می‌گردد:

```
for(int i=0; i<100; i++)
```

۲. شمارنده را از ۱۰۰ تا ۱۰ در گام‌هایی به طول ۲ کم می‌کند:

```
for(int i=100; i>=10; i=i-2)
```

۳. سه حلقه `for` بی‌نهایت:

```
for(int i=1, j; ; i++, j*=i%k)
for(;;)
for(k=8; 7<=12; k++)
```

۴. حلقه تکرار با یک عبارت شرطی مرکب:

```
for(int i=10, j=75;
(i<=105&& j>50) || k!=42;
i++, k=k/i)
```

مثال ۵-۲: برنامه‌ای که مربعات و مکعبات اعداد بین یک تا ۱۰ را چاپ می‌کند.

```
1. //This program calculates square and cube of some numbers
2. #include "iostream"
3. using namespace std;
4. int main()
5. {
6.     for(int i=1;i<11;i++)
7.         cout<<i<<"\t"<<i*i<<"\t"<<i*i*i<<endl;
8.     return 0;
9. }
```

توضیح مثال: در این برنامه تنها از یک حلقه `for` ساده استفاده شده است. به دلیل وجود تنها یک دستور از قرار دادن آکولاد باز و بسته خودداری کردیم.

مثال ۵-۳: برنامه‌ای که با دریافت یک عدد فاکتوریل آن را حساب می‌کند.

```
1. //This program calculates factorial of an integer numbers
2. #include <iostream>
3. using namespace std;
4. int main()
5. {
6.     short n;
7.     cout<<"Please enter an integer number : ";
8.     cin>>n;
9.     unsigned int result=1;
10.    for(int fact=n;fact>0;fact--)
11.        result*=fact;
12.    cout<<"\nThe factorial of "<<n<<"is = "<<result;
13.    return 0;
14. }
```

مثال ۵-۴: برنامه‌ای که به کارگیری شمارندهٔ اعشاری را در حلقهٔ `for` نشان می‌دهد.

```
1. //This program shows how you can use float number as
2. //index of loop.
3. #include "iostream"
4. using namespace std;
5. int main()
6. {
7.     int j=2;
8.     for(float f=5.0 ; f<100 ; cout<<f*j<<"\n" ,f *=1.5);
9.     {
10.        cout<<"In the block.\n";
11.    }
12.    cout<<"Out of  block.\n";
13.    return 0;
14. }
```

توضیح مثال: اصولاً در حلقه‌های `for` از شمارنده‌هایی از نوع `int` استفاده می‌شود. چرا که سرعت دستیابی به نوع `int` از هر نوع دیگری بیشتر است و این خود می‌تواند سرعت اجرای برنامه را افزایش دهد. پیشنهاد می‌شود حتی در مواردی که نوع `char` یا `short` برای شمارنده کافی است نیز از نوع `int` استفاده کنید زیرا تفاوت سرعت دستیابی حتی اگر چند میلیونیم ثانیه هم باشد با توجه به وجود تکرار در حلقه‌ها، می‌تواند در مجموع این تفاوت، تا چند ثانیه هم افزایش یابد. اما گاهی هم پیش می‌آید که به دلیل محاسبات داخلی حلقه تکرار و به کار گرفته شدن شمارنده در این محاسبات مجبور هستیم شمارنده‌هایی از نوع اعشاری را به کار ببریم، لذا استفاده از انواعی غیر از نوع صحیح نیز به عنوان شمارنده حلقه امکانپذیر است.

نکته قابل توجه دیگر در این مثال به کارگیری دستور خروجی `cout` در داخل خود حلقه تکرار است. چنانکه می‌بینید در هر بار گذر از حلقه این دستور نیز اجرا می‌گردد. همچنین هرگاه پس از پرانتز بسته در دستور `for` علامت سمیکالن قرار دهیم دستورات داخل بلاک بی اعتبار شده و در تکرارهای حلقه اجرا نمی‌گردند. اما پس از خارج شدن از حلقه دستورات داخل بلاک تنها یکبار اجرا می‌شود. با توجه به نکات مطرح شده خروجی این برنامه به صورت زیر است:

```
10
15
22.5
33.75
50.627
75.9375
113.906
170.859
In the block.
out of block.
```

مثال ۵-۵: برنامه‌ای که جدول کدهای اسکی را به همراه نماد مربوط به هر کد چاپ می‌کند.

```
1. //This program prints ASCII code table.
2. #include "iostream.h"
3. int main()
4. {
5.     for(char ch=-128; ch<127; ch++)
6.         cout<<"character="<<ch
7.             <<"\tASCII Code="<<(int)ch<<"\n";
8.     cout<<"character="<<ch<<"\tASCII Code="<<(int)ch<<endl;
9.     return 0 ;
10. }
```

توضیح مثال: برای به دست آوردن کد اسکی داخل یک متغیر از نوع `char` باید آن متغیر را به یک عدد از نوع `int` نسبت داد، و یا آنکه مانند مثال فوق از تبدیل نوع استاتیک استفاده کرد. نکته دیگر آنکه چون دامنه اعداد `char` از ۱۲۸- تا ۱۲۷ می‌باشد باید کلیه کاراکترهای این بازه را چاپ کنیم، اما اگر شرط حلقه را به صورت `(ch<=127)` بنویسیم با حلقه بی نهایت روبرو خواهیم شد؟! لذا مجبور به چاپ کد اسکی ۱۲۷ در یک خط جداگانه (خط هشتم) شده ایم. این برنامه را اجرا کنید و خروجی آن را ببینید.

حلقه‌های تودرتو

چنانکه می‌دانید اگر در داخل بدنه یک حلقه از حلقه دیگری استفاده شود می‌گوییم که حلقه‌های تودرتو ایجاد شده‌اند. برای به‌کارگیری یک حلقه `for` در داخل حلقه `for` دیگر بهتر است نام شمارنده‌ها را متفاوت در نظر بگیریم. چنانکه حلقه داخلی تنها دستور حلقه خارجی باشد، نیازی به قرار دادن آکولاد باز و بسته برای حلقه خارجی نیست. در زیر مثال ساده‌ای از کاربرد حلقه‌های تودرتو را می‌بینید.

مثال ۵-۶: برنامه‌ای که جدول ضرب را در صفحه نمایش چاپ می‌کند.

```
1. //This program generates multiplication chart.
2. #include "iostream"
3. using namespace std;
4. int main()
5. {
6.     for(int i=1;i<=10;i++)
7.     {
8.         for(int j=1;j<=10;j++)
9.             cout<<i*j<<"\t";
10.        cout<<endl;
11.    }
12.    return 0;
13. }
```

توضیح مثال: در این برنامه از حلقه‌های تودرتو استفاده شده است. به ازای هر بار تکرار حلقه `i` حلقه درونی به تعداد ۱۰ بار اجرا می‌گردد. در خط نهم برای منظم کردن خروجی پس از چاپ هر عدد به میزان هشت خانه به جلو می‌رویم. همچنین پس از چاپ شدن هر ۱۰ عدد خروجی در یک خط، در خط دهم کد دستکاری‌کننده `endl` خط جاری را رد می‌کند، تا ده عدد بعدی در خط جدیدی چاپ شود.

کاردرکلاس ۵-۱:

به نظر شما فروبی برنامه زیر چیست؟

```
1. //This program shows how we use counter in the complex loop.
2. #include "iostream"
3. using namespace std;
4. int main()
5. {
6.     for(int i=1;i<=10;i++){
7.         for(int i=1;i<=10;i++)
8.             cout<<i*i<<"\t";
9.         cout<<"\n";
10.    }
11.    return 0 ;
12. }
```

ساختار تکرار while

با نماد حلقه‌های **while** در الگوریتم‌نویسی در فصل قبل آشنا شدید. در C++ نیز ساختاری با نام **while** برای ایجاد حلقه‌های دارای نگهدارنده وجود دارد. وقتی کنترل اجرای برنامه به دستور **while** می‌رسد، ابتدا شرط حلقه چک شده و در صورت درست بودن شرط، اجرای دستورات داخل بلاک آغاز می‌شود، و تا زمانی که شرط حلقه دارای ارزش درستی باشد، این روند ادامه پیدا می‌کند. شکل کلی این دستور به صورت زیر است:

```
( شرط حلقه ) while
{
    دستور یکم
    ⋮
    دستور n-ام
}
```

در این ساختار نیز اگر تنها یک دستور در بدنه حلقه داشته باشیم، نیازی به قرار دادن آکولاد باز و بسته نیست، و همچنان که می‌دانید در حلقه‌هایی که از مکانیسم نگهدارنده استفاده می‌کنند، برای توقف حلقه باید شرایطی را مهیا کرد که شرط حلقه در داخل خود حلقه نقض گردد. اگر شرط نقض نشود با حلقه بی‌نهایت روبرو می‌شویم.

مثال ۵-۷: برنامه‌ای که جملات دنباله فیبوناچی را تا آنجایی که ممکن باشد محاسبه می‌کند. این برنامه قبل از آنکه با سرریز شدن داده مواجه شود، متوقف می‌گردد.

```
1. //This program uses WHILE loop for Fibonacci series.
2. #include <iostream>
3. #include <limits.h>
4. using namespace std;
5. int main()
6. {
7.     unsigned long next=0,last=1;
8.     while(next < ULONG_MAX / 2)
9.     {
10.         cout<<last<<endl;
11.         unsigned long sum=next+last;
12.         next=last;
13.         last=sum;
14.     }
15.     return 0;
16. }
```

توضیح مثال: چنانکه می‌بینید شرط حلقه تکرار را به گونه‌ای انتخاب کرده‌ایم که قبل از بروز سرریز محاسبات متوقف گردد. مقدار **ULONG_MAX** در سرفایل **limits.h** تعریف شده است. نکته دیگر آنکه تعریف یک متغیر در داخل یک حلقه تکرار موجب نمی‌شود، هر بار که کنترل اجرای برنامه به اعلان متغیر می‌رسد متغیر جدیدی تعریف کند. بلکه تنها در اولین اجرای حلقه متغیر **sum** تعریف می‌شود.

مثال ۵-۸: برنامه‌ای که دو عدد را به‌عنوان صورت و مخرج یک کسر متعارفی دریافت کرده و خارج قسمت و باقی‌مانده آن دو عدد را به واسطه عملگرهای / و % به خروجی می‌برد. به نحوه کنترل حلقه تکرار توجه کنید.

```
1. //This program demonstrates WHILE loop.
2. #include <iostream>
3. using namespace std;
4. int main()
5. {
6.     long int dividend, divisor;
7.     char ans='y';
8.     while(ans=='y' || ans=='Y')
9.     {
10.        cout<<"Enter dividend : ";
11.        cin>>dividend;
12.        cout<<"Enter divisor : ";
13.        cin>>divisor;
14.        cout<<"Quotient is : "<<dividend / divisor
15.            <<" , remainder is : "<<dividend % divisor<<endl;
16.        cout<<"Do you want to try again?(Y/N):";
17.        cin>>ans;
18.    }
19.    return 0 ;
20. }
```

توضیح مثال: تنها نکته قابل توجه در این برنامه خطوط ۱۴ و ۱۵ است که به شما نشان می‌دهد می‌توانید یک دستور را در بیش از یک خط نیز بنویسید چرا که کامپایلر C++ نسبت به فضاهاى خالی بی‌توجه است.

ساختار تکرار do...while

ساختار تکرار do...while دقیقاً مانند ساختار تکرار while عمل می‌کند با این تفاوت که شرط حلقه در پایان حلقه می‌آید، بنابراین دستورات داخل بلاک حداقل یکبار قبل از چک شدن شرط حلقه اجرا می‌گردد. شکل کلی این ساختار به‌صورت زیر است:

```
do {
    دستور یکم ;
    ⋮
    دستور n-ام ;
} while (شرط حلقه) ;
```

در این ساختار نیز هرگاه تنها یک دستور داشته باشیم نیازی نیست که آکولاد باز و بسته قرار دهیم. اما نکته بسیار مهمی که اکثر دانشجویان آن را فراموش می‌کنند علامت سمیکالنی است که در انتهای این دستور آمده است.

مثال ۵-۹: برنامه‌ای که تعداد نامشخصی عدد را از ورودی خوانده مقلوب آنها را به خروجی می‌برد. به چگونگی به کارگیری حلقه‌های تودرتو در این مثال توجه کنید.

```

1. //This program calculates inverted of some numbers.
2. #include <iostream>
3. using namespace std;
4. int main()
5. {
6.     int number, digit;
7.     while(true)
8.     {
9.         cout<<"\nEnter an integer number for convert :";
10.        cin>>number;
11.        cout<<"Inverse is : ";
12.        do{
13.            digit=number % 10;
14.            cout<<digit;
15.            number /=10;
16.        }while(number!=0);
17.    }//end of first loop
18.    return 0;
19. }
```

توضیح مثال: چنانکه می‌بینید در این برنامه از حلقه‌های تودرتو استفاده شده است. حلقه بیرونی دارای شرطی است که همواره درست است، بنابراین یک حلقه تکرار بی‌نهایت را ساخته و برای پایان اجرای این حلقه باید دکمه‌های **Ctrl+Break** را بفشارید. برای نمایش مقلوب عدد نیز تک‌تک ارقام آن را از سمت راست به دست آورده و چاپ می‌کنیم، در نتیجه تنها مقلوب عدد در صفحه چاپ می‌شود. آیا می‌توانید مثال فوق را به گونه‌ای تغییر دهید که ابتدا عدد مقلوب را در داخل یک متغیر صحیح ذخیره کند و در نهایت آن متغیر را چاپ کند.

چه وقت بهتر است از کدام یک از این حلقه‌ها استفاده کنیم؟

اگر از قبل تعداد دفعات مورد نیاز اجرای حلقه را بدانیم، بهتر است از ساختار **for** برای ایجاد حلقه استفاده کنیم. گرچه می‌توان از حلقه **while** نیز برای ایجاد حلقه تکرار با شمارنده استفاده کرد! اما از انجام این کار به شدت خودداری کنید، زیرا از خوانایی برنامه می‌کاهد. اما هرگاه تعداد دفعات تکرار حلقه را ندانیم باید به دنبال یک نگهبان برای ساخت شرط حلقه باشیم تا بتوانیم از یکی از دو ساختار **while** یا **do...while** استفاده کنیم. همچنین هرگاه نیاز داشته باشیم دستورات داخل بلاک حلقه حداقل یک بار اجرا گردد از ساختار **do...while** استفاده می‌کنیم، در غیر این صورت از ساختار **while** باید استفاده کرد.

نکته پایانی که باید متذکر شویم آن که بسیاری از دانشجویان به اشتباه به جای علامت **&&** در قسمت شرط حلقه **for** از علامت **kama** استفاده می‌کنند، و یا آنکه به جای علامت سمیکالن در حلقه‌های **for** از علامت **kama** استفاده می‌کنند. هر دوی این موارد را در یکی از مثال‌های فوق اعمال کنید و خطای صادر شده را مشاهده نمایید.

۵-۲: انتقال کنترل غیر شرطی در C++

در C++ دو دسته دستور وجود دارند که به واسطه آنها می‌توان کنترل اجرای برنامه را به خطوط مختلف کد منتقل کرد. دسته‌ای از این دستورات با چک کردن شرط یا شروطی مبادرت به انتقال کنترل اجرای برنامه می‌کنند که به ساختارهای تصمیم‌گیری یا انتقال کنترل شرطی معروف هستند، و دسته‌ای دیگر بدون چک کردن هیچ شرطی کنترل اجرای برنامه را به نقطه‌ای خاص انتقال می‌دهند که به کنترل غیرشرطی موسوم شده‌اند. در این قسمت به معرفی این کنترل‌ها می‌پردازیم اما مثال‌های متنوع آنها را در صفحات آتی خواهید دید.

دستور break

این دستور در داخل حلقه‌های تکرار قرار گرفته و موجب خروج از حلقه تکرار می‌شود. نحوه کاربرد این دستور به‌صورت زیر است:

```
break;
```

اگر چند حلقه تودرتو وجود داشته باشد این دستور موجب خروج از داخلی‌ترین حلقه می‌شود. کاربرد دیگر این دستور در ساختار switch است که در ادامه بحث خواهد شد.

دستور continue

این دستور نیز در حلقه‌های تکرار به‌کار می‌رود و موجب انتقال کنترل اجرای برنامه به ابتدای حلقه تکرار می‌شود. این دستور به‌صورت زیر به‌کار می‌رود:

```
continue;
```

پس از انتقال کنترل به ابتدای حلقه، شرط حلقه چک شده و در صورت درست بودن شرط مذکور، اجرای دستورات داخل حلقه از اولین دستور بلاک از سرگرفته می‌شود. به عبارت دیگر اگر دستور یا دستوراتی پس از دستور continue وجود داشته باشد هیچگاه اجرا نخواهند شد. بنابراین اصولاً هر دو دستور break و continue باید در داخل ساختارهای تصمیم‌به‌کار گرفته شوند در غیر این صورت اجرای دستورات حلقه تکرار با مشکل مواجه خواهد شد.

مثال ۵-۱۰: در زیر برنامه‌ای ارائه شده که عملکرد دستور continue را نشان می‌دهد.

```
1. //This program demonstrates continue statement.
2. #include <iostream.h>
3. void main()
4. {
5.     int i=0;
6.     do{
7.         i++;
8.         cout<<"before the continue\n";
9.         continue;
10.        cout<<"after the continue, should never print\n";
11.    }while(i<3);
12.    cout<<"after the do loop\n";
13. }
```

توضیح مثال: در این برنامه با توجه به وجود دستور `continue` خط دهم برنامه هیچ‌گاه اجرا نخواهد شد، و خروجی این برنامه به صورت زیر خواهد بود:

```
before the continue
before the continue
before the continue
after the do loop
```

دستور goto

این دستور سبب انتقال کنترل اجرای برنامه به نقطه‌ای می‌شود که با یک برچسب^۱ مشخص شده است. روش کاربرد این دستور به صورت زیر است:

برچسب `goto` ;

برچسب عبارت است از یک نام دلخواه که برای انتخاب آن باید از روش نامگذاری متغیرها پیروی کرد. هر خطی را که بخواهیم برچسب‌گذاری کنیم باید در ابتدای آن خط نام برچسب را بیاوریم و سپس یک علامت کولن (:) پس از نام برچسب قرار دهیم. به عنوان مثال می‌توان هر یک از عبارات زیر را به عنوان برچسب در جلوی یک خط از برنامه به کار ببریم:

`Loop:` یا `Label:` یا `G1:`

به دلیل این که استفاده از دستور `goto` می‌تواند برنامه را از ساخت یافتگی خارج کند، استفاده از این دستور در تمامی منابع برنامه‌نویسی نهی شده است. حتی برای ایجاد حلقه‌های تکرار هم به هیچ‌عنوان نباید از دستور `goto` استفاده نمود. لذا ما نیز به دلیل عدم کاربرد این دستور در برنامه‌های ساخت یافته به آن نمی‌پردازیم.

کاردر کلاس ۵-۲:

با استفاده از دستور `goto` برنامه معادل کارنمای شکل ۴-۳ را به زبان C++ بنویسید.

۵-۳: ساختارهای تصمیم در C++

در هر زبان برنامه‌نویسی ساختارهایی را جهت اعمال تصمیم‌گیری در برنامه‌ها که معادل جعبه‌های تصمیم در کارنما هاست فراهم می‌کنند. در C++ نیز تعدادی ساختار جهت اعمال تصمیم‌گیری در برنامه‌ها فراهم شده است که در ادامه به بررسی این ساختارها می‌پردازیم.

ساختار تصمیم‌گیری if

ساختار تصمیم‌گیری if دارای دو شکل کلی است. چنانکه در زیر می‌بینید در حالت اول اگر شرط ساختار if دارای ارزش درستی باشد دستورات داخل بلاک اجرا می‌گردد و در صورت نادرست بودن شرط مذکور، کنترل اجرای برنامه به بعد از علامت } می‌رود. اما حالت دیگر نحوه عملکرد این ساختار بدین گونه است که اگر شرط ساختار if درست باشد دستورات داخل بلاک پس از دستور if اجرا می‌گردد، و در صورت نادرست بودن شرط، دستورات داخل بلاک پس از کلمه else اجرا می‌گردد. فرم کلی این ساختار به صورت‌های زیر است:

ا- صورت کلی اول:

```
if (شرط)
{
    دستور یکم
    ⋮
    دستور n-ام
}
```

ب- صورت کلی دوم:

```
if (شرط)
{
    دستور یکم
    ⋮
    دستور n-ام
}
else
{
    دستور یکم
    ⋮
    دستور n-ام
}
```

در این ساختار نیز در صورت وجود تک دستور در یک بلاک نیازی به قرار دادن آکولاد باز و بسته نیست.

مثال ۵-۱۱: برنامه‌ای که از کاربر رمز ورود به برنامه را می‌خواهد و در صورتی که رمز به‌طور صحیح وارد شود پیغام خوش‌آمد دریافت می‌کند.

```
1. //This program demonstrates IF statement.
2. #include "iostream"
3. using namespace std;
4. int main ( )
5. {
6.     const int password=62541893;
7.     int ans;
8.     cout<<"Please enter the password for login : ";
9.     cin>>ans;
10.    if (ans==password)
11.    {
12.        cout<<"Welcome to MJZ program.\n";
13.        return 0;
14.    }
15.    cout<<"Unfortunately you entered a wrong password.";
16.    cout<<"\nLogin failed.\n";
17.    return 0;
18. }
```

توضیح مثال: اولین نکته بسیار مهم در این مثال تعریف یک متغیر از نوع صحیح به‌صورت ثابت است. که تضمین می‌کند مقدار password در طول برنامه سهواً تغییر نمی‌کند.

نکته قابل توجه در این برنامه وجود دو دستور return در خطوط ۱۳ و ۱۷ برنامه است. چنانکه در فصل سوم در خصوص دستور return عنوان شد این دستور نه تنها مقداری را به فراخواننده تابع برمی‌گرداند بلکه کنترل اجرای برنامه را نیز به فراخواننده تابع برمی‌گرداند. لذا اگر دستوری پس از دستور return قرار داشته باشد هیچ‌گاه اجرا نخواهد شد. در صورتی که کاربر رمز صحیح، یعنی عدد 62541893 را وارد کند، رمز مورد قبول واقع شده و پیغام "Welcome to MJZ program." را در صفحه می‌بیند و با رسیدن به دستور return کنترل اجرای برنامه نیز به سیستم عامل برگردانده می‌شود و لذا پیغام خطا چاپ نخواهد شد. اما در صورت وارد کردن هر عدد دیگری پیغام زیر در صفحه نمایش مشاهده می‌شود.

"Unfortunately you entered a wrong password. Login failed."

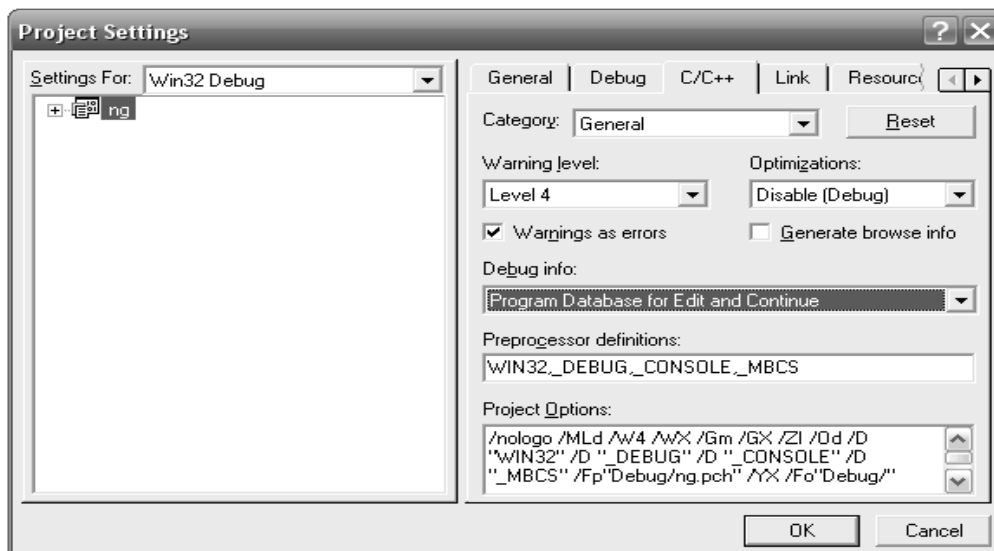
کار در کلاس ۵-۳:

مثال ۵-۱۱ را یک بار دیگر به واسطه if...else به گونه ای بازنویسی کنید که نیازی به دو دستور return نداشته باشد.

تذکره مهم: بسیاری از دانشجویان به اشتباه در عبارت شرطی مربوط به دستور `if` به جای استفاده از علامت `==` برای نشان دادن برابری از علامت مربوط به انتساب یعنی `=` استفاده می‌کنند. مسلماً چنین عملی موجب می‌شود با نتایج غیرمنتظره‌ای روبرو بشویم. به‌عنوان یک آزمایش در خط دهم همین برنامه علامت `==` را با علامت `=` جایگزین کنید و سپس یکبار دیگر برنامه را کامپایل و اجرا کنید. چه نتیجه‌ای مشاهده می‌کنید؟

نتیجه‌ای که مشاهده خواهید کرد این است که با ورود هرگونه عدد شما پیام خوش‌آمد را مشاهده می‌کنید. یعنی رمزی که ما برای ورود به برنامه خود گذاشتیم هیچگاه عمل نمی‌کند. این در یک برنامه بزرگ یعنی یک فاجعه! در حقیقت یک عمل انتساب صورت می‌گیرد و در صورت موفقیت‌آمیز بودن این عمل مقدار `true` به‌جای عبارت انتساب قرار می‌گیرد، و از آنجا که عمل انتساب همواره با موفقیت همراه است همیشه شرط دستور `if` برقرار است و در نتیجه پیام خوش‌آمد چاپ می‌شود.

اما متأسفانه اگر دقت کرده باشید کامپایلر حتی هیچگونه هشداري هم در خصوص این خطا نمی‌دهد. با این اوصاف چگونه می‌توان چنین مشکل بزرگی را در یک برنامه چند هزار خطی تشخیص داد؟ جواب این سؤال در تنظیمات کامپایلر است. برای این که کامپایلر بتواند چنین خطاهایی را تشخیص دهد باید سطح هشداردهی کامپایلر را بر روی `maximum warning` قرار دهید. نحوه انجام این تنظیم در کامپایلرهای مختلف متفاوت است. برای این منظور می‌توانید در مستندات مربوط به کامپایلر خود، عبارت `warning level` را جستجو کنید. اما در محیط `Visual Studio 6.0` می‌توانید از طریق انتخاب گزینه `setting` از منوی `project` پنجره‌ای مطابق شکل ۵-۱ موسوم به پنجره `Project Setting` را باز کنید. در این پنجره صفحه `C/C++` را انتخاب کنید. در این صفحه سطح هشدارها را بر روی `level 4` تنظیم کنید و جلوی گزینه `warning as error` علامت تیک بزنید. حال یکبار دیگر کد قبلی را کامپایل کنید این بار چه نتیجه‌ای می‌بینید.



شکل ۵-۱: پنجره تنظیمات سطح هشدار

مثال ۵-۱۲: برنامه‌ای که با دریافت یک پاراگراف تعداد حروف و کلمات آن را می‌شمارد. انتهای ورود داده ها نیز با زدن کلید **enter** مشخص می‌شود.

```

1. //This program counts the number of words and characters.
2. #include <iostream>
3. #include <conio.h> //included for getch() and getche()
4. using namespace std;
5. int main()
6. {
7.     int char_counter=0, word_counter=0;
8.     char ch=0;
9.     cout<<"Enter a paragraph(and press Enter for end):\n";
10.    while((ch=getche())!='\r')
11.    {
12.        char_counter++;
13.        if(ch==' ')
14.            word_counter++;
15.    } //end of while
16.    cout<<"\nChar count= "<<char_counter
17.        <<" ,Word count= "<<word_counter+1;
18.    getch();
19.    cout <<endl;
20.    return 0;
21. }
```

توضیح مثال: در این برنامه توسط تابع **getche** شروع به گرفتن کاراکترهای ورودی می‌کنیم. این تابع در هدر فایل **conio.h** قرار دارد و یک کاراکتر را از کاربر گرفته و در صفحه نمایش نشان می‌دهد. هر بار که حلقه اجرا می‌شود ابتدا این تابع اجرا می‌گردد. همچنین تابع **getche** کداسکی کاراکتر دریافتی را برمی‌گرداند. لذا در این مثال کداسکی برگردانده شده توسط این تابع، را به داخل یک متغیر از نوع **char** ریخته‌ایم. کاراکتر کنترلی **\n** کلید **enter** را مشخص می‌کند. کداسکی کلید **enter** برابر ۱۳ می‌باشد و تا زمانی که این کداسکی توسط تابع برگردانده نشود حلقه به تکرار خود ادامه می‌دهد. می‌توان به جای **'\n'** عدد 13 را قرار داد. هر کاراکتر که خوانده می‌شود یک واحد به شمارنده حروف اضافه می‌گردد. اما از آنجا که کلمات با فضای خالی (**space**) از یکدیگر جدا می‌شوند در خطوط ۱۳ و ۱۴ برنامه در صورتی که کاراکتر ورودی برابر فضای خالی باشد یک واحد نیز به شمارنده کلمات اضافه می‌شود. اما آخرین نکته در خصوص این برنامه قرار دادن تابع **getch** در انتهای برنامه است. این تابع نیز یک کاراکتر را از کاربر می‌گیرد ولی در صفحه چاپ نمی‌کند. برای جلوگیری از خاتمه برنامه و ایجاد یک وقفه جهت مشاهده نتایج از این تابع استفاده کرده‌ایم. با آنکه در کامپایلر **VC6** یک عبارت **press any key to continue** پس از اجرای برنامه قرار داده می‌شود، و شما می‌توانید نتایج را به راحتی مشاهده کنید، اما در صورت اجرا کردن برنامه‌ها پس از مرحله کامپایل این عبارت در انتهای برنامه نمی‌آید و در نتیجه برنامه با رسیدن به دستور **return** خاتمه می‌یابد. لذا این دستور یک وقفه قبل از اجرای دستور **return** ایجاد می‌کند. هنگام استفاده از سرفایل **conio.h** در کامپایلر **VC6** به هیچ عنوان از **iostream.h** استفاده نکنید.

مثال ۵-۱۳: برنامه‌ای که مثال ۵-۱۱ را به‌واسطه توابع `getch` و ساختار `if...else` پیاده‌سازی می‌کند.

```
1. //This program demonstrates IF...ELSE statement.
2. #include "iostream"
3. #include <conio.h>
4. using namespace std;
5. int main()
6. {
7.     const int password=62541893;
8.     int ans=0;
9.     char ch=0;
10.    cout<<"Please enter the password for login : ";
11.    while( (ch=getch()) != 13)
12.        if(ch>='0' && ch<='9')
13.        {
14.            cout.put('*');
15.            ans *=10;
16.            ans +=(ch-48);
17.        }
18.    if(ans==password)
19.        cout<<"\nWelcome to MJZ program.\n";
20.    else
21.        cout<<"\nUnfortunately you entered a wrong password."
22.        <<"\nLogin failed.\n";
23.    return 0;
24. }
```

توضیح مثال: با عبارتی که در خط ۱۱ آمده در مثال قبل آشنا شدید. اما به جهت آنکه می‌خواهیم رمز وارد شده توسط کاربر بر روی صفحه نمایش چاپ نشود از تابع `getch` استفاده کرده‌ایم. این تابع نیز کد اسکی کلید فشرده شده را برمی‌گرداند و در نتیجه این کد داخل `ch` قرار می‌گیرد. در خط ۱۲ چک می‌شود که اگر کد اسکی داخل کاراکتر `ch` شامل کد اسکی یکی از ارقام ۰ تا ۹ می‌باشد، وارد بلاک مربوط به دستور `if` بشویم. توجه کنید که ارقام ۰ تا ۹ هنگامی که به‌صورت یک کاراکتر فرض می‌شوند دارای کدهای اسکی بین ۴۸ تا ۵۷ هستند لذا در خط ۱۶ برای رسیدن به رقم ریاضی معادل کاراکترهای ۰ تا ۹ مقدار ۴۸ را از کاراکتر `ch` کم کرده‌ایم. اما قبل از آن، در خط ۱۵ متغیر `ans` را در ۱۰ ضرب کرده‌ایم، تا به این واسطه هنگامی که در خط ۱۶ رقم وارد شده جدید را با `ans` جمع می‌کنیم این رقم در انتها الیه سمت راست عدد منظور شود.

اما در خط ۱۴ پس از آنکه کاربر یک رقم را به‌عنوان ورودی وارد کرد یک علامت * بر روی صفحه چاپ می‌کنیم تا کاربر بداند رقم وارد شده با موفقیت ثبت شده است. این درحالی است که اگر به‌عنوان مثال کاربر حرفی را وارد کند علامت * چاپ نمی‌شود. برای چاپ این علامت از تابع `put` که یک تابع عضو شیء `cout` می‌باشد استفاده کرده‌ایم. این تابع با دریافت کد اسکی یک کاراکتر آن کاراکتر را بر روی صفحه چاپ می‌کند، و دارای شکل کلی زیر است:

`cout.put ( کد اسکی یک کاراکتر )` ;

ساختار تصمیم‌گیری else if

اگر در داخل قسمت else یک دستور if، دستور if جدیدی به کار ببریم به ساختار else if می‌رسیم. این ساختار به ما کمک می‌کند تا شروط مختلفی را چک کنیم. شکل کلی این دستور به صورت زیر است:

```
if ( شرط ۱ )
{
    نخستین بلاک
}
else
    if ( شرط ۲ )
    {
        دومین بلاک
    }
    else
    {
        آخرین بلاک
    }
```

هنگامی که کنترل اجرای برنامه به اولین دستور if می‌رسد شرط آن چک می‌شود، اگر شرط دارای ارزش درستی باشد دستورات داخل نخستین بلاک اجرا شده و سپس کنترل اجرای برنامه به بعد از آکولاد بسته آخرین بلاک می‌رود. اما اگر شرط درست نباشد به قسمت else آمده و براساس درست یا غلط بودن شرط ۲ یکی از بلاک‌های دوم یا سوم اجرا می‌گردد. این روند یعنی قرار دادن یک دستور if در داخل قسمت else یک دستور if دیگر می‌تواند تا هر جایی که نیاز ما را برطرف سازد و تمامی شروط ما را پوشش دهد ادامه یابد. اصولاً دو دستور else و if را در ساختار فوق در یک خط می‌نویسند تا به خوانایی برنامه افزوده شود. بنابراین داریم:

```
if ( شرط ۱ )
{
    نخستین بلاک
}
else if ( شرط ۲ )
{
    دومین بلاک
}
```

دستورات else if متعدد

↓

```
else
{
    آخرین بلاک
}
```

پاک کردن صفحه نمایش

در کامپایلرهای شرکت بورلند امکان استفاده از چهار تابع مفید و پرکاربرد وجود دارد که در VC6 امکان استفاده از آنها مهیا نیست، چرا که کامپایلر VC6 از این چهار تابع که به ترتیب عبارتند از: `clrscr`، `gotoxy`، `random` و `randomize` پشتیبانی نمی‌کند. از بین این چهار تابع دو تابع اول بسیار مهم و دارای کاربردهای متعددی هستند، که نمونه‌ای از کاربرد آنها را در همین فصل خواهید دید و در مورد کاربرد دو تابع دوم اندکی بعد صحبت خواهیم کرد. با توجه به احساس نیاز برنامه‌نویسان به این توابع تلاش‌هایی برای پیاده‌سازی این توابع در VC6 صورت گرفته که در این قسمت به ذکر آنها برای پیاده‌سازی `clrscr` و `gotoxy` می‌پردازیم. ذکر جزئیات و عملکرد کدهای ارائه شده در این قسمت فراتر از معلومات فعلی شماست. لذا فعلاً تنها به نحوه پیاده‌سازی این توابع و چگونگی به‌کارگیری و استفاده این توابع توجه کنید.

در کامپایلرهای شرکت بورلند با اضافه کردن سرفایل `conio.h` قادر به استفاده از تابع `clrscr` برای پاک کردن صفحه نمایش هستید. این تابع به صورت زیر به کار می‌رود:

```
clrscr();
```

هنگامی که کنترل اجرای برنامه به این تابع می‌رسد محتویات صفحه نمایش را پاک می‌کند. اما اگر چنین دستوری را در داخل VC6 بنویسید با خطا روبرو می‌شوید. برای آنکه بتوانید این دستور را در VC6 نیز همانند کامپایلرهای بورلند استفاده کنید مراحل زیر را باید انجام دهید.

۱. پیش از تابع `main` و بعد از دستور `using` خط زیر را تایپ کنید:

```
void clrscr(void);
```

۲. پس از آکولاد بسته تابع `main` نیز کد زیر را تایپ کنید:

```
void clrscr()
{
    system("cls");
}
```

در هر کجای کد هر برنامه‌ای که در حالت کنسول نوشته می‌شود می‌توانید دستورات سیستم عامل `Dos` را توسط تابع `system` به خط فرمان فرستاده، و اجرا کنید. به عنوان مثال در عبارت فوق دستور `cls` که یکی از فرامین سیستم عامل `Dos` می‌باشد و موجب پاک شدن محتویات صفحه نمایش می‌گردد را به خط فرمان داس فرستاده‌ایم تا صفحه نمایش را برای ما پاک کند. نکته بسیار مهم آنکه دستور موردنظر را باید به صورت یک رشته (در داخل گیومه) بنویسید. به عنوان مثالی دیگر اگر بخواهیم رنگ پس زمینه را به رنگ آبی و رنگ متن نوشتاری را به رنگ سبز در بیاوریم کافی است در ابتدای تابع `main` دستور زیر را تایپ کنید:

```
system("color 12");
```

برای یافتن اطلاعات بیشتر در خصوص فرامین `Dos` باید به قسمت `help` سیستم عامل `Dos` مراجعه کنید. برای این منظور ابتدا در گزینه `Run` از منوی `Start` ویندوز خود عبارت `cmd` را تایپ کرده و کلید `enter` را بزنید تا پنجره `command prompt` باز شود. سپس در جلوی خط فرمان عبارت `help` را تایپ کرده و `enter` بزنید تا مستندات سیستم عامل `Dos` نمایان گردد. در این قسمت فرامین بیشتری را می‌توانید بیابید.

انتقال مکان نما در صفحه نمایش

اصولاً برای ایجاد یک خروجی مناسب و شکل در صفحه نمایش نیازمند آن هستیم که به راحتی مکان نما را در صفحه حرکت داده و در هر نقطه که لازم باشد خروجی چاپ شود. این امکان در کامپایلرهای شرکت بورلند به واسطه تابع `gotoxy` فراهم شده است. برای استفاده از این تابع باید هدر فایل `conio.h` را به برنامه اضافه کرد. این تابع دارای شکل کلی زیر است:

```
gotoxy(int X, int Y);
```

در حالت کنسول صفحه نمایش به صورت ۸۰ ستون و ۲۵ ردیف فرض می‌شود. که گوشه بالا سمت چپ، معادل نقطه (0, 0) و گوشه پایین سمت راست، معادل نقطه (80, 25) در نظر گرفته می‌شود. نکته قابل توجه آن که در دستور `gotoxy` ابتدا شماره ستون و سپس شماره ردیف ذکر می‌گردد. به عنوان مثال دستور `gotoxy(5, 10)` مکان نما را به ستون پنجم و ردیف دهم منتقل می‌کند.

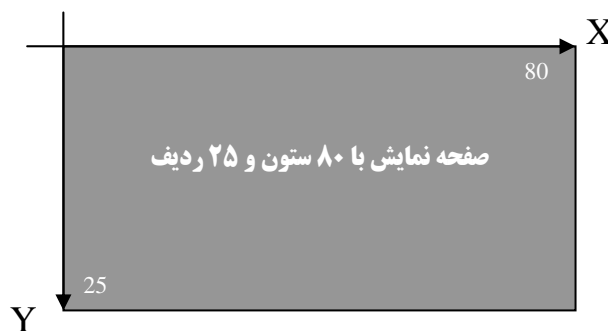
اما برای آنکه بتوانید در VC6 نیز از این تابع استفاده کنید باید مراحل زیر را انجام دهید:

۱. قبل از تابع `main` و پس از دستور `using` خطوط زیر را تایپ کنید:

```
#include <windows.h>
void gotoxy(int, int);
```

۲. پس از آکولاد بسته تابع `main` نیز دستورات زیر را تایپ کنید:

```
//function definition -- requires windows.h
void gotoxy(int x, int y)
{
    HANDLE hConsoleOutput;
    COORD dwCursorPosition;
    cout.flush();
    dwCursorPosition.X = x;
    dwCursorPosition.Y = y;
    hConsoleOutput = GetStdHandle(STD_OUTPUT_HANDLE);
    SetConsoleCursorPosition(hConsoleOutput, dwCursorPosition);
}
```



شکل ۵-۲: نمای صفحه کنسول

مثال ۵-۱۴: برنامه‌ای که با فشردن دکمه‌های U و D و L و R یک علامت ☺ را در صفحه نمایش به ترتیب به سمت بالا، پایین، چپ و راست می‌برد. هدف از این برنامه نشان دادن نحوه به‌کارگیری ساختار `else...if` و توابع `gotoxy` و `clrscr` است.

```

1. //This program demonstrates ELSE...IF statement
2. #include <iostream>
3. #include <conio.h>
4. using namespace std;
5. #include <windows.h>
6. void gotoxy(int, int); //prototype
7. void clrscr(); //prototype
8. int main()
9. {
10. //move ☺ in to four corners of the screen.
11. clrscr(); //function call
12. gotoxy(0,25); //move to begin of the last line
13. char U=24,D=25,R=26,L=27; //ASCII code of 4 vectors
14. cout<<"Use these keys for move:\t"
15. <<"U="<<U<<"\tD="<<D<<"\tR="<<R<<"\tL="<<L;
16. char sg=1,ch=0; //sg has the ASCII code of @
17. int x=39,y=11; //x and y store position of cursor.
18. gotoxy(x,y); //move to center of screen.
19. cout.put(sg);
20. gotoxy(x,y); //return to previous position.
21. while((ch=getch())!=27)
22. {
23. //start getting command of user.
24. if((ch=='U' || ch=='u') && y>1)
25. {
26. cout.put(' ');
27. gotoxy(x,--y);
28. cout.put(sg);
29. gotoxy(x,y);
30. }
31. else if((ch=='D' || ch=='d') && y<24)
32. {
33. cout.put(' ');
34. gotoxy(x,++y);
35. cout.put(sg);
36. gotoxy(x,y);
37. }
38. else if((ch=='R' || ch=='r') && x<79)
39. {
40. cout.put(' ');
41. gotoxy(++x,y);
42. cout.put(sg);
43. gotoxy(x,y);
44. }

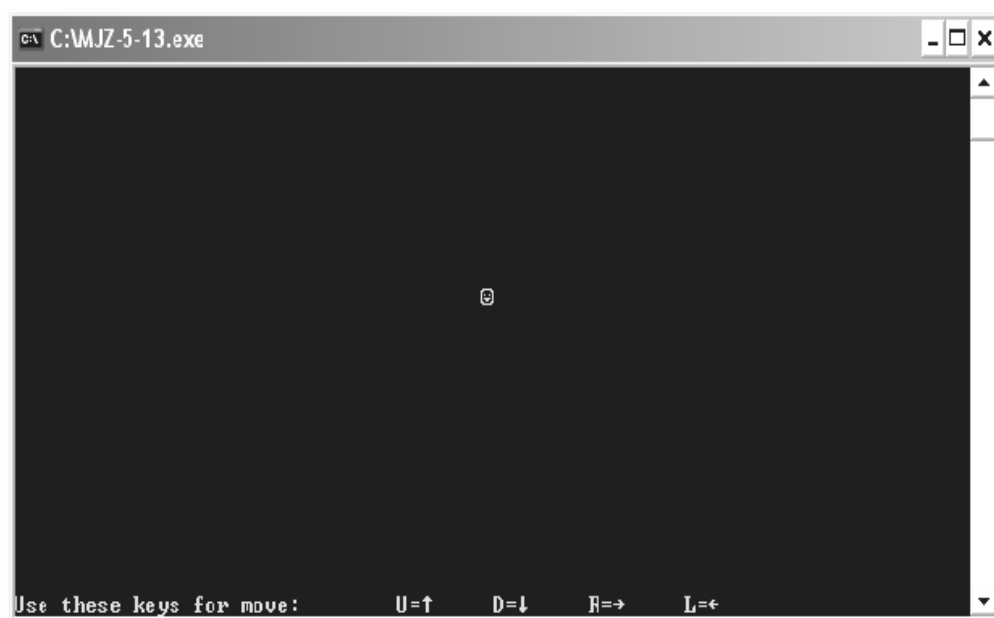
```

```

45.         else if((ch=='L' || ch=='l') && x>0)
46.         {
47.             cout.put(' ');
48.             gotoxy(--x,y);
49.             cout.put(sg);
50.             gotoxy(x,y);
51.         }
52.     } //end of while
53.     clrscr();
54.     return 0;
55. }
56. // function definition -- requires windows.h
57. void gotoxy(int x, int y)
58. {
59.     HANDLE hConsoleOutput;
60.     COORD dwCursorPosition;
61.     cout.flush();
62.     dwCursorPosition.X = x;
63.     dwCursorPosition.Y = y;
64.     hConsoleOutput = GetStdHandle(STD_OUTPUT_HANDLE);
65.     SetConsoleCursorPosition(hConsoleOutput,dwCursorPosition);
66. }
67. //function definition.In some compilers require process.h
68. void clrscr()
69. {
70.     system("cls");
71. }

```

توضیح مثال : در این قسمت از خط ۹ تا ۵۳ که مربوط به تابع اصلی است را توضیح می‌دهیم. از خط ۱۱ تا ۱۴ ابتدا مکان‌نما را به ابتدای آخرین خط برده و توضیحاتی را برای کاربر چاپ می‌کنیم، که از چه کلیدهایی قادر است استفاده کند. متغیر sg حاوی کد اسکی علامت ☺ است و برای چاپ این علامت از sg استفاده می‌کنیم. متغیرهای x و y حاوی آخرین موقعیت مکان‌نما هستند. در خطوط ۱۶ تا ۱۸ مکان‌نما را به وسط صفحه برده و علامت ☺ را چاپ می‌کنیم. در خط ۱۹ مکان‌نما را دوباره به محل چاپ علامت دایره‌ای هدایت می‌کنیم تا در دفعه بعد که می‌خواهیم این علامت را حرکت دهیم ابتدا آن را پاک کنیم و سپس علامت را در مکان جدیدی چاپ کنیم. در حلقه while تا زمانی که کاربر کلید Esc را فشار ندهد فرامین کاربر را دریافت می‌کنیم. کد اسکی کلید Esc برابر ۲۷ می‌باشد. از بین بلوک‌های else...if به دلیل تشابه کد تنها خطوط ۲۲ تا ۲۸ را توضیح می‌دهیم. در خط ۲۲ ابتدا چک می‌کنیم که اگر کاربر کلید U را فشار داده است و مکان‌نما به بالای صفحه نرسیده است، وارد بلوک نخست می‌شویم. در خط ۲۴ این بلوک ابتدا علامت را از مکان قبلی آن پاک می‌کنیم و سپس با کاهش مقدار y و هدایت مکان‌نما به یک خانه بالاتر علامت ☺ را در محل جدید چاپ می‌کنیم. و در خط ۲۷ دوباره اشاره‌گر را به زیر علامت دایره‌ای هدایت می‌کنیم.



شکل ۵-۳: نمونه خروجی مثال ۵-۱۴

تذکره: آخرین نکته‌ای که در خصوص دستورات `if` بیان خواهیم کرد این است که، همیشه یک دستور `else` با آخرین دستور `if` که دارای هیچ `else` نیست متناظر می‌شود. به عنوان مثال شما فکر می‌کنید با اجرا شدن قطعه کد زیر کدام یک از خطوط ۴ یا ۶ اجرا می‌گردد؟

```
1. int a=5, b=7, c=7;
2. if (a==b)
3.     if (b==c)
4.         cout<<"a and b and c are the same.\n";
5.     else
6.         cout <<"a and b are different.\n";
```

احتمالاً به این نتیجه رسیده‌اید که خط ۶ اجرا می‌شود. اما در حقیقت هیچ یک از دو پیغام فوق چاپ نمی‌گردد. چرا که `else` موجود در خط ۵ با `if` موجود در خط ۳ متناظر می‌شود، نه `if` موجود در خط ۲. برای آنکه `else` موجود در خط ۵ با `if` موجود در خط ۲ متناظر گردد باید از بلوک‌بندی استفاده کنیم. در این صورت باید بنویسیم:

```
1. int a=5, b=7, c=7;
2. if (a==b)
3. {
4.     if (b==c)
5.         cout<<"a and b and c are the same.\n";
6. }
7. else
8.     cout<<"a and b are different.\n";
```

تولید اعداد تصادفی

در C++ تابع کتابخانه‌ای به نام `rand` وجود دارد که با فراخوانی آن یک عدد تصادفی از نوع صحیح تولید کرده و عدد مذکور را برمی‌گرداند. الگوی این تابع در فایل `stdlib.h` قرار دارد که باید به برنامه اضافه گردد. شکل کلی به‌کارگیری این تابع به‌صورت زیر است.

```
int magic = rand();
```

اما چنانکه پیشتر گفته شد در بورلند C++ دو تابع `randomize` و `random` وجود دارند که در VC6 قابل استفاده نیستند. تابع `random` موجب می‌شود که عدد تصادفی تولید شده در یک بازه خاص قرار داشته باشد. شکل کلی به‌کارگیری این تابع به‌صورت زیر است:

```
int magic = random(num);
```

عدد تصادفی تولید شده توسط این تابع در بازه صفر تا `num` قرار دارد. `num` باید یک عدد غیر اعشاری باشد. همچنین الگوریتم تولید اعداد تصادفی نیاز به یک عدد مبنا دارد. اگر تابع `random` را بدون تعیین مبنای تولید اعداد تصادفی بکار ببرید هر بار که از این تابع استفاده می‌کنید نتایج مشابهی را مشاهده خواهید کرد. زیرا عدد مبنا به‌طور خودکار برابر صفر فرض می‌شود. لذا تابع اعداد تصادفی همواره نتایج مشابهی را به‌دست خواهد داد. اما برای تعیین یک مبنا برای تولید اعداد واقعاً تصادفی از تابع `randomize` استفاده می‌شود. اصولاً قبل از تولید اعداد تصادفی با اجرای تابع `randomize` یک عدد تصادفی برحسب زمان سیستم تولید می‌شود و به‌عنوان مبنای تولید اعداد تصادفی بعدی قرار می‌گیرد. این تابع نیز به‌صورت زیر به‌کار می‌رود:

```
randomize();
```

اما برای شبیه‌سازی این توابع در محیط VC6 باید خطوط زیر را در ابتدای برنامه اضافه کنید:

```
#include <cstdlib>
#include <ctime>
#define randomize() (srand(time(0)))
#define random(x) (rand() % x)
```

مثال ۵-۱۵: برنامه‌ای که نحوه استفاده از این توابع را نشان می‌دهد. این برنامه را چندین بار اجرا کنید و نتایج آن را با هم مقایسه نمایید. خروجی خط ۹ همواره ثابت اما خروجی خط ۱۲ متغیر است.

```
1. //This program generates random number.
2. #include <cstdlib>
3. #include <ctime>
4. #define randomize() (srand(time(0)))
5. #define random(x) (rand() % x)
6. #include <iostream.h>
7. int main ( )
8. {
9.     for(int i=0; i<10; cout<<random(25)<<endl , i++);
10.    randomize();
11.    cout<<"After call randomize function.\n";
12.    for(i=0; i<10; cout<<random(25)<<endl , i++);
13.    return 0;
14. }
```

ساختار switch

یکی از ساختارهایی که می‌تواند معادل جعبه‌های تصمیم‌چندگانه یا درخت‌های انتخاب به کار رود ساختار switch است. با این تفاوت که در ساختارهای چند انتخابی قبلی مانند else if قادر بودیم هرگونه شرطی را مورد ارزیابی قرار دهیم ولی ساختار switch تنها برای ارزیابی "مساوی بودن" یک عبارت با مقادیر گوناگون، به کار می‌رود. شکل کلی این ساختار به صورت زیر است:

```
switch (عبارت)
{
    case مقدار ۱ :
        مجموعه دستورات ۱
        break;
    case مقدار ۲ :
        مجموعه دستورات ۲
        break;
        ...
    case مقدار آخر :
        مجموعه دستورات آخر
        break;
    default:
        مجموعه دستورات حالت استثناء
}
```

در ساختار فوق در جلوی دستور switch یک عبارت محاسباتی یا نام یک متغیر قرار می‌گیرد. نتیجه حاصل از ارزیابی عبارت محاسباتی یا متغیری که در بین دو پرانتز قرار می‌گیرد باید از نوع عدد صحیح باشد، مثلاً می‌تواند int یا char باشد ولی از انواع ممیز شناور مثل float نمی‌تواند باشد. در جلوی هر یک از کلمات case باید یک عدد صحیح یا یک کد اسکی قرار گیرد. هنگامی که کنترل اجرای برنامه به دستور switch می‌رسد مقدار داخل پرانتز ارزیابی می‌شود، سپس این مقدار با مقدار اولین case مقایسه شده در صورت برابر بودن مجموعه دستورات ۱ اجرا شده و با رسیدن به دستور break از ساختار switch خارج می‌شویم. در صورتی که مقدار داخل پرانتز با مقدار جلوی اولین case برابر نباشد، کنترل اجرای برنامه به سراغ case بعدی می‌رود و عمل مقایسه عبارت داخل پرانتز را با مقدار جلوی این case انجام می‌دهد. این روند تا آخرین case ادامه پیدا می‌کند تا آنکه بالاخره یک case مشابه مقدار داخل پرانتز پیدا شود. اگر مقدار هیچ یک از case‌ها با عبارت داخل پرانتز برابر نباشد و ساختار switch دارای قسمت default باشد دستورات داخل این قسمت اجرا شده و در نهایت از ساختار switch خارج می‌شویم. اگر تمایل دارید بلوک واحدی از دستورات برای case‌های متفاوتی اجرا شود کافی است از قرار دادن break خودداری کنید تا مقادیر case‌ها با هم بشوند.

مثال ۵-۱۶: برنامه‌ای که با تولید اعداد تصادفی چگونگی ریخته شدن تاس را شبیه‌سازی می‌کند. در این برنامه تا زمانی که کاربر کلید **enter** را بزند، تاس ریخته شده و عدد مربوط به آن نمایش داده می‌شود. با زدن کلید **Esc** از برنامه خارج می‌شویم.

```

1. //This program can imagery dice rolling.
2. #include <cstdlib>
3. #include <ctime>
4. #define randomize() (srand(time(0)))
5. #define random(x) (rand()%x)
6. #include <conio.h>
7. #include <iostream>
8. using namespace std;
9. void main()
10. {
11.     cout<<"Press Enter for roll the dice and press Esc for exit.";
12.     int n=1;
13.     do{
14.         char ch=getch();
15.         if(ch==13)
16.         {
17.             system("cls");
18.             randomize();
19.             switch(random(6)+1)
20.             {
21.                 case 1:
22.                     cout<<n++<<"- The dice is 1.";
23.                     break;
24.                 case 2:
25.                     cout<<n++<<"- The dice is 2.";
26.                     break;
27.                 case 3:
28.                     cout<<n++<<"- The dice is 3.";
29.                     break;
30.                 case 4:
31.                     cout<<n++<<"- The dice is 4.";
32.                     break;
33.                 case 5:
34.                     cout<<n++<<"- The dice is 5.";
35.                     break;
36.                 default :
37.                     cout<<n++<<"- The dice is 6.";
38.             }//end of switch
39.         }//end of if
40.         else if(ch==27)
41.             break;//break the loop if user press the Esc.
42.     }while(1);//end of DO..WHILE loop.
43. }
```

توضیح مثال: در این مثال در یک حلقه تکرار بی‌نهایت در خط ۱۳ شروع به دریافت فرامین کاربر کرده‌ایم. اگر کاربر کلید enter را بفشارد در خطوط ۱۸ و ۱۹ یک عدد تصادفی بین ۱ تا ۶ تولید می‌شود. سپس در caseهای مختلف بر حسب اینکه عدد تصادفی تولید شده کدام یک از اعداد ۱ تا ۶ باشد پیغام مناسب، چاپ می‌شود. هرگاه کاربر کلید Esc را بفشارد دستور break موجود در خط ۴۱ موجب شکستن حلقه تکرار شده و اجرای برنامه خاتمه می‌یابد.

مثال ۵-۱۷: برنامه‌ای که طریقه or کردن caseهای مختلف دستور switch (ایجاد caseهای چندگانه) را نشان می‌دهد. در این برنامه نیز یک عدد تصادفی بین ۱ تا ۶ به‌عنوان عدد تاس ریخته شده تولید می‌شود و برنامه تنها زوج یا فرد بودن عدد را اعلام می‌کند.

```
1. //This program shows what happen when you delete BREAK.
2. #include <cstdlib>
3. #include <ctime>
4. #define randomize() (srand(time(0)))
5. #include <iostream>
6. using namespace std;
7. int main()
8. {
9.     randomize();
10.    switch (rand()%6+1)
11.    {
12.        case 1:
13.        case 3:
14.        case 5:
15.            cout<<"The dice is odd.\n";
16.            break;
17.        case 2:
18.        case 4:
19.        case 6:
20.            cout<<"The dice is even.\n";
21.            break;
22.    }
23.    return 0;
24. }
```

عملگر شرطی (عملگر سه‌تایی Ternary Operator)

در C++ می‌توان به واسطه عملگر ؟ یک ساختار تصمیم‌گیری ساده ولی پرکاربرد را پیاده‌سازی کرد که به این واسطه در کدنویسی صرفه‌جویی می‌شود. شکل کلی به‌کارگیری این عملگر به‌صورت زیر است:

؛ عبارت محاسبه‌ای ۲ : عبارت محاسبه‌ای ۱ ؟ عبارت شرطی = متغیر

عملکرد این عملگر به این صورت است که اگر عبارت شرطی دارای ارزش درستی بود آن‌گاه مقدار عبارت محاسبه‌ای ۱ ارزیابی شده و در متغیر سمت چپ دستور انتساب قرار می‌گیرد، و الا اگر عبارت شرطی دارای ارزش نادرستی باشد مقدار عبارت محاسبه‌ای ۲ ارزیابی شده و در متغیر سمت چپ دستور انتساب قرار می‌گیرد.

به عبارت دیگر عملگر سه‌تایی معادل کد زیر عمل می‌کند:

```
if (عبارت شرطی)
    عبارت محاسباتی ۱ = متغیر ;
else
    عبارت محاسباتی ۲ = متغیر ;
```

مثال ۵-۱۸: برنامه‌ای که نحوه عملکرد عملگر سه‌تایی را نشان می‌دهد. در این برنامه می‌توانید ستون‌بندی پرشی (Tab Stop) را در صفحه نمایش ملاحظه کنید.

```
1. //This program prints 'X' every 8 columns
2. //by means of conditional operator.
3. #include <iostream>
4. using namespace std;
5. int main()
6. {
7.     for(int j=0; j<80; j++)
8.     {
9.         char ch=(j%8) ? ' ' : 'X';
10.        cout<<ch;
11.    }
12.    return 0;
13. }
```

توضیح مثال: در این برنامه هرگاه باقی‌مانده j بر ۸ برابر صفر گردد کاراکتر **X** و در غیر این صورت فضای خالی در صفحه نمایش چاپ می‌شود.

کاردرکلاس ۵-۴:

مثال ۵-۱۴ را به وسیله دستور **switch** بازنویسی کنید. توجه داشته باشید که در این مورد مقادیر **case** های مختلف کد اسکی هستند.

مثال ۵-۱۹: برنامه‌ای که یک عملگر و دو عملوند صحیح را از ورودی گرفته و عملگر را بر روی عملوندهای خود اجرا می‌کند.

```

1. //This program uses SWITCH with char variable.
2. #include <iostream>
3. #include <conio.h>
4. using namespace std;
5. int main()
6. {
7.     int num1, num2;
8.     bool control=true;
9.     char opt;
10.    while(control)
11.    {
12.        system("cls");
13.        cout<<"Enter num1, operator , num2:";
14.        cin>>num1>>opt>>num2;
15.        switch(opt)
16.        {
17.            case '+' :
18.                cout<<"\n sum = "<<num1+num2;
19.                break;
20.            case '-' :
21.                cout<<"\n minus = "<<num1-num2;
22.                break;
23.            case '/' :
24.            case '\\':
25.                cout<<"\n division = "<<(float)num1/num2;
26.                break;
27.            case '*' :
28.                cout<<"\n multiply = "<<num1*num2;
29.                break;
30.            default:
31.                cout<<"\nOperator is illegal.Press a key to end.";
32.                control=0;//It's similar to write control=false;
33.        }//end of switch
34.        getch();
35.    }//end of while
36.    return 0;
37. }
```

توضیح مثال: اولین نکته جدید این مثال استفاده از یک متغیر از نوع `bool` به عنوان نگهدارنده حلقه تکرار است. زمانی که کاربر کاراکتر اشتباهی را به عنوان عملگر وارد کند، متغیر کنترلی مقدار `false` به خود گرفته و موجب اتمام کار حلقه می‌شود. همچنین در خطوط ۲۳ و ۲۴ برنامه هر دو کاراکتر `/` و `\` به عنوان عملگر تقسیم با `or` کردن دو `case` متوالی پذیرفته شده است. همچنین به به کارگیری تبدیل نوع استاتیک در خط ۲۵ توجه کنید.

۵-۴: تمرین

۱. خطای قطعه کدهای زیر را بیان کنید.

I.

```
1.  if (x=y)
2.      cout<< ++(x+y);
3.  else;
4.      x--;
```

II.

```
1.  int z = 2, sum;
2.  while (z >= 0)
3.      sum += z;
```

III.

```
1.  switch (value%2)
2.  {
3.      case 0:
4.          cout<<"Even integer."<<endl;
5.      case 1:
6.          cout<<"Odd integer."<<endl;
7.  }
```

۲. کدام یک از عبارات زیر درست و کدام یک نادرست است.

الف- وجود حالت default در دستور switch الزامی است.

ب- وجود دستور break در دستور default الزامی است.

ج- دستور default حتماً باید به عنوان آخرین قسمت دستور switch قرار گیرد.

د- اگر متغیرهایی را داخل بلاک تعریف کنیم در آن صورت خارج از بلاک قابل دسترسی نیستند.

۳. برنامه‌ای به زبان C++ بنویسید که یک مربع و یک لوزی توخالی را به واسطه دستورات gotoxy و for در صفحه

رسم کند. سپس برنامه را به گونه‌ای گسترش دهید که طول ضلع شکل را هم دریافت کند.

۴. برنامه‌ای بنویسید که دو عدد تصادفی تولید کند و سپس براساس آن دو عدد یک علامت مثل ☺ را در صفحه

جابه‌جا کند و به محلی که آن دو عدد تصادفی اعلام می‌کند برود. دو عدد تصادفی باید در محدوده مجاز صفحه باشند.

۵. برنامه‌ای بنویسید که یک عدد مبنای ۱۰ را دریافت کند. سپس عدد دریافتی را به هر مبنای مورد نظری برود.

۶. برنامه‌ای بنویسید که با دریافت یک عدد در مبنایی غیر از ۱۰، آن را به مبنای ۱۰ برود و نتیجه را چاپ کند.

۷. برنامه‌ای به زبان C++ بنویسید که مقدار عدد π را از رابطه زیر محاسبه و چاپ کند.

$$\pi = 4 - \frac{4}{3} + \frac{4}{5} - \frac{4}{7} + \frac{4}{9} - \frac{4}{11} + \dots$$

۸. خروجی برنامه زیر را مشخص کنید.

```
1. #include <iostream>
2. using namespace std;
3. int main()
4. {
5.     int count =1;
6.     while(count <= 10)
7.     {
8.         cout<<(count % 2 ? "****" : "++++++")
9.         <<endl;
10.        count++;
11.    }
12.    return 0;
13. }
```

۹. برنامه‌ای به زبان C++ بنویسید که مقدار e^x را با استفاده از فرمول‌های زیر تا ۹ رقم اعشار حساب کند.

$$e = 1 + \frac{1}{1!} + \frac{1}{2!} + \frac{1}{3!} + \dots$$

$$e^x = 1 + \frac{x}{1!} + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots$$

۱۰. با استفاده از قوانین دموگن معادل هریک از عبارات منطقی زیر را بیان کنید.

الف - $!(x<5) \ \&\& \ (y==32)$

ب - $!(a==b) \ || \ !(h!=8)$

ج - $!(v<=43) \ \&\& \ f>3$

د - $!(i>3 \ || \ a<=3)$

۱۱. یکی از اشکالات دستور **break** و دستور **continue** آن است که این دستورها، ساخت یافته نیستند. در عمل،

دستورهای **break** و **continue** را می‌توان به صورت دستورهای ساخت یافته شبیه‌سازی کرد که می‌تواند با

اشکالاتی نیز همراه باشد. در حالت کلی توضیح دهید چگونه می‌توانید هر دستور **break** یا **continue** در یک

برنامه را از حلقه تکرار حذف کنید در عین حالی که روند اجرایی حلقه با مشکلی مواجه نشود.

۱۲. شرکتی می‌خواهد انتقال داده‌ها را از طریق خطوط تلفن انجام دهد اما این کار تنها می‌تواند از طریق داده‌های

چهار رقمی انجام شود. برنامه‌ای بنویسید که داده‌های شرکت را به رمز در آورد و آنها را به شکل مطمئن‌تری

انتقال دهد. برنامه باید یک عدد صحیح چهار رقمی را بخواند و آن را به این صورت رمز کند که به جای هر رقم،

باقیمانده تقسیم بر ۱۰ آن عدد را قرار دهد، سپس جای رقم اول را با رقم سوم و رقم دوم را با رقم چهارم جابه‌جا

کند. و نتیجه را در خروجی چاپ کند. همچنین برنامه را به اینگونه گسترش دهید که قادر باشد یک عدد رمز شده را رمزگشایی کند.

۱۳. برنامه‌ای به زبان C++ بنویسید که ب.م.م و ک.م.م دو عدد دریافتی را حساب کند.

۱۴. برنامه‌ای بنویسید که حرف B را به صورت زیر در خروجی بنویسید.

```
* * * * *
*           *
*           *
* * * * *
*           *
*           *
* * * * *
```

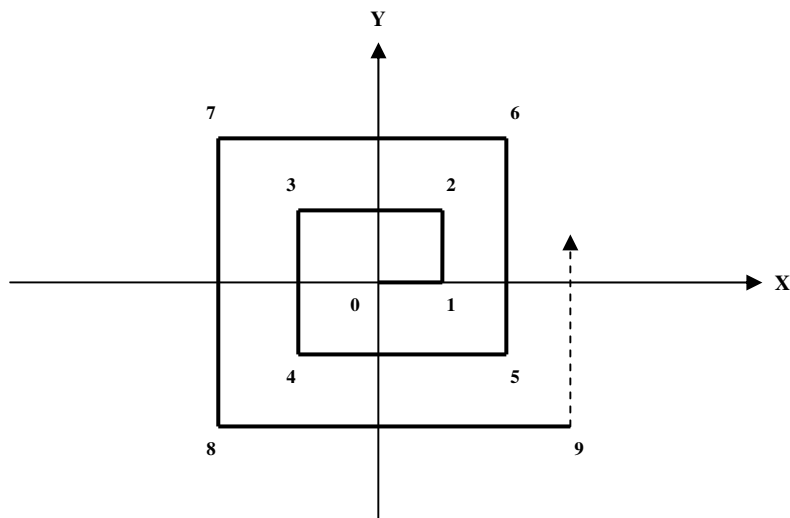
۱۵. برنامه‌ای به زبان C++ و با استفاده از دستور sizeof بنویسید که مشخص کند یک سیستم کامپیوتری برای هر یک از ۱۲ نوع پایه‌ای موجود در C++ چقدر فضا برحسب بیت استفاده می‌کند.

۱۶. برنامه‌ای بنویسید که با دریافت عدد صحیح مثبت n مثلث متساوی الساقینی با قاعده $2n-1$ ستاره و ساق‌هایی در n خط در خروجی چاپ کند. بهتر است از عملگر سه تایی در حل این مسئله استفاده کنید.

۱۷. برنامه‌ای بنویسید که شماره دانشجویی و معدل تعداد n دانشجو را از ورودی بخواند، و دانشجویی را که دومین معدل را از نظر بزرگی دارد، پیدا کند و به خروجی ببرد. (برای حل این مسئله نباید از آرایه‌ها استفاده کنید).

۱۸. برنامه‌ای بنویسید که اعداد چهار رقمی را بیابد که مضربی از مقلوب خود باشند.

۱۹. برنامه‌ای بنویسید که عدد n را بگیرد و با توجه به شکل زیر موقعیت نقطه n -ام را به صورت یک دوتایی مرتب شامل x و y چاپ نماید.



۲۰. برنامه‌ای بنویسید که عدد را بخواند و بگوید آن عدد کامل هست یا نه. عددی کامل است که مجموع مقسوم‌علیه‌های آن به جزء خودش برابر با آن عدد باشد.
۲۱. برنامه‌ای بنویسید که رنگ پس زمینه و رنگ متن را از کاربر بپرسد و آن‌گاه به‌واسطهٔ دستور `system` و با استفاده از دستورات `DOS` رنگ پس‌زمینه و متن صفحهٔ کنسول را تغییر دهد.
۲۲. برنامه‌ای بنویسید که یک عدد اعشاری را خوانده و وارون (مقلوب) آن را به خروجی ببرد.
۲۳. برنامه‌ای بنویسید که یک عدد اعشاری را خوانده و قسمت‌های صحیح و اعشاری آن را به‌صورت دو عدد صحیح مجزا به خروجی ببرد.
۲۴. برنامه‌ای بنویسید که n عدد اعشاری با ۴ رقم اعشار به‌صورت تصادفی تولید کند، سپس هر عدد را با توجه به آنچه در خصوص گرد کردن اعداد در قسمت ۲-۱۰ فراگرفتید تا سه رقم اعشار گرد کند.
۲۵. برنامه‌ای بنویسید که یک سکهٔ ۱۰۰ ریالی را با سکه‌های ۲، ۵، ۱۰، ۲۰ و ۵۰ ریالی خرد کند. برنامه باید تمامی حالت‌های ممکن را چاپ کند.
۲۶. برنامه‌ای بنویسید که توزیع اعداد اول را به‌صورت یک نمودار گرافیکی نشان دهد. در این نمودار هر مکان در صفحه نمایش ۸۰ ستون و ۲۵ سطری از ۰ تا ۹۹۹ به‌طور فرضی شماره‌گذاری شده است. اگر شمارهٔ یک مکان خاص از صفحه نمایش، اول باشد آن مکان با رنگ سفید، باید رنگ آمیزی شود، و در صورت اول نبودن باید به رنگ خاکستری در آید. جهت رنگ‌آمیزی به رنگ سفید از کد اسکی ۲۱۹ و جهت رنگ‌آمیزی به رنگ خاکستری از کد اسکی ۱۷۶ استفاده کنید.
۲۷. یکی از جالب‌ترین خصوصیات دنبالهٔ فیبوناچی رابطهٔ آنها با عدد طلایی است، به‌طوری که هرچه اعداد فیبوناچی را تولید کنیم و نسبت دو جملهٔ آخر دنباله را به‌دست آوریم (نسبت عدد کوچکتر به عدد بزرگتر) تقریب بهتری از نسبت طلایی به‌دست خواهیم آورد. برنامه‌ای بنویسید که عدد طلایی را با بهترین تقریب ممکن به‌دست آورد.
۲۸. در برنامهٔ مثال ۵-۸ ممکن است یک اشکال منطقی در حین اجرای برنامه پیش آید. ضمن کشف این اشکال، برنامه را دوباره به‌صورت صحیح بازنویسی کنید.
۲۹. برنامه‌ای بنویسید که عملیات چهارگانهٔ ریاضی را بر روی دو کسر پیاده‌سازی کند.
۳۰. مردی اصفهانی سرمایه‌ای معادل ۱۰ میلیون تومان دارد. او سرمایهٔ خود را در یک بانک خصوصی با بهرهٔ مرکب ۱۵٪ در یک حساب پس‌انداز بلند مدت قرار داده. او می‌خواهد مقدار سرمایهٔ خود را به‌صورت سالانه و تا ۱۰ سال محاسبه کند. فرض کنید وی هیچگونه سودی تا پایان ده سال از بانک دریافت نکند و سود پول او توسط بانک در یک حساب کوتاه مدت دیگر ریخته شود، برای حساب کوتاه مدت سود سپرده یک سوم سود حساب پس‌انداز بلند مدت محاسبه می‌گردد. سرمایهٔ وی در پایان هر سال برابر مجموع پول‌های موجود در این دو حساب است. برنامه‌ای بنویسید که سود هر سال را در یک خط خروجی برای ۱۰ سال چاپ کند.

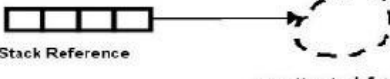
۵-۵: موارد مطالعاتی

۱. (پروژه برنامه‌نویسی): با توجه به آنچه در مسئله ۲۶ بخش ۴-۴ دیدید بازی چرخ گردان را با تولید اعداد تصادفی شبیه‌سازی کنید.
۲. (پروژه برنامه‌نویسی): برنامه‌ای بنویسید که بازی دوز را اجرا کند. بازی دوز به این صورت است که از یک جدول 3×3 تشکیل می‌شود. و دو بازی‌کن دارد. به قید قرعه یک بازی‌کن شروع‌کننده می‌شود. بازی به این صورت است که هر بازی‌کن ۳ مهره جداگانه دارد و به ترتیب یکی پس از دیگری مهره‌های خود را وارد جدول می‌کنند. پس از آنکه هر دو بازی‌کن تمامی مهره‌های خود را در خانه‌های جدول قرار دادند و تنها سه خانه آزاد ماند از آن پس تنها مجاز به جابه‌جا کردن مهره‌ها در بین خانه‌های آزاد جدول هستند. هر کس که زودتر مهره‌های خود را در یک خط راست (چه به صورت قطری و چه به صورت ردیفی یا ستونی) قرار داد، برنده محسوب می‌شود و بازی خاتمه می‌یابد. برنامه شما باید با ریختن دو تاس آغاز شود و هر بازی‌کن که تاس بالاتری بیاورد آغازکننده خواهد بود. برنامه در هر بار باید از بازی‌کنی که مجاز به انجام بازی است بپرسد که کدام مهره‌اش را می‌خواهد جابه‌جا کند. همچنین باید محل جدید مهره را از بین محل‌های ممکن بپرسد. در صورت درست بودن باید حرکت را انجام دهد. هر موقع که یک بازی‌کن بازی را برد باید پیام مناسب را چاپ کند و بازی را خاتمه دهد. برنامه باید موقعیت مهره‌ها را در جدولی 3×3 در مانیتور نمایش دهد. بهتر است هر بار که نوبت بازی بین بازی‌کن‌ها عوض می‌شود رنگ پس زمینه نیز تغییر یابد. مثلاً رنگ سبز برای بازی‌کن شماره یک و رنگ آبی برای بازی‌کن شماره دو. همچنین هنگام ورود داده، نباید جزء ارقام مجاز چیز دیگری در صفحه چاپ گردد.
۳. (پروژه برنامه‌نویسی): سعی کنید پروژه قبلی را به گونه‌ای گسترش دهید که یکی از بازی‌کن‌ها کامپیوتر باشد.
۴. دقت اعداد اعشاری float تا ۷ رقم اعشار است. بنابراین کسر عدد $9/90$ از $10/100$ می‌تواند نتیجه متفاوتی با $0/10$ داشته باشد. برای مثال ممکن است عدد $0/9999999$ در خروجی چاپ شود. لذا در محاسبات پولی و مالی که دقت اعداد در آنها اهمیت دارد بهتر است از نوع float استفاده نکنید. با مراجعه به شبکه جهانی اینترنت یا مستندات MSDN تحقیق کنید که در کتابخانه C++ چه امکاناتی برای محاسبات پولی و مالی فراهم شده است. مثلاً به دنبال نوع decimal بگردید.
۵. از مدرس خود بخواهید منابعی را در خصوص نحوه تولید اعداد تصادفی در کامپیوتر به شما معرفی کند. با مراجعه به آن منابع مقاله‌ای در خصوص این موضوع تهیه و به کلاس ارائه دهید. سعی کنید شما نیز روش جدیدی را برای تولید اعداد تصادفی ابداع کنید.

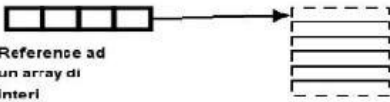
i ; // Alloca 4 bytes per contenere un intero

i 

Stack s ; // Alloca 4 bytes per tracciare un oggetto stack

s 
Stack Reference oggetto stack futuro

numbers[] ; // Alloca 4 bytes per tracciare un array di interi

numbers 
Reference ad un array di interi Futuro array di interi

فصل ششم

نقش آرایه‌ها و متغیرهای لیستی در الگوریتم

اهداف فصل و چکیده مطالب :

۱. آرایه چیست و چه کاربردی دارد؟
۲. متغیر لیستی (زیرنویس دار)
۳. آشنایی با جستجوی ترتیبی و جستجوی دودویی
۴. آشنایی با مرتب‌سازی حبابی و مرتب‌سازی درجی
۵. عملیات اشتراک و اجتماع و ادغام بر روی مجموعه‌ها
۶. کاربرد متغیرهای وضعیت (سوئیچ)
۷. آرایه‌های دو بعدی و ماتریس‌ها
۸. کاربرد حلقه‌های تودرتو در پردازش ماتریس‌ها
۹. جدول خوانی با روش‌های تطابق و نصف کردن

۶-۱: آرایه و کاربرد آن

مقدمه

فرض کنید برای هر یک از داوطلبان کنکور امسال پس از تصحیح پاسخ‌نامه‌ها و منظورکردن تمامی ضرایب، نمره‌ای بین ۱ تا ۱۰۰۰ به‌دست آمده است. نمرات ۱.۴۵۳.۲۸۶ نفر داوطلب شرکت‌کننده در آزمون امسال، بر روی یک نوار مغناطیسی به‌صورت یک سری عدد ثبت و ضبط شده است. حال سازمان سنجش کشور می‌خواهد با پردازش این تعداد رکورد، آماری از داوطلبان امسال تهیه کند و در جدولی ۱۰۰۰ ردیفی در پیک سنجش چاپ نماید، که شامل تعداد دانشجویانی باشد که هر یک از این نمرات را کسب کرده‌اند. به‌عنوان مثال در ردیف صدم تعداد افرادی که نمره ۱۰۰ گرفته‌اند مشخص باشد و به همین ترتیب ادامه یابد. شما چه روشی را برای حل این مسئله پیشنهاد می‌کنید؟

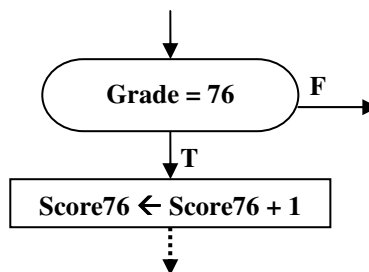
اگر به‌خاطر بیاورید پیشتر در بخش ۲-۶ کارنمایی را در شکل ۲-۲۲ برای شمارش نمرات موجود در چند محدوده خاص پیشنهاد کردیم. شاید در نخستین تلاش‌ها اینگونه به نظر آید که، از مکانیسمی مشابه کارنمای شکل ۲-۲۲ بتوان برای حل این مسئله نیز استفاده کرد. اما آیا واقعاً چنین است؟ اگر بخواهیم از روش مذکور برای حل این مسئله استفاده کنیم با مهمترین مشکلی که برخورد می‌کنیم، چگونگی تعریف هزار متغیر برای ذخیره تعداد هر یک از این هزار نمره است؟! مسلماً هزار حرف الفبا نداریم که چنین کاری را بتوانیم به‌راحتی انجام دهیم، پس باید به فکر ترکیب حروف و ارقام جهت ساخت هزار نام متفاوت باشیم. مثلاً یک نام را بگیریم `ljdas` و دیگری را بگیریم `hGas` و همین‌طور ادامه دهیم تا به هزار نام متفاوت برسیم. اما آیا واقعاً کار کردن با این اسامی جورواجور و متفاوت آسان است؟! از کجا به خاطر داشته باشیم که کدام متغیر مربوط به کدام نمره است؟! پس باید اصلاحاتی در شیوه نامگذاری متغیرها ایجاد کنیم تا بر مشکلات مطرح شده فائق آییم. پیشنهادی که می‌توان به واسطه آن، این مشکل را حل کرد، قرار دادن یک دنباله عددی به دنبال نام هر متغیر است تا ارتباط هر شمارنده را با نمره مربوطه‌اش نشان دهد. مثلاً اگر قرار است متغیر `hGas` مربوط به نمره ۴۶۲ باشد نام این متغیر را به‌صورت زیر تغییر دهیم و بنویسیم `hGas426`. اما اگر توجه کرده باشید، با تغییر کردن دنباله نام متغیرها، دیگر احتیاجی نیست به دنبال ترکیب حروف مختلف برای ساخت نام‌های متفاوت باشیم. چرا که تغییرات دنباله خود تضمین‌کننده متفاوت بودن نام متغیرها است. بنابراین می‌توانیم شروع نام تمامی متغیرها را با یک نام مشابه مثل `Score` در نظر بگیریم. با توجه به نکات مطرح شده می‌توان هزار نام متفاوت را برای شمارنده‌های موردنیاز این مسئله به‌صورت زیر در نظر بگیریم:

`Score1 , Score2 , Score3 , ... , Score999 , Score1000`

شکی نیست که این هزار نام متفاوت مشکل نامگذاری را برای ما هموار می‌سازند اما آیا حالا دیگر می‌توانیم مسئله را به‌راحتی حل کنیم؟ فقط به نخستین جعبه کارنمای شکل ۲-۲۲ نگاه کنید اگر بخواهیم معادل این جعبه را برای کارنمای این مسئله رسم کنیم باید ۱۰۰۰ دستور انتساب قرار دهیم که هر یک از این متغیرها را با صفر مقدار اولیه بدهد. پس از خواندن هر نمره باید هزار جعبه تصمیم تشکیل دهیم تا با خواندن هر نمره، شمارنده مربوط به آن نمره را یک واحد افزایش دهیم. حال به نظر شما چگونه می‌توان این الگوریتم را با این گستردگی پیاده‌سازی کرد؟ حتی نوشتن برنامه ++C آن نیز به ۱۰۰۰ دستور `else if` نیاز دارد! بنابراین باید راه دیگری برای این مسئله پیدا کنیم. از طرف دیگر این

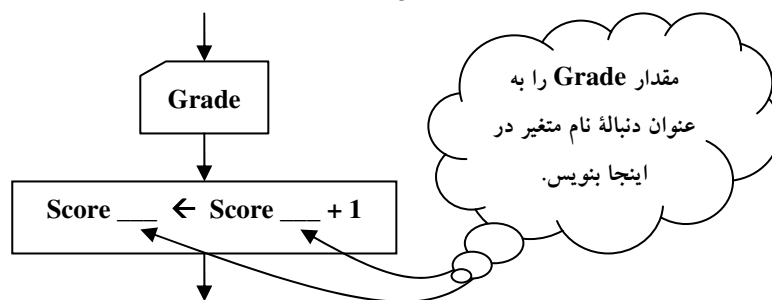
روش الگوریتم‌نویسی یک مشکل اساسی دیگر هم دارد، به‌طوری که اگر بازه نمرات از ۱ تا ۱۰۰۰ به ۱ تا ۵۰۰۰ تغییر کند باید کل الگوریتم را تغییر دهیم تا ۴۰۰۰ نمره جدید را پوشش دهد و اگر بازه نمرات به ۱ تا ۵۰۰ تقلیل یابد، دوباره نیز باید الگوریتم را تغییر دهیم تا ۵۰۰ جعبه اضافی حذف گردد. پس پاسخ این الگوریتم با کوچکترین تغییری در بازه نمرات ناکارآمد می‌شود. بنابراین بهتر است روشی پیدا کنیم که الگوریتم نه تنها تعداد نمرات را از کاربر دریافت کند، بلکه بازه نمرات را هم از وی دریافت کند و بر اساس آن بازه روند الگوریتم پیش‌رفته، و نتایج حاصل آید.

برای حل این مشکلات بیایید یکبار دیگر ارتباط بین داده‌های ورودی و شمارنده‌ها را مد نظر قرار دهیم. الگوریتم ما بر این مبنا استوار است که اگر بر فرض عدد ۷۶ خوانده شود یعنی متغیر مربوط به داده ورودی (Grade) حاوی عدد ۷۶ باشد به متغیر Score76 به‌عنوان شمارنده تعداد افراد دارای نمره ۷۶ یک واحد افزوده شود. قسمتی از الگوریتم که این کار را انجام می‌دهد در شکل ۱-۶ آمده است.



شکل ۱-۶: نمایی از مراحل الگوریتم شمارش نمرات کنکور

اما نکته بسیار مهمی که می‌خواهیم شما را در شکل ۱-۶ معطوف به آن سازیم، برابر بودن عدد ۷۶ خوانده شده، با دنباله شمارنده این عدد، است. با توجه به نکته اخیر می‌توان کارنمای شکل ۱-۶ را به زبان ساده‌تری توصیف کرد. شکل ۱-۶ به ما می‌گوید اگر عدد ورودی برابر ۷۶ می‌باشد به متغیری که نام آن Score و دنباله نام آن ۷۶ است یک واحد اضافه کن. احتمالاً اگر بتوانیم این توصیف را به شکلی پیاده‌سازی کنیم مشکل ما نیز در حل این مسئله رفع خواهد شد. یک شیوه برای پیاده‌سازی این توصیف در شکل ۲-۶ آمده است.



شکل ۲-۶

واقعیت آن است که ایده مطرح شده در شکل ۲-۶ همان چیزی است که می‌توان به کمک آن این مسئله و مسائل مشابه آن را به راحتی حل کرد. اما بیایید روش خود را به شیوه‌ای بهتر و سیستماتیک پیاده‌سازی کنیم. پیشتر در ریاضیات

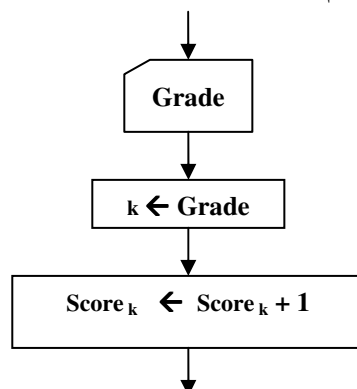
با متغیرهای زیرنویس‌دار آشنا شده‌اید، و چنانکه در ریاضیات دوره دبیرستان نیز دیده‌اید در برخی از فرمول‌های ریاضی مثل $\sum$ از متغیرهایی استفاده می‌کنیم که دارای زیرنویس هستند. به عنوان مثال با فرمول زیر که بیانگر میانگین حسابی چند عدد است از قبل آشنایی دارید:

$$\bar{X} = \frac{1}{n} \times \sum_{i=1}^n X_i$$

در الگوریتم‌ها هم می‌توانیم چنین متغیرهایی را تعریف کنیم. به طوری که یک نام مشترک برای تعدادی متغیر در نظر می‌گیریم و به ترتیب به آنها یک زیرنویس اختصاص می‌دهیم. با تغییر زیرنویس دستیابی ما به متغیر دیگری که دارای آن زیرنویس است، میسر می‌شود. بنابراین می‌توانیم نامگذاری را که در قسمت قبل انجام دادیم به صورت زیر تغییر دهیم:

$Score_1, Score_2, Score_3, \dots, Score_k, \dots, Score_{999}, Score_{1000}$

در برخی از کتب به مجموعه متغیرهایی که به این روش به دست می‌آید متغیرهای لیستی گفته می‌شود، اما در این کتاب به جهت هماهنگی با مباحث زبان C++ بیشتر از لفظ آرایه به جای متغیرهای لیستی استفاده شده است. تا اینجا یک گام دیگر در حل این مسئله به جلو رفتیم. چرا که حالا می‌توانیم به راحتی متغیر Grade که حاوی نمره ورودی است را به عنوان زیرنویس متغیر لیستی خود معرفی کنیم، و با دسترسی مستقیم به شمارنده مربوطه یک واحد بیفزاییم. تکه کارنمایی برای این الگوریتم در شکل ۶-۳ با استفاده از یک متغیر لیستی ارائه شده است:

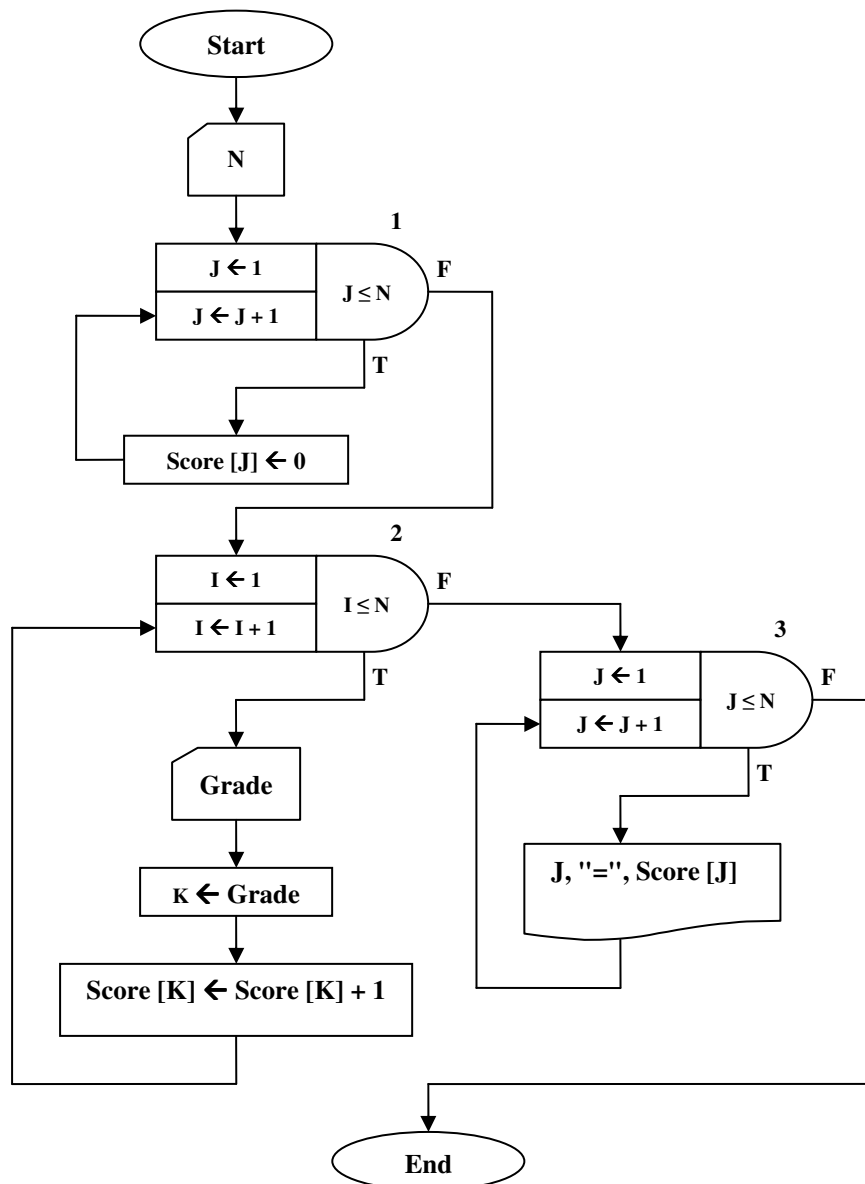


شکل ۶-۳: چگونگی استفاده از متغیرهای لیستی در الگوریتم

اما به دلایلی که پیشتر در قسمت ۲-۸ مطرح کردیم نمادهایی که از حالت خطی خارج می‌شوند برای کامپیوتر قابل خواندن نیستند، بنابراین مجبوریم زیرنویس‌ها را نیز به صورت خطی بنویسیم. لذا به جهت تمایز دادن زیرنویس‌ها از نام متغیر لیستی زیرنویس را بین دو علامت کروشه یعنی [] محصور می‌کنیم. بنابراین از این پس نامگذاری متغیرهای لیستی (آرایه‌ها) را مطابق مثال زیر انجام می‌دهیم.

$Score[1], Score[2], Score[3], \dots, Score[k], \dots, Score[999], Score[1000]$

در شکل ۶-۴ کارنمای نهایی برای این الگوریتم با استفاده از آرایه‌ها ارائه شده است.

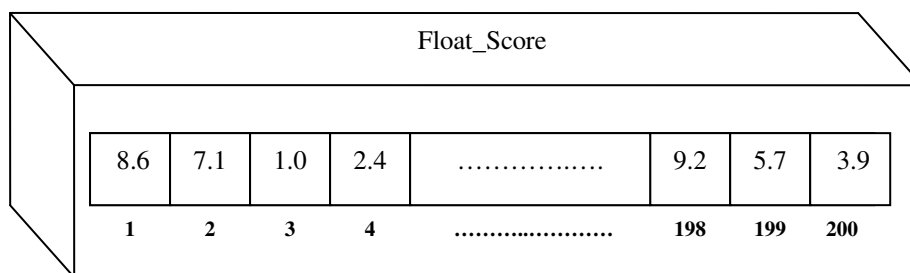
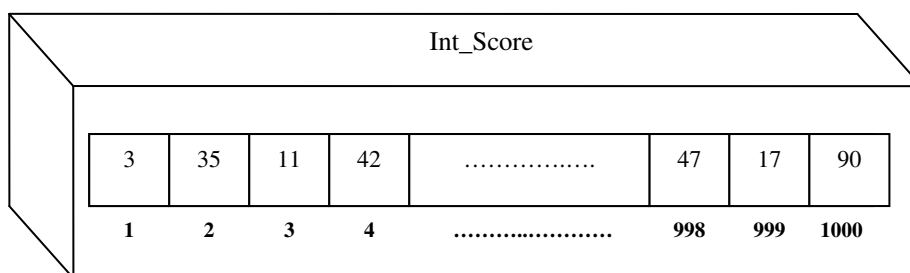


شکل ۶-۴: کارنمای شمارش تعداد نمرات مختلف کنکور

توضیح کارنما: در اولین حلقه تکرار در کارنمای فوق کلیه متغیرهای آرایه را به صفر مقدار اولیه داده‌ایم سپس در حلقه دوم شروع به خواندن تک تک نمرات کرده‌ایم و چنانکه پیشتر گفته شد با خواندن هر نمره شمارنده معادل آن را یک واحد افزایش داده‌ایم. و در حلقه سوم نیز مقادیر متغیرهای لیست را چاپ کرده‌ایم.

در خصوص آرایه‌ها نکات زیر را مد نظر قرار دهید:

۱. اگر بخواهیم یک نماد برای آرایه‌ها در نظر بگیریم اینگونه مطرح می‌سازیم که: آرایه عبارت است از تعدادی خانه از حافظه که به‌صورت متوالی در نظر گرفته می‌شوند. در شکل ۶-۵ نماد آرایه را مشاهده می‌کنید.



شکل ۶-۵: نمادی برای خانه‌های آرایه به همراه زیرنویس آنها

۲. محتوای آرایه‌ها منحصر به مقادیر نوع صحیح نیست، چنانکه در شکل ۶-۵ می‌بینید، می‌توان مقادیری از نوع اعشاری یا رشته‌ای یا کاراکتری را نیز در آنها ذخیره کرد. ولی همواره همه مقادیر مربوط به عناصر یک آرایه از یک نوع هستند. به‌عنوان مثال نمی‌توان در داخل یک آرایه هم مقادیر صحیح و هم مقادیر اعشاری را ذخیره کرد.

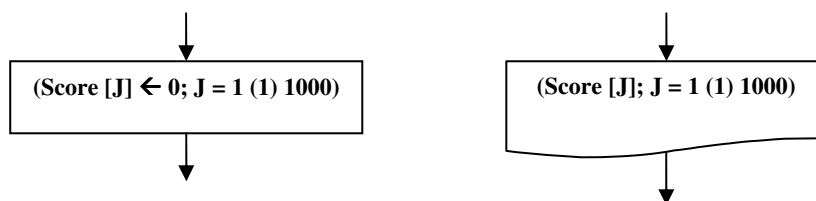
۳. زیرنویس آرایه می‌تواند به‌صورت یک عبارت محاسباتی هم باشد. لذا در کارنما به‌کاربردن زیرنویس‌هایی از قبیل:

$$A[n+3] \quad , \quad B[2*J*(K-1)+7] \quad , \quad C[FLR(J\%k^I)]$$

کاملاً مجاز است.

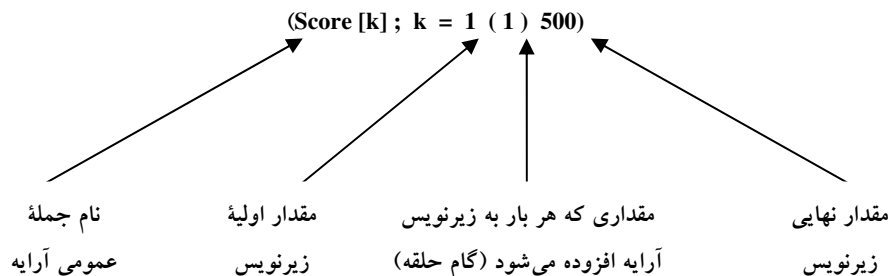
۴. چنانکه در کارنمای شکل ۶-۴ دیدید، یکی از مهمترین مزایای لیست‌ها (آرایه‌ها) پردازش جمعی تمامی عناصر یا بخشی از عناصر لیست در داخل یک حلقه تکرار است. این مزیت ما را قادر می‌سازد که به‌عنوان مثال به‌جای نوشتن تعداد زیادی دستور انتساب، تنها یک دستور انتساب بنویسیم و توسط یک حلقه تکرار آن دستور را برای تمامی عناصر لیست تکرار کنیم. اما اگر قرار باشد برای هر عملیات ترتیبی که می‌خواهیم بر

روی لیست‌ها انجام دهیم مجبور باشیم یک حلقه تکرار در کارنمای خود بگنجانیم، کارنمای ما از صفحات کاغذ بیرون خواهد رفت. بنابراین بهتر است برای عملیات پردازش ترتیبی لیست‌ها نمادهایی را در نظر بگیریم، تا از گنجاندن حلقه‌های تکرار متعدد، معاف گردیم. مقصود ما از پردازش ترتیبی لیست‌ها، هر گونه عملیاتی است که شمارنده حلقه تکرار در حین محاسبات، به عنوان زیرنویس آرایه منظور شود. به عنوان مثال حلقه‌های تکرار اول و سوم در کارنمای شکل ۶-۴ از نوع پردازش ترتیبی می‌باشند ولی حلقه تکرار دوم از نوع پردازش ترتیبی نیست. برای حلقه‌های اول و سوم کارنمای شکل ۶-۴ می‌توان جعبه‌های زیر را پیشنهاد کرد.



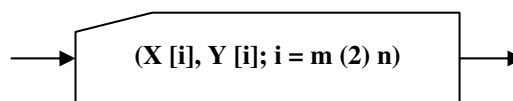
شکل ۶-۴: نمادی برای پردازش ترتیبی لیست‌ها

چنانکه در این شکل‌ها می‌بینید نوع عملیات توسط همان جعبه‌هایی که پیشتر یاد گرفتید بیان می‌شود. سپس در داخل جعبه، عبارتی مطابق نمونه زیر نوشته می‌شود.



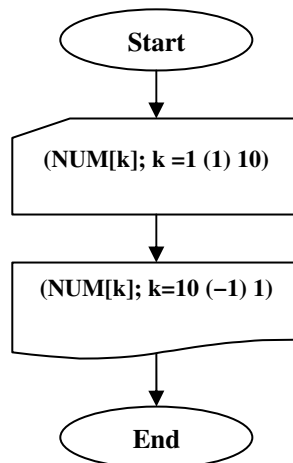
شکل ۶-۷: جمله داخل جعبه‌ها برای پردازش ترتیبی لیست‌ها

در عبارت فوق، در ابتدا نام لیست نوشته می‌شود، سپس یک علامت سمیکالن قرار می‌گیرد، و معنای قسمت دوم عبارت فوق این است که زیرنویس k از ۱ تا ۵۰۰ تغییر می‌کند. میزان تغییر زیرنویس آرایه در هر مرحله برابر مقداری است که در بین دو پراونتز ذکر شده است. بنابراین ممکن است بخواهیم مقدار آرایه را یکی در میان پردازش کنیم، در این صورت مقدار گام را برابر ۲ در نظر می‌گیریم. همچنین امکان پردازش چند لیست در یک جعبه نیز وجود دارد. به عنوان مثال جعبه زیر را ببینید:



مثال ۱-۶: الگوریتمی که ۱۰ عدد را از ورودی خوانده و سپس عناصر آرایه را از آخرین عنصر به اولین

عنصر به خروجی می‌برد.



شکل ۱-۶: کارنمای مثال ۱-۶

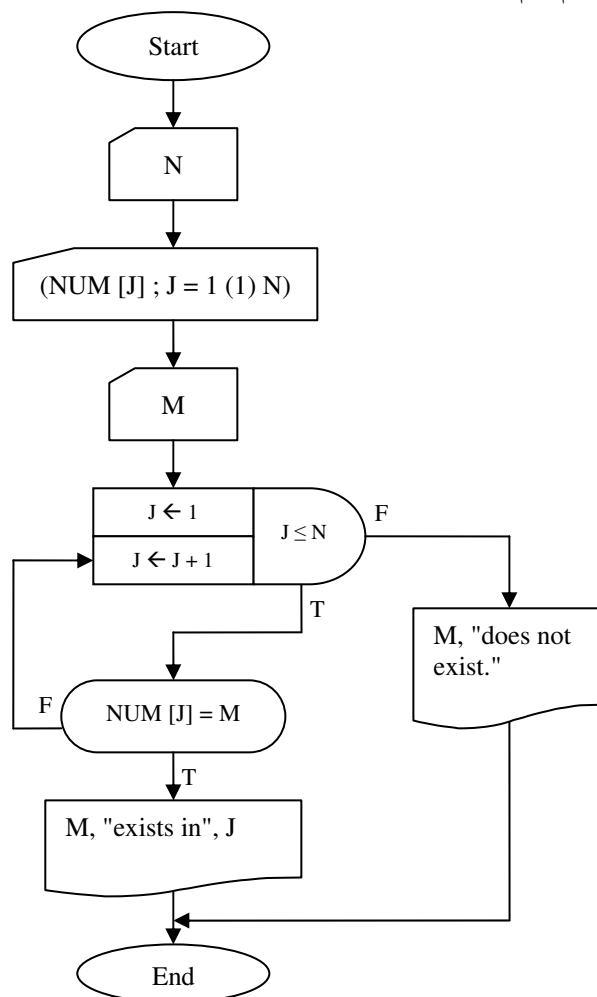
کار در کلاس ۱-۶:

کارنمای شکل ۱-۴، ۱، یکبار دیگر با حذف حلقه‌های تکرار اول و سوم بازنویسی کنید. آیا امکان حذف حلقه تکرار دوم نیز وجود دارد؟

جستجوی ترتیبی در یک لیست

پیشتر در فصل دوم با مقدماتی در خصوص الگوریتم‌های جستجو آشنا شدیم، حال در این قسمت با مبحث جستجوی ترتیبی در آرایه‌ها آشنا می‌شویم. مقصود از جستجوی ترتیبی، پردازش عناصر یک لیست از اولین عنصر تا آخرین عنصر آن برای یافتن مکان یک نمونه خاص در آن لیست می‌باشد. در مثال ۶-۲ الگوریتمی در این خصوص ارائه شده است.

مثال ۶-۲: الگوریتمی که لیستی از اعداد صحیح را دریافت می‌کند و سپس با دریافت یک عدد مشخص می‌کند که این عدد در کجای آرایه قرار دارد. و در صورتی که تا آخر آرایه چک شود و عدد مورد نظر در آرایه یافت نشود، پیغام عدم وجود عدد در لیست چاپ می‌شود.



شکل ۶-۹: کارنمای مثال ۶-۲

۶-۲: تمرین

۱. الگوریتمی که برای پیدا کردن بزرگترین عدد از میان N عدد، در شکل ۲-۲۱ نشان داده شد، احتیاج ندارد که در هیچ لحظه‌ای بیش از دو مقدار از N مقدار داده را در حافظه نگهداری کند. در اکثر موارد لیستی شامل N مقدار داده در حافظه آماده داریم و مسئله، پیدا کردن بزرگترین آنها است. گاهی هم نه فقط می‌خواهیم بزرگترین عدد را به‌دست آوریم بلکه می‌خواهیم تعیین کنیم که بزرگترین عدد در کجای لیست واقع شده است (زیرنویس آن، چه بوده است).

الف- قطعه کارنمایی بسازید که در صورت اجرا، بزرگترین مقدار را در لیست A شامل N عنصر حقیقی پیدا کرده و آن را چاپ کند. فرض کنید که مقادیر A و N در حافظه آماده هستند.

ب- قطعه کارنمایی را که در قسمت (آ) ساخته شده است طوری تغییر دهید که نه فقط بزرگترین مقدار، بلکه «محل» عنصر انتخاب شده، یعنی زیرنویس آن را نیز چاپ کند. چنانچه بزرگترین مقدار بیش از یک بار در داخل لیست آمده باشد زیرنویس اولین مورد یعنی کوچکترین زیرنویس را چاپ کنید.

۲. کارنمای شکل ۶-۱۰ الگوریتمی است که به سه مرحله زیر صورت عمل می‌بخشد.

■ عدد c را به داخل می‌آورد.

■ یک صد عدد $b[1], b[2], \dots, b[100]$ را به داخل می‌آورد.

■ لیست مقادیر $b[i]$ را که در رابطه $b[i] \geq c$ صدق می‌کند تعیین و چاپ می‌کند.

کارنما را به دقت مطالعه کنید و سؤال‌های زیر را پاسخ دهید.

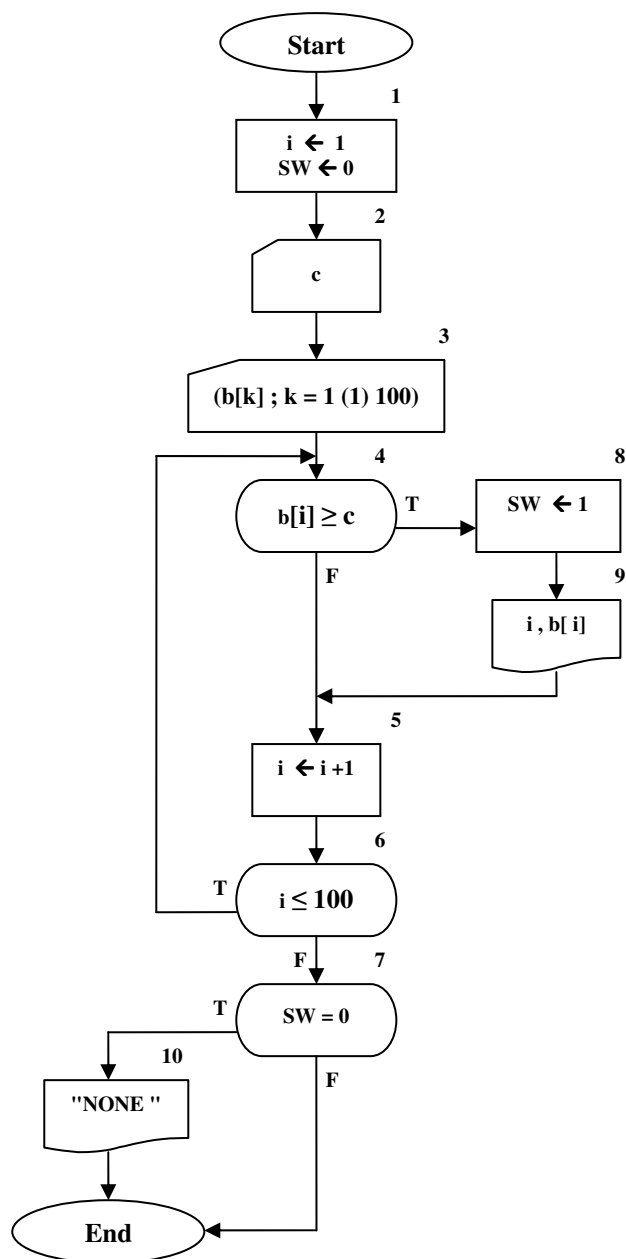
الف- جعبه ۶ چند بار اجرا می‌شود؟

ب- جعبه ۸ چند بار اجرا می‌شود؟

ج- تحت چه شرایطی جعبه ۱۰ اجرا خواهد شد؟ اشاره می‌شود به اینکه SW یک «متغیر وضعیت یا سوئیچ» است و مورد استعمال آن شبیه استفاده سوزن‌بانان از سوزن خطوط قطار، (جهت کنترل حالات مسئله) است. در این خصوص در بخش ۶-۷ بیشتر صحبت خواهیم کرد.

د- آیا واقعاً برای کسب منظور این برنامه نیاز به این هست که در هر لحظه بیش از یکی از مقادیر b در حافظه باشد؟ به عبارت دیگر «آیا وجود آرایه» واقعاً در این الگوریتم لازم است؟ اگر جوابتان منفی است، کارنما را بر آن اساس دوباره رسم کنید و در کنار هریک از جعبه‌هایی که تغییر داده‌اید علامتی برای نشان دادن این امر بگذارید.

و- برای این که به جای خواندن ۱۰۰ عنصر برای b هر تعداد معین n از آنها خوانده شود، کارنمای شکل ۶-۱۰ و یا صورت تغییر یافته آن را که در قسمت (ث) ترسیم کرده‌اید چگونه تغییر خواهید داد؟



شکل ۶-۱۰: کارنمای تمرین ۲

۳. سرمایه‌گذاری که مایل است در امر معامله سهام، یک روش «الگوریتمی» را اتخاذ کند، برای خرید و فروش سهم به قواعد زیر رسیده است.

أ) هرگاه ارزش یک نوع سهم برای دو هفته متوالی صعود کرد، ۱۰۰ سهم از آن را بخر (حتی اگر در آن حال صاحب سهامی از آن نوع باشی).

ب) اگر ارزش سهامی که داری برای دو هفته متوالی نزول کرد، همه دارایی خود را از آن سهم بفروش.

او به منظور امتحان این خط مشی، دسته داده‌ای شامل قیمت ۵۲ هفته یک نوع سهام خاص جمع‌آوری کرده است. تکلیف شما این است که کارنمایی بسازید که با فرض این که همواره پول لازم برای خریدها موجود باشد، سود یا زیان خالص او را در پایان ۵۲ هفته محاسبه کند. سود یا زیان هنگامی حاصل می‌شود که سهام فروخته می‌شود و چون سهام همیشه ۱۰۰ تایی خریده می‌شود، سود حاصل (از هر ۱۰۰ سهم) برابر $100 \times (\text{قیمت خرید} - \text{قیمت فروش})$ خواهد بود. (به این ترتیب، سود منفی، زیان است). اگر سهام در چند نوبت خریداری شود و سپس به فروش برسد آنگاه لازم است سابقه قیمت هر خرید نگهداری شود تا بتوان هنگام فروش، سود یا زیان را به‌طور صحیح حساب کرد. (آیا آرایه مفید خواهد بود؟) اگر در پایان هفته پنجاه و دوم سهامی داشته باشیم همگی در آن هنگام فروخته و در سود و زیان حاصله محسوب می‌شود. احتمالاً انتخاب متغیری، مثلاً **OWN**، برای نشان دادن اینکه آیا در حال حاضر دارای سهمی هستیم یا نه، مفید واقع خواهد بود.

۴. کارنمایی رسم کنید که مقدار n و لیست $a_1, a_2, \dots, a_n$ را به داخل بیاورد. a ها ضرایب چندجمله‌ای

$$a_0 + a_1 X^1 + a_2 X^2 + \dots + a_n X^n$$

و n درجه ظاهری چندجمله‌ای است. البته برخی و یا تمامی ضرایب ممکن است صفر باشند. کارنمایی بسازید که درجه حقیقی (m) چند جمله‌ای را تعیین کند. البته واضح است که $m \leq n$ و m را می‌توان با جستجو در مجموعه ضرایب، برای پیدا کردن عنصر غیر صفر دارای بزرگترین زیرنویس، تعیین کرد. مقدار m و ضرایب $a_0, a_1, \dots, a_m$ را در خروجی بنویسید. اگر همه ضرایب صفر باشند، هیچ ضریبی چاپ نشود و مقدار m برابر -1 چاپ شود.

۵. برای مسئله زیر یک راه حل کارنمایی تهیه کنید. فهرست علائم و اختصارات برای متغیرها فراموش نشود. متغیرها را با تأمل انتخاب کنید و نام‌های با معنی برای آنها به کار ببرید.

در دانشگاه x ، برای درس Y دو امتحان میان دوره‌ای و یک امتحان نهایی وجود دارد. حداکثر تعداد امتیازهای هر امتحان ۱۰۰ و حداقل آن صفر است. نمره نهایی درس برای هر دانشجو میانگین امتیازات امتحان هاست که با منظور کردن ضریب دو برابر برای امتحان نهایی نسبت به امتحان‌های میان دوره‌ای محاسبه می‌شود. لذا نمره درس نیز عددی در محدوده صفر الی ۱۰۰ (شامل این دو) است. هر دانشجو دارای یک شماره دانشجویی

اختصاصی از ۰۰۱ تا ۱۰۰ است. تعدادی داده برای پردازش نمرات آماده شده است. این داده‌ها شامل یکسری (یا جریانی از) اعداد دوازده رقمی مانند آنچه در زیر آمده است، هستند.

0 4 5 0 7 5 0 6 2 0 8 9

سه رقم اول هر عدد دوازده رقمی شماره دانشجویی، سه رقم دوم امتیاز امتحان میان دوره‌ای اول، سه رقم سوم امتیاز امتحان میان دوره‌ای دوم، و بالاخره سه رقم آخر امتیاز امتحان نهایی آن دانشجو است. بعضی دانشجویان موفق به شرکت در هیچ امتحانی نشده‌اند و لذا شماره دانشجویی آنها جزء داده‌ها نیست. اجرای کارنامی شما باید موجب شود که کل داده‌ها خوانده و نمره نهایی، در موارد ممکن، محاسبه و لیست نمرات دانشجویان کلاس چاپ شود. ورودی برحسب شماره دانشجویی مرتب نشده است. لیست نمرات باید به ترتیب شماره دانشجویی و به‌صورتی شبیه آنچه که در زیر نشان داده شده است، باشد.

Student Code Number	Grade
001	96
002	70
003	No scores found
004	85
.	.
.	.
.	.
100	75

شکل ۶-۱۱: نمونه خروجی تمرین ۵

در بالای هر ستون باید عنوانی برای مشخص کردن آن چاپ شود. اگر برای یک شماره دانشجویی نمره‌ای یافت نشود باید در محل مربوط به نمره نهایی پیام «No scores found» چاپ شود. از آنجا که تعداد داده‌ها ممکن است دقیقاً ۱۰۰ تا نباشد، داده نگهبانی شامل صفر در انتهای داده‌ها قرار داده خواهد شد. ابتدا درباره مسئله تأمل کنید تا داده‌ها و خواسته‌های آن کاملاً بر شما معلوم شود. کارنامی شما باید ورودی ۱۲ رقمی را در چهار گروه سه رقمی از هم جدا کند.

گرچه در مباحث آتی به‌طور مفصل در خصوص مرتب‌سازی صحبت خواهیم کرد، اما مسئله حاضر را با استفاده از مرتب کردن حل نکنید چون که راه حل بسیار ساده‌تری برای آن وجود دارد.

۶. با استفاده از یک جعبه WHILE، کارنمایی برای بیان پردازش زیر رسم کنید. لیستی از اعداد مثبتی به صورت $(B[i] ; i = 1 (1) n)$ داده شده است، لیست را از $B[1]$ تا $B[n]$ به ترتیب جستجو و اولین عنصر $B[j]$ را که حداقل دو برابر عنصر بلافاصله قبل از خود یعنی $B[j-1]$ است، پیدا کرده و مقدار آن را چاپ کن. در صورتی که چنین عنصری یافت نشود کلمه «نیست» را چاپ کن.

۷. با استفاده از نوعی جعبه کنترل حلقه، الگوریتم کاملی بنویسید که همه کارهای زیر را انجام بدهد.
الف- مقدار عددی برای لیست VEC به داخل بیاورد. (این مقادیر ورودی از قبل به ترتیب عددی غیر صعودی مرتب شده‌اند).

ب- با شروع از $VEC[1]$ ، لیست را برای پیدا کردن یک جفت عدد تکراری مورد جستجو قرار دهد.
ج- اگر یک جفت عدد تکراری پیدا شد، مقدار تکرار شده و زیر نویس اولین عنصر جفت را چاپ کند و متوقف شود.

د- اگر جفت عدد تکراری پیدا نشد، با چاپ پیام مقتضی متوقف شود.

۸. از آنجا که در کامپیوتر هیچ روش سخت‌افزاری برای محاسبه ریشه n -ام وجود ندارد اصولاً ریشه n -ام عدد A را با استفاده از روش تکراری زیر محاسبه می‌کنند. الگوریتمی بنویسید که مقدار A و n را از ورودی خوانده و ریشه n -ام عدد A را با تقریب 0.001 محاسبه و چاپ کند. X_0 را برابر یک انتخاب کنید.

$$X_{i+1} = \frac{1}{n} \times (n-1) \times X_i + \frac{A}{X_i^{n-1}}$$

۹. الگوریتمی بنویسید که دو عدد مبنای ۲ را با هم جمع کرده و سپس به خروجی ببرد. طول دو عدد مبنای دو ۳۲ رقم است.

۱۰. الگوریتمی برای تفریق دو عدد ۱۰ رقمی در مبنای ۱۰ طراحی کنید. این مسئله را یک بار برای تفریق معمولی و یکبار با استفاده از روش مکمل-۹ حل کنید.

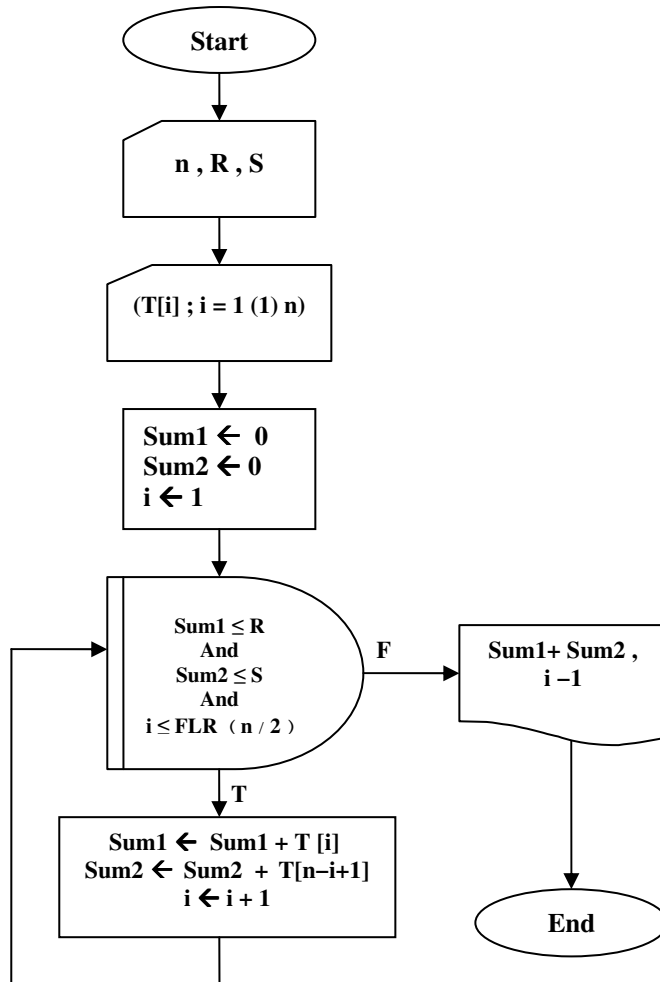
راهنمایی: برای اطلاع از روش مکمل-۹ می‌توانید از استاد خود کمک بگیرید و یا آن که به فصل اول کتاب "طراحی دیجیتال" نوشته موريس مانو مراجعه کنید.

۱۱. الگوریتمی بنویسید که ضرایب و توان‌های دو چند جمله‌ای را به صورت ۴ لیست مجزا خوانده و سپس حاصل جمع و حاصل ضرب آنها را در لیست‌های جدیدی ذخیره کند و به خروجی بفرستد.

۱۲. الگوریتمی بنویسید که روز اول یک سال (از نظر روز هفته) خوانده و سپس تقویم این سال را تولید کند و در خروجی چاپ کند. آیا استفاده از لیست در این الگوریتم ضروری است؟

الگوریتمی بنویسید که در یک لیست از اعداد، عددی را که بیشتر از همه تکرار شده را مشخص کند و به-همراه تعداد دفعات تکرار آن به خروجی ببرد.

۱۳. به زبان ساده فارسی توضیح دهید که با اجرای کارنمای زیر چه عملی انجام می‌شود.



شکل ۶-۱۲: کارنمای تمرین ۱۳

۱۴. آرایه‌ای مرکب از n عدد صحیح داریم، این آرایه را در دو آرایه جدید کوچکتر چنان افراز کنید که اندازه هر یک $n/2$ باشد، به طوری که اختلاف میان حاصل جمع اعداد صحیح موجود در دو آرایه کوچکتر، حداقل باشد. الگوریتم باید هم برای n های زوج و هم برای n های فرد درست عمل کند.

۱۵. مسئله قبل را برای حالتی که اختلاف بین دو آرایه کوچکتر حداکثر باشد نیز حل کنید.

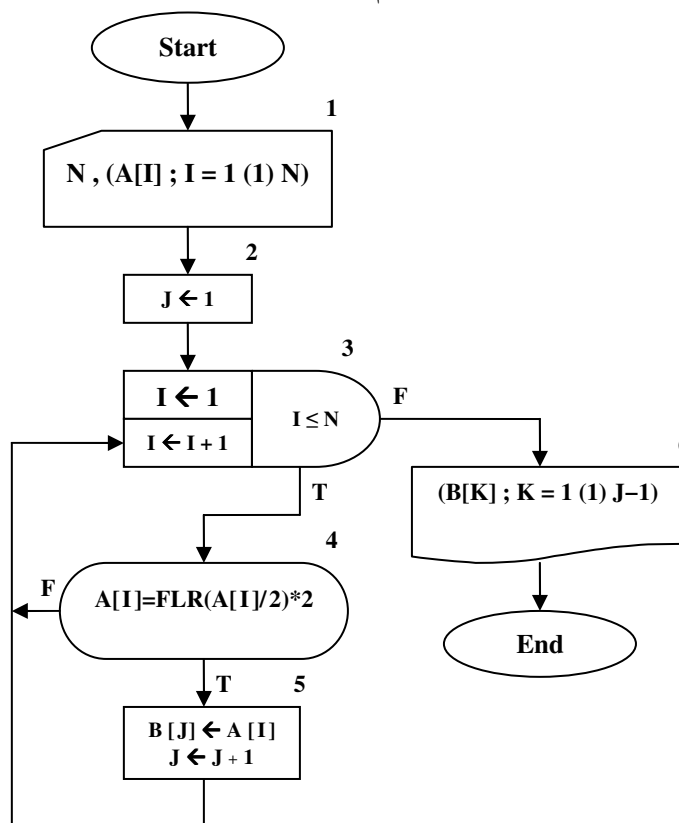
۱۶. در شکل ۶-۱۳ کارنمایی به شما داده شده است.

الف- به زبان ساده بگویید این الگوریتم چه کاری انجام می‌دهد؟ توضیح شما واضح و روشن باشد.

ب- فرض کنید در جعبه ۱ شش مقدار ورودی زیر به لیست A منسوب شده باشد.

۱۲، ۷، ۶، ۶، ۴، ۳

چه مقادیری (در صورت وجود) هنگام اجرای جعبه ۶ چاپ می‌شود؟



شکل ۶-۱۳: کارنمای تمرین ۱۶

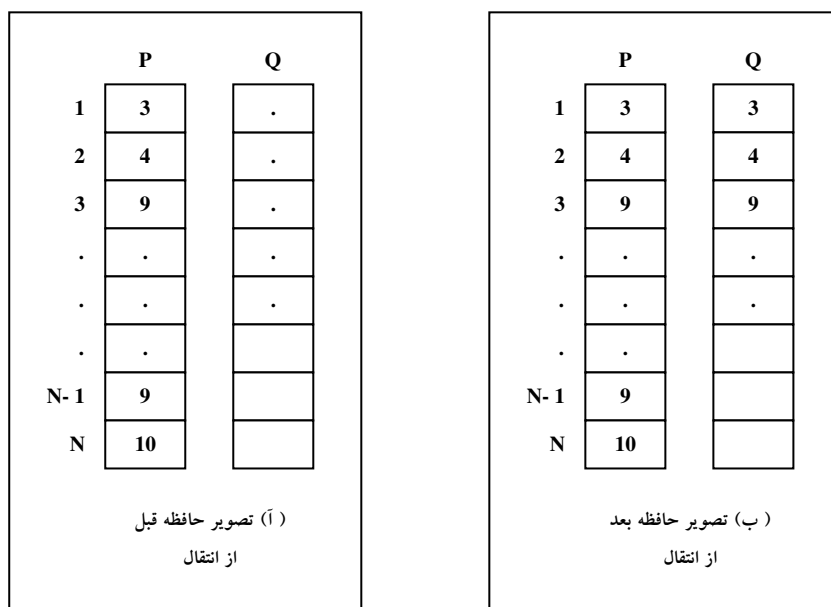
در مسائل ۱۷ تا ۲۳ دو لیست که هر کدام شامل N عدد است در حافظه دارید. یک لیست را P و دیگری را Q نامیده‌ایم. وظیفه شما این است که بیان کلامی هر مسئله را به قطعه کارنمایی که معادل آن باشد تبدیل کنید. جعبه تکرار را در کارنما مفید خواهید یافت. شما ممکن است بخواهید به این شکل عمل کنید که ابتدا قسمت محاسبه حلقه (الگوریتم واحد) را به کارنما تبدیل کنید و سپس آن را به جعبه تکرار مناسبی وصل کنید و بالاخره در صورت لزوم قبل از جعبه تکرار، جعبه تعیین مقادیر اولیه را قرار دهید.

۱۷. مقادیر عناصر i -ام لیست‌های P و Q را به صورت جفت $P[i]$ و $Q[i]$ تصور کنید. مقادیر موجود در هریک از این جفت‌ها را با هم عوض کنید.

۱۸. کارنمایی را که برای مسئله قبل رسم کرده‌اید طوری تغییر دهید که عمل تعویض فقط بر روی جفت‌های با زیرنویس زوج انجام گیرد. آیا زوج یا فرد بودن N تأثیری دارد؟

۱۹. کارنمایی را که برای مسئله ۱۸ رسم کرده‌اید، با این فرض که می‌خواهید از جفت پنجم شروع و مقادیر جفت‌ها را دو در میان با هم عوض کنید، به نحو مقتضی تغییر دهید.

۲۰. نسخه‌ای از تعداد $FLR(N/2)$ عنصر اول لیست P را به لیست Q منتقل کنید. به شکل ۶-۱۴ نگاه کنید.



شکل ۶-۱۴: تصویر انتقال عناصر لیست

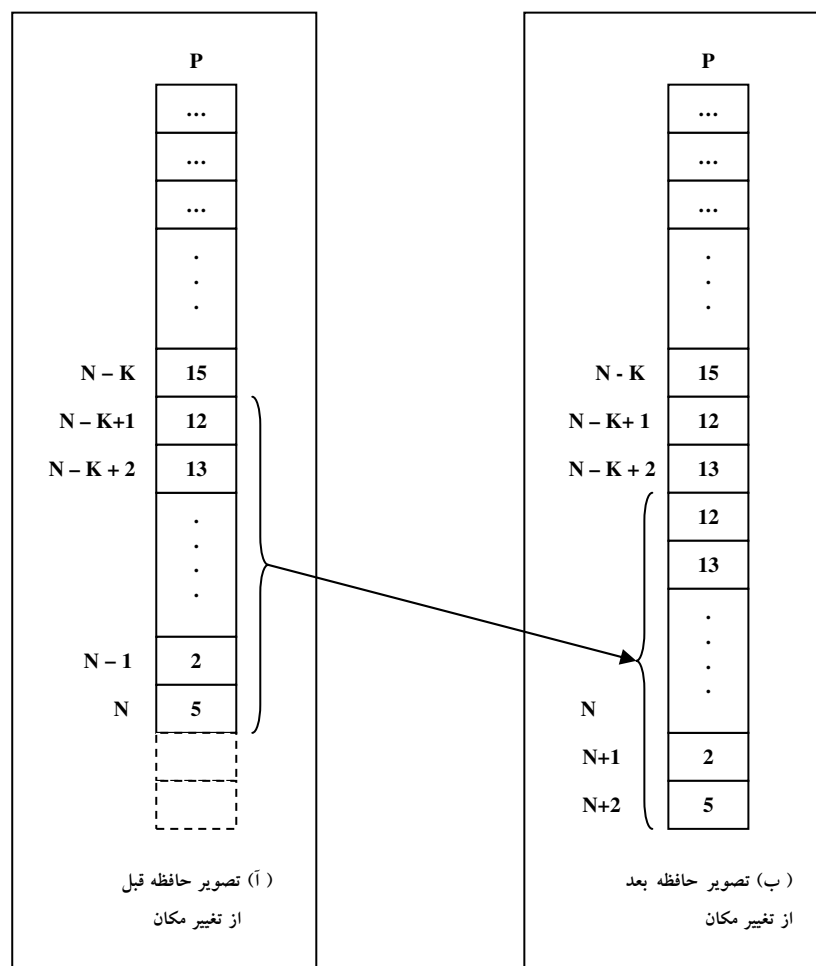
۲۱. تعداد $FLR(N/2)$ عنصر آخر لیست P را به $FLR(N/2)$ محل اول لیست Q منتقل کنید. فرض کنید N زوج است.

توجه: زیرنویس اولین عنصری از P که باید منتقل شود چیست؟

۲۲. مسئله قبل را بدون فرض زوج بودن N یک بار دیگر حل کنید.

۲۳. K عنصر آخر لیست N عنصری P را دو محل به پایین «تغییر مکان» دهید تا جا برای فرار دادن مقادیر جدید در

محل‌های $N-K+1$ و $N-K+2$ باز شود (شکل ۶-۱۵ را ببینید).



شکل ۶-۱۵: تصویر تغییر مکان عناصر لیست

۲۴. الگوریتمی بنویسید که تعدادی شماره حساب، موجودی این شماره حساب‌ها و کد نوع حساب (۱: جاری و ۲: پس انداز) را خوانده و در یک سری لیست قرار دهد. حساب‌های جاری و پس انداز را تفکیک کند، ابتدا حساب‌های جاری و سپس حساب‌های پس انداز را به خروجی ببرد.

۲۵. الگوریتمی بنویسید که اعمال ریاضی ترکیب و انتخاب را برای دو ورودی a و b انجام دهد. بر روی کارآمدترین الگوریتم بحث کنید. مسلماً محاسبهٔ تعدادی فاکتوریل و سپس ساده‌سازی عاقلانه نیست، چرا که ممکن است فاکتوریل‌های به‌دست آمده از محدودهٔ اعداد قابل قبول برای `int` فراتر رود. لذا باید به روشی فاکتوریل‌ها را ساده کنید. احتمالاً آرایه‌ها کمک فراوانی می‌توانند به حل این مسئله بکنند. مراقب باشید که ترکیب و انتخاب رابطهٔ ریاضی با یکدیگر دارند! حال به نظر شما بهتر است کدام یک را با روش الگوریتمی و کدام یک را با روابط ریاضی از روی دیگری به‌دست آوریم؟

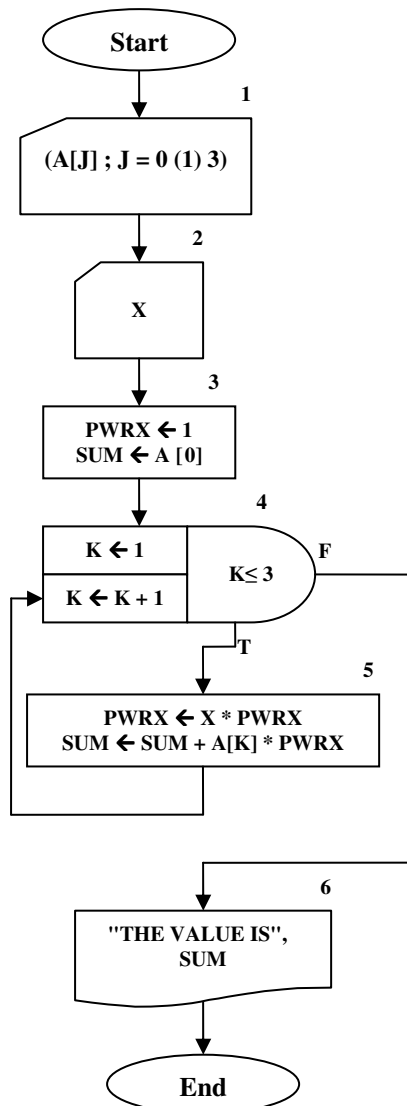
۲۶. دو کارنمای شکل ۶-۱۶ را که برای تعیین مقدار چند جمله‌ای درجه سه داده شده‌اند، مطالعه کنید و سؤالات زیر

را پاسخ دهید:

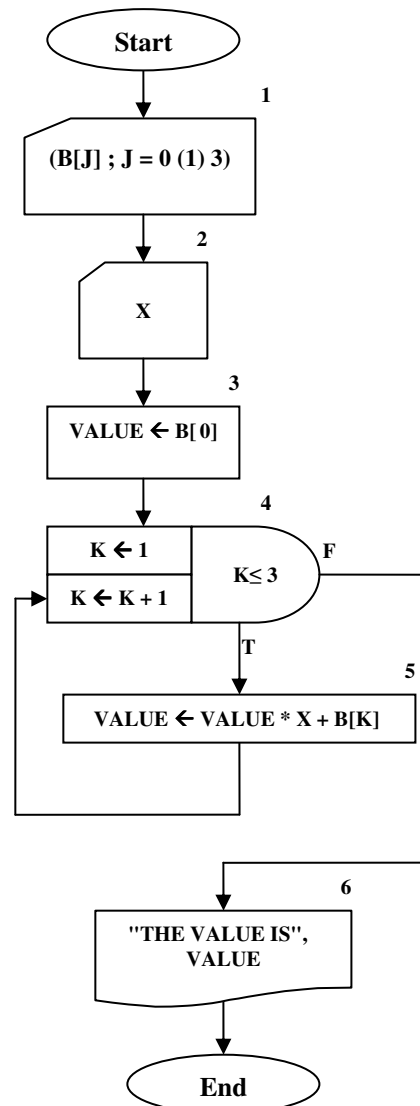
الف- آیا دو کارنمای مزبور از نظر اثر نهایی معادل‌اند؟ شرح دهید.

ب- چه تغییری در هریک از این کارنها لازم است داده شود تا تعیین مقدار چند جمله‌ای‌ها از هر درجه N را میسر شود؟

ج- کدام کارنما کارتر است؟ (یعنی تعداد کمتری محاسبه لازم دارد؟) شرح دهید.



(آ) تعیین مقدار چند جمله‌ای، روش معمولی



(ب) تعیین مقدار چند جمله‌ای، روش ابتکاری

شکل ۶-۱۶: کارنماهای تمرین ۲۶

در هریک از سه تمرین زیر، هر جا که مناسب باشد از جعبه WHILE استفاده و فرض کنید که قبلاً N مقدار به لیست P منسوب شده و در حافظه موجود است.

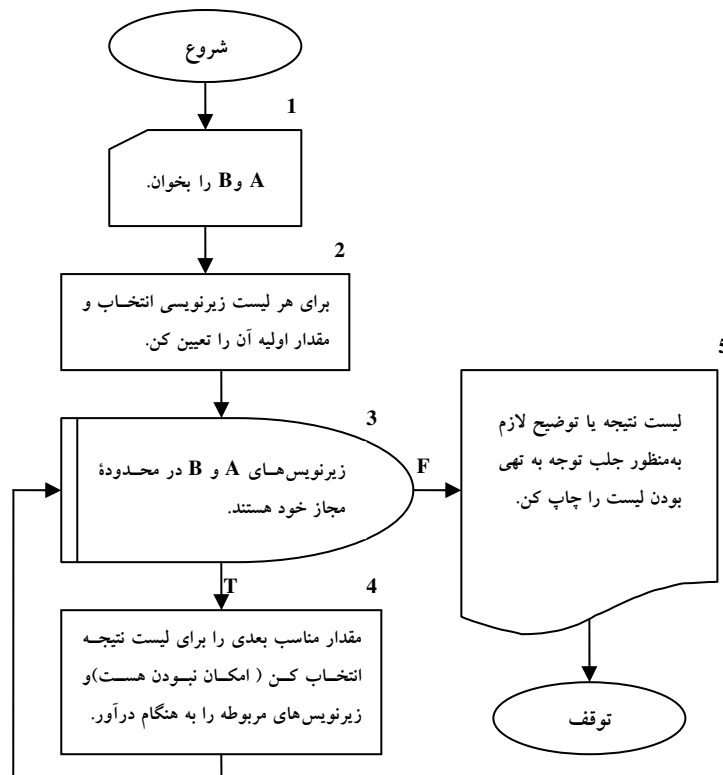
۲۷. لیست را در جهت صعودی زیرنویس‌ها، یعنی $P[1]$ ، $P[2]$ و ... برای پیدا کردن اولین مقداری که قدرمطلق آن بزرگتر از ۵۰ باشد جستجو کرده و در صورت یافتن چنین مقداری آن را به W و عدد ۱ را به ANY منسوب کن. در صورتی که یک چنین مقداری یافت نشد مقدار صفر را به ANY منسوب کن، در هر صورت، پس از این مرحله به نقطه مشترکی از کارنما برو.

۲۸. لیست را به‌طور وارونه، یعنی $P[N]$ ، $P[N-1]$ و ... برای پیدا کردن اولین عنصری که قدرمطلق آن بزرگتر از ۵۰ باشد مورد جستجو قرار بده و در صورت پیدا شدن آن را به W منسوب کن. اگر چنین مقداری یافت نشد، مقدار ۵۰ را به W منسوب کن. در هر دو صورت پس از این مرحله، به نقطه مشترکی از کارنما برو.

۲۹. تمام N عنصر فهرست P را برای پیدا کردن عنصر مخالف صفری که دارای بزرگترین قدرمطلق کوچکتر از $|M|$ باشد مورد جستجو قرار بده. فرض کن که مقدار M از قبل در حافظه ذخیره شده است. مقدار عنصری را که در این جستجو پیدا می‌شود به T منسوب کن. در صورتی که چنین عددی پیدا نشد پیغام «پیدا نمی‌شود» را چاپ کن و متوقف شو.

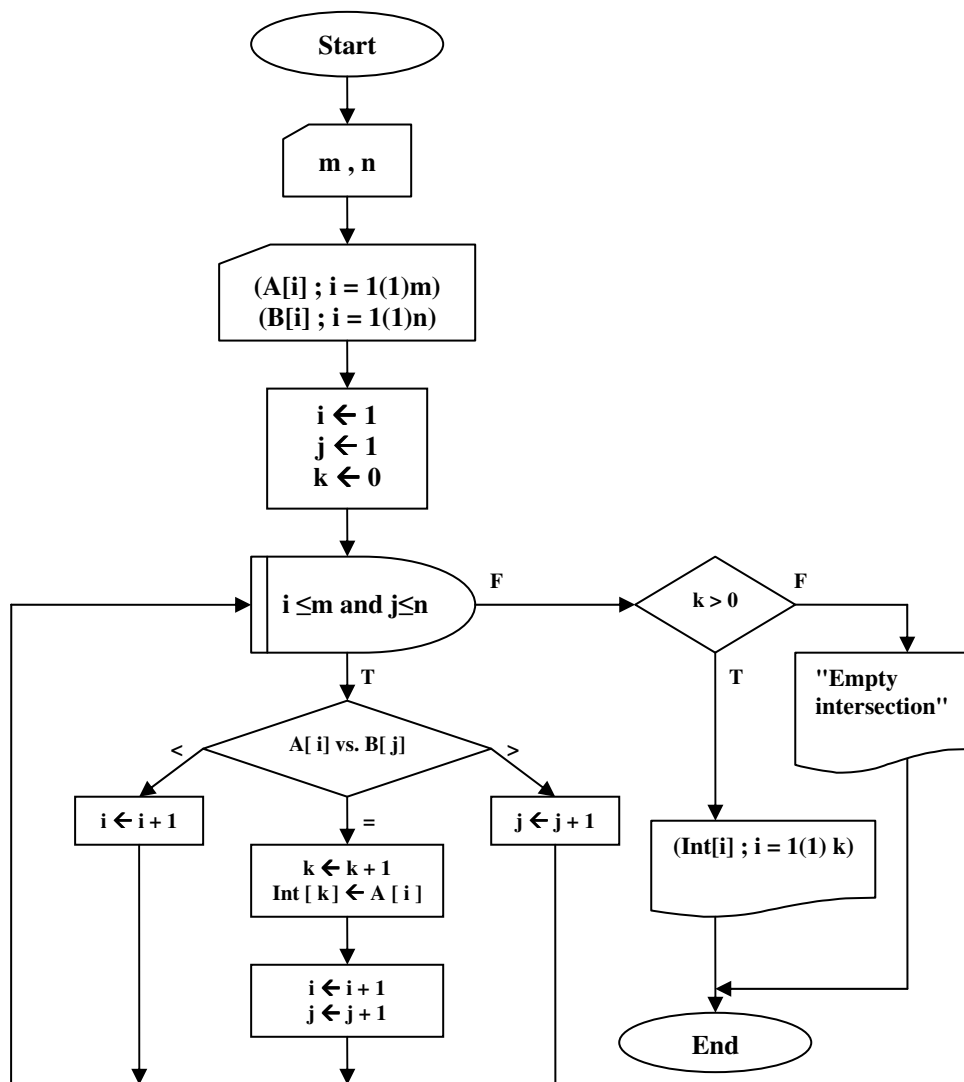
۶-۳: عملیات بر روی مجموعه‌ها

در اینجا توجه شما را به یک دسته از عملیاتی که در دنیای واقعی پردازش داده‌ها (اعم از این که بر روی کامپیوتر باشد یا نه) از اهمیت بنیادی برخوردار است، معطوف می‌نیم. مثال‌ها و مسائلی را در این قسمت خواهید دید که مبین عملیات اشتراک، اجتماع و ادغام دو مجموعه هستند. در تمامی مباحث این قسمت فرض کنید، A یک لیست m عنصری از اعداد و B یک لیست n عنصری از اعداد است. به علاوه مقادیر موجود در لیست‌ها قبلاً از نظر عددی به ترتیب صعودی مرتب شده‌اند. الگوریتم‌های اشتراک، اجتماع و ادغام دارای شباهت‌های ساختی معینی هستند، لذا کارنمای تشریحی زیر روند کلی حل این سه دسته مسئله را نشان می‌دهد. از طرفی به جهت شباهت مذکور ما در اینجا تنها عملیات اشتراک دو لیست را به عنوان نمونه حل می‌کنیم و عملیات اجتماع و ادغام به عنوان تمرین بر عهده خود دانشجویان گذاشته می‌شود.



شکل ۶-۱۷: کارنمای تشریحی عملیات مجموعه‌ای

مثال ۳-۶: اشتراک: دو لیست A و B داده شده است. کارنمای الگوریتمی را بسازید که لیست جدید Int را طوری تشکیل دهد که عناصر آن دربرگیرنده اشتراک مجموعه‌هایی باشد که توسط A و B تعریف شده‌اند. در اینجا فرض می‌کنیم که در هیچیک از دو لیست مقادیر تکراری وجود ندارد. منظور ما از اشتراک این است که Int شامل نسخه‌ای از مقادیری است که در هر دو لیست A و B مشترک هستند. مقادیری که در Int قرار داده می‌شود باید دارای ترتیب عددی صعودی باشد. توجه داشته باشید که حداکثر تعداد اجزایی که می‌تواند در لیست اشتراک Int بیاید محدود است به حداقل m و n که به ترتیب ابعاد A و B هستند.



شکل ۶-۱۸: کارنمای مثال ۳-۶

۶-۴: تمرین

۱. اجتماع: دو لیست مرتب شده A و B داده شده است، کارنمای الگوریتمی را بسازید که لیست جدید U را طوری تشکیل دهد که عناصر آن دربرگیرنده اجتماع مجموعه‌هایی باشد که توسط A و B تعریف شده‌اند. فرض بر این است که در هیچیک از دو لیست مقادیر تکراری وجود ندارد. بدیهی است منظور ما از اجتماع این است که U شامل یک نسخه از هر مقدار متمایزی است که در ضمن پوشش همه عناصر A و همه عناصر B پیدا شود. مقادیری که در U قرار می‌گیرند باید به ترتیب صعودی باشند. توجه کنید که اجتماع U حداکثر شامل $n + m$ عنصر خواهد بود.

۲. ادغام: دو لیست مرتب شده A و B داده شده است. کارنمای الگوریتمی را بسازید که لیست جدید W را مرکب از همه مقادیر A و B به گونه‌ای تشکیل دهد که نتیجه نیز به صورت مرتب شده باشد. به عنوان مثال، اگر

$$A = \{1, 2, 3, 4, 5, 10\}$$

و

$$B = \{2, 6, 10, 12\}$$

آنگاه:

$$W = \{1, 2, 2, 3, 4, 5, 6, 10, 10, 12\}$$

الف- در حل مسئله ابتدا فرض کنید که هریک از دو لیست فاقد مقادیر تکراری است.

ب- حال تصمیم بگیرید که در صورت لزوم، چه تغییراتی باید برای کار در مواردی که مقادیر تکراری در A یا B یا در هر دو موجود باشد، داده شود.

۳. اجتماع و اشتراک به طور همزمان: در مواردی که برای یک جفت لیست A و B ، اجتماع U و اشتراک Int هر دو را بخواهیم، می‌توانیم با ترکیب قسمت‌هایی از الگوریتم‌های قبلی و تشکیل الگوریتم واحدی که طی یک پوشش A و B هر دو لیست U و Int را ایجاد کند، صرفه‌جویی قابل ملاحظه‌ای به عمل آوریم. ابتدا کارنمای تفصیلی این الگوریتم دو منظوره را بسازید. و سپس کارنمای عادی آن را ترسیم کنید.

۵-۶: مرتب‌سازی و جستجو

مرتب‌سازی^۱ لیستی از داده‌ها و جستجو^۲ برای یافتن یک نمونه خاص در یک لیست دو عمل بسیار مهم در علوم کامپیوتر هستند. شاید سخنی گزاف نباشد که بگوییم ۹۰٪ عملیات بسیاری از مسائل کامپیوتری را مرتب‌سازی و جستجو تشکیل می‌دهد. چیزی که در این زمینه بسیار مهم است یافتن کارآمدترین الگوریتم‌ها برای این دو عمل کلیدی در برنامه‌های کامپیوتری است. به عنوان مثال پیشتر با الگوریتم جستجوی ترتیبی که اصولاً در لیست‌های نامرتب استفاده می‌شود آشنا شدید. آیا فکر می‌کنید این الگوریتم برای یافتن یک نمونه خاص در یک لیست، الگوریتمی کارآمد است؟ واقعیت این است که اگر بخواهیم تحلیلی کوتاه بر روی الگوریتم جستجوی ترتیبی انجام دهیم، با فرض اینکه عمل مقایسه انجام شده در جعبه تصمیم در کارنامای شکل ۶-۹ تنها عمل وقتگیر در این الگوریتم باشد، چنین مطرح می‌سازیم که؛ بهترین حالتی که ممکن است برای الگوریتم جستجوی ترتیبی رخ دهد، هنگامی است که نمونه مورد نظر در ابتدای لیست قرار داشته باشد. در این صورت اگر زمان لازم برای یک عمل مقایسه در این الگوریتم برابر α باشد، بهترین زمان ممکن برای انجام این الگوریتم برابر 1α در نظر گرفته می‌شود. از طرفی بدترین حالت زمانی رخ می‌دهد که نمونه مورد نظر یا در لیست نباشد و یا آنکه در انتهای لیست قرار داشته باشد. در هر دوی این حالات عمل مقایسه در الگوریتم جستجوی ترتیبی باید n بار صورت پذیرد تا الگوریتم به حالت توقف برسد، که n طول لیست مورد پردازش می‌باشد. در این صورت زمان لازم برای اجرای این الگوریتم برابر $n.\alpha$ می‌باشد. لذا نتیجه می‌گیریم به‌طور میانگین برای یافتن یک نمونه خاص باید نیمی از لیست را مورد پردازش قرار دهیم. حال تصور کنید مقدار α برابر 0.0001 ثانیه و طول لیست مورد پردازش برابر یک میلیون نمونه باشد. به‌طور متوسط برای یافتن یک نمونه خاص در این لیست باید ۵۰ ثانیه وقت صرف کنیم، چیزی در حدود یک دقیقه! اما اگر در موتورهای جستجوی اینترنتی مثل Google مطلبی را Search کنید، زمان جستجو در بین چند ده میلیارد سایت موجود در اینترنت را چیزی کمتر از حتی یک ثانیه اعلام می‌کند. بنابراین می‌توانید تصور کنید که زمان جستجو تا چه حد در عملیات کامپیوتری می‌تواند مؤثر باشد. چه بسا اگر قرار بود، ما برای جستجوی یک مطلب در اینترنت بیش از ده ثانیه معطل می‌شدیم از جستجوی خود به کلی منصرف می‌گشتیم.

الگوریتم‌های متنوعی برای عملیات جستجو کشف شده و حتی بسیاری از آنها به دلیل رقابت شرکت‌های تجاری در دسترس عموم قرار نگرفته است، اما بسیاری از آنها فراتر از سطح این کتاب است و در دروسی همچون ساختمان‌داده‌ها و طراحی الگوریتم‌ها مطرح می‌شود. لذا در این کتاب تنها به بیان جستجوی دودویی بسنده می‌کنیم، و مطالعه بیشتر در این زمینه را به خوانندگان عزیز محول می‌سازیم. اما نکته بسیار مهمتر آن که بسیاری از روش‌های جستجو از جمله جستجوی دودویی با این فرض بنا شده‌اند که لیست مورد پردازش از قبل مرتب شده است. لذا قبل از آنکه به بحث در خصوص جستجوی دودویی بپردازیم به بیان مطالبی در خصوص مرتب‌سازی می‌پردازیم.

1. Sort
2. Search

قبل از این که هر گونه الگوریتمی را در خصوص مرتب‌سازی و جستجو بیان کنیم از شما می‌خواهیم که کاردر کلاس زیر را تنها با تکیه بر خلاقیت‌های فردی و آموزه‌هایی که تا اینجا فراگرفتید انجام دهید.

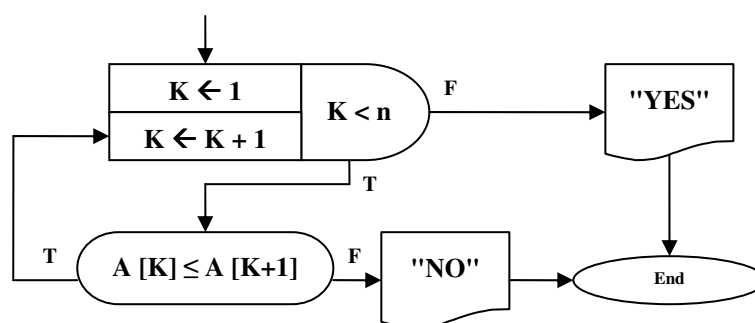
کاردر کلاس ۶-۲:

کارنمایی برای مرتب‌کردن یک آرایه از اعداد حقیقی به طول n رسم کنید.

نخستین تلاش برای ساخت یک الگوریتم مرتب‌سازی

شما بارها و بارها در زندگی خود عمل مرتب‌سازی را انجام داده‌اید. از مرتب‌سازی ورق‌هایی بازی که در یک دست به شما می‌افتند گرفته تا مرتب‌سازی اسکناس‌های داخل کیف پولتان و یا مرتب‌سازی کتاب‌های درسیتان. هر کدام از این مرتب‌سازی‌ها بر اساس یک معیار خاص انجام می‌شود. اما مسلماً هدف و منظور ما از مرتب‌سازی آن مرتب‌سازی نیست که به منظم کردن اتاق یا کتاب‌های درسی و امثال آن اطلاق می‌شود. مرتب‌سازی که مدنظر ما است مرتب‌سازی است که در نتیجه وجود «رابطه هم‌ارزی ترتیب» به‌دست می‌آید. هرگاه بر روی دسته‌ای از اشیاء بتوان رابطه هم‌ارزی $\leq$ را تعریف کرد به آن یک رابطه ترتیب گفته می‌شود. به‌عنوان مثال از آنجا که عمل «کوچکتر یا مساوی» بر روی اعداد حقیقی تعریف شده است می‌توان گفت اعداد حقیقی دارای رابطه ترتیب هستند. اما اعداد مختلط دارای این رابطه نیستند. مسلماً چنین رابطه‌ای در بین اشیاء یک اتاق نیز وجود ندارد و نظم و ترتیب آنها از یکسری قواعد بشری نشأت می‌گیرد. هر کجا که بتوان رابطه ترتیب را تعریف کرد به دنبال آن دو نوع ترتیب صعودی و نزولی نیز بین اشیاء دارای این رابطه مطرح می‌شود. به‌عنوان مثال شما به دفعات لیستی از اعداد را به‌صورت صعودی یا نزولی مرتب کرده‌اید. اما ترتیب صعودی یا نزولی بودن نمونه‌های مذکور هیچ اهمیتی برای ما ندارد، چرا که اگر به‌عنوان مثال بتوان اشیائی را به‌صورت صعودی مرتب کرد با کمی تغییرات در الگوریتم مرتب‌سازی می‌توان خروجی الگوریتم را به‌گونه‌ای تغییر داد که ترتیب لیست به‌صورت نزولی باشد. لذا در اینجا نخستین تلاش خود را برای ساخت کارنمایی که لیستی از اعداد حقیقی را به‌صورت غیرنزولی مرتب‌سازی کند معطوف می‌سازیم.

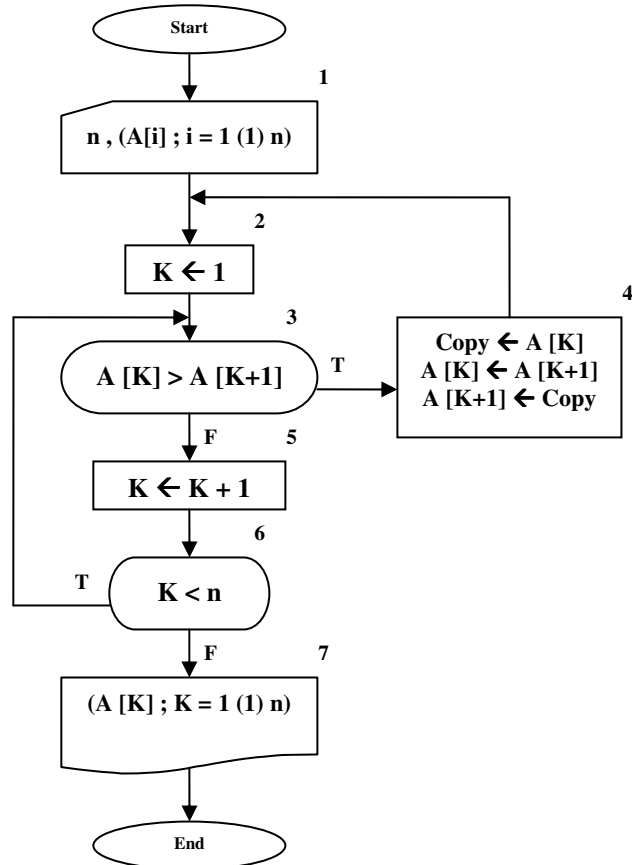
برای حل این مسئله ابتدا این سؤال را مطرح می‌کنیم که شرط مرتب بودن یک لیست به‌صورت صعودی چیست؟ پاسخ این سؤال بسیار روشن است! یک لیست از اعداد حقیقی به شرطی به‌صورت صعودی مرتب است که اگر از ابتدا تا انتهای لیست، هر عدد را با عدد بعد از خود مقایسه کنیم، عددی که در مکان پایین‌تر قرار دارد کوچکتر یا مساوی عددی که در مکان بالاتر است، باشد. در شکل ۶-۱۹ قطعه کارنمایی ارائه شده است که مرتب بودن یک آرایه از اعداد حقیقی به طول n را تشخیص می‌دهد.



شکل ۶-۱۹: قطعه کارنمای تشخیص مرتب بودن یک لیست

در شکل ۶-۲۰ کارنمایی به‌منظور مرتب‌سازی لیستی از اعداد حقیقی ارائه شده است. در این کارنما از ایده مطرح شده در تکه کارنمای شکل ۶-۱۹ استفاده کرده‌ایم. بدین ترتیب که فرض می‌کنیم لیست مذکور به‌صورت غیرصعودی

مرتب است. لذا پیمایش لیست را از ابتدا تا انتها برای تعیین غیر نزولی بودن ترتیب عناصر آن، آغاز می‌کنیم. اگر در حین پیمایش دو عدد مجاور را در لیست پیدا کردیم که عدد با زیرنویس کوچک‌تر نسبت به عدد با زیرنویس بزرگتر دارای ارزش بیشتری بود، ابتدا مکان دو عدد را مذکور را تعویض کرده تا رابطه ترتیب صعودی بین آنها برقرار گردد. سپس دوباره پیمایش لیست را برای تعیین غیر نزولی بودن آن از ابتدای لیست آغاز می‌کنیم.



شکل ۶- ۲۰: کارنمای مرتب کردن ناخوش‌آیند

این الگوریتم به دو دلیل عمده ناخوش‌آیند نام گرفته است. اول به این دلیل که این الگوریتم یک الگوریتم غیر ساخت‌یافته است؟! دوم به این دلیل که این الگوریتم برای مرتب کردن یک لیست، در بدترین حالت (که لیست از قبل به ترتیب نزولی باشد) باید $n*(n-1)/2$ بار لیستی به طول n را پیمایش کند و این زمان چندان جالبی برای یک الگوریتم مرتب‌سازی نیست. در ادامه الگوریتم‌هایی را برای مرتب‌سازی معرفی خواهیم کرد که از سرعت بالاتری نسبت به الگوریتم فوق برخوردار هستند و از همه مهمتر ساخت‌یافته نیز می‌باشند.

کار در کلاس ۳-۶:

این دسته از مسائل مربوط است به الگوریتم مرتب کردن شکل ۶-۲۰.

(۱) فرض کنید می‌خواهید با تعیین اینکه آیا لیست

۳ و ۵ و ۲ و ۷

به شکل صحیح یعنی

۷ و ۳ و ۲ و ۵

به ترتیب صعودی ردیف می‌شود یا نه، الگوریتم را امتحان کنید.

a. مقادیر ورودی جعبه یک چه می‌باشد؟

b. با استفاده از این مقادیر ورودی، پیگیری الگوریتم را از جعبه ۲ شروع کنید و شماره جعبه‌هایی را

که تا رسیدن به جعبه ۷ عملاً اجرا می‌شوند، به ترتیب نشان دهید. از جدولی مانند آنچه در اینجا

ارائه شده است استفاده کنید.

مقدار منسوب به K	۷	۶	۵	۴	۳	۲	جعبه شماره قدم
۱						√	۱

c. تعداد کل جعبه‌های کارنما که پس از قدم ورودی و پیش از رسیدن به جعبه پایان اجرا می‌شوند

چقدر است؟

d. چند بار به جعبه ۳ می‌رسیم؟

(۲) تا اینجا باید کاملاً قانع شده باشید که الگوریتم در همه موارد کار خود به طور صحیح انجام خواهد داد.

فرض کنید مقادیری که باید مرتب شوند عبارت باشند از:

۱۲ و ۹ و ۵ و ۹-

یعنی از اول به ترتیب صعودی باشند. قبل از رسیدن به جعبه ۷، جعبه ۳ چند بار اجرا خواهد شد؟

(۳) در صورتی که مقادیر ورودی از ابتدا به ترتیب معکوس مرتب باشند مانند:

۹- و ۵ و ۹ و ۱۲

جعبه ۳ چند بار اجرا می‌شود؟

(۴) آیا می‌توانید بگویید رابطه $\frac{n(n-1)}{2}$ چگونه برای بدترین حالت به دست آمده است؟

روش مرتب‌سازی حبابی

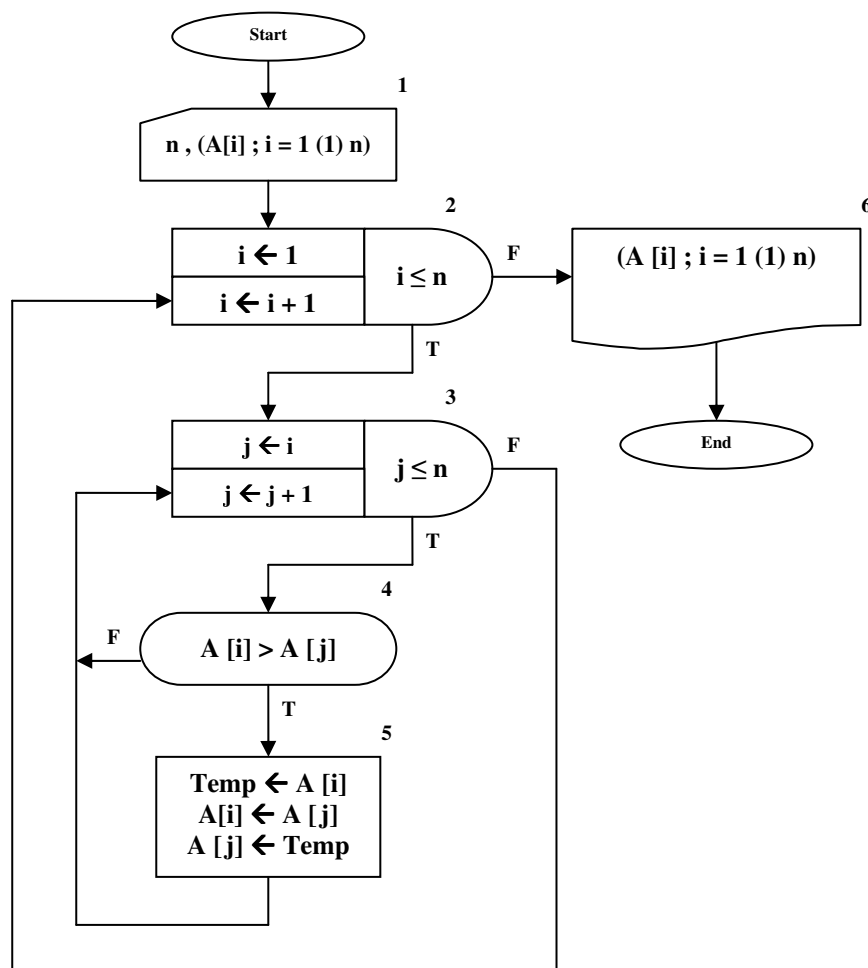
چنانکه پیشتر مطرح شد، الگوریتم شکل ۶-۲۰ الگوریتم چندان کارآمدی نیست. زیرا هر بار که یک جفت عدد را به ترتیب صعودی مرتب می‌سازیم، دوباره به ابتدای لیست باز می‌گردیم و لیست را به‌عنوان یک لیست جدید از نو مورد پردازش قرار می‌دهیم. در حالی که به خوبی می‌دانیم عناصر قبلی تا حدودی مرتب هستند. لذا در اینجا الگوریتمی را تحت‌عنوان الگوریتم مرتب‌سازی حبابی مطرح می‌سازیم که از نظر سرعت و کارآمدی تا حدودی بهتر از الگوریتم غیر ساخت‌یافته قبلی است.

پیشتر با الگوریتم یافتن بزرگترین عدد از بین N عدد در شکل ۲-۲۱ آشنا شدید. الگوریتم مرتب‌سازی حبابی ارتباط تنگاتنگی با الگوریتم مذکور دارد. اگر یک لیست مرتب را در نظر بگیرید نخستین عنصر لیست کوچکترین عنصر لیست نیز می‌باشد، و یا آخرین عنصر لیست بزرگترین عنصر لیست نیز می‌باشد. حال اگر یک لیست را به جهت یافتن کوچکترین عنصر جستجو کنیم و مکان نخستین عنصر لیست را با کوچک‌ترین عنصر لیست تعویض کنیم یک قدم به لیست مرتب شده نزدیک شده‌ایم. اگر این عمل را برای دومین عنصر لیست انجام دهیم یک قدم دیگر به لیست مرتب شده نزدیک‌تر گشته‌ایم. زیرا از دومین عنصر تا آخر لیست را به جهت یافتن کوچک‌ترین عنصر بعدی مورد جستجو قرار می‌دهیم و مکان آن عنصر را با دومین عنصر لیست تعویض می‌کنیم، بنابراین حالا دوتا از عناصر لیست مرتب در جای قطعی خود قرار گرفته‌اند. اگر این عمل را تا تعیین موقعیت آخرین عنصر لیست انجام دهیم، در نهایت می‌توان ادعا کرد که لیست مذکور مرتب شده است. همین عمل را می‌توان از انتهای لیست و با پیدا کردن بزرگترین عنصر لیست به شکل معکوس انجام داد.

برای پیاده‌سازی الگوریتم مرتب‌سازی حبابی به جای یافتن کوچکترین یا بزرگترین عنصر لیست و قرار دادن آن در محل موردنظر از روش دیگری برای پیاده‌سازی این ایده استفاده کرده‌ایم. در کارنمای شکل ۶-۲۱ که مربوط به مرتب‌سازی حبابی است، عنصر اول با عنصر دوم مقایسه شده در صورت بزرگتر بودن عنصر اول از عنصر دوم جای آنها تعویض می‌شود. سپس عنصر اول با عنصر سوم مقایسه شده و در صورت بزرگتر بودن عنصر اول، جای آنها با یکدیگر تعویض می‌شود. پس از آنکه مقایسه عنصر اول با تمامی عناصر بعد از خودش پایان یافت کوچکترین عنصر به ابتدای لیست منتقل شده است. همین عمل را برای عنصر دوم با عناصر بعد از خودش انجام می‌دهیم، و این روند برای تمامی عناصر لیست تکرار می‌شود تا آنکه لیست به‌صورت مرتب درآید.

اگر بخواهیم یک تحلیل ریاضی‌وار از زمان لازم برای اجرای کارنمای شکل ۶-۲۱ داشته باشیم باید تعداد دفعات اجرای جعبه شماره ۴، درون حلقه داخلی (j) را برآورد کنیم. از آنجا که حلقه بیرونی (n-1) بار اجرا می‌شود، حلقه داخلی نیز (n-1) بار از نو اجرا می‌شود. حلقه داخلی نیز، در اولین مرتبه (n-1) بار و در دومین مرتبه (n-2) بار و ... تا در آخرین مرتبه که یک بار اجرا می‌گردد. در مجموع جعبه شرط به تعداد دفعات زیر انجام می‌شود:

$$\sum_{i=1}^{n-1} i = \underbrace{(n-1) + (n-2) + \dots + 1}_{\text{عدد } (n-1)} = \frac{(n-1)(n-2)}{2}$$



شکل ۶- ۲۱: کارنمای مرتب‌سازی به روش حبابی

کاردر کلاس ۴-۶:

کارنمای شکل ۶- ۲۱ را به اِزای مقادیر زیر آزمایش کنید.

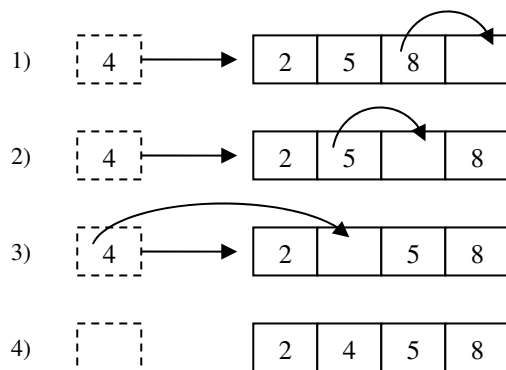
۹۸ و ۱۲ و -۸ و ۱۲۷ و ۴۳ و ۱۵ و ۷۴ و ۹۶ و ۴۰ و ۱۰

کار در کلاس ۵-۶:

- ا- کارنمایی رسم کنید که مرتب‌سازی هبابی را به نخستین شیوه مطرح شده در صفحات قبل، یعنی یافتن کوچکترین عنصر لیست و قرار دادن آن در ابتدای آرایه انجام دهد.
- ب- زمان لازم برای اجرای الگوریتم خود را مطابق آنچه که دیدید تخمین بزنید.
- ت- سعی کنید با استفاده از یک ابتکار زمان لازم برای اجرای الگوریتم را به نصف کاهش دهید. (راهنمایی: بروی مرتب‌سازی لیست از هر دو سمت فکر کنید).

مرتب‌سازی درجی

روش مرتب‌سازی درجی بدین‌گونه است که هنگام خواندن داده‌ها به داخل حافظه، آنها را مرتب می‌کنیم. با توجه به آن که اصولاً در عملیات ورودی و خروجی مقداری زمان جهت تبادل داده‌ها بین ترمینال‌های سخت‌افزاری کامپیوتر به هدر می‌رود، می‌توان عملیات مرتب‌سازی درجی را برای لیست‌های کوچک در این زمان مرده انجام داد. مرتب‌سازی درجی به اینگونه عمل می‌کند که عناصر لیست را تک تک از ورودی می‌خواند و همزمان در مکان مناسبی از لیست قرار می‌دهد. جهت ایجاد فضای مناسب در بین عناصر لیست برای قرار دادن عنصر مورد نظر در مکان مناسب خود؛ عناصر لیست را از انتهای لیست تا مکان موردنظر، یکی یکی، یک خانه به سمت راست حرکت (شیفت) می‌دهیم. در آن صورت، فضای لازم در بین عناصر آرایه ایجاد می‌شود. به‌عنوان مثال فرض کنید می‌خواهیم عدد ۴ را در آرایه زیر قرار دهیم. در این صورت ابتدا باید اعداد ۵ و ۸ را یک خانه به سمت راست شیفت دهیم. سپس عنصر ۴ را در فضای ایجاد شده قرار می‌دهیم. به اشکال رسم شده در تصویر ۶-۳۱ توجه کنید.

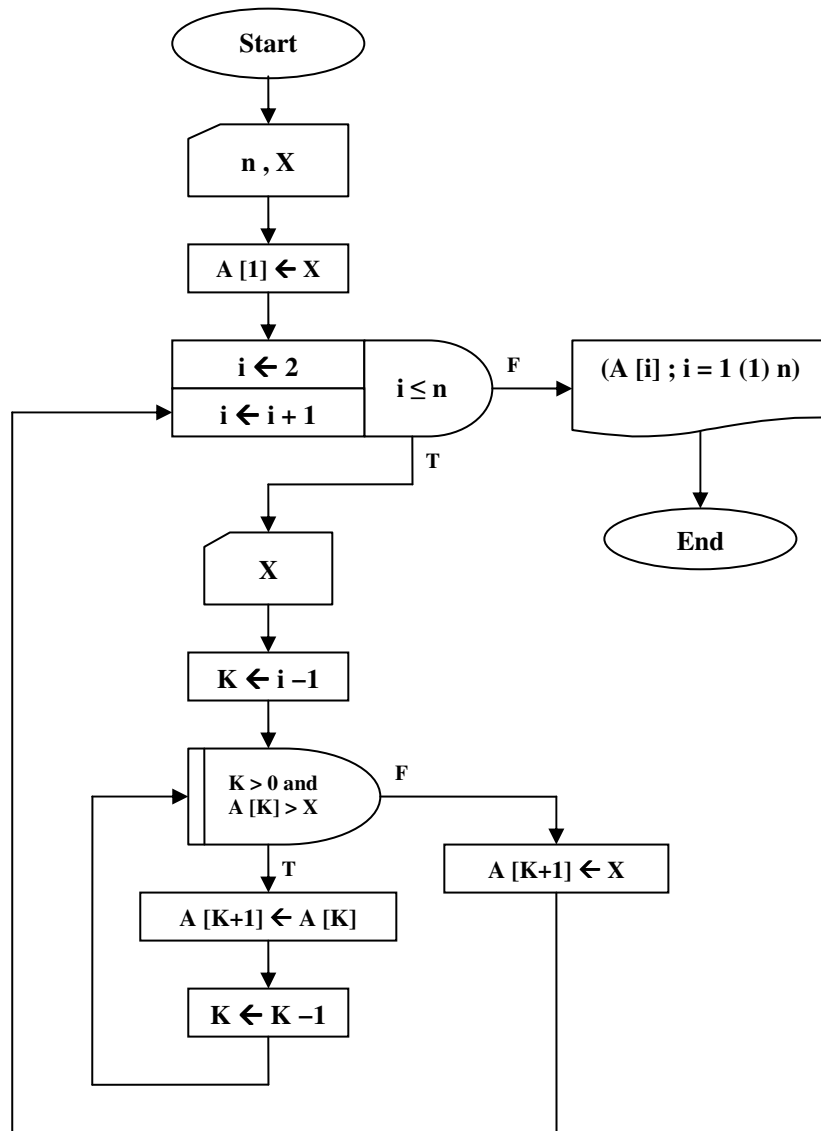


شکل ۶-۲۲: روند مرتب‌سازی درجی

به‌کار بردن این روش برای لیست‌های بلند توصیه نمی‌شود، چرا که در آن صورت به دلیل عملیات وقت‌گیر شیفت دادن عناصر، زمان مرتب‌سازی بیش از حد انتظار افزایش می‌یابد. در شکل ۶-۲۲ کارنمای مرتب‌سازی درجی نمایش داده شده است.

کار در کلاس ۶-۶:

کارنمای شکل ۶-۲۱ چه تغییری باید بکند تا پس از خواندن کامل یک لیست، آن را در لیستی دیگر به روش درجی مرتب‌سازی کند.



شکل ۶-۲۳: کارنمای مرتب‌سازی درجی

از نظر تحلیل زمانی این الگوریتم دقیقاً مشابه الگوریتم مرتب‌سازی حبابی می‌باشد. آیا می‌توانید مراحل تحلیل را به‌طور دقیق در خصوص این کارنما شرح دهید؟

جستجوی دودویی

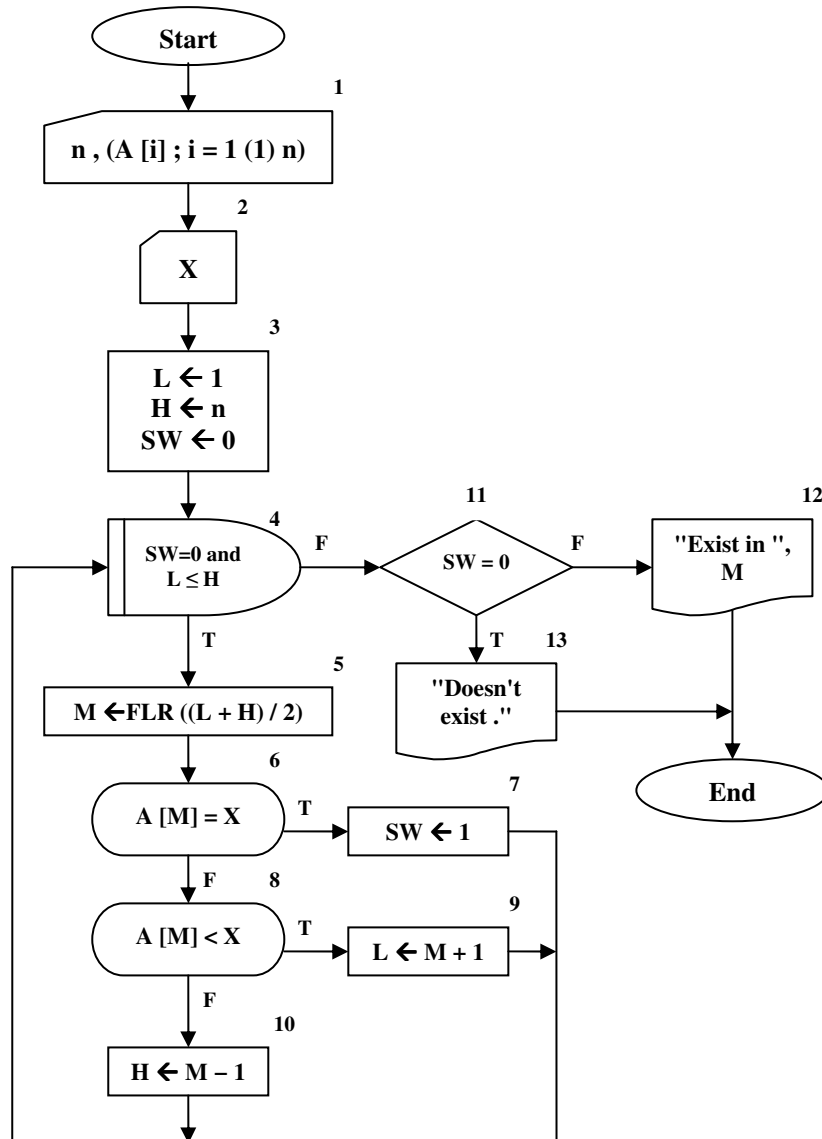
برای آنکه درک خوبی از جستجوی دودویی پیدا کنید این سؤال را مطرح می‌سازیم که، برای یافتن نام یک نفر از دوستان خود در یک دفترچه تلفن چگونه عمل می‌کنید؟

برای پاسخ به این سؤال فرض کنید یک دفترچه تلفن که براساس ۳۲ حرف الفبای فارسی یعنی از «الف» تا «ی» صفحه‌بندی شده است در اختیار داریم. در ابتدا دفترچه تلفن را از وسط آن باز می‌کنیم. یعنی مرز بین دو حرف «ش» و «ص». حال نگاه می‌کنیم حرف اول نام خانوادگی فرد مورد نظر در کدام نیمه دفترچه تلفن قرار می‌گیرد. مثلاً فرض کنید حرف مذکور در نیمه بالایی قرار گیرد، به راحتی نیمه پایینی را کنار گذاشته و عمل نصف کردن را برای نیمه بالایی دفترچه تلفن ادامه می‌دهیم. اگر این عمل را حداکثر ۵ مرتبه انجام دهیم در نهایت به انتخاب یک حرف از بین دو حرف می‌رسیم، و به این ترتیب در نهایت به نام فرد مورد نظر خود می‌رسیم.

جستجوی دودویی بر مبنای همین روش نصف کردن بنا شده است. برای اجرای عمل جستجوی دودویی حتماً باید آرایه مربوطه از قبل مرتب شده باشد. در این روش یک آرایه مرتب را مدام نصف می‌کنیم و با تشخیص اینکه نمونه مورد نظر ما، در کدام نیمه از آرایه قرار دارد، نیمه دیگر را کنار گذاشته و عمل نصف کردن را برای نیمه باقی‌مانده ادامه می‌دهیم.

اگر می‌خواهید به ارزش جستجوی دودویی پی‌برید فرض کنید لیستی داریم که شامل ۲۰ نمونه است. برای انجام یک جستجوی ترتیبی در این لیست حداکثر باید ۱۰۴۸.۵۷۶ عمل مقایسه (پیگرد) انجام داد. در حالی که برای انجام همان عمل جستجو به شیوه دودویی در این لیست، حداکثر به ۲۰ عمل مقایسه نیازمندیم. به‌طور کلی برای صورت دادن یک عمل جستجوی دودویی در یک لیست به طول n ، نیازمند $\log_2^n$ پیگرد هستیم.

در شکل ۶-۳۳ کارنمایی ترسیم شده که چگونگی اجرای یک جستجوی دودویی را بر روی یک لیست n عنصری نشان می‌دهد. این کارنما کمی با روش مطرح شده در خصوص دفترچه تلفن متفاوت است. در این کارنما متغیر X عدد مورد جستجو را در خود ذخیره می‌کند. متغیرهای L و H اندیس ابتدا و انتهای محدوده مورد پردازش از لیست را در خود نگه می‌دارند. متغیر SW یک متغیر وضعیت (سوئیچ) است. از این متغیر جهت کنترل حلقه تکرار استفاده شده است. از آنجا که عمل جستجو باید تا زمان یافتن نخستین نمونه ادامه یابد متغیر SW را به مقدار صفر مقدار اولیه داده‌ایم، در صورت یافته شدن نمونه مورد نظر این متغیر به مقدار یک تغییر وضعیت می‌دهد، تا از اجرای دوباره حلقه تکرار جلوگیری کند. شرط $L \leq H$ تا زمانی برقرار است که عناصر مورد پردازش در لیست بیش از یک مورد باشد در جعبه شماره ۵ اندیس مربوط به عنصر میانی در محدوده مورد پردازش در داخل متغیر M قرار می‌گیرد. در جعبه شماره ۶ چک می‌کنیم عنصر میانی محدوده مورد پردازش برابر نمونه مورد جستجو هست یا نه؟ اگر عنصر موردنظر یافت شود در جعبه ۷ مقدار SW را به مقدار یک تغییر داده و از حلقه خارج می‌شویم. در غیر این صورت در جعبه‌های ۸ و ۹ و ۱۰ با مقایسه نمونه مورد جستجو با عنصر میان محدوده در حال پردازش نیمه پایینی و یا نیمه بالایی محدوده را کنار می‌گذاریم. این کار به واسطه مقداردهی جدید به یکی از متغیرهای L یا H میسر می‌شود. این روند تا هنگامی که نمونه مورد نظر در آرایه یافت شود و یا تمامی آرایه مورد پردازش قرار گیرد و نمونه موردنظر در آرایه وجود نداشته باشد ادامه پیدا می‌کند. در جعبه‌های ۱۱ و ۱۲ و ۱۳ بر حسب مقدار SW پیام مناسب چاپ می‌گردد.



شکل ۶-۲۴: کارنمای جستجوی دودویی

۶-۶: تمرین

۱. فرض کنید مجموعه داده‌هایی که در رابطه با کارنمای پایین همین صفحه مورد استفاده قرار می‌گیرد متشکل از مقادیر زیر باشد.

۱ و ۲ و ۳ و ۴ و ۵ و ۶ و ۷ و ۸

الف- پس از اولین اجرای جعبه ۳ مقادیر لیست‌های A و B عبارت‌اند از (یکی را انتخاب کنید):

(1) A : ۱, ۲, ۳, ۴ B : ۵, ۶, ۷, ۸

(2) A : ۱, ۲, ۳, ۵ B : ۴, ۶, ۷, ۸

(3) A : ۵, ۲, ۳, ۴ B : ۱, ۶, ۷, ۸

(4) A : ۸, ۲, ۳, ۴ B : ۵, ۶, ۷, ۱

(5) A : ۸, ۲, ۳, ۵ B : ۱, ۶, ۷, ۴

ب- مقادیری که به خارج داده می‌شود (به ترتیب) عبارت خواهند بود از (یکی را انتخاب کنید):

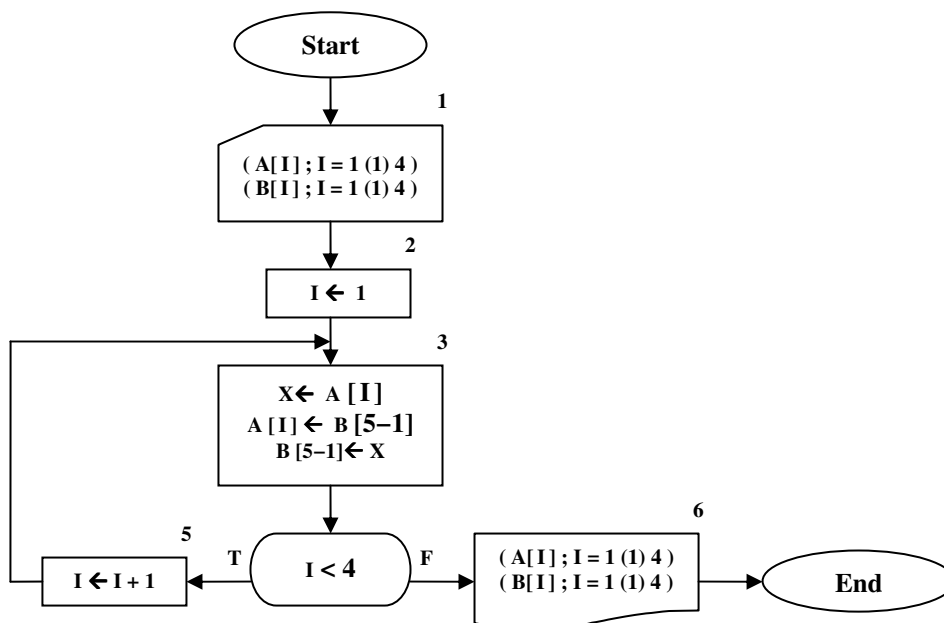
(1) ۸, ۷, ۶, ۵, ۴, ۳, ۲, ۱

(2) ۱, ۲, ۳, ۴, ۵, ۶, ۷, ۸

(3) ۵, ۶, ۷, ۸, ۱, ۲, ۳, ۴

(4) ۶, ۵, ۳, ۴, ۱, ۲, ۷, ۸

(5) هیچکدام



شکل ۶-۲۵: کارنمای تمرین ۱

۲. فرض کنید داده‌های ورودی کارنمای شکل ۶-۲۶ از مقادیر زیر تشکیل شده باشد.

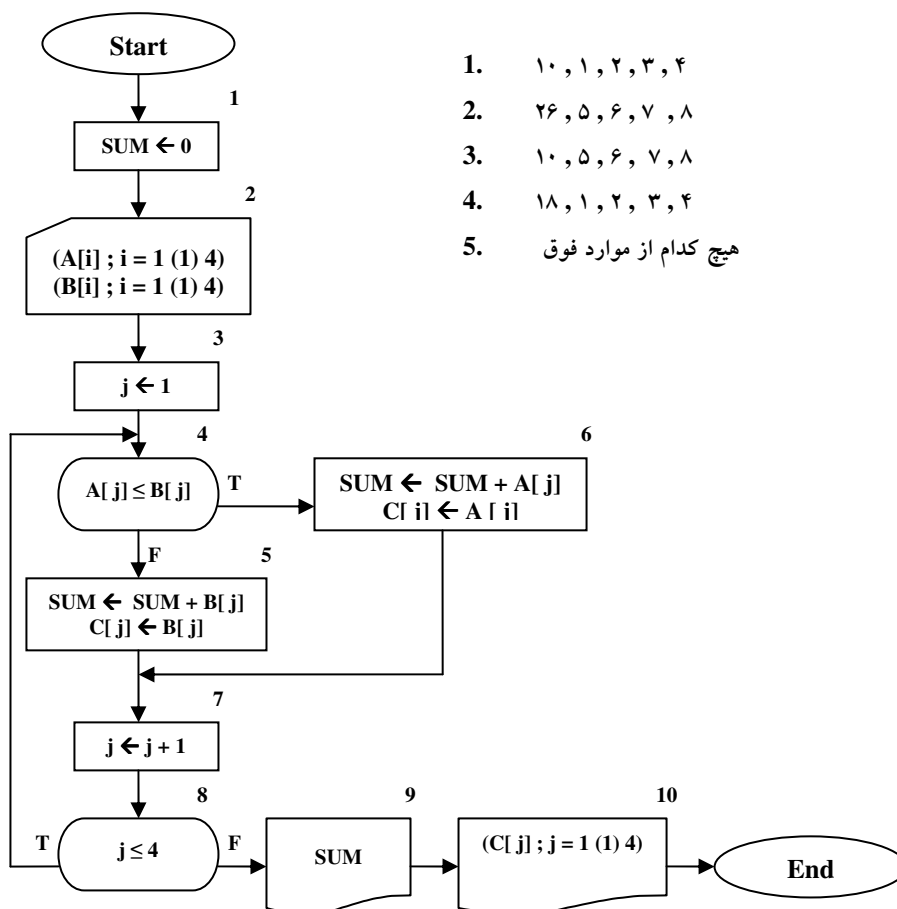
۸ و ۷ و ۶ و ۵ و ۴ و ۳ و ۲ و ۱

الف- درست یا نادرست: پس از اولین اجرای جعبه ۷ مقادیر فهرست C به جز C[1] نامعین هستند.

ب- درست یا نادرست: پس از آخرین اجرای جعبه ۷ مقادیر عناصر لیست B عبارتند از:

۸ و ۷ و ۶ و ۵

ج- در اجرای جعبه‌های ۹ و ۱۰ مقادیری که به خارج داده می‌شود به ترتیب به قرار زیرند (یکی را انتخاب کنید):



شکل ۶-۲۶: کارنمای تمرین ۲

۳. یک ارکستر سمفونی جوانان، که عازم مسافرتی برای اجرای کنسرت است، دارای صد و یک عضو است.

خبرنگاری از رهبر ارکستر سؤال می‌کند «میانۀ سن اعضای شما چه قدر است؟» رهبر ارکستر جواب می‌دهد،

«نمی‌دانم، ولی این، فهرست الفبایی اسامی و سنین نوازندگان است.»

الف- تعریفی که در این مسئله برای میانه یک مجموعه مرتب از اعداد می‌کنیم این است که عنصر «میانی» مجموعه باشد. به عبارت دیگر قبل از آن، به همان تعداد عنصر وجود داشته باشد که بعد از آن وجود دارد. برای مثال، در مجموعه زیر میانه ۳۵ است.

$$۹۷ \text{ و } ۸۱ \text{ و } ۶۷ \text{ و } ۳۵ \text{ و } ۲۴ \text{ و } ۷ \text{ و } ۳$$

کارنمایی رسم کنید که میانه سن نوازندگان را از فهرست مذکور، که بنا به فرض به ترتیب عددی نیست، پیدا کند.
ب- در مواردی که تعداد اعداد مورد نظر زوج باشد، تعریف فوق‌الذکر برای میانه صادق نیست. مثلاً دنباله زیر دارای عنصر «میانی» نیست.

$$۶۸ \text{ و } ۵۷ \text{ و } ۴۳ \text{ و } ۳۱ \text{ و } ۱۷ \text{ و } ۴$$

در این صورت می‌توان میانه را به صورت میانگین دو عدد میانی تعریف کرد. در دنباله فوق میانه عبارت است از: $۳۷ = (۳۱ + ۴۳) \div ۲$.

رهنمایی: برای تعیین میانه مجموعه شامل n عدد $a_1, a_2, \dots, a_n$ عبارتی تهیه کنید. پاسخ شما باید چنان عبارتی باشد که جواب صحیح را برای هر n ، اعم از زوج یا فرد، به دست دهد.
کارنمایی شبیه کارنمای بند (الف)، ولی طرح شده برای ارکستری شامل تعداد دلخواهی نوازنده، چنان تهیه کنید که نه فقط میانه سن بلکه سن جوانترین و پیرترین نوازنده را نیز پیدا کند.

۴. کارنمای شکل ۶-۲۷ را مطالعه کنید و بند (الف) و (ب) را پاسخ دهید.

الف- فرض کنید مجموعه داده‌هایی که در اثر اجرای جعبه ۱ کارنما به داخل آورده می‌شود عبارت باشد از:

$$۱۶/۷ \text{ و } ۴/۴ \text{ و } ۰ \text{ و } -۴/۱ \text{ و } ۲/۱ \text{ و } ۱۹ \text{ و } ۶$$

هنگامی که اجرای کارنما به جعبه تعریف می‌رسد، مقدار منسوب متغیر K کدام یک از مقادیر زیر خواهد بود.

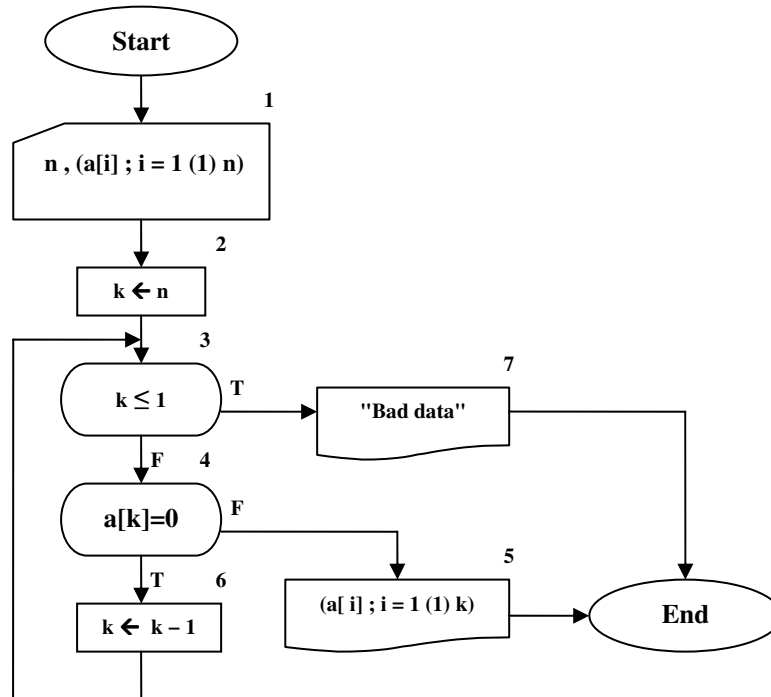
۱. ۰

۲. ۶

۳. -۱

۴. ۴

۵. ۵



شکل ۶-۲۷: کارنمای تمرین ۴

ب- اگر مجموعه داده‌هایی که در نتیجه اجرای جعبه ۱ به داخل آورده می‌شود عبارت باشد از:

۰ و ۰ و ۴ و ۰ و ۴ و ۵

خروجی‌ای که به وسیله الگوریتم تولید می‌شود، توسط کدام یک از جواب‌های زیر نشان داده شده؟

۱. ۴ ، ۰ ، ۴

۲. «داده خراب»

۳. ۳ ، ۴ ، ۰ ، ۴ ، ۰ ، ۰

۴. ۳ ، ۳ ، ۰ ، ۴

۵. ۲ ، ۴ ، ۰ ، ۴

۵. فرض کنید کارنمای شکل ۶-۲۸ با استفاده از مجموعه داده‌های زیر اجرا شود.

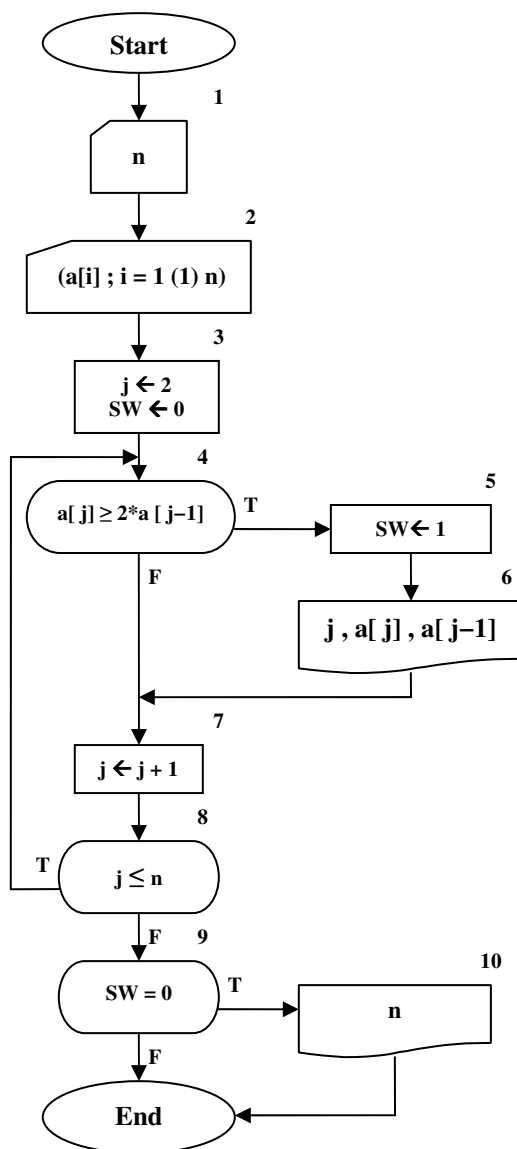
۹ و ۳ و ۷ و ۲۲ و ۱۰ و ۱۲ و ۶ و ۳ و ۸

الف- جعبه ۸ چند بار اجرا خواهد شد؟

ب- جعبه ۵ چند بار اجرا خواهد شد؟

ج- کلیه مقادیری را که چاپ خواهد شد نشان بدهید. فرض کنید هر بار اجرای جعبه ۶ یا جعبه ۱۰، موجب عمل چاپ بر روی خط جدیدی می‌شود (وقتی جعبه ۶ اجرا شود سه مقدار و وقتی جعبه ۱۰ اجرا شود یک مقدار بر روی خط چاپ خواهد شد).

د- آیا استفاده از زیرنویس واقعاً برای مسئله ضروری است؟ اگر جوابتان منفی است، به‌منظور حذف زیرنویس‌ها، کارنما را مجدداً ترسیم کنید و در کنار هر جعبه‌ای که تغییر می‌دهید علامت تیک (✓) قرار دهید.



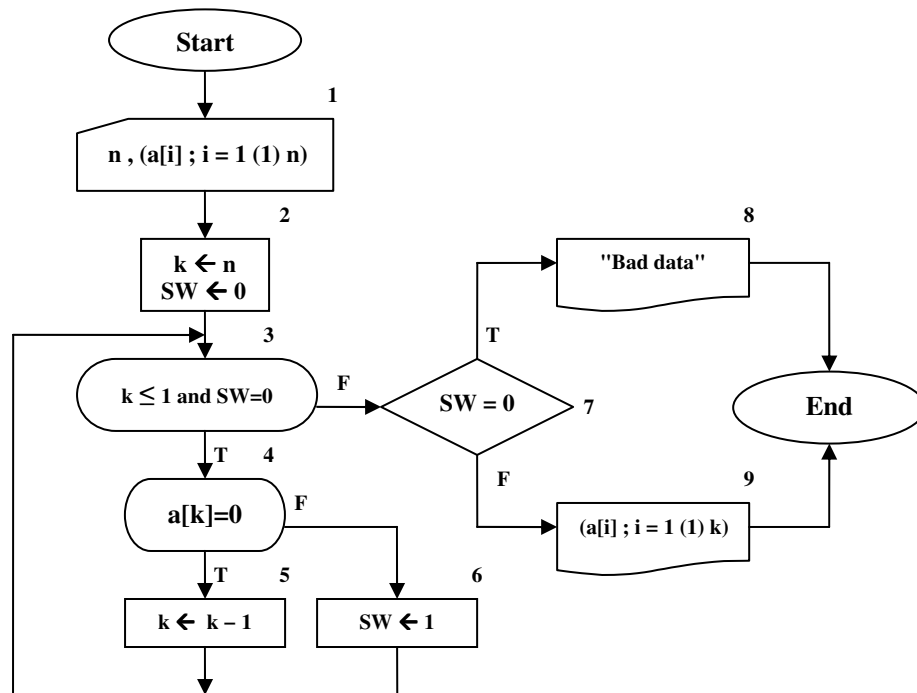
شکل ۶- ۲۸: کارنمای تمرین ۵

۶. آیا کارنمای شکل ۶- ۲۸ ساخت یافته است؟ اگر جواب شما منفی است آن را دوباره به‌صورت ساخت یافته ترسیم کنید و اگر جواب شما مثبت است دلیل خود را شرح دهید.

۶-۷: کاربرد متغیرهای وضعیت (سوئیچ)

در برخی مسائل مطرح شده تا اینجا با متغیرهای سوئیچ آشنا شدید و برخی از کاربردهای آنها را دیدید. اصولاً در الگوریتم‌ها از متغیرهای وضعیت به نام سوئیچ می‌توان استفاده کرد تا تصمیم‌گیری در دستورات شرطی و حلقه‌های تکرار آسان بشود. به عنوان مثال پیشتر در کارنمای شکل ۶-۲۴ با متغیر وضعیت SW برخورد کردید. از این متغیر جهت کنترل حلقه تکرار و تصمیم‌گیری استفاده کردیم. لذا از ارائه مثال تکراری در خصوص این کاربرد متغیرهای سوئیچ خودداری می‌کنیم و به بررسی کاربرد مهمتر این متغیرها می‌پردازیم.

یکبار دیگر به کارنمای شکل ۶-۲۷ توجه کنید. فارق از این که نحوه تعریف آرایه در زبان C++ چگونه است، اگر به شما بگویند برنامه C++ مربوط به این کارنما را با معلومات فعلی خود بنویسید قطعاً به مشکل برخورد خواهید خورد^۱. علت این امر، رفتن به سوی دکمه پایان از دو محل متفاوت در کارنما یعنی از جعبه‌های ۵ و ۷ است، که هر کدام مربوط به یک جعبه تصمیم جداگانه هستند. برای رفع این مشکل از متغیرهای وضعیت باید استفاده کرد. نحوه استفاده از این متغیرها در کارنمای شکل ۶-۲۹ که بازنویسی کارنمای شکل ۶-۲۷ است نشان داده شده است. البته راه حل زیر را می‌توان به نوعی حرکت به سوی ساخت یافتگی بیشتر، در برنامه‌ها دانست.



شکل ۶-۲۹: کارنمای بازنویسی شده با متغیر سوئیچ

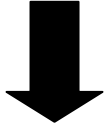
۱. البته همانگونه که مطرح شد معلومات فعلی شما کم است، و گرنه می‌توان برنامه معادل کارنمای شکل ۶-۲۴ را با استفاده از تابع کتابخانه‌ای exit نوشت. اما در همه زبان‌های برنامه‌نویسی چنین امکانی وجود ندارد.

۸-۶: آرایه‌های دو بعدی

در ریاضیات دوره متوسطه با ماتریس‌ها آشنا شده‌اید. با عملیات مختلف جمع، تفریق، ضرب، دترمینان و معکوس‌گیری ماتریس‌ها به‌خوبی آشنایی دارید و برخی کاربردهای آنها را دیده‌اید. یکی از مهمترین کاربردهای ماتریس‌ها یعنی حل دستگاه‌های چند معادله و چند مجهول را تجربه کرده‌اید. تمامی این عملیات از نظر علم کامپیوتر بسیار با اهمیت و پرکاربرد هستند. بسیاری از معادلات پیچیده در مدارات الکترونیکی در نهایت منجر به یک دستگاه معادلات می‌شود که حل آنها با دست بسیار وقت‌گیر و شاید ناممکن است. لذا کارکردن بر روی ماتریس‌ها و عملیات مختلف بر روی آنها بسیار حائز اهمیت می‌باشد. برای پیاده‌سازی ماتریس‌ها در حافظه کامپیوتر از آرایه‌های دو بعدی بهره می‌گیریم. فرض کنید ماتریس $A_{3 \times 4}$ که ماتریس ضرایب دستگاه معادلات زیر است، را بخواهیم در حافظه کامپیوتر نشان دهیم. برای این منظور از یک آرایه دو بعدی که علاوه بر ستون دارای تعدادی سطر نیز هست استفاده می‌کنیم، که در شکل ۶-۳۱ نشان داده شده است.

$$\begin{cases} 2X + 7Y + 63Z = 13 \\ 45X + Z = 91 \\ 6X + 29Y = 4 \end{cases} \quad \longrightarrow \quad \begin{bmatrix} 2 & 7 & 63 & 13 \\ 45 & 0 & 1 & 91 \\ 6 & 29 & 0 & 4 \end{bmatrix}$$

شکل ۶-۳۰: یک دستگاه معادلات و ماتریس ضرایب آن

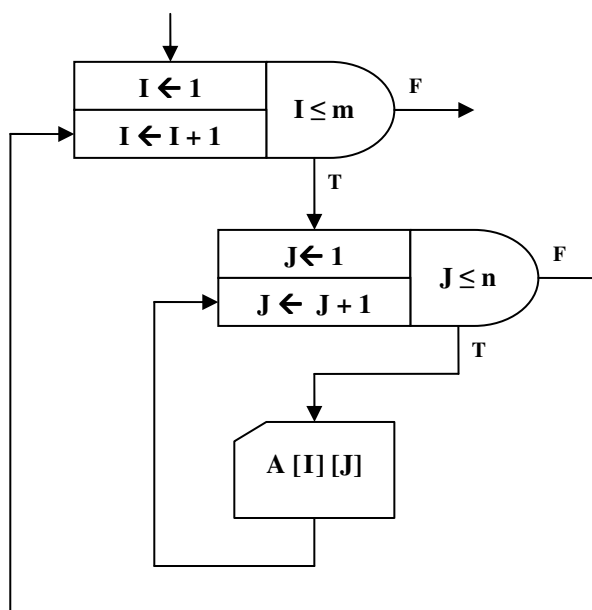


2	7	63	13
45	0	1	91
6	29	0	4

شکل ۶-۳۱: آرایه دو بعدی معادل ماتریس فوق

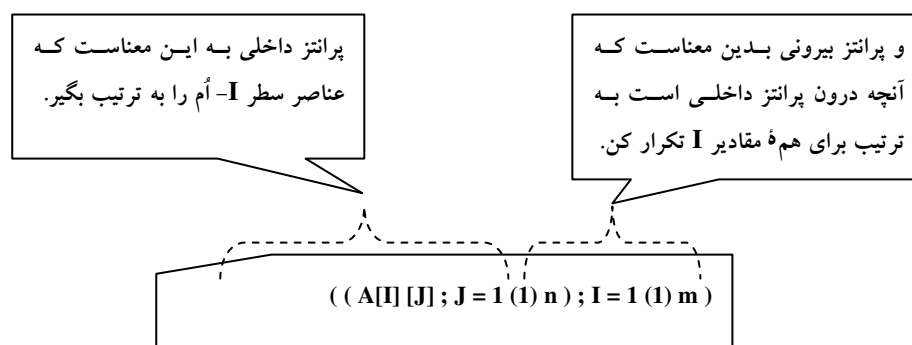
یک آرایه دو بعدی را می‌توان به‌عنوان یک جدول تصور کرد با این توصیف کاربرد آرایه‌های دو بعدی بسیار فراتر از ماتریس‌ها خواهد رفت. به‌عنوان مثال می‌توان از آرایه‌ها به‌عنوان جداول ریز قیمت اجناس یک فروشگاه استفاده کرد. که هر ردیف نام یک کالا و قیمت آن را نگه دارد، و یا آن که می‌توان جدول میهمانان یک هتل را به همراه شماره اتاق آنها در یک آرایه دو بعدی ذخیره کرد که در ستون اول هر ردیف نام میهمان و در ستون دوم، شماره اتاق آن را ذخیره کرد. رفته رفته کاربردهای متنوع‌تری از آرایه‌های دو بعدی را خواهید دید.

چنانکه در ریاضیات برای مراجعه به درایه واقع در سطر I - اُم و ستون J - اُم می‌نوشتیم $A_{I,J}$ در اینجا به‌طور معادل می‌نویسیم $A[I][J]$. نکته قابل توجه آنکه ابتدا شماره سطر و سپس شماره ستون ذکر می‌شود. از طرفی برای خواندن اطلاعات یک ماتریس باید از دو حلقه تکرار به‌صورت زیر استفاده کنیم:



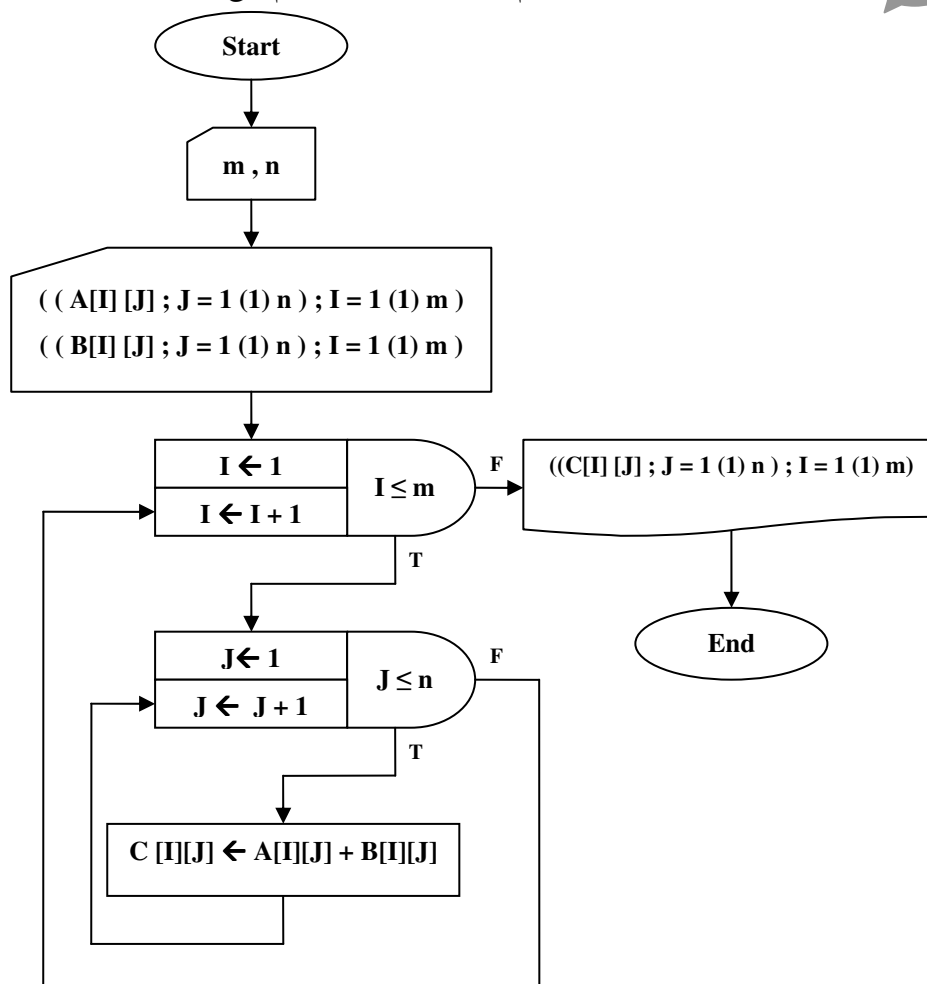
شکل ۶-۳۲: نحوه خواندن درایه‌های آرایه دو بعدی

اما چنانکه برای خواندن و نوشتن بر روی تمامی درایه‌های آرایه یک بعدی نماد خاصی را قرارداد کردیم برای خواندن و یا نوشتن تمامی درایه‌های یک آرایه دو بعدی به روش زیر عمل می‌کنیم. به نحوه پرانتزگذاری‌ها توجه ویژه داشته باشید.



شکل ۶-۳۳: قرارداد جدید خواندن درایه‌های آرایه دو بعدی

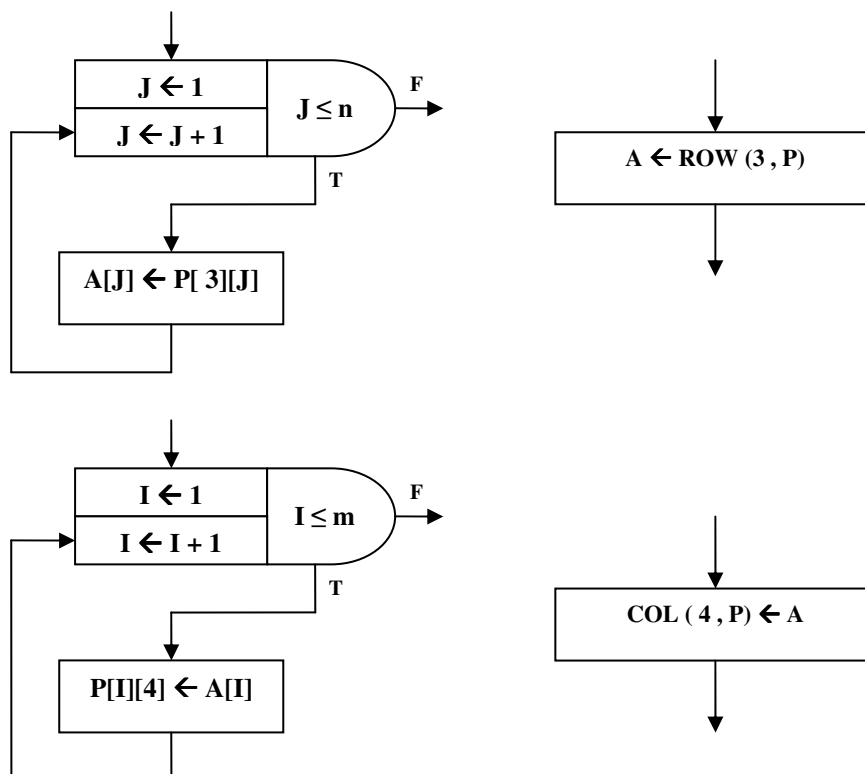
مثال ۴-۶: کارنمایی که دو ماتریس هم اندازه با ابعاد $m \times n$ را با هم جمع می‌کند.



شکل ۴-۶: کارنمای مثال ۴-۶

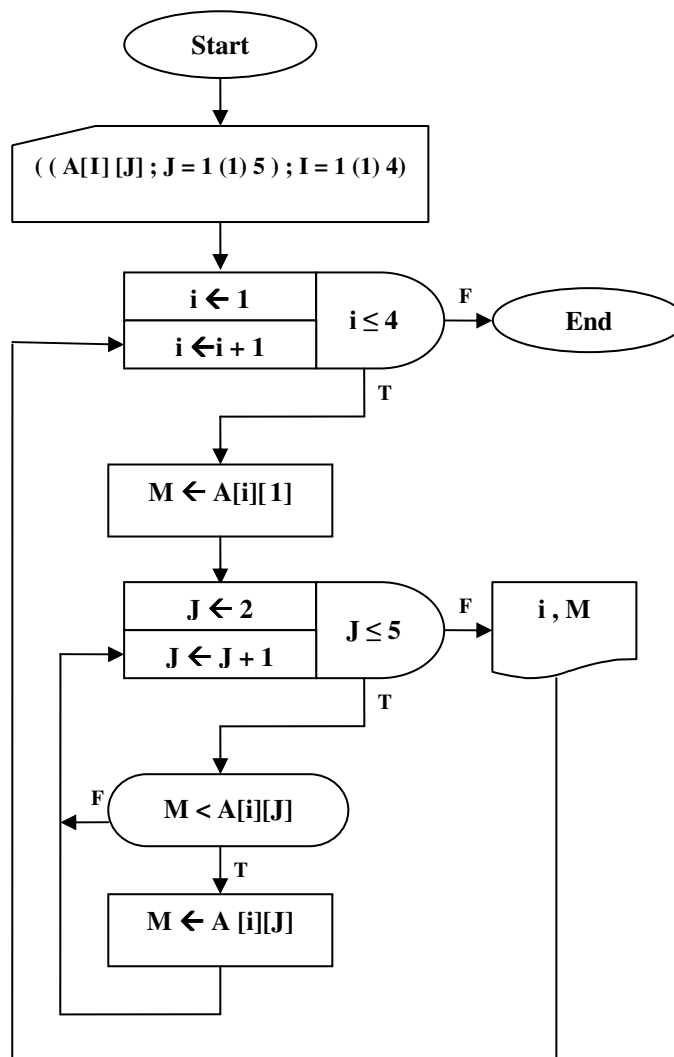
توضیح مثال: جمع دو ماتریس تنها با انجام جمع جبری بر روی درایه‌های نظیر به نظیر ماتریس محقق می‌گردد. لذا پس از خواندن درایه‌های هر دو ماتریس A و B، در یک حلقه تودرتو شروع به جمع درایه‌های نظیر به نظیر دو ماتریس کرده‌ایم.

اما به‌عنوان آخرین نکته در زمینه کار با آرایه‌ها این که گاهی اوقات می‌خواهیم درایه‌های یک سطر از یک ماتریس دو بعدی را در داخل یک آرایه یک بعدی بریزیم، و یا بر عکس ممکن است بخواهیم درایه‌های یک آرایه یک بعدی را در یک سطر از یک ماتریس دو بعدی قرار دهیم. در اینگونه موارد به جای استفاده از حلقه‌های تکرار از نمادهای مندرج در شکل ۶-۴۶ استفاده می‌کنیم. به چگونگی کاربرد دو کلمه ROW و COL به دقت توجه کنید.



شکل ۶-۳۵: نماد انتساب تک ردیفی یا تک ستونی

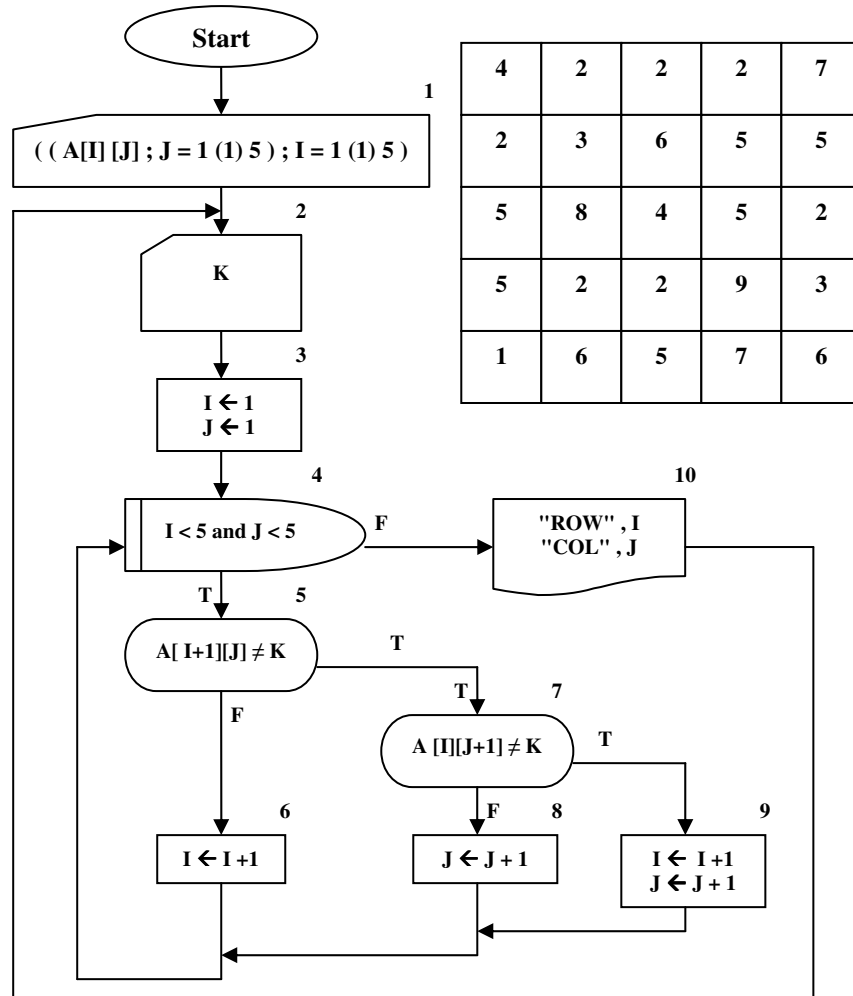
مثال ۵-۶: الگوریتمی که عناصر یک آرایه 4×5 را خوانده و بزرگترین عنصر هر سطر را چاپ می‌کند.



شکل ۶-۳۶: کارنمای مثال ۵-۶

۹-۶: تمرین

۱. فرض کنید با اجرای جعبه ۱ در کارنمای زیر مقادیر ماتریس زیر به داخل A وارد شود:



شکل ۶-۳۷: کارنمای تمرین ۱

الف- فرض کنید اولین اجرای جعبه ۲، مقدار ۲ را به k منسوب کند. در این صورت نتیجه‌ای که در جعبه ۱۰ برای این مقدار از k چاپ می‌شود مطابق زیر خواهد بود.

۱. ROW 5 COL 3

۲. ROW 2 COL 5

۳. ROW 3 COL 5

۴. ROW 5 COL 4

۵. هیچکدام از موارد فوق.

ب- فرض کنید دومین اجرای جعبه ۲ مقدار ۵ را به K منسوب کند. در این صورت نتیجه‌ای که در جعبه ۱۰ برای این مقدار از K چاپ می‌شود مطابق زیر خواهد بود.

۱. ROW 5 COL 3

۲. ROW 5 COL 2

۳. ROW 4 COL 5

۴. ROW 2 COL 5

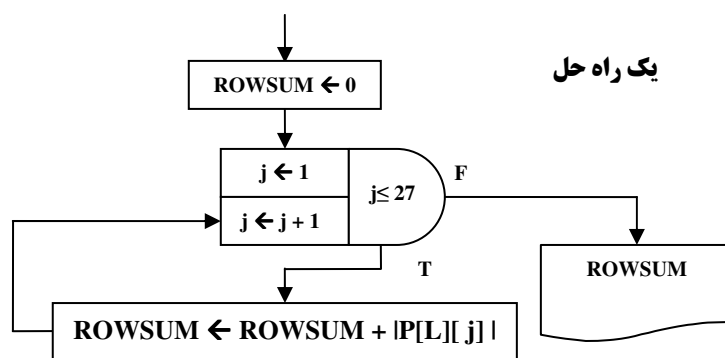
۵. هیچکدام از موارد فوق.

۲. با استفاده از آرایه ۵ سطری و ۲۵ ستونی A از رشته‌های تک نویسه‌ای، کارنمایی جهت چاپ حروف درشت MJZ، مطابق آنچه در زیر نشان داده شده، ترسیم کنید. این حروف درشت با فاصله‌ای با عرض ۵ نویسه خالی از یکدیگر جدا شده‌اند و مقادیر ورودی منحصراً شامل نویسه‌های «M»، «J»، «Z» هستند.

M		M		J	J	J	J	J		Z	Z	Z	Z	Z
M	M		M	M				J						Z
M		M		M				J						Z
M			M			J		J					Z	
M			M					J		Z	Z	Z	Z	Z

در مسائل ۳ الی ۸ که در ادامه آمده، فرض کنید که به همه متغیرها یا درایه‌های ماتریس‌هایی که ذکر شده‌اند قبلاً مقادیر اولیه‌ای منسوب شده است. وظیفه شما ترسیم کارنمای عملی است که توضیح داده شده است. (اینها عملیات ابتدایی هستند که غالباً بر روی ماتریس‌ها انجام می‌شوند و معمولاً قسمت‌هایی از مسائل بزرگتراند).

مثال ۲: ماتریس P دارای ۲۲ سطر و ۲۷ ستون است. مجموع قدرمطلق‌های همه درایه‌های سطر L این ماتریس را پیدا کنید. قبلاً مقدار اولیه‌ای به L منسوب شده است.



شکل ۶-۳۸: کارنمایی برای مثال فوق

۳. برای همان ماتریس P ، مجموع همه درایه‌های ستون K -ام را به استثنای یک درایه پیدا کنید. درایه‌ای که مستثنی می‌شود درایه سطر ۱۲ است. مجموعی را که بدین وسیله به دست می‌آید COLSUM بنامید.
۴. برای همان ماتریس P ، به هر درایه از ردیف L مقدار درایه نظیر آن (در همان ستون) از ردیف M را بیفزایید. به عنوان یک مثال واقعی و با یک ماتریس به مراتب کوچک‌تر Q خواهیم داشت:

ماتریس فرضی Q قبل از عملیات

$$\begin{array}{l} \text{سطر } L\text{-ام} \\ \text{سطر } M\text{-ام} \end{array} \begin{bmatrix} 3 & 4 & 2 & 5 & -4 \\ 3 & 9 & 1 & 2 & -4 \\ 1 & 1 & 2 & 3 & 2 \end{bmatrix}$$

ماتریس فرضی Q بعد از عملیات

$$\begin{array}{l} \text{سطر } L\text{-ام} \\ \text{سطر } M\text{-ام} \end{array} \begin{bmatrix} 3 & 4 & 2 & 5 & -4 \\ 4 & 10 & 3 & 5 & -2 \\ 1 & 1 & 2 & 3 & 2 \end{bmatrix}$$

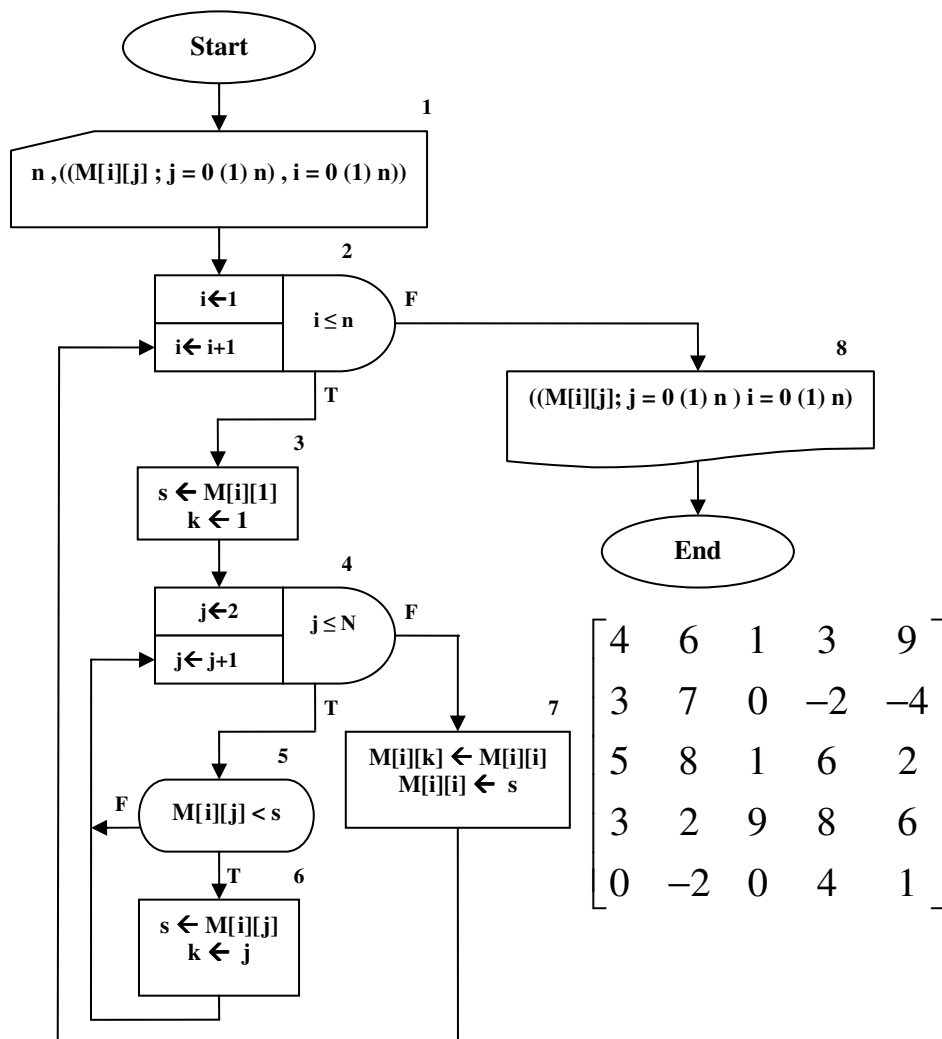
۵. برای همان ماتریس P ، به هر درایه از سطر L ، به استثنای درایه K -ام آن، دو برابر درایه نظیر آن در سطر M -ام را بیفزایید.
۶. برای همان ماتریس P ، جای دو سطر L و M را با هم عوض کنید.
۷. برای همان ماتریس P ، آن درایه از سطر L را که دارای بزرگترین قدرمطلق است پیدا کنید. به فرض صفر نبودن آن، هر درایه از سطر L را به آن تقسیم کنید.
۸. در مباحثی که بلافاصله قبل از این مجموعه تمرین‌ها آمد، ما قراردادهای مربوط به بیان سطح بالاتر را مورد بحث قرار دادیم. برای مثال، در جایی که A یک لیست و P ماتریسی با ابعاد مناسب باشد، توافق شد که نماد انتساب

$$A \leftarrow \text{ROW}(2, P)$$

معنایش این باشد که: مقادیر سطر دوم P را به A منسوب کن.

- اگر تاکنون این کار را نکرده‌اید، با به کار گرفتن یک بیان سطح بالاتر راه‌حل خود را برای تمرین ۷ ساده کنید. سپس ببینید آیا می‌توانید با ابتکار خود حلقه‌های بقیه تمرینات ۴ تا ۸ را نیز حذف کنید.

۹. این سؤال مربوط به کارنمای زیر می‌شود. فرض کنید با اجرای جعبه ۱ مقادیر ماتریس زیر خوانده شود. هنگامی که اجرا به جعبه ۸ برسد مقدار M چیست؟



شکل ۶-۳۹: کارنمای تمرین ۹

۱۰. الگوریتمی بنویسید که لیست اسامی n دانشجو را به همراه معدل آنها بخواند و سپس بدون این که ترتیب دانشجویان دو لیست را بهم بزند، به آنها رتبه بدهد. مثلاً به دانشجویانی که معدل ۲۰ دارند رتبه ۱ بدهد و به همین ترتیب ادامه دهد. در نهایت لیست دانشجویان را به ترتیب رتبه آنها چاپ کند.

۱۱. فرض کنید ماتریس Q با M سطر و N ستون در حافظه موجود است. سطر L -ام Q را برای پیدا کردن کوچکترین مقدار مورد جستجو قرار دهید. این مقدار را به SMALL منسوب کنید.

۱۲. فرض کنید ماتریس Q با M سطر و N ستون در حافظه موجود است. ستون R -ام را به‌طور وارونه (یعنی از پایین به بالا) به‌منظور پیدا کردن اولیه درایه (در صورت وجود) که حداقل به بزرگی مقدار جاری T باشد مورد جستجو قرار دهید. شماره سطر این درایه را به ROW و مقدار آن را به BIG منسوب کنید. در صورتی که چنین مقداری یافت نشد، مقدار صفر را به ROW منسوب کنید.

۱۳. فرض کنید که مقادیر مربوط به یک ماتریس $N \times N$ به نام p قبلاً در حافظه منسوب شده باشند. قطعه‌کارنمایی ترسیم کنید که (بدون نگرانی از ورودی و خروجی) قدم‌هایی را که در زیر توضیح داده شده است (به ترتیب) انجام بدهد.

قدم ۱: به جای هر درایه از قطر اصلی p (از گوشه بالای سمت چپ تا گوشه پایین سمت راست) مربع مقدار آن درایه را قرار دهد.

قدم ۲: بزرگترین درایه سطر اول را به هریک از درایه‌های سطر آخر بیفزاید و نتایج حاصل را به‌عنوان مقادیر جدید در سطر آخر قرار دهد.

۱۴. فرض کنید یک جدول کلمات متقاطع در ماتریسی به نام CWP ذخیره شده باشد به‌طوری که هر عنصر از جدول یک نویسه تنها یا یک جای خالی (که یک مربع سیاه را نمایش می‌دهد) باشد. شکل ۶-۴۰ نمایش ماتریسی یک جدول قابل قبول است. در این مورد یک ماتریس 9×9 مورد نیاز است.

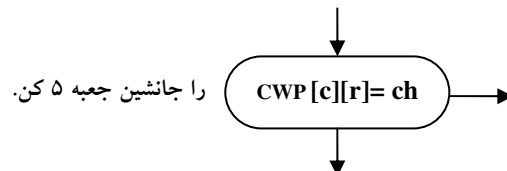
جعبه‌های ۳ الی ۹ کارنمای شکل ۶-۴۰ قدم‌های لازم برای جستجو در جدول و تعیین این که آیا اولین حرف آن کلمه «افقی» که در سطر r از ستون C شروع می‌شود، مطابق با مقدار (تک) حرفی متغیر ورودی ch هست یا نه را نمایش می‌دهد. بعضی از جعبه‌های کارنما ناتمام هستند. کارنمای مزبور را مطالعه و تعیین کنید که آیا گزاره‌های (أ) تا (ت) درست هستند یا نادرست.

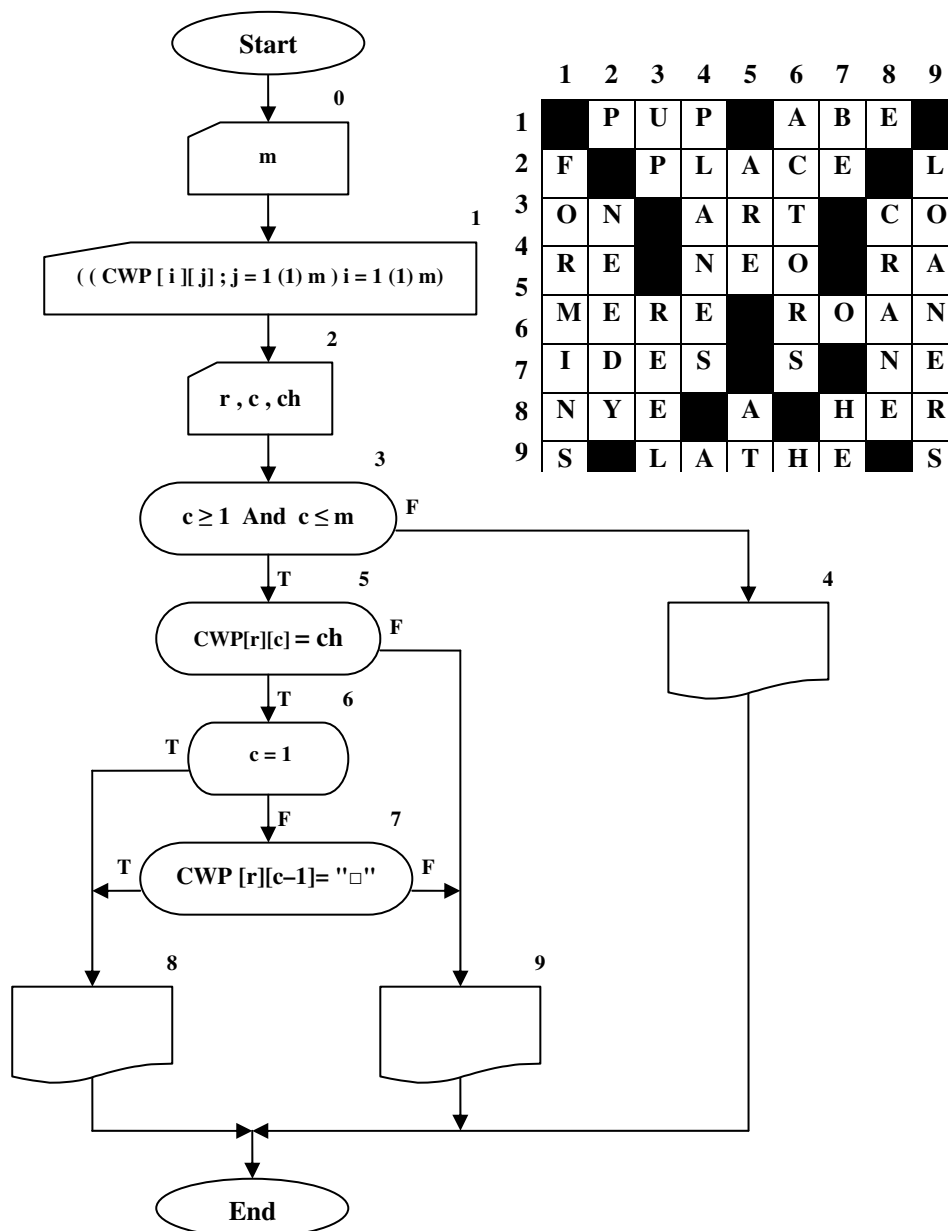
الف- جعبه‌های ۱ و ۲ کارنما برای این منظورند که بترتیب جدول کلمات متقاطع تکمیل شده و داده‌های لازم برای جستجو را به داخل بیاورد.

ب- جعبه ۳ صحت شماره ستون، c ، را مورد بررسی قرار می‌دهد، لکن بررسی مشابهی در مورد صحت شماره سطر، r ، به عمل نمی‌آید.

ج- فرض کنید که جدولی که در جعبه ۱ داده شده است به داخل آورده شده است. به علاوه، فرض کنید که مقادیر داده‌های ۳، ۵، 'R' در جعبه ۲ به داخل آورده شده‌اند. جریان کنترل در نهایت به جعبه ۸ خواهد رسید.

د- اگر قرار باشد الگوریتم به نحوی تغییر پیدا کند که بتواند برای جستجو کلمات عمودی که با مقدار حرفی متغیر ch شروع می‌شوند به کار رود، تغییر زیر کفایت می‌کند.





شکل ۶-۴۰: کارنمای تمرین ۱۴

۱۵. کارنمایی رسم کنید که در مورد ماتریس p با m سطر و n ستون، در سطرهاى شماره فرد و ستون‌های شماره زوج جستجو و عنصری را که دارای کوچکترین مقدار جبری و مخالف صفر است پیدا و آن را به LEAST منسوب کند. در حین انجام این جستجو، حساب تعداد عنصرهای مساوی صفر را که به آن برمی‌خورید در متغیر

۲۱. صفحه شطرنج را می‌توان به صورت یک الگوی نویسه ای 8×8 تجسم کرد. کارنمایی بسازید که دو عدد صحیح (M و N) را به عنوان محل قرار گرفتن وزیر (Q) بخواند و نقش صفحه شطرنج را طوری چاپ کند که در محل‌های تحت پوشش وزیر (محل‌هایی که در همان سطر یا ستون یا قطر قرار گرفته‌اند) علامت ستاره (*) و در محل‌های دیگر علامت منها چاپ شود. شکل زیر حل مسئله را برای حالتی که وزیر در محل (۴ و ۳) قرار گرفته است نشان می‌دهد.

الف- حل کارنمایی مسئله را بدون استفاده از متغیر لیستی یا آرایه بسازید.

ب- حل کارنمایی مسئله را بدون استفاده از آرایه ولی با استفاده از متغیر لیستی بسازید.

ج- حل کارنمایی مسئله را با مجاز بودن استفاده از آرایه بسازید.

```

- * - * - * - -
- - * * * - - -
* * * Q * * * *
- - * * * - - -
- * - * - * - -
* - - * - - * -
- - - * - - - *
- - - * - - - -

```

۲۲. این سؤال مربوط به الگوریتمی برای تولید صورت رمزی یک پیام است. این الگوریتم از دو لیست IN و OUT،

مانند آنچه در زیر نشان داده شده است، استفاده می‌کند. هر لیست شامل تمام ۲۶ حرف الفبای انگلیسی است ولی

ترتیب حروف در آنها متفاوت است.

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
I	C	T	Q	Y	J	W	E	V	A	O	K	B	H	P	R	G	L	S	X	D	N	Z	M	U	F

الگوریتم به طریق زیر کار می‌کند.

الف- مقادیر اولیه برای لیست‌های IN و OUT به داخل آورده می‌شوند.

ب- m، یعنی تعداد نویسه‌های پیام به داخل آورده می‌شود.

ج- پیام را خوانده و هر کاراکتر (نویسه) آن به ترتیب در یک عنصر لیست MSG قرار داده می‌شود. (فرض کنید

پیام فقط از حروف تشکیل شده است و جای خالی، عدد و غیره در آن وجود ندارد)

د- هر حرف پیام، به رمز در می‌آید بدین ترتیب که محل آن در لیست IN مشخص می‌شود و سپس حرف متناظر

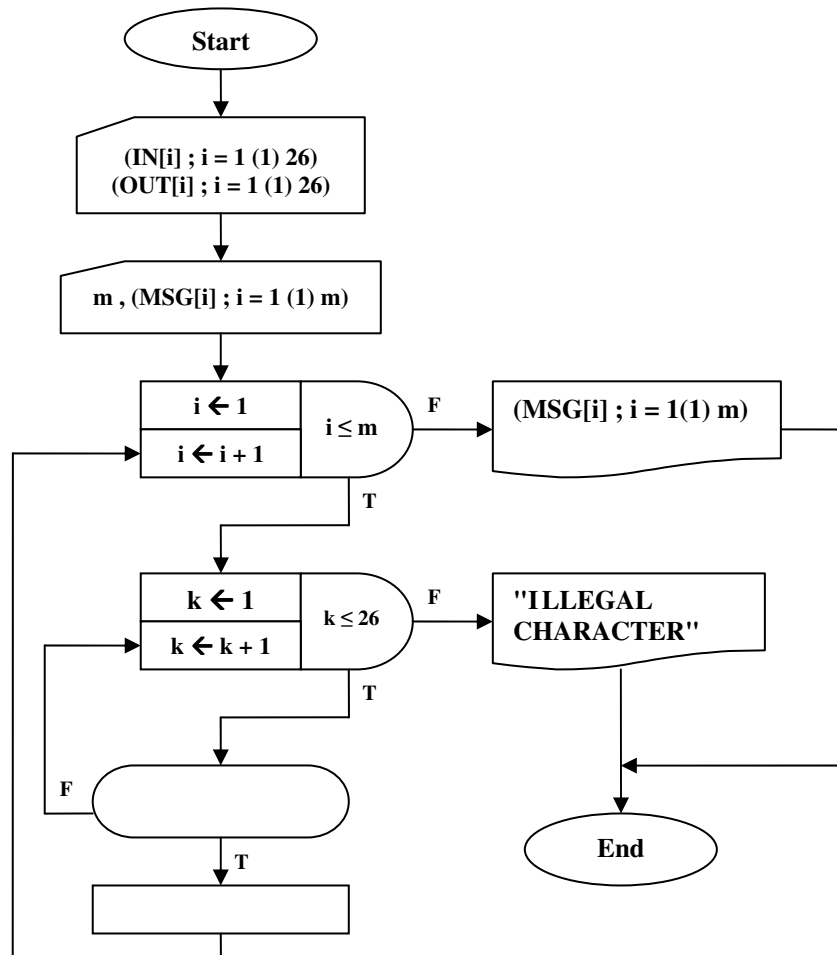
آن در لیست OUT جایگزین آن حرف می‌شود. در اینجا «عنصر متناظر» یعنی عنصری که دارای همان زیرنویس

باشد. (مثلاً در مورد IN و OUT که در زیر نشان داده شده است، هر بار که D در MSG باشد به جای آن Q

قرار خواهد گرفت زیرا D، IN[4] است و Q، OUT[4] است.)

و- شکل رمزی پیام چاپ می‌شود.

در زیر، کارنمای ناتمامی برای این الگوریتم نشان داده شده است. وظیفه شما آن است که این کارنما را تکمیل کنید.



شکل ۶-۴۱: کارنمای تمرین ۲۲

۲۳. پس از حل مسئله قبل کارنمای، شکل ۶-۴۱ را مطابق روش مطرح شده در بخش ۶-۷ به گونه‌ای بازنویسی کنید که تنها از یک نقطه به سمت دکمه پایان برود.

۲۴. یک صفحه شطرنج را در نظر بگیرید. یک مهره در خانه گوشه سمت چپ و پایین این صفحه قرار دارد. این خانه را «خانه شروع» می‌نامیم و با مختصات 1×1 نمایش می‌دهیم. این مهره در هر بار حرکت فقط می‌تواند یک خانه به سمت راست یا یک خانه به سمت بالا برود. الگوریتمی بنویسید که کلیه مسیرهای ممکن برای رسیدن این مهره از خانه شروع به خانه‌ای با مختصات $i \times j$ را تولید کند. هر مسیر با دنباله‌ای از R و U مشخص می‌شود که در آن هر حرکت به سمت راست با R و هر حرکت به سمت بالا با U نشان داده می‌شود.

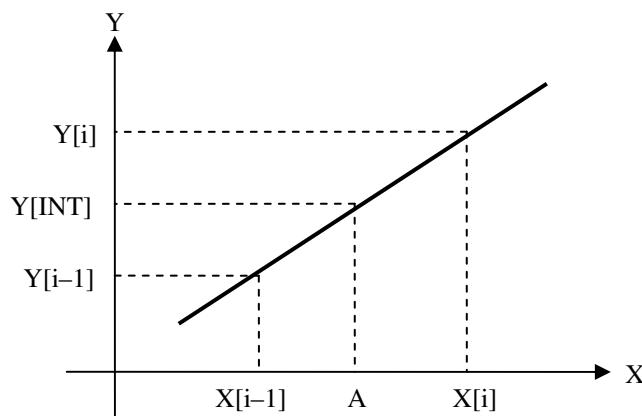
۲۵. جدول مرتبی از مقادیر X و $\sin X$ به شما داده شده است. کار شما ساختن کارنمایی است که:

الف- این مقادیر را به داخل کامپیوتر بخواند،

ب- آنگاه مقداری از A را که ممکن است مطابق یکی از مقادیر X در جدول باشد یا نباشد، را بخواند،

ج- در صورت امکان مقدار نظیر $\sin X$ را چاپ کند. در صورتی که A با هیچ یک از مقادیر X در جدول مطابقت نکند، اگر مقدار A خارج از قلمرو تحت پوشش جدول باشد پیام مناسبی مبنی بر این امر چاپ کند و اگر A بین دو مقدار X قرار می‌گیرد، دو مقدار مذکور و مقادیر $\sin$ نظیر آنها را چاپ کند.

۲۶. شکل ۶-۴۲ را مطالعه کنید و سپس کارنمای مسئله قبل را توسعه دهید به‌طوری که با استفاده از درون‌یابی خطی از دو مقدار دربرگیرنده $\sin X$ در جدول، مقادیر $Y[INT]$ یعنی درون‌یابی خطی $\sin A$ را محاسبه کند.



شکل ۶-۴۲: نمایش درون‌یابی مستقیم

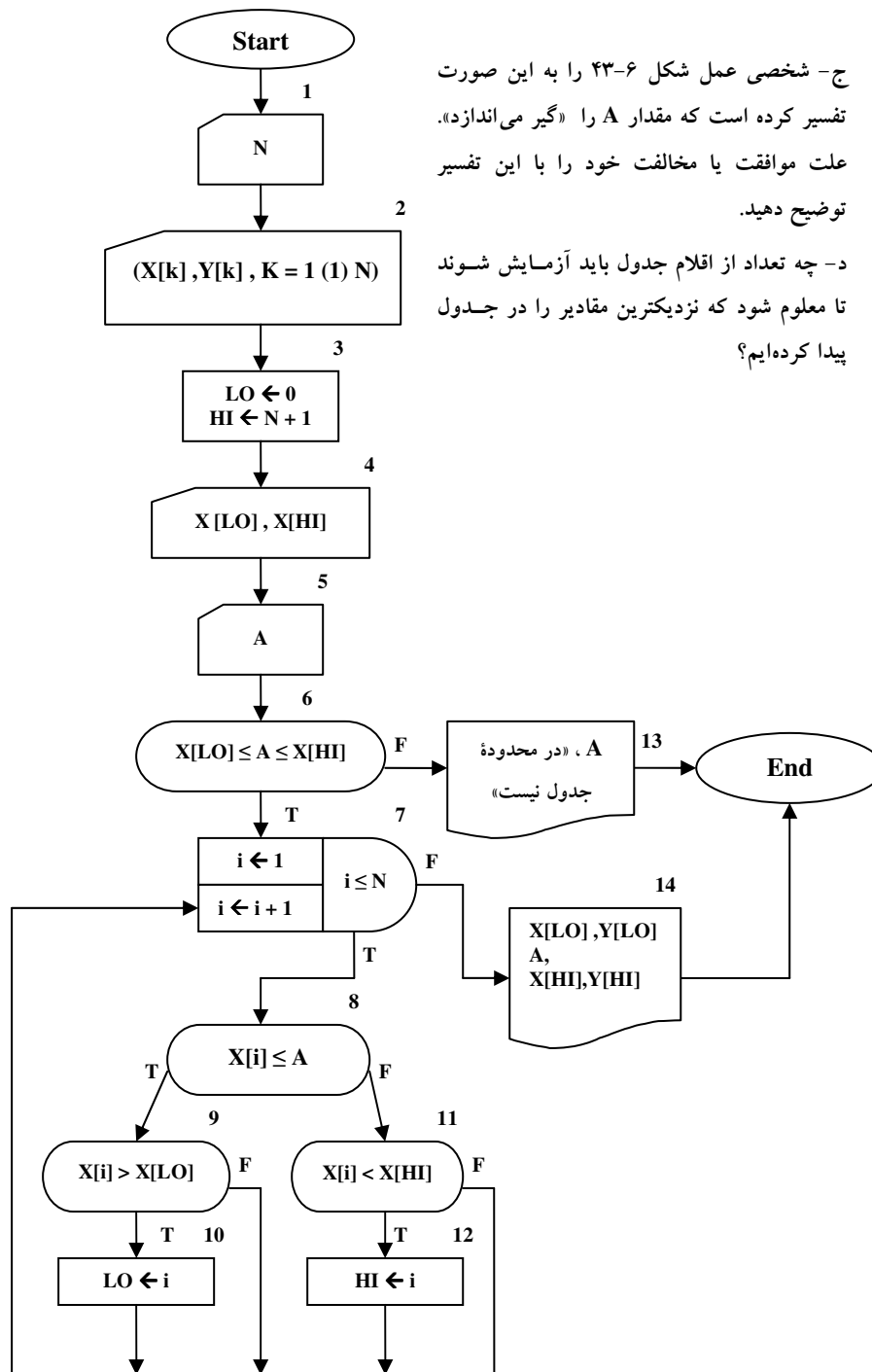
۲۷. جدول مرتب نشده‌ای از مقادیر $(X[k], Y[k] ; k = 1(1) N)$ به ما داده شده است. حداقل و حداکثر مقادیر X معلوم است. می‌خواهیم به دفعات زیاد به این جدول مراجعه کنیم. دو راه در جلوی پای ما باز است:

۱. ابتدا جدول را مرتب کنیم و آنگاه جدول خوانی با روش جستجوی دودویی را انجام دهیم. و یا
۲. کارنمای جدیدی برای جستجو در جدول مرتب نشده بسازیم. شکل ۶-۴۳ راه حلی است برای مورد دوم. در این شکل، مقادیر معلوم حداقل و حداکثر X در جعبه ۴ به داخل خوانده می‌شوند.

شکل ۶-۴۳ را مطالعه کنید و به سؤالات زیر پاسخ دهید.

الف- برای جدول‌های با تعداد ارقام کم، روش (۱) بهتر است یا روش (۲)؟ کدام یک برای جدول‌های بزرگ بهتر است؟

ب- فرض کنید A مساوی $X[LO]$ ، یعنی مقدار حداقلی که برای X داده شده است، باشد. در اولین اجرای جعبه ۸، تحت چه شرایطی مسیر خروجی درست اختیار می‌شود؟



شکل ۶-۴۳: کارنمای تمرین ۲۷